



前 言

《数据通信与计算机网络(第2版)》是面向21世纪课程教材,同时也是普通高等教育“十五”国家级规划教材。该书自出版以来,已被许多学校采用。为了方便大家使用,我们编写了教学参考书,与教材《数据通信与计算机网络(第2版)》配套。全书由两部分组成:第一部分是习题解答,对教材每章练习中的习题都给出参考答案,因为有些题目的答案并不是唯一的,故仅供参考;第二部分是实验指南,共有14个实验。每个实验均由描述、目的、基础知识和要求4部分组成。在基础知识部分介绍了一些与实验相关的知识。有些实验最后还有“提示”,这部分内容是对实验作进一步的深入介绍,或给出一些关键的指示,或提供部分参考的程序源代码。这些实验绝大多数都是与计算机网络及其应用方面编程有关的实验,并不要求配置特殊的网络专用设备,以求为广大师生提供更大的选择余地。相信本书对使用《数据通信与计算机网络(第2版)》一书作为培训教材或自学的人员都是有帮助的。限于时间与水平,书中难免有不妥处,敬请批评指正。

编 者

2005年6月

郑重声明

高等教育出版社依法对本书享有专有出版权。任何未经许可的复制、销售行为均违反《中华人民共和国著作权法》，其行为人将承担相应的民事责任和行政责任，构成犯罪的，将被依法追究刑事责任。为了维护市场秩序，保护读者的合法权益，避免读者误用盗版书造成不良后果，我社将配合行政执法部门和司法机关对违法犯罪的单位和个人给予严厉打击。社会各界人士如发现上述侵权行为，希望及时举报，本社将奖励举报有功人员。

反盗版举报电话：(010) 58581897/58581896/58581879

传 真：(010) 82086060

E - mail: dd@hep.com.cn

通信地址：北京市西城区德外大街4号

高等教育出版社打击盗版办公室

邮 编：100011

购书请拨打电话：(010)58581118

目 录

第一部分 习题解答	1
第一章 概述	2
第二章 物理层与数据通信基础	11
第三章 数据链路层	19
第四章 局域网	32
第五章 高速网络技术	49
第六章 无线网络	56
第七章 网络层	81
第八章 运输层	99
第九章 高层应用	109
第十章 网络管理与信息安全	122
第十一章 多媒体网络	132
第十二章 服务质量控制 QoS	138
第二部分 实验指南	147
实验一 异步串行通信接口实验	148
实验二 数据链路层的帧封装协议	154
实验三 数据链路层的滑动窗口协议	157
实验四 局域网模拟:模拟 CSMA/CD	161
实验五 模拟网络层路由算法	163
实验六 NetBIOS 编程	166
实验七 UNIX 下的 Socket 编程	175
实验八 Winsock 编程(1)	185
实验九 Winsock 编程(2)	193
实验十 支持内容过滤的 HTTP 及 FTP 代理	199
实验十一 模拟 FTP 的客户端	209
实验十二 Internet/Intranet Web 编程	212
实验十三 网络服务器	215
实验十四 网络管理软件	216

第一部分

习题解答

第一章 概 述

1.1 计算机网络的发展经历了几个阶段？各阶段有什么特点？

答：

经历了面向终端的计算机网络、计算机-计算机网络、开放式标准化网络和网络计算的新时代等4个阶段。

面向终端的计算机网络的特点是以单个计算机为中心，连接多个终端，组成一个远程联机系统。只有中心计算机具有自主处理信息的能力。

计算机-计算机网络的特点是将多台计算机主机通过通信线路互连起来为用户提供服务，这里的多台计算机都有自主处理能力，不存在主从关系。

开放式标准化网络的特点是计算机互连，并具有统一的体系结构，遵守统一的国际标准化协议，这样可以使不同的计算机方便地互连在一起。

网络计算的新时代的特点是网络的发展和应用达到了一个非常高的水平，计算机已经进入了以网络为中心的时代，每台计算机必须以某种形式连网，并共享信息或协同工作，否则就无法充分发挥其效用。

1.2 请从计算机网络的发展历史，展望一下今后计算机网络的发展趋势。

答：

首先，计算机网络向高速宽带化发展。早期的以太网的数据速率只有10 Mb/s，目前已被速度高10倍的100 Mb/s的以太网所替代，而速度再提高10倍，就可达到Gb/s（即1 000 Mb/s，又称千兆网），Gb/s的产品已是不少主干网络的首选，10 Gb/s（即万兆网）的产品也已经面世。从远距离的网络来看，早期按照CCITT X.25建议组建的公用分组交换数据网的数据速率只有64 kb/s，后来采用了帧中继技术可提高至2 Mb/s，近年来出现的ATM又可达到155 Mb/s、622 Mb/s甚至2.5 Gb/s的数据速率，目前已有高达6 400 Gb/s的密集波分多路复用DWDM产品，即在一根光纤上传输640路光波，每路光波为10 Gb/s。高速的网络必然需要新的高速交换和路由技术支持。从端用户（如家用计算机）的网络接入速率看，也从早期的2 400 b/s低速调制解调器过渡到56 kb/s的高速调制解调器，可提供128 kb/s上网的窄带综合业务数字网N-ISDN（即俗称“一线通”），直至目前端用户可得到数百kb/s上网速率的ADSL和有线电视经双向改造后的HFC（即俗称“有线通”）接入。未来VDSL（超高速数字用户线）和光纤到户则可达到更高的网络接入速率。

其次，早期计算机网络中传输的主要是数字、文字和程序等数据，但随着应用的扩展，提出了越来越多的图形、图像、声音和影像等多媒体信息在网络中传输的需求。网络的多媒体应用不但

要求网络有更高的数据速率或者说带宽,而且对延迟时间(实时性)及时间抖动(等时性)等方面都提出了更高的要求。传统的 Internet 是一个只能提供尽力(Best-effort)服务的网络,换句话说,它不能保证服务质量(Quality of Service, QoS),这促进了集成服务(Integrated Service, Int-Serv)、区分服务(Differentiated Service, DiffServ)和多协议标签交换(Multi-Protocol Label Switching, MPLS)等网络技术的研究和发展。另外,许多多媒体应用,如视频会议(Video Conference)和远程教育等,都要求从一个源发送的信息同时通过网络传输到多个目标,从而使得组播(Multicast)也成为目前网络研究和发展的方向之一。一方面,网络技术的发展使得其应用的范围与领域越来越广泛,可以说已涉及人类生活的各个方面;另一方面,应用五花八门的需求反过来又成为推动网络技术发展的强大动力,使得各种新的技术与进展层出不穷,如虚拟局域网(Virtual Local Area Network, VLAN)、虚拟专网(Virtual Private Network, VPN)、无线局域网(Wireless Local Area Network, WLAN)、新一代的网际协议 IPv6、对等计算(Peer-to-peer Computing)、网格(Grid)、移动自组网(Mobil Ad Hoc Network)和主动网(Active Network)等,在此不再一一详述。可以预计无线化以实现移动连网也是一个重要的发展方向,随着计算机网络的发展,有线网络设备也在一定程度上限制了网络的应用,而无线网络设备的推广,则更加加强了网络数据资源共享的优势。现在 11 Mb/s 的 IEEE 802.11b WLAN 已有许多成熟的商品并得到广泛的应用,更高数据速率的 WLAN 也正在逐步推出。在广域的范围,号称第二代半(2.5G)的移动通信网:窄带 CDMA(Code Division Multiple Access)和架构在现有的第二代(2G)移动通信网 GSM(Global System for Mobile communication)上的通用无线分组业务(General Packet Radio Service, GPRS)已经能支持一定的数据传输业务。未来的第三代(3G)的移动通信网:宽带 CDMA 可以对多媒体的数据传输提供更好的支持。随着无线移动网络的发展,也促进了接入设备的多样化,早期的网络终端设备主要是计算机,未来各种手持式的设备,如手机、个人数字助理(Personal Digital Assistant, PDA)、MP3 等以及传统的家用电器设备都会拥有连网的功能。

目前,电话、有线电视和数据等都有各自不同的网络,随着多媒体网络的建立和日趋成熟,三网融合甚至多网融合会是一个重要的发展方向。处于核心的是能传输各种多媒体信息的高速宽带主干网(Backbone),外连许多汇集点(Point Of Presence, POP)。端用户(User)可以通过电话线、电视电缆、光缆、无线信道等不同的传输媒体和技术组成不同的接入网(Access Network),再由汇集点集中后连入主干网。整个网络中的核心协议将采用 Internet 的网际协议 IP,通过它把下面各种各样的通信子网络互连在一起,并向上支持多种多媒体应用,这就是所谓的统一的 IP 网络。网络覆盖的地理范围会不断扩大,向全球延伸,并逐步深入到每个单位、每个办公室或每个家庭。有人设想未来通信和网络的目标是实现 5W 的个人通信,即任何人(Whoever)在任何时间(Whenever)、任何地方(Wherever)都可以和任何一个其他人(Whomever)通过网络进行通信,以传送任何信息(Whatever)。

1.3 数据通信子网是计算机网络的一个组成部分。试列出在计算机网络的工作中哪些是由数据通信子网完成的,哪些不是。

答:

数据通信子网的作用只是进行信息的传输,例如,将某一个数据分组从源地址经过物理媒体、转发器、交换机和路由器等通信设备传输到目的地址就是由数据通信子网完成的,而数据通信子网并不对数据分组中所包含的信息进行处理,后者是由作为资源子网的一部分并通过通信子网互连在一起的计算机来负责处理的。

1.4 计算机网络有哪些应用功能?

答:

计算机网络的功能可归纳为资源共享、提供人际通信手段、提高可靠性、节省费用、便于扩充、分担负荷及协同处理等方面。这些方面的功能相互之间也是相辅相成的。正因为计算机网络有如此多的功能,使得它在工业、农业、交通运输、邮电通信、文化教育、商业、国防、科学研究和电子政务等各个领域日益获得越来越广泛的应用。现在的计算机网络的应用已经深入到人类社会生活的各个方面,甚至对每个人的日常生活或娱乐方式都会产生很大影响。

1.5 试观察并举出一个日常生活中你所接触到的计算机网络应用的例子。

答:

如网络浏览、电子邮件、IP 电话、视频会议或网上购物等(具体略)。

1.6 简述开放系统互连参考模型及各层的主要功能。

答:

开放系统互连参考模型是由国际标准化组织(ISO)制定的一个标准化开放式的计算机网络层次结构模型。它共有 7 层,从下到上依次是物理层、数据链路层、网络层、运输层、会话层、表示层和应用层。各层的主要功能分别如下:

- 1) 物理层:在物理媒体上传输原始数据的比特流。
- 2) 数据链路层:通过校验、确认和反馈重发等手段将原始的、肯定会出错的物理连接改造成无差错的数据链路。
- 3) 网络层:关心的是通信子网的运行控制,主要任务是把网络协议数据单元从源端传送到目的端。
- 4) 运输层:建立一个端到端的连接,为上层用户提供端到端的、透明优化的数据传输服务。
- 5) 会话层:允许不同主机的各个进程之间进行会话,它组织并同步进程间的对话。
- 6) 表示层:为上层用户提供共同需要的数据或信息的语法表示变换。
- 7) 应用层:开放系统互连环境的最高层。它直接面向网络的应用程序,不同的应用层为特定类型的网络应用提供访问 OSI 环境的手段。

1.7 为了使得协议数据单元能够被正确地传送到目的地的对等层,在每层协议附加的报头

中通常包含一些什么样的信息?

答:

首先,为了能够正确地将协议数据单元传送到目的端,在报头中必须包含该层虚通信源端和目的端信息。例如,在 IP 分组中包含的源 IP 地址和目的 IP 地址,在 Ethernet 数据帧中的源 MAC 地址和目的 MAC 地址等。其次,必须包含一些和相邻层有关的说明信息,以说明本层的服务和相邻层的服务。再者,还需要一些对于本身数据的描述和校验,例如指出该协议数据单元的性质和格式、长度信息、校验码等。

1.8 判断题:ISO's OSI 协议层的辨析。

- 1) 物理层不分辨传输的内容;
- 2) 运输层是由下到上第一个进行流量控制的层;
- 3) 数据链路层进行的差错控制主要是通过帧序号来进行的;
- 4) 会话层进行数据加密和数据压缩。

答:

- 1) 正确。物理层只传输原始比特流,定义机械、电气等的功能,不分辨传输的内容。
- 2) 错误。在数据链路层已经有基本的流量控制功能,它负责和一条数据链路直接相连的两个节点间的流量控制,只不过运输层是由下到上第一个进行端到端流量控制的层。
- 3) 错误。在数据链路层的差错控制主要是通过检错码和纠错码(统称校验码)来完成的,帧序号只是在反馈重发中的一种辅助手段。
- 4) 错误。数据加密和数据压缩都可以看成是一种数据或信息语法表示的变换,在 ISO's OSI 中属于表示层而不是会话层的功能。

1.9 填空题

- 1) 面向终端的计算机网络又称为 _____;第二代计算机网络的典型代表是 _____;在开放式标准化网络发展中曾发挥重要作用的 CCITT 现已改组更名为 _____;未来三网融合的宽带主干网将是 _____。
- 2) 开放系统互连参考模型(OSI)是由 _____ 制定的国际标准。OSI 模型中,低层在相邻层接口的 _____ 处通过服务原语向高层提供服务,有四种类型的服务原语,其中 _____ 和 _____ 由高层发往低层,前者用于互动的主动方、后者用于被动方;而 _____ 和 _____ 则由低层发往高层,同样前者用于主动方。服务过程中传递的信息单元称为 _____。
- 3) Internet 参考模型也是一个层次模型,但比 OSI 参考模型简单,依次包括子网层、_____、_____和_____。Internet 协议套中属于 OSI 模型网络层的著名协议为_____。Internet 中相当于 OSI 运输层最常用的两种协议为_____和_____,后者较适合于可靠性要求高的通信。

4) 接入分组交换公用数据网的 DTE 必须遵循原 CCITT _____ 建议与公用数据网的 _____ 连接。非智能终端必须通过称为 _____ 的装置才能接入公用数据网,在 _____ 中规定了其功能和参数;两个公用数据网互连的标准是 _____,它规定了分属于该两个网络中的 _____ 设备间的通信协议。

答:

1) 面向终端的计算机网络又称为远程联机系统;第二代计算机网络的典型代表是ARPA 网 (ARPANET);在开放式标准化网络发展中曾发挥重要作用的 CCITT 现已改组更名为ITU-T;未来三网融合的宽带主干网将是IP 网。

2) 开放系统互连参考模型(OSI)是由国际标准化组织 ISO 制定的国际标准。OSI 模型中,低层在相邻层接口的服务访问点 SAP处通过服务原语向高层提供服务,有四种类型的服务原语,其中请求和响应由高层发往低层,前者用于互动的主动方、后者用于被动方;而证实和指示则由低层发往高层,同样前者用于主动方。服务过程中传递的信息单元称为服务数据单元 SDU。

3) Internet 参考模型也是一个层次模型,但比 OSI 参考模型简单,依次包括子网层、互联网层、运输层和应用层。Internet 协议套中属于 OSI 模型网络层的著名协议为IP。Internet 中相当于 OSI 运输层最常用的两种协议为UDP和TCP,后者较适合于可靠性要求高的通信。

4) 接入分组交换公用数据网的 DTE 必须遵循原 CCITT X.25 建议与公用数据网的 DCE 连接。非智能终端必须通过称为 PAD 的装置才能接入公用数据网,在 X.3 中规定了其功能和参数;两个公用数据网互连的标准是 X.75,它规定了分属于该两个网络中的 STE 设备间的通信协议。

1.10 什么是网络协议(Protocol)? 请例举一些你所知道的网络协议并说明它们的相关功能。

答:

协议,是计算机网络对等层通信时,对传送的信息内容的理解、信息表示形式以及各种情况下的应答信号都必须遵循的一个共同的约定。在计算机网络的每个层次上都有若干协议。在日常的网络应用过程中,常见的协议如下:

- 1) FTP (文件传输协议)——用来传输文件;
 - 2) SMTP(简单邮件传输协议)——用来发送邮件;
 - 3) HTTP(超文本传输协议)——Web 网页浏览的基础;
 - 4) TCP(传输控制协议)——面向连接的运输层协议;
 - 5) UDP(用户数据报协议)——无连接的运输层协议;
 - 6) IP(网际协议)——网络层互联的基础;
 - 7) DHCP(动态主机配置协议)——动态获得主机 IP 地址等信息;
 - 8) ARP(地址解析协议)——将 IP 地址翻译成 MAC 地址;
- 等等。

1.11 试比较证实的服务和非证实的服务,并举例说明何时采用。

答:

证实的服务要使用请求、指示、响应和证实四类服务原语,而非证实的服务只使用请求和指示两类服务原语。证实的服务需在对等方向来回一次,花费较多的时间,但增加了可靠性。非证实的服务只用两个原语,不需来回应答,实现简单,但可靠性不高。建立连接的服务通常都是证实的服务,被叫方既可以同意建立连接,也可以拒绝建立连接。数据传送以及断开连接服务根据需要可采用证实的服务或非证实的服务方式。

1.12 比较 OSI 参考模型和 Internet 参考模型的异同点。

答:

相同点:都是层次模型;都有运输层,且它在两个参考模型中都是第一个提供端到端数据传输服务的层次,都能提供面向连接或无连接的运输服务;其最高层都是向各种用户应用进程提供服务的应用层等。

不同点:两者划分的层次数不同;对比 OSI 模型,Internet 中没有表示层和会话层;Internet 没有明确规定通信子网的协议,也不再区分通信子网中的物理层、数据链路层和网络层;Internet 中特别强调了互连网层,其中运行的 IP 协议是 Internet 的核心协议,且互连网层向上只提供无连接的服务,而不提供面向连接的服务等。

1.13 举出 3 个在数据通信和计算机网络标准化方面最有影响的国际组织,并比较其工作范围的异同。

答:

国际标准化组织 ISO、国际电信联盟电信标准化部 ITU-T 和因特网工程特别任务组 IETF。

ISO 的工作范围很广,除了数据通信和计算机网络以外,还包括其他很多领域,所有有关信息处理的标准都属于其工作范围,而且只是其工作范围的一部分。

ITU 是电信界最有影响力的标准化组织,早在 1865 年就开始运作。当时还主要是电报标准化的工作,当电话开始提供国际服务时,ITU 又接管了电话标准化的工作。1947 年 ITU 成为联合国下属的一个官方组织。ITU 有 3 个主要部门(Sector),即无线通信(Radio Communications)部 ITU-R、开发(Development)部 ITU-D 和 ITU-T。ITU-T 负责在电话和数据通信领域提出称为建议的标准,其前身是国际电话电报咨询委员会 CCITT。

IETF(Internet Engineering Task Force)致力于解决 Internet 近期发展的工程问题,制定 Internet 中采用的标准是其工作任务之一。IETF 还负责维护 Internet 中被称为“请求评估”(Request For Comments, RFC)的在线存放的技术文件。Internet 的标准都有一个 RFC 的编号,如著名的 IP 协议和 TCP 协议的文件最早分别为 RFC 791 和 RFC 793。但是,反之则不然,并不是每个 RFC 文件都是 Internet 的标准。有的 RFC 文件只是提出一些新的思想和建议,也可以对原有的一些 RFC 文件进行增补和修订。

1.14 原 CCITT 制定了一系列与公用分组交换数据网有关的建议。试问哪些建议分别规定了

- 1) 数据终端设备与公用数据网的接口;
- 2) 两个公用数据网互连时 STE 间的接口;
- 3) 分组组装/拆卸器的功能和参数;
- 4) 普通终端和 PAD 之间的协议。

答:

- 1) X.25;
- 2) X.75;
- 3) X.3;
- 4) X.28。

1.15 TCP 和 UDP 是 Internet 模型中运输层的两种协议,试概述两者的差异和优缺点。

答:

TCP 提供的是面向连接的服务。面向连接的服务需要在用户之间建立连接后,再传输数据,而当数据发送完毕后,必须关闭连接。它能够提供一定的差错和流量控制,从而保证按顺序传输。而 UDP 是面向无连接的服务,所以它不提供上述机制,从而也不能保证报文不丢失和一定按序到达。在网络拥挤的情况下,如果要求更为可靠的顺序报文传输,则需要使用 TCP 协议;而在网络状态良好的情况下,使用 UDP 协议则能够达到较高的效率。

1.16 列出你所知道的 Internet 的主要功能。

答:

例如,远程登录、文件传输、电子邮件、电子公告牌(BBS)与网络新闻、文件查找、人员查找、菜单式信息浏览(Gopher)、按内容自动查找以及超媒体 Web 信息浏览等。

1.17 Internet 中,通过路由器、交换机等网络中间设备将数据包从源端传送到目的端,那么路由器是否知道所转发的数据包属于哪两台通信的端主机?

答:

路由器、交换机等网络中间设备,主要工作在下 3 层,即物理层、数据链路层和网络层。因此,理论上说,路由器并不知道所传送的数据是属于哪两台端主机。运输层才是第 1 个端对端的层,也就是说,只有工作在该层以上的设备才能够意识到端的存在。不过,目前有些高级的路由交换机产品也可提供第 4 层(即运输层)甚至更高层的交换功能,在运输层上它就可看到所转发的数据包属于哪两台通信的端主机。

1.18 各举一个你所知道的局域网、校园网、城域网、广域网和全球网的例子。

答:

局域网——某机关一幢楼内的办公网;

校园网——某部门大院内的网络(园区网)或某大学校园内的网络;

城域网——上海市公务网;

广域网——中国公用数据网;

全球网——Internet。

1.19 简述可以从几种不同角度来对计算机网络分类。

答:

按地理范围分类:局域网 LAN、校园网 CAN(又称园区网)、城域网 MAN、广域网 WAN 和全球网 GAN;

按拓扑构型分类:星形、总线型、环形和不规则网形;

按物理传输媒体分类:双绞线网、同轴电缆网、光纤网、无线网和卫星网等;

按使用范围分类:公用网和专用网。

1.20 在由 5 个 IMP 构成的数据通信子网中,若每对 IMP 之间可选择采用一条高速线路、中速线路、低速线路或不设线路,问一共有多少种可能的不同拓扑结构?

答:

5 个 IMP 之间可有 $C_5^2 = (5 \times 4) / (2 \times 1) = 10$ 条线路,每条边线路有 4 种可能性,所以共有 4^{10} 种不同的拓扑结构。

1.21 数据链路层通常通过反馈重发的手段来处理原始物理连接的错误。设帧在物理连接上发生差错的概率为 P ,问在确认反馈不出错的情况下,正确地发一帧平均需发送多少次?

答:

正确地发送一帧平均需要的发送次数:

$$n = 1 \times (1 - p) + 2 \times p \times (1 - p) + 3 \times p \times (1 - p) + \dots = \frac{1}{1 - p}$$

1.22 在一个 n 层的网络系统中,每层协议分别要求加上 H_n 字节长的报头。若送往该网络的应用数据长度为 A 字节,问在物理媒体的带宽中有多少百分比是用来传输有效应用数据的?

答:

$$\frac{A}{H_n + H_{n-1} + \dots + H_2 + H_1 + A} \times 100\%$$

1.23 若某运载工具能以 80 km/h 的速度一次传递 10 盒 8 mm 的磁带,每盒 3 盘带,每盘带

容量为 7 GB。问在什么距离范围内,该运载工具的数据传送速率比 155 Mb/s 的 ATM 网络要高?

答:

$$(10 \times 3 \times 7 \times 1\,024 \times 8) / (3\,600 \times S/80) > 155$$

在小于

$$S = 80 \times \frac{10 \times 3 \times 7 \times 1\,024 \times 8}{155 \times 3\,600} \text{ km} \approx 246.4 \text{ km}$$

的距离范围内该运载工具的数据传送速率比 155 Mb/s 的 ATM 网络要高。

1.24 现代化的小区正在发展一种“智能化住宅”,其设想是形成一种家庭网络(Home Network),使房屋中的各种电器设备能进行相互通信,并可以连上 Internet。试举例家中的哪些设备可被连网。

答:

- (1) 计算机类:台式计算机、笔记本电脑、PDA 和共享的外设(如打印机、扫描仪等)等。
- (2) 通信类:固定电话、手机、传真机和对讲等。
- (3) 监测类:水电表、烟雾报警器、防盗报警器和温度调节装置等。
- (4) 应用类:微波炉、冰箱、空调、时钟和电灯等。
- (5) 娱乐设备类:电视机、DVD 机、录像机、摄像机、照相机、音响和 MP3 等。

1.25 对于一幅 $1\,024 \times 768$ 像素的图像,每个像素需要 3 B。假设图像没有经过压缩,也不考虑网络传输中增加的额外开销,那么用 56 kb/s 的调制解调器传输需要多长时间?用 500 kb/s 的有线宽带呢?用 10 Mb/s 的以太网呢?用 155 Mb/s 的 ATM 网呢?

答:

$$1\,024 \times 768 \times 3 \times 8 = 18\,874\,368 \text{ b}$$

$$\text{调制解调器: } 18\,874\,368 / (56 \times 1\,000) = 337.04 \text{ s}$$

$$\text{有线宽带: } 18\,874\,368 / (500 \times 1\,000) = 37.75 \text{ s}$$

$$\text{以太网: } 18\,874\,368 / (10 \times 1\,000\,000) = 1.89 \text{ s}$$

$$\text{ATM 网: } 18\,874\,368 / (155 \times 1\,000\,000) = 0.12 \text{ s}$$

1.26 在有 n 个节点的星形拓扑结构中,有多少个直接连接?全连接呢?

答:

星形: $n - 1$

全连接: $n \times (n - 1) / 2$

第二章 物理层与数据通信基础

2.1 简述通信系统的模型。

答：

教材《数据通信与计算机网络(第2版)》中图2.1概括了一般点到点的通信系统模型。该模型中包含信源、变换器、信道、噪声源、反变换器和信宿。信源和变换器构成发送端,发送端信源将各种信息转换成原始电信号,再由变换器转换成适合在信道上传输的信号,向信道发送。信道是信号的传输媒体及有关的设备。噪声源是信道中噪声以及分散在通信系统其他各处的噪声的集中表示。信宿和反变换器构成接收端。通过信道传输到远地的电信号由接收端的反变换器转换复原成原始的电信号,再送给信宿,然后由信宿将其转换成各种信息。

2.2 什么是数据通信?

答：

简单地说,数据通信就是数字计算机或其他数字终端装置之间的通信。

2.3 计算函数 $f(t) = t (0 \leq t \leq 1)$ 的傅里叶系数。

答：

$$c = 2 \int_0^1 t dt = 1$$

$$a_n = 2 \int_0^1 t \cdot \sin(2n\pi \cdot t) dt = -\frac{1}{n\pi}$$

$$b_n = 2 \int_0^1 t \cdot \cos(2n\pi \cdot t) dt = 0$$

2.4 填充题

1) 数字信号通过模拟信道传输时要使用称为_____的装置,而模拟信号通过数字信道传输时则要使用_____。

2) 根据光线在光纤内部的传输方式,光纤分为两种,分别是_____和_____,后者能传输更长的距离和达到更高的数据速率。

答：

1) 数字信号通过模拟信道传输时要使用称为调制解调器 (Modem)的装置,而模拟信号通过数字信道传输时则要使用编码解码器 (CODEC)。

2) 根据光线在光纤内部的传输方式,光纤分为两种,分别是多模光纤和单模光纤,后者能传输更长的距离和达到更高的数据速率。

2.5 普通电话的一路话频线路带宽通常为 4 kHz,如果用其直接传输数据速率为 9 600 b/s 的数字信号,可以通过几次谐波? 若速率仅为 1 200 b/s,则情况是否改善?

答:

先看数据速率为 9 600 b/s 的情况:

以发送 1 B(8 b)数据的时间为周期:

$$T = 8 \text{ b} / 9\,600 \text{ b/s} \approx 0.83 \text{ ms}$$

基波频率为

$$f = 1/T = 1/0.83 = 1\,200 \text{ Hz}$$

所以 4 kHz 的带宽能够通过谐波次数为

$$4/1.2 \approx 3(\text{次})$$

若数据速率为 1 200,则

发送 8 b 数据时间为

$$T = 8 \text{ b} / 1\,200 \text{ b/s} \approx 6.67 \text{ ms}$$

基波频率为:

$$f = 1/T = 1/6.67 \approx 150 \text{ Hz}$$

所以 4 kHz 的带宽能够通过谐波次数为:

$$4/0.15 \approx 26(\text{次})$$

2.6 对于带宽为 4 kHz 的信道,若有八种不同的物理状态来表示数据,信噪比为 30 dB。问:按奈奎斯特定理,最大限制的数据速率是多少? 按香农定理,最大限制的数据速率是多少?

答:

按题意, $H = 4 \text{ kHz}$, $L = 8$, $S/N = 10^{(30/10)}$ 。

按奈奎斯特定理,最大限制的数据速率是

$$C = 2 \times 4 \times \log_2 8 = 24 \text{ kb/s}$$

按香农定理,最大限制的数据速率是

$$C = 4 \times \log_2 (1 + 10^{30/10}) \approx 40 \text{ kb/s}$$

2.7 若电视信道的带宽为 6 MHz,假定无热噪声并使用 4 电平的数字信号,每秒钟能发送的比特数不会超过多少?

答:

按题意, $H = 6 \text{ MHz}$, $L = 4$ 。按奈奎斯特定理,每秒钟能发送的比特数不会超过

$$C = 2 \times 6 \times \log_2 4 = 24 \text{ Mb}$$

2.8 一个数字信号通过两种物理状态经信噪比为 20 dB 的 3 kHz 信道传送。其数据速率不会超过多少？

答：

按题意, $H = 3 \text{ kHz}$, $L = 2$, $S/N = 10^{\frac{20}{10}}$ 。

按照奈奎斯特定理, 其数据速率不会超过

$$C = 2H = 2 \times 3 \text{ kHz} = 6 \text{ kb/s}$$

而按照香农定理, 数据速率不会超过

$$C = 3 \times \log_2 (1 + 10^{\frac{20}{10}}) \approx 20 \text{ kb/s}$$

所以最大的数据速率为 6 kb/s。

2.9 若要在一条 50 kHz 的信道上传输 1.544 Mb/s 的 T1 载波, 信噪比至少要多大？

答：

$$1.544 \times 1000 = 50 \times \log_2 (1 + 10^{\frac{K}{10}})$$

$$\Rightarrow K = \lg \left(2^{\frac{1.544 \times 1000}{50}} - 1 \right) \times 10 \approx 90 \text{ dB}$$

2.10 要在带宽为 4 kHz 的信道上用 2 秒钟发送 80 kb 的数据块, 按照香农 (Shannon) 公式, 信道的信噪比最小应为多少分贝 (取整数值)？

答：

按照香农 (Shannon) 公式：

$$80 \text{ kb}/2 = 4 \text{ Kb} \times \log_2 (1 + S/N)$$

$$S/N = 2^{10} - 1 = 1023$$

由此可得

$$\text{信噪比} = 10 \lg (S/N) > 30 \text{ dB}$$

故至少要 31 dB。

2.11 若使用红外线来传输一系列的电脑动画, 假设每帧的大小是 640×480 , 色彩为 24 位真彩, 每秒传送 60 帧, 每 Hz 调制 1 位, 则对于波长为 $1.3 \mu\text{m}$ 的红外线, 其波动范围为多少才能承载该传输？

答：

传输速率的要求：

$$640 \times 480 \times 24 \times 60 \text{ b/s} = 442368 \text{ kb/s} = 442.368 \text{ Mb/s}$$

相应的要求信道带宽为:

$$\Delta f = 442.368 \text{ MHz}$$

波动范围:

$$\Delta \lambda = (\lambda^2) \times \Delta f / c = (1.3 \times 10^{-6})^2 \times (4.42368 \times 10^8) / (3 \times 10^8) = 2.49 \times 10^{-6} \mu\text{m}$$

2.12 某微波的波长为 $1 \mu\text{m}$, 波动范围为 $0.1 \mu\text{m}$, 则该微波信道的带宽是多少?

答:

带宽 $\Delta f = c \times \Delta \lambda / \lambda^2$, 按题意 $\lambda = 1 \mu\text{m}$, 而 $\Delta \lambda = 0.1 \mu\text{m}$

故有带宽为 $(3 \times 10^8) \times (0.1 \times 10^{-6}) / (1 \times 10^{-6})^2 \text{ Hz} = 3 \times 10^{13} \text{ Hz}$

即带宽为 30 THz 。

2.13 简述各种物理媒体的优缺点。

答:

双绞线: 优点是价格便宜, 易于铺设和安装; 缺点是性能较差, 相对而言数据速率低、传输距离短, 且易受干扰。

同轴电缆: 优点是与双绞线相比数据速率较高、传输距离较长、抗干扰能力较强, 因其能屏蔽大部分电磁干扰; 缺点是价格相对较贵, 且铺设和安装不及双绞线方便。

光纤: 优点是轻便、传输距离远和大容量 (即数据速率高), 且不受电磁干扰; 缺点是光纤之间不易连接, 抽头分支困难, 光电转换设备不便宜。

无线通信: 优点是无需布线, 支持移动性; 缺点是易受干扰。

卫星: 优点是无需布线, 且能进行全球范围内的长距离通信; 缺点是延迟时间长, 且难以防止非法接收。

2.14 若在相隔 1000 km 的两地间要传送 3 kb 位的数据。可以通过地面电缆以 4.8 kb/s 的数据速率传送或通过卫星信道以 50 kb/s 的数据速率传送。问从开始发送到接收方收到全部数据为止用哪种方式传送时间较短?

答:

从开始发送到接收方收到全部数据为止的时间 = 传播时间 + 传输时间

通过地面电缆:

$$T = 1000 / 5000000 + 3 / 4.8 = 0.6252 \text{ s}$$

通过卫星:

$$T = 0.27 + 3 / 50 = 0.33 \text{ s}$$

因此, 由卫星传送时间较短。

2.15 比较模拟传输和数字传输的不同。

答:

模拟传输是不考虑传输内容的传输方式。在传输过程中由于噪声的干扰和能量的损失会使信号发生失真和衰减。为了避免这种情况,需要在传输过程中通过放大器来放大信号的强度,但放大信号的同时也放大了噪音。

而数字传输需要考虑传输内容。传输过程中每间隔一定距离使用转发器来代替放大器。转发器通过判别内容而重新产生信号发送,所以又称为再生器(Regenerator)。它不引起失真,也不会放大噪音,特别适合于长距离的传输。

2.16 一个简单的电话系统由两个终端局和一个长途局组成,每个终端局用 1 MHz 的全双工中继线连到长途局。平均每台电话在 8 小时的工作日中使用 4 次,每次通话平均持续 6 分钟,其中 10% 的通话是长途电话(即经过长途局的)。若设每路话音带宽为 4 kHz,问每个终端局能够支持的电话机最多台数是多少?

答:

按频分多路复用,每个终端局最多能够支持

$$1 \text{ MHz} / 4 \text{ kHz} = 250 \text{ 台电话}$$

按时分多路复用,每个终端局最多能够支持

$$(8 \times 60) / (4 \times 6 \times 10\%) = 200 \text{ 台电话}$$

故总共每个终端局最多能够支持 $250 \times 200 = 50\,000$ 台电话。

2.17 某调制解调器同时使用幅移键控和相移键控。采用 0 、 $\pi/2$ 、 π 和 $3\pi/2$ 四种相位,每种相位又都有两个不同的幅值。问在波特率为 1 200 的情况下数据速率是多少?

答:

$$C = B \log_2 L$$

按题意给定时刻信号可能取的离散值的个数为 $4 \times 2 = 8$, 即 $L = 8$,

$$\text{故数据速率} = 3\,600 \text{ b/s}$$

2.18 调制解调器和编码器的功能有何异同点? 能否用调制解调器的解调部分来代替编码解码器的编码部分?

答:

调制解调器用于在发送端将数字信号调制成模拟信号,在接收端将模拟信号复原成数字信号;而编码解码器在发送端将模拟信号变换成数字信号,在接收端将收到的数字信号复原为模拟信号。调制解调器的解调部分不适宜用来代替编码解码器的编码部分,因为解调处理的模拟信号是按照一定规则(如调幅、调频或调相)经过调制而得到的信号,而编码部分所接收的模拟信号是无规则的,必须经过严格的采样、量化再进行编码。

2.19 一个 4 kHz 的模拟信号若采用 4 位编码的 DPCM 方式进行数字传输,需要多大的数据速率?若改采用增量调制的方法,需要多大的数据速率?

答:

4 kHz 模拟信号的采样频率为 8 kHz。

若采用 4 b 编码的 DPCM 方式进行数字传输,数据速率为

$$8 \text{ kHz} \times 4 \text{ b} = 32 \text{ kb/s}$$

若改用增量调制的方法,每个采样点仅用 1 b 编码,数据速率为

$$8 \text{ kHz} \times 1 \text{ b} = 8 \text{ kb/s}$$

2.20 某单位总机共有 K 条中继出口线。若外线电话是随机发生的,且服从普阿松分布,两次相邻外线电话间的平均间隔时间为 $1/\lambda$ s,且每次外线电话的通话时间也服从负指数分布,平均为 $1/\mu$ s。在中继出口线不空时,电话在单一队列中按先到先服务的规则排队等候。试推导出因所有中继线都被占用而电话需排队等候的概率的表达式。

答:

可以将此问题看成是一 $M/M/1$ 排队系统。平均 $1/\lambda$ s 就有电话产生(进队),平均 $1/(k\mu)$ s 就有电话结束(出队),即平均到达率为 λ 、平均服务率为 $k\mu$,所以

$$\rho = \lambda / (k\mu)$$

因所有中继线都被占用而电话需排队等候的概率就是系统中外线电话大于 K 的概率,它等于

$$\begin{aligned} P_{K+1} + P_{K+2} + \cdots &= (\rho^{K+1} + \rho^{K+2} + \cdots) \times P_0 = \rho^{K+1} \times 1/(1-\rho) \times (1-\rho) \\ &= \rho^{K+1} \end{aligned}$$

2.21 对一个振幅为 A 的正弦波进行增量调制,若输出 1 和 0 分别表示 $+A/8$ 和 $-A/8$ 的信号变化。若采样频率为 X ,且没有累积误差,问可以跟踪的最高频率是多少?

答:

正弦波信号在 $1/4$ 周期($T/4$)内的变化幅度为 A ,为了增量调制时可以跟踪该信号,在 $1/4$ 周期($T/4$)内必须有 8 个采样点,或者说采样频率 $X > 32/T$ 。因此,采样频率为 X 时可以跟踪的最高频率是

$$X/32$$

2.22 若在带宽为 4 kHz 的普通电话线上,按照美国标准,每次采样为 8 位,其中有 1 位用于控制,那么理论上,利用传统电话线,经过脉码调制后,能达到的数据速率为多少?

答:

根据奈奎斯特定理,若模拟信号带宽为 H ,那么采样频率为 $2H$ 即可,因此在 4 kHz 电话线上,采样频率为 $4 \text{ kHz} \times 2 = 8 \text{ kHz}$ 次,每次的采样为 8 b,除去一位用来控制,传输数据的信号为

7 b, 所以理论速率为 $7 \times 8 \text{ k} = 56 \text{ kb/s}$ 。

2.23 现有 10 个信号, 共同频分复用同一个信道, 每个信号需要 4 000 Hz, 假设两边各要用 200 Hz 的保护带宽保证不受相邻信道的干扰, 那么所需的最小带宽为多少?

答:

最小带宽: $4\,000 \times 10 + 200 \times 2 \times 10 = 44\,000 \text{ Hz} \approx 44 \text{ kHz}$

2.24 QAM-16 使用四种幅移和四种相移, 共 $16 = 2^4$ 种组合, 使得每波特可传送 4 位的数据。那么在 2 400 波特率的线路上, 使用 QAM-128 能够达到多少比特率?

答:

$128 = 2^7$, 所以每波特可传输 7 b 的数据, 因此比特率为 $2\,400 \times 7 = 16\,800 \text{ b/s}$

2.25 绘出比特流 011000101111 的基本曼彻斯特编码波形图。

答:

如图 2.1 所示。

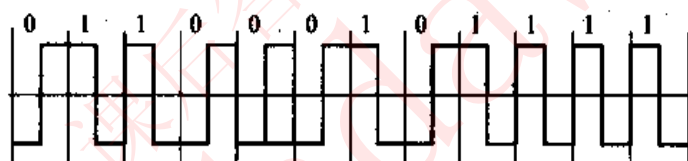


图 2.1

2.26 绘出上题比特流的差分曼彻斯特编码波形图。

答:

如图 2.2 所示。



图 2.2

2.27 写出比特流 011000101111 经 4B/5B 的 5 位编码后的码组流。

答:

如图 2.3 所示。

011000101111 $\xrightarrow{4B/5B}$ 011101010011101

图 2.3

2.28 绘出上题比特流的 4B/5B 编码后的 NRZI 波形图。

答:

如图 2.4 所示。

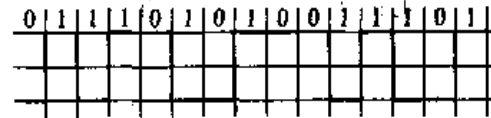


图 2.4

2.29 试述 EIA - RS - 232C 与 CCITT V. 24 和 V. 28 以及 ISO 2110 标准之间的异同点。

答:

EIA - RS - 232C 作为一项物理层协议的规范,包括机械特性、功能特性和电气特性等方面。ISO2110 标准所规定的机械特性和 EIA - RS - 232C 的机械特性是兼容的;CCITT 的 V. 24 建议所规定的功能特性和 EIA - RS - 232C 的功能特性是兼容的;V. 28 则规定了电气特性,它和 EIA - RS - 232C 的电气特性是兼容的。

2.30 RS - 232C 是由美国电子工业协会 EIA 指定的一种串行物理接口标准,在 DTE 和 DCE 的互连与 DTE 设备自行对连这两种情况下,连线的方法有何不同?为什么?

答:

在 DTE 与 DCE 的互连中,相应的引脚直接相连;而同种 DTE 设备互连,则需要交换 2、3 和 6、20 这两对针号的引脚线,并且每个 DTE 设备的 4、5 针号引脚自连。

2 为 BA,发送数据,从 DTE 发出信号;3 为 BB,接收数据,DTE 接收信号。在连接 DTE 和 DCE 的时候,两条连线分别负责将 DTE 的数据发送给 DCE 以及将 DCE 的数据发送给 DTE;而当同种 DTE 互连时,通过交叉正好可以把一方的发送端连到另一方的接收端。

6、20 的情况同上。

4 为 CA,请求发送,5 为 CB,清除待发送。当 DTE 要向 DCE 发送数据的时候,置 CA 为 ON, DCE 收到后在 CB 上回答,取消 DTE 的待发送状态,然后 DTE 开始发送。当 DTE 之间互相连接时,不需要等待 DCE 的回答,因此将 4 导入 5,一旦发送置位,就允许发送。

第三章 数据链路层

3.1 试比较书中介绍的几种帧定界方法的优缺点。

答:

书中共介绍了四种帧定界方法:字节计数法、使用字符填充的首尾定界符方法、使用比特填充的首尾标志方法和违例编码方法。

字节计数法中,“字节计数”字段是十分重要的,必须采取措施来保证它不会出错。因为它一旦出错就会失去帧尾的位置,特别是其错误值变大时不但会影响本帧,而且会影响随后的帧,造成灾难性的后果。比特填充的首尾标志方法优于字符填充的首尾定界符方法。后者依赖于字符编码集,而且硬件实现较麻烦;而前者不依赖于字符编码集,硬件实现较方便,不必以字符为单位,也不必等待一个字符到达后进行判别再决定是否要插入或删除转义字符;而可以连续发送或接收,通过简单地计数就可以决定是否要进行比特的插入或删除,还有其控制功能较完善。违例编码方法不需要任何填充技术,但它只适合于采用了冗余编码的特殊编码方法。所以,比特填充的首尾标志方法使用最为广泛。

3.2 简述数据链路层流量控制的功能。

答:

实际上是限制发送方的数据流量,使其发送速率不要超过接收方所能处理的程度。在这个过程中,也需要通过某种反馈机制使发送方知道接收方是否能跟得上。要有一些规则使得发送方知道在什么情况下可以接着发送下一帧;在什么情况下必须禁止发送,停下来等待收到某种新的信息后再继续发送等。

3.3 某信道误码率为 10^{-5} ,每帧长度为 10 kb,那么

1) 若差错都是单个错,则在该信道上传送的帧的平均出错率是多少?

2) 若差错大多为突发错,平均突发长度为 100 b,则在该信道上传送的帧的平均出错率是多少?

答:

$$1) \frac{1}{10^5} \times 10\,000 = 10\%$$

$$2) \frac{1}{10^5 \times 100} \times 10\,000 = 0.1\%$$

3.4 利用编码方法进行差错控制的方式主要有哪些？分别使用什么样的差错控制编码？

答：

基本上有两类：自动请求重发(Automatic ReQuest for repeat, ARQ)和前向纠错(FEC, Forward Error Correction)。在 ARQ 方式中,接受端检测出有差错时,就设法通知发送端重发,直到收到正确的码字为止,所以采用检错码。在 FEC 方式中,接受端不但能发现差错,而且能确定二进制出错码元的位置,从而就可以加以纠正,所以采用纠错码。

3.5 检错码和纠错码有什么不同？试比较在网络通信中使用时的优缺点。

答：

检错码只能检查出传输中出现的错误,然后要求发送方重传;而纠错码不但可以检测错误,还可以确定二进制错码元的位置,从而可以纠正差错,避免重传。检错码的优点是编码效率较高(即使用较少的冗余位),且实现设备简单;但只在具有反向反馈信道的情况下才能采用,同时发送方需要有数据缓冲区用于存储已经发送的数据,而且在误码率高时大量反馈重传要浪费不少时间。纠错码则反之,其优点是不需要反向反馈信道来传递请求重发的信息,发送方也不需要数据缓冲区,且可以避免重传;缺点是比检错码使用更多的冗余位。其编码效率较之检错码要低,同时纠错的设备也要比检错的设备复杂。

3.6 信息段为 1001110,如果加 1 位奇偶校验码,若为奇校验,是什么? 若为偶校验呢?

答：

奇校验:1

偶校验:0

3.7 什么是定比码? 对于一个 7 中取 5 的定比码来说,其编码效率是多少?

答：

定比码是指每个码字中均含有相同数目的“1”(码字长一定,“1”的数目固定后,所含“0”的数目也必然相同)。

$$\text{编码效率 } R = (\log_2 C_n^r) / n = (\log_2 C_7^5) / 7 = (\log_2 21) / 7 \approx 0.63$$

3.8 若信息位是 11001,写出其正反码的冗余位。如果信息位有一位差错,校验码组可能是什么?

答：

正反码的冗余位:11001

如果信息位有一位差错,校验码组可能为:11110, 11101, 11011, 10111, 01111

3.9 若信息位为 7 位,要构成能纠正 1 位错的海明码,则至少要加上多少位冗余位? 并写

出其监督关系式。

答:

一般来说,信息位为 k 位,增加 r 位冗余位,构成 $n = k + r$ 位码字。若希望用 r 个监督关系式产生的 r 个校正因子来区分无错和在码字中 n 个不同位置的一位错,则要求

$$2^r \geq n + 1$$

或者

$$2^r \geq k + r + 1$$

这里 $k = 7$,满足上式的最小 $r = 4$,即至少要加上 4 位冗余位。

记信息位为 $a_{10} a_9 a_8 a_7 a_6 a_5 a_4$,加上 4 位冗余位为 $a_3 a_2 a_1 a_0$,若规定 $s_3 s_2 s_1 s_0$ 值与错码位置的对应表如表 3.1 所示(自然,也可以规定成另外的对应关系,这并不影响讨论的一般性)。

表 3.1

$s_3 s_2 s_1 s_0$	0000	0001	0010	0100	1000	0011
出错码	无错	a_0	a_1	a_2	a_3	a_4
$s_3 s_2 s_1 s_0$	0101	0110	0111	1001	1010	1011
出错码	a_5	a_6	a_7	a_8	a_9	a_{10}

由此得监督关系式:

$$s_3 = a_3 \oplus a_8 \oplus a_9 \oplus a_{10}$$

$$s_2 = a_2 \oplus a_5 \oplus a_6 \oplus a_7$$

$$s_1 = a_1 \oplus a_4 \oplus a_6 \oplus a_7 \oplus a_9 \oplus a_{10}$$

$$s_0 = a_0 \oplus a_4 \oplus a_5 \oplus a_7 \oplus a_8 \oplus a_{10}$$

若规定成另外的对应关系,则得出的监督关系式也相应不同。

3.10 若信息位为 1001000,要构成能纠正 1 位错的海明码,利用上题中的监督关系式求出其冗余位。

答:

令上题的监督关系式中 s_3, s_2, s_1 和 s_0 都为 0,则可求出其冗余位为

$$a_3 = a_8 \oplus a_9 \oplus a_{10} = 1$$

$$a_2 = a_5 \oplus a_6 \oplus a_7 = 1$$

$$a_1 = a_4 \oplus a_6 \oplus a_7 \oplus a_9 \oplus a_{10} = 0$$

$$a_0 = a_4 \oplus a_5 \oplus a_7 \oplus a_8 \oplus a_{10} = 0$$

3.11 已知海明码的监督关系式为

$$s_0 = a_0 \oplus a_3 \oplus a_5 \oplus a_6$$

$$s_1 = a_1 \oplus a_4 \oplus a_5 \oplus a_6$$

$$s_2 = a_2 \oplus a_3 \oplus a_4 \oplus a_5$$

接收端收到的码字 $a_6 a_5 a_4 a_3 a_2 a_1 a_0 = 1011110$, 在最多 1 位错的情况下, 问发送端发送的码字是什么?

答:

将 a_i 的值代入题中 S_i 的表达式, 我们得到

$$s_0 = 0 \oplus 1 \oplus 0 \oplus 1 = 0$$

$$s_1 = 1 \oplus 1 \oplus 0 \oplus 1 = 1$$

$$s_2 = 1 \oplus 1 \oplus 1 \oplus 0 = 1$$

不全为 0, 必有错, 若为 1 位错, 在 s_1 和 s_2 表达式中, 但又不在 s_0 表达式中的只有 a_4 , 故可判断为 a_4 错, 发送端发送的码字为 1001110。

3.12 9 位信息位采用纠正 1 位错的海明码至少要加 _____ 位冗余位。数据 101111110000011111 通过面向比特的同步规程传输时, 经比特填充后在物理信道上为 _____。

答:

9 位信息位采用纠正 1 位错的海明码至少要加 4 位冗余位。数据 101111110000011111 通过面向比特的同步规程传输时, 经比特填充后在物理信道上为 10111110100000111110。

3.13 若生成多项式为 $x^3 + 1$, 信息位多项式为 $x^6 + x^4 + 1$, 则 CRC 码的冗余多项式是什么?

答:

$$G(x) = x^3 + 1 \quad \leftrightarrow \quad 1001$$

$$K(x) = x^6 + x^4 + 1 \quad \leftrightarrow \quad 1010001$$

$$x^3 K(x) = x^9 + x^7 + x^3 \quad \leftrightarrow \quad 1010001000$$

用 1001 来除 1010001000, 得余数 10, 即 CRC 码的冗余多项式是 $R(x) = x$ 。

3.14 已知循环冗余码的生成多项式 $G(x) = x^5 + x^4 + x + 1$, 若接收方收到的码字为 1010110001101, 问传输中是否有差错?

答:

$$G(x) = x^5 + x^4 + x + 1 \quad \leftrightarrow \quad 110011$$

用 110011 来除 1010110001101, 不能整除, 即余数不为 0, 由此可知传输中有差错。

3.15 CRC 码的生成多项式 $G(x) = x^6 + x^5 + x + 1$, 若信息位为 1101110010001, 求 $R(x)$ 。

答:

余数为 101010, 即

$$R(x) = x^5 + x^3 + x$$

3.16 某循环冗余码的生成多项式 $G(x) = x^{12} + x^{11} + x^3 + x^2 + x + 1$, 它能检测出所有突发长度小于 _____ 比特的突发错, 而对于突发长度大于该比特的突发错的漏检率为 _____。

答:

某循环冗余码的生成多项式 $G(x) = x^{12} + x^{11} + x^3 + x^2 + x + 1$, 它能检测出所有突发长度小于 13 比特的突发错, 而对于突发长度大于该比特的突发错的漏检率为 2^{-12} 。

3.17 试画出 $G(x) = x^9 + x^6 + x^5 + x^4 + x^3 + 1$ 的编码电路。

答:

如图 3.1 所示。

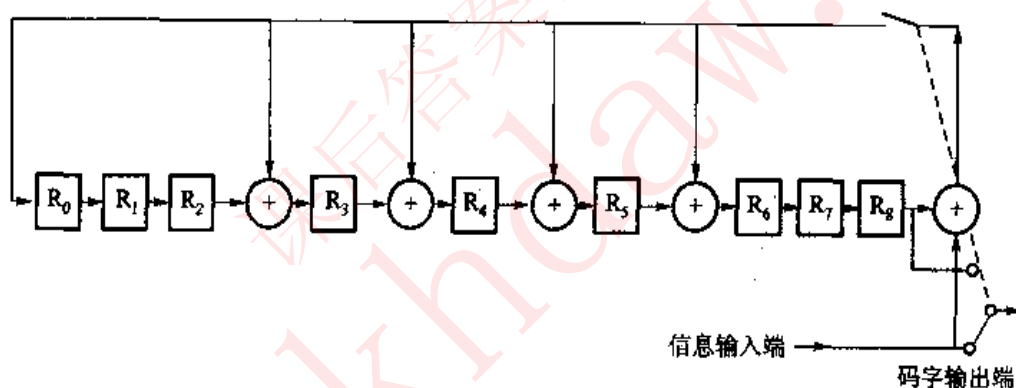


图 3.1

3.18 数据链路层差错控制的方法是什么? 对于遇到一帧整个消失和同一帧被重复发送, 又是如何分别处理的?

答:

通常采用反馈重发的方法来纠正错误。要求每帧传送后接收方向发送方提供是否已正确接收的反馈信息, 从而发送方可以据此决定是否要重发, 发送方只有收到接收方已正确接收的反馈信息后才能认为该帧已正确传送完毕。

对于帧丢失的情况, 可以引入计时器来解决。当发送方计时器超过指定的时间间隔, 即计时器超时后还未收到确认正确接收的返回信息时, 就可认为传出的帧已出错或丢失, 因而要重新发送。

对于多次收到同一帧的情况, 可以采用对发送的帧编号, 即给每帧一个序号, 从而使得接收方能从该序号来区分是新发送的帧, 还是已被接收但发送方又重新发送来的帧, 从而决定是不是

交给上一层。

3.19 一个信道的数据速率为 4 kb/s, 单向传播延迟时间为 20 ms, 问帧长在什么范围内, 才能使停等协议的效率至少是 50%?

答:

这里, $B = 4 \text{ kb/s}$, $R = 20 \text{ ms} = 0.02 \text{ s}$, 要使效率

$$u = \frac{L/B}{L/B + 2R} \geq 50\%$$

$$\Rightarrow L \geq 2BR = 2 \times 4 \times 0.02 \text{ kb} = 0.16 \text{ kb}$$

3.20 使用回退 n 协议在 3 000 km 长的 1.544 Mb/s 的 T1 干线上发送 64 字节的帧, 若信号传播速度是 6 $\mu\text{s/km}$, 问帧的顺序号应有多少位?

答:

这里, $B = 1.544 \text{ Mb/s}$, $R = 6 \times 10^6 \times 3 \text{ 000 s}$, $L = (64 \times 8) \text{ b}$, 故帧的顺序号

$$n \geq \left\lceil \log_2 \left(\frac{L + 2BR}{L} \right) \right\rceil = \left\lceil \log_2 \left(\frac{64 \times 8 + 2 \times 1.544 \times 10^6 \times 6 \times 10^{-6} \times 3 \text{ 000}}{64 \times 8} \right) \right\rceil = 7$$

3.21 设一种滑动窗口协议使用的帧序号位数足够多, 以至于不会出现重复, 那么在窗口的 4 个边界及窗口大小间必须满足什么关系?

注: 记发送窗口为 $[S_t, S_u]$ 、接收窗口为 $[R_l, R_u]$ 、发送窗口及接收窗口的大小皆为 W , 此题即问 S_t, S_u, R_l, R_u 和 W 之间应满足什么关系?

答:

发送窗口的大小大于等于 0, 但小于等于 W , 有

$$0 \leq S_u - S_t \leq W - 1$$

接收窗口的大小固定为 W , 有

$$R_u - R_l = W - 1$$

接收窗口下边界应大于等于发送窗口下边界, 但小于等于发送窗口上边界加 1, 有

$$S_t \leq R_l \leq S_u + 1$$

3.22 若帧号位数为 3, 窗口尺寸为 2, 请对选择重传协议画出由初始状态出发下列事件依次发生时的滑动窗口图: 发送帧 0、发送帧 1、接收帧 0、接收确认帧 0、发送帧 2、接收反向确认帧 1、接收帧 2、重发帧 1、接收帧 1、接收确认帧 2。

答:

如图 3.2 所示。

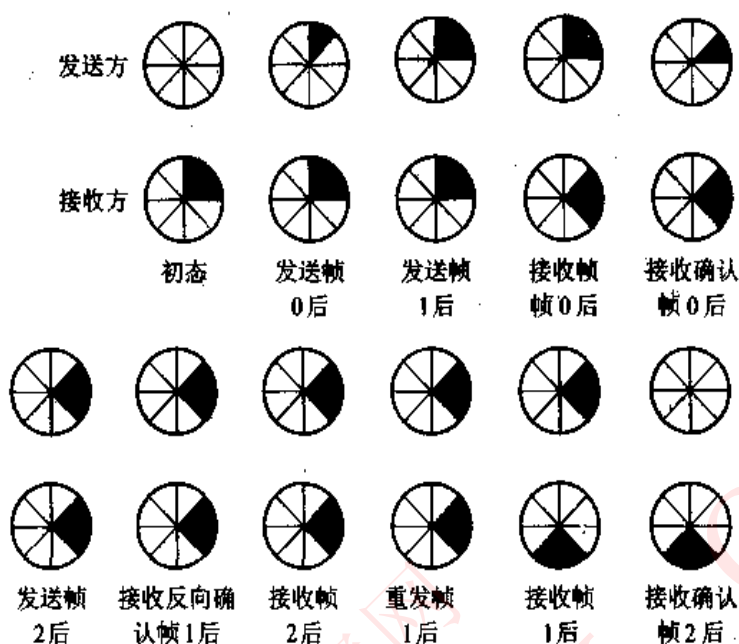


图 3.2

3.23 通过举反例来说明,对“回退 n ”协议,必须使发送窗口 $\leq 2^m - 1$ (这里, m 是帧号数)。

答:

假定 $m = 3, 2^m = 8$ 。若发送窗口不小于等于 7, 而为 8, 假设 t_1 时刻发送方 A 发送帧 0 ~ 7, t_2 时刻接收方 B 接收到帧 0 ~ 7, 并发送帧 7 的确认帧, 但该确认帧丢失了。A 在 t_3 时重发帧 0 ~ 7。而此时 B 正在等待帧 7 之后的新的帧 0。在 t_4 时 B 接收到帧 0 时, 它将误以为该帧就是新的帧 0, 并接收, 因此协议失败。

3.24 通过举反例来说明,对选择重传协议,必须使发送窗口 $\leq 2^{m-1}$ 。

答:

假定 $m = 3, 2^{m-1} = 4$ 。若发送窗口尺寸不小于等于 4, 而为 5, 接收窗口尺寸为 4。假设 t_1 时刻发送方 A 发送帧 0 ~ 4。 t_2 时刻 B 收到帧 0 ~ 3, 并使接收窗口下滑至 4, 5, 6, 7, 并发送对帧 0 ~ 3 的确认 ACK 3。随后又接收了帧 4, 窗口再次下滑为 5, 6, 7, 0, 并发送 ACK 4。但 ACK 3 和 ACK 4 都丢失了, 所以 A 在 t_3 时刻重发帧 0 ~ 4。此后 B 在 t_4 时刻收到重发的帧 0, 由于帧 0 在其接收窗口范围内, 将被错误地当作新的帧 0 而接收, 因此协议失败。

3.25 已知信道的数据速率为 1 Mb/s, 往返传播延迟时间为 4 ms, 帧长度为 1 000 位, 帧号用 3 位, 并假定确认不采用捎带方式且不占时间。问若不考虑差错重发和帧头所造成的信道损失时, 采用选择重传协议信道可能达到的最大有效利用率是多少?

答:

假设发送窗口数取最大值 $W = 2^{n-1} = 4$ 。发送方首先用 $4 \times L/B$ s 时间连续发送 4 个帧, 然后等待确认。 L/B s 第一帧发送完再经 R s 到达接收方后, 接收方立即发送确认帧, 确认帧经过 R s 之后到达发送方。发送周期为 $L/B + 2R$ s, 其中用于发送数据的时间为 $4 \times L/B$ s。本题中帧长 $L = 1\,000$ b, 信道的数据速率 $B = 10^6$ b/s, $R = 2 \times 10^{-3}$ s, 在这种情况下, 信道可能达到的最大有效利用率为

$$u = \frac{4 \times L/B}{L/B + 2R} = \frac{4 \times 1\,000/10^6}{1\,000/10^6 + 4 \times 10^{-3}} \times 100\% = 80\%$$

3.26 在 50 kb/s 的卫星信道上发送 1 kb 长的帧, 确认信号总是由数据帧捎带。帧头很短, 使用 3 位顺序号。对下述三种协议, 最大可能达到的信道有效利用率是多少?

- 1) 停等协议;
- 2) 回退 n 协议;
- 3) 选择重传协议。

答:

本题中 $L = 1\,000$ b, $B = 50$ kb/s, 对卫星信道来说可以近似认为 $2R = 0.5$ s, 并注意到确认信号总是由数据帧捎带, 有

1) 停等协议中, 发送方首先用 L/B s 时间发送 1 个帧, 然后等待确认。帧经 R 秒到达接收方之后, 接收方立即用 L/B s 时间发送一个数据帧, 其中捎带了对发送方发送帧的确认, 该数据帧经过 R s 时间之后到达发送方。发送周期为 $(2 \times L/B + 2R)$ s, 其中用于发送数据的时间为 L/B s。在这种情况下, 信道所能达到的最大信道有效利用率为

$$u_1 = \frac{L/B}{2 \times L/B + 2R} = \frac{1\,000/50\,000}{2 \times 1\,000/50\,000 + 0.5} \times 100\% = \frac{1}{27} \times 100\% \approx 3.7\%$$

2) 回退 n 协议中 (W 取最大值 $2^n - 1 = 7$), 发送方首先连续发送 7 个帧, 然后等待确认。第一帧到达接收方之后, 接收方立即发送一个数据帧, 其中捎带了对发送方发送的第一帧的确认, 该数据帧再经过 R s 之后到达发送方。发送周期仍然为 $(2 \times L/B + 2R)$ s, 其中用于发送数据的时间为 $7 \times L/B$ 。在这种情况下, 信道所能达到的最大信道有效利用率为

$$u_2 = \frac{7 \times L/B}{2 \times L/B + 2R} = \frac{7 \times 1\,000/50\,000}{2 \times 1\,000/50\,000 + 0.5} \times 100\% = \frac{7}{27} \times 100\% \approx 25.9\%$$

3) 由于所求的是最大可能达到的信道有效利用率, 必然在信道无错时达到, 此时选择重传协议和 2) 中的回退 n 协议唯一的区别是 W 取的最大值是 $2^{n-1} = 4$, 其余完全相同。在这种情况下, 信道所能达到的最大信道有效利用率为

$$u_3 = \frac{4 \times L/B}{2 \times L/B + 2R} = \frac{4 \times 1\,000/50\,000}{2 \times 1\,000/50\,000 + 0.5} \times 100\% = \frac{4}{27} \times 100\% \approx 14.8\%$$

3.27 在重负荷的 50 kb/s 的卫星信道上, 用选择重传协议发送含 40 位帧头和 3 960 位数

据的帧。假定无确认帧,NAK 帧为 40 位,数据帧的出错率为 1%,NAK 帧的出错率可忽略不计,顺序号是 7 位,问由于帧头和差错重发而浪费的信道带宽占百分之几?

答:

数据长度 $D = 3\,960$ b,帧头长度 $H = 40$ b,帧长度 $L = D + H = 4\,000$ b,NAK 帧长度 $A = 40$ b,出错率 $q = 0.01$,正确发送率 $p = 0.99$ 。平均发送一个数据帧由于出错需要传输的次数为

$$\sum_{k=1}^{\infty} kq^{k-1}p = p + 2qp + 3q^2p + \cdots = \frac{1}{p}$$

也就是说,其中 $(1/p) - 1 = (1-p)/p$ 次出错要传输 $(L + A)$ b,再加上一次正确发送 L b 才能发送有效的 D b 数据。信道利用率为

$$u = \frac{\text{有效数据}}{\text{发送数据总量}} = \frac{D}{(H + D) + \frac{1-p}{p}(H + D + A)} \times 100\% \approx 98\%$$

所以,浪费的信道带宽大约占 2%。

3.28 设信道容量为 1 Mb/s,误码率为 10^{-2} ,等待 ACK 的超时间隔时间为 1 ms,帧头长 32 B,对于停等协议,帧的数据部分最佳长度近似多少?

答:

帧头长 $H = 32 \times 8$ b,信道容量 $B = 1$ Mb/s,等待 ACK 的超时间隔时间 $T = 10^{-3}$ s,误码率 $E = 10^{-2}$ 。按照教材《数据通信与计算机网络》中第 91 页的公式可算出帧的数据部分最佳长度为

$$D_{\text{opt}} \approx \sqrt{\frac{H + BT}{E}} = \sqrt{\frac{32 \times 8 + 10^6 \times 10^{-3}}{10^{-2}}} \approx 354 \text{ b}$$

3.29 与一个 IMP 相连的 N 个主机中,每个主机以每秒钟 λ 个报文的平均值产生信息,并服从普阿松规律。一个报文长度为 m 位的概率是 P_m 。在该 IMP 的唯一 IMP-IMP 输出线上,每个报文要加 h 位的帧头。该线的数据速率是 b b/s。接收 1 位出错的概率是 e 。确认信息将立即返回,并且所占带宽可忽略不计。如果所有的通信都是与远地主机同的通信,问 N 的最大值是多少?

答:

先看 IMP-IMP 输出线,其上 1 位出错的概率是 e ,那么长度为 $(m + h)$ 位的帧不出错的概率为

$$\alpha = (1 - e)^{(m+h)}$$

故,一帧成功发送平均所需次数为

$$\sum_{k=1}^{\infty} [K(1 - \alpha)^{(k-1)} \alpha] = 1/\alpha$$

相当于每帧要发送

$$L_m = (m + h) \times (1/\alpha) = (m + h)/\alpha$$

位。再考虑报文长度为 m 位的概率是 P_m , 可得到平均发送一个报文相当于在 IMP - IMP 输出线上发送

$$\sum L_m P_m$$

位。因此, 相当于输入 IMP 的数据速率为每秒

$$N\lambda \sum (L_m P_m)$$

位。要使通信顺利进行而不发生拥塞, 输入 IMP 的数据速率必须小于等于该 IMP 的唯一 IMP - IMP 输出线的数据速率, 即

$$N\lambda \sum (L_m P_m) < b$$

或者说

$$N < b / (\lambda \sum (L_m P_m))$$

3.30 对教材图 3.14 中的状态变迁路径: $(000) \rightarrow (01A) \rightarrow (01S) \rightarrow (010) \rightarrow (01A)$, 给出教材图 3.16 中相应的 Petri 网变迁触发序列, 并用文字解释此序列所代表的含义。

答:

教材图 3.16 中相对应的 Petri 网的变迁触发序列为:

$$ACG - 1 \rightarrow ADF - 10 \rightarrow AF - 7 \rightarrow ACF - 5 \rightarrow ADF$$

此序列代表发送方发出 0 号帧, 接收方收到之后改为期待 1 号帧, 并发送 ACK 帧; ACK 帧在传输中丢失; 发送方计时超时之后重发第 0 帧; 接收方收到之后, 因与期待帧号不符而拒收, 但仍要再次发送 ACK 帧。

3.31 已知某 Petri 网的变迁规则为 $AC \rightarrow B$ 、 $B \rightarrow AC$ 、 $CD \rightarrow E$ 和 $E \rightarrow CD$, 请画出该 Petri 网图以及与该 Petri 网相对应的由初始状态 ACD 可达到的有限状态机图。这些变迁规则是什么著名的计算机科学概念的模型化?

答:

Petri 网图如图 3.3 所示。

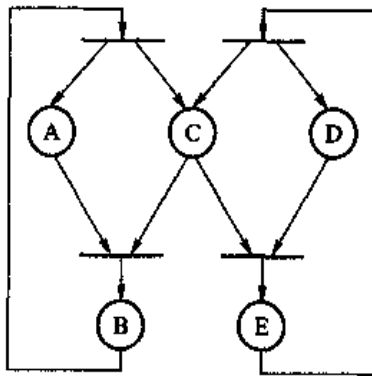


图 3.3

相应于初始状态 ACD 的有限状态机图如图 3.4 所示。

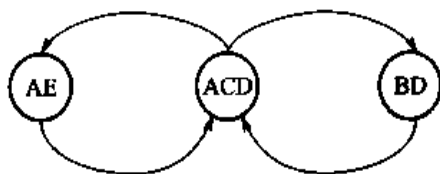


图 3.4

这是操作系统中两个进程 A 和 D 竞争访问共享临界区 C 概念的模型化,或者 A 访问 C,即状态 B,或者 D 访问 C,即状态 E,两者不能并存。

3.32 分别简述协议描述语言 SDL、ESTELLE 和 LOTOS 的特点。

答:

SDL:有两种语法表示形式:图形表示的 SDL/GR(Graphical Representation)和文字短语表示的 SDL/PR(textual Phrase Representation)。前者是一种类似于程序框图的图形语言,而后者采用了语法形式定义,可用来定义具体的语法结构。SDL 本质上仍是一种扩充的有限状态机,但为了不使状态空间太大,它还允许使用辅助存储区,用以保存那些不必出现在状态之中的信息,并定义了两种辅助操作:判决(Decision)和任务(Task)。

ESTELLE:是一种基于状态变迁模型和高级程序设计语言 PASCAL 的协议形式描述语言。它的描述支持层次的分解、模块化和分别编译。它描述的协议比较容易通过自动编译转化为 PASCAL 语言的协议实现源程序。

LOTOS:它不关心系统(进程)内部的状态和动作,而只描述其外部可见行为的时态次序。它为协议系统中各进程的行为定义了一套运算规则和推理系统,使得此种形式描述建立在强有力的数学基础上,从而使描述更加简洁并更容易进行协议正确性验证。

3.33 比较通信协议中的异步协议和同步协议。

答:

异步协议把每个字符看作一个独立的信息,在每个字符起始处同步,但各个字符之间的间隔时间是可以变化的。由于发送器和接收器近似于同一频率的两个时钟,能够在一段短时间内保持同步,故可以用字符起始处同步的时钟来采样字符中的各位,而不需要每位同步了。

同步协议则把许多字符组织成一个数据块(通常称为帧),在该数据块的起始处同步,而后面维持固定的时钟,实际上是发送端通过某种技术将时钟混合到数据中一起发送出来,而接收端又从输入数据中分离出时钟来。该时钟不但用来定时字符内的各位,也用来定时字符本身。

可见,异步和同步的主要区别在字符之间。

3.34 简述面向字符、面向比特和面向字节计数三种同步规程分别是如何做到“数据透明”的。

答：

面向字符：将每个独立的控制字符都作为普通的数据字符对待，因而它可以出现在数据正文段中，而不具有通信控制的含义。只有在这些字符紧随着一个 DLE 转义字符后面时才具有通信控制的含义。若数据段正文本身出现 DLE 字符，则在向传输线路发送前在它的前面再插入一个 DLE。当接收到连续两个 DLE，就删去前一个，并把第二个作为普通数据字符对待而不是转义字符。

面向比特：通过比特填充来实现的。因为是以一个特定的比特模式“01111110”作为标志来定时帧的开头和结束，发送器对除了标志以外的其余字段，每连续出现 5 个“1”之后就自动插入一个“0”。这样就能把数据字段和标志区分开来。当然接收器在接收数据的过程中还要在连续出现 5 个“1”之后删掉后面的那个“0”。

面向字节计数：用 Count 字段来指示帧中数据字段中含有多少个字节，数据字段的长度必须为字节的整数倍。这种采用字节计数的方法用来确定帧结束边界，不会引起数据和其他标志或字符混淆，因而不需要采用任何填充技术就可达到“数据透明”。

3.35 若某面向字符同步规程的帧数据段中出现下列字符串，问字符填充后的输出是什么？

A DLE STX B C DLE DLE DLE DLE ETB

答：

字符填充之后的输出为

A DLE DLE STX B C DLE DLE DLE DLE DLE DLE DLE DLE ETB

3.36 若某面向比特同步规程的帧数据段中出现下列位串，问比特填充后输出是什么？

0100000111110101111110

答：

比特填充之后的输出为

0100000011111001011111010

3.37 某 PPP 帧的信息部分为十六进制 7C 7D 5E 5D 69 3A 7D 5D 7D 5D 6A 5D，问其原始信息是什么？

答：

7C 7E 5D 69 3A 7D 7D 6A 5D

3.38 试比较帧中继和 X.25 分组交换网。

答：

1) 帧中继协议简化了 X.25 分组级(即网络层)功能,在体系结构上只有两个层次:物理层和数据链路层,从而简化了网内节点的处理过程。在帧中继网络中,一个节点收到一个帧,大约只需执行 6 个检测步骤,因而提高了帧中继网络的处理效率;

2) 帧中继传送的单元是帧,其长度是可变的,最大长度为 1 600 B,比 X.25 网络的缺省分组 128 B 要长,这特别适合于封装局域网的数据单元,减少了分段与重组的处理开销;

3) 帧中继在数据链路层完成统计复用、帧透明传输和差错检测功能。它与 X.25 网络不同的是帧中继网络节点对待差错,并不采用重传机制,而将出错帧丢弃,这就减少了帧编号和应答等开销,由此减少了节点机处理时间,提高了网络吞吐量,降低了网络延迟。例如,X.25 网络节点机的处理时延为 5 ~ 10 ms,而帧中继节点机的处理时延小于 2 ms;

4) 帧中继技术提供了一套有效的拥塞控制机制,既能控制网络拥塞,又能使用户在允许的条件下合理地传送超出约定带宽的突发性数据,从而充分利用网络资源。

第四章 局 域 网

4.1 N 个站点共享一条 56 kb/s 的 ALOHA 信道,每个站点以平均每 100 s 输出一个 1 000 b 的帧,而不管前面的帧发送出去没有(即站点有缓冲功能)。 N 的最大值为多少?

答:

对于 ALOHA 协议,信道的最大可用带宽 $= 0.184 \times 56 \text{ kb/s}$

每个站点以平均每 100 s 输出一个 1 000 b 的帧,即每个站点所占用的带宽为

$$1\,000/100 \text{ b/s} = 10 \text{ b/s}$$

所以最大值 $N = 0.184 \times 56 \times 1\,000/10 = 1\,030$

4.2 在 ALOHA 和分槽 ALOHA 协议中,哪一种协议的延迟更短一些? 请说出原因。

答:

ALOHA 的延迟更短一些。这是因为:在 ALOHA 协议中,站点有数据传输时马上开始;而分槽 ALOHA 协议中,传输只有在时槽的边界处开始进行,即必须等待下一个时槽,这样分槽 ALOHA 协议中,延迟平均需要等待半个时槽。

4.3 1 万个航空订票站竞争使用一条分槽 ALOHA 信道,每个站平均每小时发出 18 个请求,一个时槽为 125 μs 。信道的负载大约是多少?

答:

每个站点平均每小时发出 18 个请求,这样 1 万个站点产生的请求的速度为

$$\frac{10\,000 \times 18}{1 \text{ 小时}} = \frac{10\,000 \times 18}{3\,600} = 50 \text{ 请求/秒}$$

每个时槽长度为 125 μs ,这样信道的负载为

$$50 \times 125 \times 10^{-6} = \frac{1}{160}$$

4.4 有一个无限用户的分槽 ALOHA 系统,其帧时等于 k 个时槽时间。一个帧可以在任何一个时槽中开始。信道负载为每帧时 G 帧。吞吐量为每帧时多少帧? 当 $k=1$ 或 $k \rightarrow \infty$ 时,给出相应的吞吐量值并解释这些结果。

答:

如图 4.1 所示,帧只有在时槽的边界开始进行传输,而帧一旦在某个时槽开始传输,将会占用 k 个时槽,这样对于用阴影部分表示的帧,与阴影帧碰撞的冲突危险区为 $2k-1$ 个时槽。

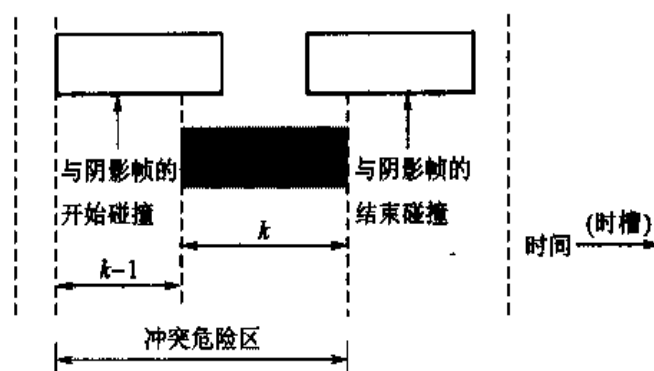


图 4.1

在任一帧时内无其他帧发送的概率为 $e^{-(2k-1)G/k}$, 这样吞吐量 S 为

$$S = Ge^{-(2k-1)G/k}$$

当 $k=1$ 时, $S = Ge^{-G}$, 实际上相当于一个标准的分槽 ALOHA 系统, 即只有在同一个时槽开始处进行传输的站点间发生碰撞。

当 $k \rightarrow \infty$ 时, $S = Ge^{-2G}$, 因为时槽无限小, 相当于每个帧在任何时间都可开始传输, 实际上就是一个标准的 ALOHA 系统。

4.5 对一个无限用户分槽 ALOHA 信道的测量显示, 10% 的时槽是空闲的。请问信道的负载 G 是多少? 吞吐量是多少? 信道是负载不足还是过载?

答:

10% 的时槽是空闲的, 即在任一时槽内没有帧发送的概率 $P_0 = 0.1$, 而对于分槽 ALOHA 来说:

$$P_0 = e^{-G} = 0.1$$

所以信道负载为

$$G = -\ln P_0 = -\ln 0.1 = 2.3$$

吞吐量 S 为

$$S = Ge^{-G} = 2.3 \times 0.1 = 0.23$$

因为 $G = 2.3 > 1$, 所以信道是过载的。

4.6 标准的 10 Mb/s 802.3 LAN 的波特率是多少?

答:

10 Mb/s 以太网采用曼彻斯特编码, 效率为 50%, 即每个比特用两个信号来表示, 这样波特率为比特传输速度的 2 倍, 即 20 Mb/s。

4.7 一个 1 km 长的 10 Mb/s 的 CSMA/CD LAN (不是 802.3), 其传播速度为 200 m/μs。数据帧长为 256 位, 包括 32 位头部、检验和以及其他开销在内。传输成功后的第一个时槽保留给接收方以捕获信道来发送一个 32 位的确认帧。假定没有冲突, 那么不包括开销的有效数据传输速率为多少?

答:

不考虑冲突, 数据的传输实际上包括两个阶段: 发送者传输数据帧和接收者传输确认帧。整个传输过程中所用的时间包括:

数据帧的传输延迟

发送者传输数据帧的最后一位到达接收者的传播延时 = τ

确认帧的传输延迟

接收者传输确认帧的最后一位到达发送者的传播延时 = τ

在这一段时间内传输的有效数据为 $256 - 32 = 224$ 位。

信号传播速度为 200 m/μs, 网段的长度为 1 km, 这样信号的传播延迟 $\tau = \frac{1\ 000}{200} = 5\ \mu\text{s}$ 。注意到确认帧的传输延时小于往返传播延时, 为了保证冲突检测机制能够正常进行, 要进行填充使其传输延时等于往返传播延时。

所以在不考虑冲突的情况下的有效数据传输率为

$$\frac{224}{\tau + \frac{256}{10 \times 10^6} + 2\tau + \tau} = \frac{224 \times 10^6}{5 + 25.6 + 10 + 5} = 4.9\ \text{Mb/s}$$

4.8 局域网竞争方案的一个缺点是由于多个站点试图同时访问信道而导致的带宽浪费。假设将时间分割成离散的时槽, 每个时槽中有 N 个站点以概率 p 试图发送帧。请问由于多个站点试图同时发送而造成的浪费时间是多少?

答:

时间被分割成离散的时槽, 以一个时槽为单位。在某个时槽中信道空闲, 意味着所有站点都不占用信道, 其概率为

$$(1-p)^n$$

只有一个站点占用信道的概率为

$$np(1-p)^{n-1}$$

因此由于冲突而浪费的时槽所占的比例等于出现冲突的概率

$$1 - np(1-p)^{n-1} - (1-p)^n$$

4.9 CSMA/CD LAN 中两个站点正在试图发送长(多帧)文件。在发出每一帧后, 它们采用二进制退避算法竞争信道。正好竞争 k 次成功的概率为多少? 每个竞争周期的平均竞争次数为

多少?

答:

首先只有两个站点传输,考虑第 i 次竞争信道的情况:

当 $1 \leq i \leq 10$ 时,根据二进制退避算法,等待的时槽数从 $0 \sim 2^{i-1} - 1$ 中随机选择,这样,第 i 次竞争信道失败的概率 = 两个站点都选择同一个时槽的概率 = $2^{-(i-1)}$ 。

当 $11 \leq i \leq 16$ 时,根据二进制退避算法,等待的时槽数从 $0 \sim 1\ 023$ 随机选择,这样,第 i 次竞争信道失败的概率 = 两个站点都选择同一个时槽的概率 = 2^{-10} 。

这样正好竞争 k 次成功的概率 = 前面 $k-1$ 次失败而第 k 次成功的概率,所以

$$P_k = (1 - 2^{-(k-1)}) \prod_{i=1}^{k-1} 2^{-(i-1)} = (1 - 2^{-(k-1)}) 2^{-\frac{(k-1)(k-2)}{2}} = (2^{k-1} - 1) / 2^{k(k-1)/2} \quad 1 \leq k \leq 10$$

$$P_k = (1 - 2^{-10}) \prod_{i=1}^{10} 2^{-(i-1)} 2^{-10(k-11)} = (1 - 2^{-10}) 2^{-45} 2^{-10(k-11)} = (2^{10} - 1) / (2^{10k-55}) \quad 11 \leq k \leq 16$$

$$P_k = 0 \quad k > 16$$

每个竞争周期的平均竞争次数为: $\sum_{k=1}^{16} k P_k$, 把上面的式子代入化解后,有

$$\begin{aligned} K &= \sum_{k=1}^{16} k \times P_k = 1 \times 0 + 2 \times \frac{1}{2} + 3 \times \frac{3}{2^3} + 4 \times \frac{7}{2^6} \\ &\quad + 5 \times \frac{15}{2^{10}} + 6 \times \frac{31}{2^{15}} + 7 \times \frac{63}{2^{21}} + 8 \times \frac{127}{2^{28}} \\ &\quad + 9 \times \frac{255}{2^{36}} + 10 \times \frac{511}{2^{45}} + 11 \times \frac{1\ 023}{2^{55}} + 12 \times \frac{1\ 023}{2^{65}} \\ &\quad + 13 \times \frac{1\ 023}{2^{75}} + 14 \times \frac{1\ 023}{2^{85}} + 15 \times \frac{1\ 023}{2^{95}} + 16 \times \frac{1\ 023}{2^{105}} \\ &= 2.641\ 632 \end{aligned}$$

4.10 令牌环上的环绕时延必须能够容纳整个令牌。如果电缆不够长,必须人为地增加时延。请说明为什么在时延只有 16 位、而令牌为 24 位的环网上必须额外地增加时延。

答:

因为一个环网的时延只有 16 位,令牌持有人在传输完 16 位之后,第一位绕环一周又回到了令牌持有者,因为 24 位的令牌还没有传输完毕,令持有者不能把该比特再放到环上。站点可以在内部人工增加 8 位的延时,即有 8 位的缓冲,从而继续把令牌的传递完成。但是这样该站点的内部缓冲中始终有令牌的某 8 位,并等待再次放到环上,如果一直保有令牌的话,这个过程将不断继续下去,站点没有办法继续传输帧。

4.11 在一个 4 Mb/s 的令牌环中,令牌持有时间为 10 ms。可在环上发送的最大帧有多长?

答:

可发送的最大帧长就是站点在握有令牌后连续发送的数据帧的长度,因此最大帧长为

$$4 \text{ Mb/s} \times 10 \text{ ms} = 4 \times 10^6 \times 10 \times 10^{-3} = 40\,000 \text{ b} = 5\,000 \text{ B}$$

4.12 在令牌环中,由发送者移走帧。如果改由接收者取走帧,则需做哪些修改? 其结果会怎样?

答:

接收者在收到帧的第一位后,无法马上知道是把帧转发还是把帧移走,因此必须有足够的缓冲以便在确认帧的地址字段之后来决定是否取走帧。这样的结果发送者没法检查到帧接收的状态,即不再有捎带确认的功能。

4.13 一个 1 km 长、10 Mb/s、负载相当重的令牌环,其传播速度为 $200 \text{ m}/\mu\text{s}$,在环上有 50 个等距的站点。数据帧为 256 位,其中包括了 32 位的额外开销。确认帧附在数据帧中,故基本上可以说没有确认帧。令牌为 8 位。请问:此环的有效数据传输速率比 10 Mb/s 的 CSMA/CD 的有效数据传输速率是高还是低?

答:

以站点抓住令牌开始到站点把令牌传递给下一个站点为一个周期,这个周期包括传输数据帧、传输令牌以及令牌到达下一个站点。在这个周期中传输的有效数据为 $256 - 32 = 224 \text{ b}$,这样环的有效数据传输率为

$$\frac{224}{\frac{256}{10 \times 10^6} + \frac{8}{10 \times 10^6} + \frac{1\,000/50}{200/10^{-6}}} \text{ b/s} = \frac{224}{26.5} \text{ Mb/s} = 8.5 \text{ Mb/s}$$

和第 4.7 题的结果相比,显然同样参数条件下的令牌环的有效数据传输率要比以太网高。

4.14 以太网有时被认为不适合于实时计算,因为在最坏情况下重发间隔是无限的。假设令牌环网中节点是固定且已知的,在何种情况下,令牌环也会出现上述情况? 在何种情况下令牌环网能已知最坏情况?

答:

如果一个站点持有令牌的时间很长时,令牌环也会出现和以太网类似的情况。如果已知站点在抓住令牌后所能持有令牌的时间有一个上限,这个时候站点可以知道等待传输的延迟的上限 = 站点个数 $\times$ 持有令牌的时间。

4.15 考虑一个有很多用户的 ALOHA 系统,它们平均每秒产生 m 个请求,包括新帧和重传帧。时间被分割成 $T \text{ ms}$ 的时槽。

- (1) 请问第一次传输就成功的概率是多少?
- (2) 一个帧正好遇到 k 次冲突然后在下一个时槽成功的概率是多少?
- (3) 一个帧成功传输的平均时槽数是多少?

答:

信道负载 $G = m \times T / 1\,000$

(1) 第一次传输成功的概率等于在一个时槽内没有其他帧发送的概率: $p = e^{-G} = e^{-\frac{mT}{1\,000}}$

(2) 一个时槽内帧遇到冲突的概率为 $1 - p$, 因此答案为

$$p(1 - p)^k$$

(3) 平均时槽数为

$$\sum_{k=0}^{\infty} kp(1 - p)^k = \frac{1}{p} \approx e^{\frac{mT}{1\,000}}$$

4.16 考虑一个基带总线 LAN 上连接的两个站点, 相距 1 km, 帧长度为 100 位, 传播速度为 2×10^8 m/s。假设每个站点产生的帧服从普阿松分布, 平均每秒产生 1 000 个帧。假设数据速率为 1 Mb/s, 对于 ALOHA 协议, 如果站点在时刻 t 开始传输一个帧, 请问冲突的概率是多少? 如果是分槽 ALOHA 协议结果又如何? 如果数据速率为 10 Mb/s, 分别采用 ALOHA 和分槽 ALOHA 的结果如何?

答:

传播延迟 τ 等于 $5 \mu\text{s}$, 假设帧时为 D 。

首先考虑 ALOHA 协议。如果 A 在时刻 t 传输帧, 在延时 τ 后到达 B, 然后直到 $t + \tau + D$ 之后完整地收到帧。因此如果 B 在 $[t + \tau, t + \tau + D]$ 传输就会冲突。另外 B 也不能在 $[t - D - \tau, t + \tau]$ 传输帧, 因为它将与 A 冲突。这样 B 在 $[t - D - \tau, t + D + \tau]$ 内都不应该传输。

因此成功传输的概率 $\approx$ 在 $2D + 2\tau$ 的时间间隔内 B 没有帧传输的概率。

而对于分槽 ALOHA, 成功的概率等于在 $D + \tau$ 时间间隔内 B 没有帧传输的概率。

站点传输帧的概率服从普阿松分布, 这样在时间 T 内正好有 k 个帧传输的概率为

$$\frac{(1\,000T)^k}{k!} e^{-1\,000T}$$

A 传输的帧冲突的概率 = T 秒内 B 至少有一个帧传输的概率 = $1 - e^{-1\,000T}$

因此:

$$1 \text{ Mb/s ALOHA}, P_c(\text{冲突}) \approx 1 - e^{-0.21} = 0.189$$

$$10 \text{ Mb/s ALOHA}, P_c(\text{冲突}) = 1 - e^{-0.03} = 0.0296$$

$$1 \text{ Mb/s 分槽 ALOHA}, P_c(\text{冲突}) = 1 - e^{-0.105} = 0.0997$$

$$10 \text{ Mb/s 分槽 ALOHA}, P_c(\text{冲突}) \approx 1 - e^{-0.015} = 0.0149$$

4.17 假设以太网的传输算法作如下改动:在每次成功的传输之后,再次发送之前主机都要等待一个或者两个时槽,除此之外采用原来的算法。

(1) 请解释为什么以太网的捕获效应出现的可能性要降低了许多?所谓捕获效应指的是那个最近成功传输的站点在遇到冲突后,相比别的指数后退了相当长的站点更有机会对媒体的访问权的现象。

(2) 请说明这种新的算法会导致一对节点轮流获得媒体的访问权,使得第三个节点一直无法传输。

答:

(1) 考虑两个站点 A 和 B。假设它们都在时槽 0 开始传输然后冲突。A 后退 0 个时槽, B 后退 1 个时槽, 因此 A 在时槽 1 成功传输。A 现在将后退一个或两个时槽(时槽 3 或者时槽 4)。B 可能在时槽 2 成功传输, 因而后退到时槽 4 或者时槽 5; B 也有可能遇到冲突, 后退到时槽 3 或者时槽 4(与 A 后退的时槽相同), 现在 A 和 B 竞争获得媒体访问权的概率相同, 因此捕获效应出现的可能性较低。

(2) 下面给出了这种情形出现的例子:

- ① 时槽 0: 站点 A 和 C 试图传输遇到冲突, A 后退 0 个时槽, C 后退 1 个时槽(时槽 2)。
- ② 时槽 1: A 获得访问权开始传输, 然后后退到时槽 4。
- ③ 时槽 2: B 开始传输, 将和 C 冲突, B 后退 0 个时槽, C 后退 2 个时槽。
- ④ 时槽 3: B 获得访问权开始传输, 后退到时槽 5。
- ⑤ 时槽 4: A 传输, 后退到时槽 6。
- ⑥ 时槽 5: B 和 C 冲突, B 后退 0 个时槽, C 后退 4 个时槽。
- ⑦ 时槽 6: 重复步骤④, 即每次 B 和 C 冲突时 B 后退 0 个时槽, C 按指数后退。

4.18 一个 LAN 上有 N 个设备, 帧的传输时间被分割成时槽, 如果同一个时槽有两个以上传输则会遇到冲突导致传输失败, 而不同时槽的传输不会互相干扰。假设每个设备在每个时槽总是有帧要传输, 但是真正进行传输的概率为 p 。

- (1) 在一个给定的时槽有帧传输成功的概率是多少?
- (2) 假设设备 1 试图进行传输, 则该传输成功的概率是多少?
- (3) 一次成功的传输的尝试次数是多少?

答:

(1) 引入一个随机变量 T , 如果成功传输则为 1, 否则为 0。这样有

$$P_r(T=1) = \binom{N}{1} \times p \times (1-p)^{N-1} \Rightarrow P_r(T=1) = N \times p \times (1-p)^{N-1}$$

(2) 引入一个随机变量 S_i , 如果设备 i 有一次成功的传输则为 1, 否则为 0, 同样随机变量 T_i , 如果设备 i 进行了一次传输为 1, 否则为 0, 这样

$$P_r(S_i = 1 | T_i = 1) = \frac{P_r(S_i = 1, T_i = 1)}{P_r(T_i = 1)} = \frac{p \times (1-p)^{N-1}}{p} = (1-p)^{N-1}$$

(3) 假设 N_s 表示成功传输的次数, 根据定义, 可知

$$\forall i, 0 < i \leq N, \text{ 有 } E[S_i] = 1 \times P[S_i = 1] + 0 \times P[S_i = 0] \Rightarrow E[S_i] = p \times (1-p)^{N-1}$$

因此有

$$\begin{aligned} N_s &= \sum_{i=1}^N S_i \Rightarrow E[N_s] = E\left[\sum_{i=1}^N S_i\right] = N \times E[S_i] = N \times p \times (1-p)^{N-1} \\ &\Rightarrow E[N_s] = N \times p \times (1-p)^{N-1} \end{aligned}$$

4.19 假设以太网的物理地址是随机选择的。请问在一个 1 024 个节点的网络中, 两个节点的物理地址相同的概率是多少?

答:

首先考察任意两个节点的物理地址都不相同的情况, 以太网的物理地址总共 48 b, 第 1 个站点任选一个地址, 第 2 个站点选择的地址必须和第一个站点选择的地址不同, 其概率为 $1 - \frac{1}{2^{48}}$, 第 3 个站点选择的地址和前面两个站点选择的地址不同, 其概率为 $1 - \frac{2}{2^{48}}$, 如此继续, 第 1 024 个站点要和前面 1 023 个站点选择的地址不同, 其概率为 $1 - \frac{1\,023}{2^{48}}$, 因此任意两个节点的物理地址都不相同的概率为

$$\left(1 - \frac{1}{2^{48}}\right) \left(1 - \frac{2}{2^{48}}\right) \cdots \left(1 - \frac{1\,023}{2^{48}}\right) \approx 1 - \frac{(1 + 2 + \cdots + 1\,023)}{2^{48}} \approx 1 - \frac{2}{2^{30}}$$

因此两个节点的物理地址相同的概率为

$$\frac{2}{2^{30}} \approx 2 \times 10^{-9}$$

4.20 在一个没有监控站的令牌环网络中, 在什么情况下一个坏帧会一直绕环传递? 监控站怎样来解决这个问题?

答:

在正常情况下, 发送者会移走其所发送的帧。但是有可能出现这些情况: 发送者在发送完帧后出现了故障, 或者帧的源地址在传输途中被改变了, 发送者无法识别到该帧实际上是由其传输的。监控站在帧第一次经过时设置监控位, 而如果发现了一个设置了监控位的帧则由监控站负责将其移走。

4.21 一个 802.5 令牌环中有 5 个站点, 环长度为 230 m。请问监控站必须插入多少位的延

迟? 假设信号传播速度为 $2.3 \times 10^8 \text{ m/s}$, 分别考虑数据速率为 4 Mb/s 和 16 Mb/s 的情形。

答:

环的传播延迟为 $\frac{230 \text{ m}}{2.3 \times 10^8 \text{ m/s}} = 1 \mu\text{s}$, 假设每个站点有一个 1 b 的转发延迟, 这样 5 个站点

就有 5 b 的延迟, 令牌的长度为 24 b , 因此对于 16 Mb/s 的令牌环, 环的比特延迟为 16 b , 监控站必须插入的比特延迟为: $24 - (16 + 5) = 3 \text{ b}$ 。而对于 4 Mb/s 的令牌环, 环的比特延迟为 4 b , 监控站必须插入 $24 - (4 + 5) = 15 \text{ b}$ 的延迟。

4.22 考虑一个令牌环网络, 站点可以拥有令牌的时间为 THT, 假设 RingDelay 表示当环中没有站点有数据要发送时令牌绕环一周需要的时间。

(1) 请问环中只有一个站点活跃时网络的利用率是多少?

(2) 请问为了使得只有一个站点活跃(总是有数据要发送)的网络的性能高, THT 的理想值应该是多少?

(3) 假设有 N 个站点活跃, 请问网络的令牌轮转时间 TRT 的上限是多少?

答:

(1) 环中只有一个站点活跃时, 站点获得令牌, 然后开始传输帧直到 THT 超时, 然后令牌绕环传递回来再获得令牌, 因此其利用率为

$$\frac{\text{THT}}{\text{THT} + \text{RingDelay}}$$

(2) 由上式可知, THT 越大, 利用率越高, 因此 THT 最理想的值是无穷大

(3) 假设有 N 个站点活跃, 则每个活跃的站点抓住令牌, 传输 THT 的时间, 因此令牌轮转时间 TRT 满足

$$\text{TRT} \leq N \times \text{THT} + \text{RingDelay}$$

4.23 考虑一个 16 Mb/s 的 IEEE 802.5 令牌环, 环的传播延迟为 $130 \mu\text{s}$ 。假设环中只有两个站点总是有帧要发送, 帧的长度为 200 B , 请问不采用早释令牌和采用早释令牌的环利用率是多少?

答:

环传播延迟 $t_p = 130 \mu\text{s}$, 帧传输延迟 $t_f = 200 \times 8 \text{ b} / 16 \text{ Mb/s} = 100 \mu\text{s}$, 活跃站点个数 $N = 2$ 。

不采用早释令牌的环的利用率为

$$\frac{t_f}{t_p + \frac{t_p}{N}} \times 100\% = 51.28\%$$

采用早释令牌的环的利用率为

$$\frac{t_f}{t_f + \frac{t_p}{N}} \times 100\% = 60.61\%$$

4.24 假设一个光纤令牌环长 400 km, 数据速率为 100 Mb/s, 发送站点在收到自己发送的帧的所有位之后重新产生一个令牌。信号传播速度为 200 000 km/s, 最大帧长度为 1 KB, 请问环的最大效率是多少?

答:

最大效率的情况应该是站点释放令牌, 下一个站点马上抓住令牌进行传输, 因此每个站点的传输时间为

$$1\ 024 \times 8\ \text{b} / 100\ \text{Mb/s} = 82\ \mu\text{s}$$

$$\text{环的传播时间} = 400\ \text{km} / 200\ 000\ \text{km/s} = 2\ \text{ms}$$

$$\text{最大效率} = \frac{82}{82 + 2\ 000} = (82 / 2\ 082) \times 100\% = 4\%$$

4.25 考虑一颗有两条上行链路和一条下行链路的卫星, 假定缓冲区无限大, 采用分槽 ALOHA 协议可以使得下行链路的利用率达到 0.736, 请问这个结果是如何得到的?

答:

对于分槽 ALOHA, 一个分组在一条链路上成功传输 (即在一个时槽内正好有一个帧的概率) 的概率为 $1/e$, 约为 0.368。因此传输失败的概率为 $1 - 0.368 = 0.632$ 。考虑两条上行链路 1 和链路 2, 在每个时槽内:

链路 1 上成功而链路 2 上成功的概率为 $0.368 \times 0.368 = 0.135$, 这时下行链路上有 2 帧发送;

链路 1 成功而链路 2 失败的概率为 $0.368 \times 0.632 = 0.233$, 这时下行链路上有 1 帧发送;

链路 1 失败而链路 2 成功的概率为 $0.368 \times 0.632 = 0.233$, 这时下行链路上有 1 帧发送;

链路 1 失败而链路 2 失败的概率为 $0.632 \times 0.632 = 0.399$, 这时下行链路上有 0 帧发送;

因此每个时槽内下行链路平均发送的分组为

$$E = 0.135 \times 2 + 0.233 \times 1 + 0.233 \times 1 + 0.399 \times 0 = 0.736$$

4.26 为什么运行在以太网上的高层协议常常需要在其头部字段中包含一个长度字段?

答:

以太网有最小帧长 (512 b) 限制, 如果发送的数据不够, 则需要填充以满足最小帧长限制, 因此为了能够区分携带的数据中哪些部分是属于填充, 高层协议经常会包括一个长度字段。

4.27 对于 10BASE5 以太网, 每个网段限制的最大长度为 500 m, 超过这个距离时由转发器

来重新生成原始的信号。对于一个 500 m 的同轴电缆,信号最多衰减为原来的 14% (8.5 dB),而经过 1 500 m,信号衰减为 $(0.14)^3 = 0.3\%$,这样一个信号即便经过 2 500 m 接收方仍然能够识别。那么为什么每 500 m 需要有一个转发器呢?

答:

以太网的媒体访问控制技术为 CSMA/CD,即不仅要求接收方能够识别收到的信号,同时也要求冲突检测机制能够发挥作用,即要求发送者在传输帧的同时也要能够检测信号是否遇到冲突,这就要求远方来的信号强度必须足够大,因此每 500 m 需要有一个转发器。

4.28 广播媒体的一个缺点是多个站点试图同时访问信道时会由于冲突而造成信道容量的浪费。假设有一个系统,时间分为离散的时槽,数据传输只允许在时槽开始处进行,总共有 n 个节点中,每个节点在每个时槽中试图占用信道的概率都为 p 。请问由于冲突而浪费的时槽所占的比例是多少?

答:

在某个时槽中信道空闲,意味着所有站点都不占用信道,其概率为

$$(1-p)^n$$

只有一个站点占用信道的概率为

$$np(1-p)^{n-1}$$

因此由于冲突而浪费的时槽所占的比例等于出现冲突的概率

$$1 - np(1-p)^{n-1} - (1-p)^n$$

4.29 考虑一个基带 CSMA/CD 网(注意不是 IEEE 802.3),电缆长度为 1 km,不使用任何转发器,支持的数据速率为 1 Gb/s,信号在电缆中的传播速度为 200 m/ μ s。请问最小帧长是多少?

答:

单程的传播延时为:距离/信号传播速度 = 1 000/200 = 5 μ s

基带系统中的往返传播延迟为 $2 \times 5 \mu\text{s} = 10 \mu\text{s}$

为了保证冲突检测机制能够正常应用,要求:帧的传输延迟 $\geq$ 帧的往返传播延时

所以帧的传输延时必须大于等于 10 μ s,即

$$\text{最小帧长} = 1 \text{ Gb/s} \times 10 \mu\text{s} = 10\,000 \text{ b} = 1\,250 \text{ B}$$

所以最小帧长为 10 000 b 或者 1 250 B

4.30 假设正在为以太网设计一种新的传输媒体。这种新的传输媒体允许任意两个端系统间最多有 6 个转发器。每个网段的最大传播延迟为 20 μ s。新媒体将工作在 10 Mb/s。请问最小的帧长度是多少?

答:

由于任意两个端系统最多有 6 个转发器,因此端系统间最多有 7 个网段,每个网段的最大传播延迟为 $20\ \mu\text{s}$,因此该以太网的最大端到端传播延迟为 $7 \times 20\ \mu\text{s}$ 。

为了保证冲突检测机制能够正常应用,要求:帧的传输延时 $\geq$ 帧的往返传播延时

所以最小帧长为 $2 \times 7 \times 20 \times 10 = 2\ 800\ \text{b} = 350\ \text{B}$

4.31 假设主机 A 和 B 在同一个 $10\ \text{Mb/s}$ 以太网网段上,并且两个节点间的传播延迟为 225 位时间。

(1) 假设主机 A 开始传输帧,在 A 完成传输之前,主机 B 开始传输帧。有没有可能 A 在它监测到 B 传输之前完成传输? 为什么?

(2) 假设主机 A 和 B 在同一时刻开始发送帧,帧遇到冲突,然后 A 和 B 进行二进制指数退避,退避的时槽数分别选择为 0 和 1。假设没有其他活跃的站点, A 和 B 的重传帧会不会冲突?

答:

(1) 在 $t=0$ 时, A 开始传输,而完成传输的时间至少应该在 576 b 时间之后 (512 b 的最小帧长 + 8 B 的前导字段)。而在最坏情况下, B 在 A 的传输到达 B 的时刻开始传输帧,即 $t=224$,然后在 $t=224+225$ 到达 A,因此 A 无法完成传输。

(2) A 和 B 的传输过程如下:

在时刻 0: A 和 B 开始传输;

在时刻 225: A 和 B 检测到冲突,开始发送 48 b 的 Jam 信号;

在时刻 273: Jam 信号传输完成, B 进行指数退避,下次传输在 $273+512=785$;

在时刻 $273+225=498$: B 的传输到达 A, A 检测到信道空闲;

在时刻 498: A 由于退避时槽位数 0,则开始重传;

在时刻 $498+225=723$: A 重传的第二个比特到达 B;

在时刻 785: B 超时,试图重新传输,发现媒体忙,因此 B 不会进行传输。

因此 B 的重传帧不会遇到冲突。

4.32 一个网桥连着 802.3 和 802.5 局域网,网桥有内存出错,这个问题在传送帧时会引起不可检测错误或在帧检验过程中被查到吗?

答:

因为在两个不同的 LAN 中进行帧的转发,必须重新计算检验和,而内存出错后,检验和的计算是根据内存中的帧来计算的,这样就会产生一个新的检验和。如果这个新帧在传输过程中没有错误,接收者会收到一个帧,检验和也是正确的,虽然这个检验和与该帧在转发前的检验和不一致,但是接收者会接收该帧,从而这个内存错有可能检测不出来。

4.33 一个大学的计算机系把 3 个以太网用两个透明网桥连接成一个线型网络。有一天,网管人员有事临时由他人替代,这个新的网管人员注意到网络两端没有相连,便使用一个新的透明

网桥进行连接以形成一个环。请问会发生什么问题？

答：

不会出任何问题。由于透明网桥采用的是生成树算法,桥之间交换 BPDU 消息,并形成一生成树,只有生成树上的桥才能进行转发。在这里新的网管人员连接后增加一个透明网桥形成一个环,该网桥会开始产生 BPDU 消息,并重新开始一个生成树的配置过程,其中某个桥可能会处于 Block 状态,但是不会影响数据的传输,不会出现数据的回路。

4.34 回想一下令牌环 MAC 协议,其中定义了 A 和 C 位可以由环中的站点设置分别表示地址识别和帧拷贝。当帧绕环一圈返回时,源站点可以利用这个信息。如果网桥收到一个帧并转发,它是否应该设置 A 和 C 位? 请给出相应的理由。

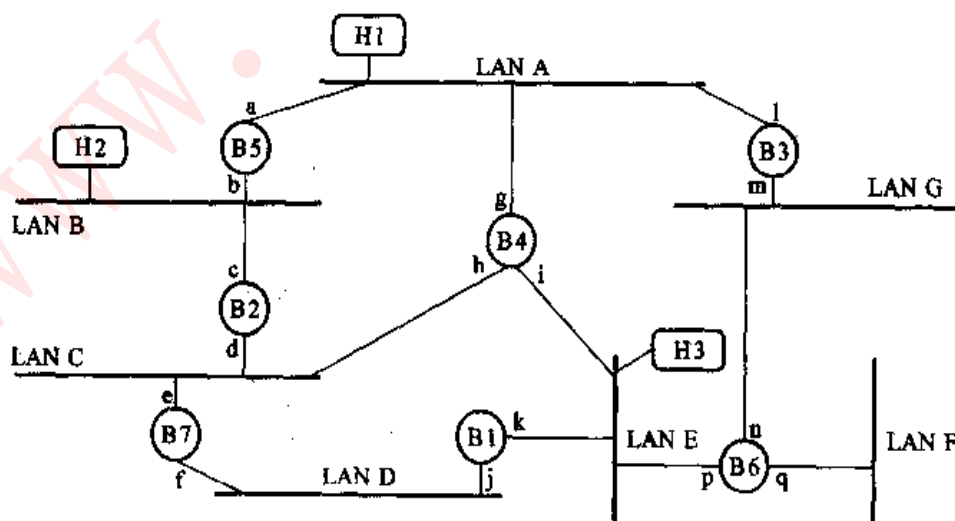
答：

A 表示地址识别,也就是目的站点是存在的,而 C 位表示目的站点已经接收到了帧,用于确认。而现在由于桥的加入,改变了 A 和 C 的语义。如果桥选择设置 A 和 C 位,但是桥虽然接收了帧,可能没法把帧转发给目的站点,所以源站点虽然看到 A 和 C 位设置了,认为帧已经经过确认,但实际上这个帧可能没有办法到达目的站点。

而如果桥不设置 A 和 C 位,源站点会因为看不到 A 和 C 位设置,而认为目的站点不存在或者出故障,从而没有办法进行确认。

4.35 如教材图 4.36 所示,7 个 LAN 通过 7 个网桥 B1 ~ B7 连接起来,桥连接到 LAN 的端口标识也在图中标出,每个网桥运行生成树算法,每个桥的 BID 就是其标识中包括的数字,有 3 个主机 H1、H2 和 H3 连接到相应的 LAN 上。请问：

(1) 哪个网桥是生成树的根,哪些端口是根端口？



教材图 4.36

(2) 请问每个 LAN 的选取桥是哪个网桥,哪个端口是选取端口?

(3) 假设当生成树算法稳定之后,主机 H1 给 H2 发送一个消息,H3 给 H1 发送一个消息,请给出这两个消息分别经过的 LAN。

答:

(1) 根桥为桥 ID 最小的桥,因此根桥为 B1,而根端口则分别为:B1 连接的 j,k 端口,B4 连接的 i 端口,B6 连接的 p 端口,B2 连接的 d 端口,B3、B5 和 B7 没有根端口。

(2) 表 4.1 列出了每个局域网对应的选取桥及相应的选取端口。

表 4.1

LAN	选取桥	选取端口
A	B4	G
B	B2	C
C	B4	H
D	B1	j
E	B1	k
F	B6	q
G	B6	n

(3) 主机 H1 发送消息给 H2 时,经过的局域网为 LAN A、LAN C 和 LAN B。而主机 H3 发送消息给 H1 时,经过的局域网为 LAN E 和 LAN A。

4.36 如果一个桥(或者路由器)的转发帧的速度不够,当负载很高时就必须丢弃帧。请问对于一个 100 Mb/s 的以太网交换机而言,为了保证不会丢失帧而要求的转发速率(每秒多少帧)至少为多少?

答:

考虑最坏的情况,所有的帧都是为最小长度的以太网帧,即帧长 64 B,再加上 8 B 的前导字段,同时也考虑到 96 b 的帧间间隔,因此最少转发速率为

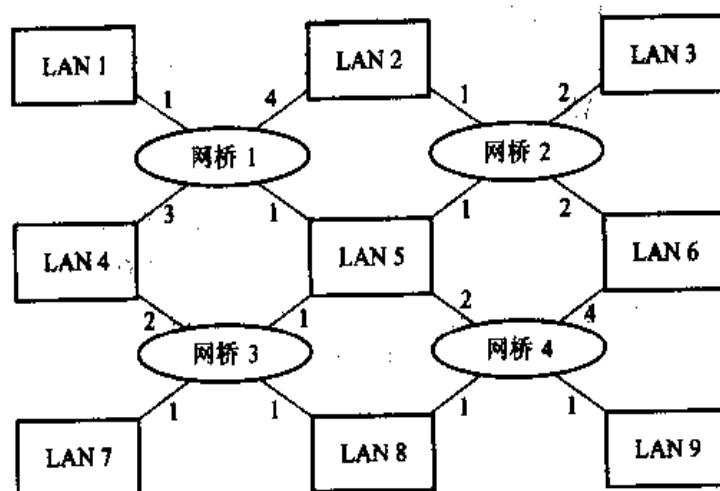
$$\frac{10 \times 10^6 \text{ b/s}}{[(64 + 8) \times 8 + 96] \text{ b/packet}} = 148\,810 \text{ packet/s}$$

4.37 教材图 4.37 给出了一个通过网桥连接的 LAN 拓扑,每个链路的值表示该链路的花费,请给出对应的生成树。假设网桥 1 关机,请问现在生成树又是怎样?

答:

对应的图的生成树的根为桥 1,如图 4.2 所示。

网桥 1 关机后,生成树的根为桥 2,如图 4.3 所示。



教材图 4.37

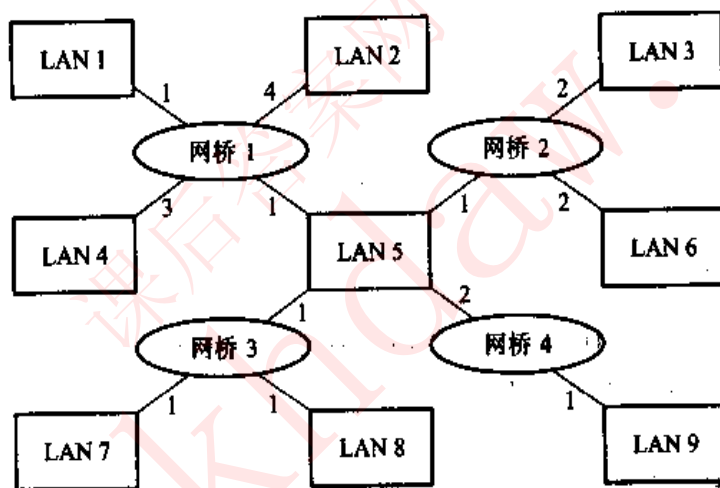


图 4.2

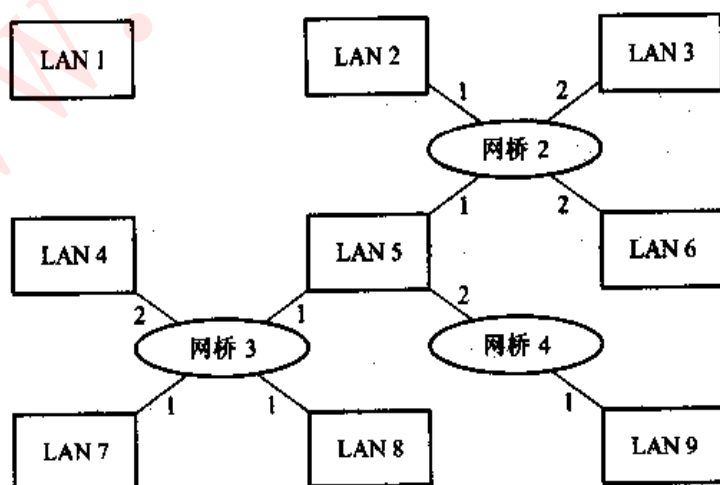
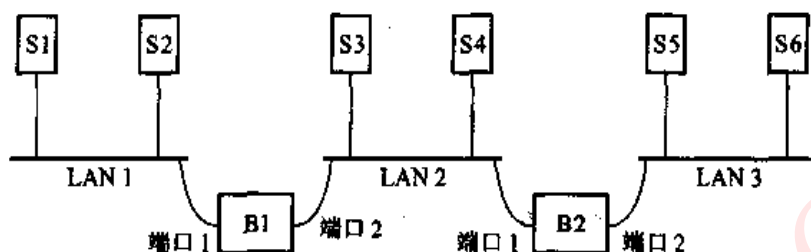


图 4.3

4.38 如教材图 4.38 所示,6 个站点通过透明网桥 B1 和 B2 连接到一个扩展的 LAN 上。刚开始桥的转发表为空,假设帧传输如下:

S2 传输帧给 S1;S5 传输给 S4;S3 传输给 S5;S1 传输给 S2;S6 传输给 S5

请给出这些帧传输完之后 B1 和 B2 的转发表。



教材图 4.38

答:

B1 转发表参见表 4.2。

表 4.2

站点	端口
S2	1
S5	2
S3	2
S1	1

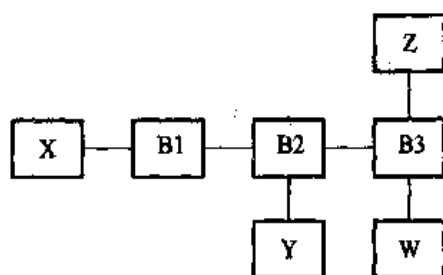
B2 转发表参见表 4.3。

表 4.3

站点	端口
S2	1
S5	2
S3	1
S1	2

4.39 如教材图 4.39 所示,其中 X、Y、Z、W 为主机,B1、B2 和 B3 为透明网桥,所有网桥的转发表初始为空。

(1) 假设 X 发送分组给 Z。哪些网桥会了解到 X 的位置? 主机 Y 的网络接口会收到该分组吗?



教材图 4.39

(2) 接着 Z 发送分组给 X。哪些网桥会了解到 Z 的位置？主机 Y 的网络接口会收到该分组吗？

(3) 接着 Y 发送分组给 X。哪些网桥会了解到 Y 的位置？主机 Z 的网络接口会收到该分组吗？

(4) 接着 Z 发送分组给 Y。哪些网桥会了解到 Z 的位置？主机 W 的网络接口会收到该分组吗？

答：

(1) X 发送分组给 Z，网桥 B1、B2、B3 会收到该分组，从而了解 X 的位置，B2 由于不知道 Z 的位置会进行扩散，因此主机 Y 会收到该分组；

(2) Z 发送分组给 X，由于 B3、B2 和 B1 已经了解到 X 的位置，因此 B3、B2、B1 都会收到该分组，了解到 Z 的位置，但是不会进行扩散，即 Y 无法收到该分组；

(3) Y 发送分组给 X，B2 收到该分组，由于已经了解到 X 的位置，所以只会向 B1 转发，因此 B2、B1 了解到 Y 的位置，而 Z 无法收到该分组；

(4) Z 发送分组给 Y，B3 尚未知道 Y 的位置，因此进行扩散，B3、B2 会了解到 Z 的位置，而 W 会收到该分组。

4.40 假设一个网桥有两个端口连接到同一个 LAN 上。网桥怎样检测到这个情况，怎样进行处理？

答：

网桥通过一个端口发送 BPDU 消息，然后该 BPDU 消息从另外一个端口接收回来，这时网桥就可以知道有两个端口连接到同一个 LAN 上。网桥将选择具有更小端口号的端口，而把另外的端口关闭。

第五章 高速网络技术

5.1 10 Mb/s 以太网与 100 Mb/s 以太网及 1 000 Mb/s 以太网的异同是什么?

答:

10 Mb/s、100 Mb/s、1 000 Mb/s 以太网的共同点是它们都采用相同的帧格式:802.3 以太网帧格式,同时这三种以太网都可以支持 CSMA/CD 协议即半双工方式。这三种以太网的主要不同点是它们支持的数据传输速率是不同的,同时下层所采用的传输媒体、编码方式等物理层协议也是不尽相同的。10 Mb/s、100 Mb/s、1 000 Mb/s 以太网都可以支持全双工的方式,同时能够和以前的协议兼容。

5.2 和快速以太网相比,为什么 100VG-AnyLan 技术不能流行?

答:

因为 100VG-AnyLan 采用了 DPP 的媒体访问协议,而不是采用 CSMA/CD。快速以太网采用共享 10 Mb/s 以太网使用的 CSMA/CD 协议,它能够与 10 Mb/s 以太网兼容,更加容易掌握和管理。

5.3 如何在以太网(10/100/1 000 Mb/s)交换机上实现 CSMA/CD?

答:

以太网交换机实际上是一个多端口的网桥,它的端口一般支持两种工作模式,分别是全双工和半双工,全双工模式中站点可以同时发送和接收,这个时候站点不需要 CSMA/CD。而半双工模式中采用 CSMA/CD 媒体访问协议,此时工作于半双工模式的以太网交换机的端口相当于一个以太网站点,它和该端口所连接的其他站点一起构成了一个冲突域。

5.4 100BaseTX、100BaseT4 和 100BaseFX 对所使用的传输媒体有什么特别的要求?各自适用于什么场合?

答:

100BaseTX 用于 5 类 UTP 和 1 类 STP 上的传输,使用两对线路,一对用于传输,一对用于接收,采用 RJ-45 接头,传输距离为 100 m。

100BaseT4 用于 3,4,5 类 UTP 线路上 100 Mb/s 的数据传输,使用 4 对线路,它采用的编码方式使得它可以在音频级的 3 类 UTP 上传输 100 Mb/s,也采用 RJ-45 接头,传输距离为 100 m。

100BaseFX 使用两芯多模 62.5/125 μm 光纤,传输距离比较长,可以达到 400 多米,常用于快速以太网主干线上网桥、路由器和交换机之间的连接。

5.5 为什么 ATM 技术可以用于局域网,又能用于广域网?

答:

ATM 制定了 LAN 仿真(LANE)协议,从而允许传统 LAN 与 ATM 之间的互通。LANE 规范包括在 ATM 上建设以太网或令牌环网仿真 LAN(ELAN)所必需的协议,提供 LAN 与 ELAN 之间的互通。另一方面,ATM 可以采用 MPOA 协议,将 ATM 层与第三层协议综合在一起,以此实现广域网中的路由等网络层服务。

5.6 用于广域网的 ATM 技术与用于局域网的 ATM 技术有什么不同?

答:

用于广域网的 ATM 技术(MPOA)致力于在 ATM 上实现第三层协议的功能,从而在 ATM 上实现多种协议(如 IP 等)。用于局域网的 ATM 技术(LANE)致力于提供实际 LAN 与 ATM ELAN 之间的无缝连接。它使用 LEC、LES、BUS、CS 等技术使原有局域网能够无需任何改动就可方便地加入到 ATM 网络中。

5.7 ATM 中 SVC 和 PVC 分别有什么特点?能用于什么场合?

答:

SVC 是交换虚电路,即需要有连接建立和释放的过程,而 PVC 是永久虚电路,预先在 ATM 交换机中配置好,不需要连接建立和释放的过程。PVC 一般用于需要一直保持连接进行数据传输的场合,这样可以减少连接建立的开销,但是如果没有数据传输,PVC 所占用的资源也不能给别人使用。而 SVC 一般用于比较灵活的场合,在有数据传输时才建立一条 SVC。

5.8 局域网仿真是 ATM 中实现 ATM 与以太网互连的一种方法,如何解决单点故障问题(即整个网络只有一个局域网仿真服务器)?

答:

ATM LANE V 1.0 规范没有提供对于冗余 LANE 部件的支持,Cisco 公司提出了一种 SSRP 协议(Simple Server Replication Protocol)来解决这个问题。这时在 ATM 交换机中配置多个 LECS 地址,每个 LECS 有不同的级别。在 LANE 中,LEC 首先要取得 LECS 的地址,这时 ATM 交换机会返回所有这些配置好的多个 LECS 地址,LEC 首先尝试和最高级别的 LECS 联系,如果失败,则试下一个。而 SSRP 协议保证所有的 LECS 服务器中只有一个主服务器,而其他服务器都处于后备状态。而对于 LES/BUS 的单点故障也采用类似的策略。

5.9 请简述 ATM 网络中 LES、LEC、BUS 等之间的关系。

答:

LEC 是指那些支持 LANE 的端系统,连接有 ATM 网卡的工作站、ATM 接口的 LAN 交换机、

支持 ATM 接入的路由器等要求实现 LEC,它通过 LES 进行地址登记、解析等服务,并且支持和其他 LEC 之间的通信,对于组播或者广播方式的通信则通过 BUS 来进行。

LES 为 LEC 服务,LES 作为管理和控制中心,主要负责动态登录和 MAC 地址转换

由于 ATM 是一种点到点的技术,不支持组播和广播,BUS 负责所有广播式点到多点数据传输服务。

5.10 MPOA 是实现 ATM 路由的一种方法,与传统路由器方法相比 MPOA 有什么好处?

答:

MPOA 提供大容量虚拟路由器的能力。它不受固定的卡槽位数,背板容量,第二层和第三层转发容量或端口位置的限制。因为 MPOA 将路由功能分配在多个物理设备上,采取分布式方式,因此提高了伸缩性。

5.11 第三层交换是当今比较流行的技术,其基本特点是既有交换机的性能又有路由器的路由功能。试问在 ATM 交换机系统上如何实现第三层交换?

答:

在 ATM 交换机系统中,ATM 交换机,MPC 和 MPS 共同构成一个虚拟路由器。每种设备都提供一个传统多协议路由器功能的一部分。MPC 提供物理连接能力,第二层拓扑结构和分组转发。交换机提供边缘设备之间的高速智能分组传送。MPS(路由器引擎)维护第三层路由选择的拓扑结构,完成第三层的转发,并进行第三层的路由计算。

5.12 两个主机之间进行通信,对于以下三种情况分别计算要经过的延时:

(1) 途中要经过一个 10 Mb/s 以太网交换机,该交换机采用存储转发交换技术,帧大小为 5 000 位。假设每个链路的传播延迟为 10 μ s,交换机在收到完整的帧后马上转发。

(2) 和(1)相同,不过途中有 3 个交换机。

(3) 和(1)相同,不过交换机采用直通方式,在收到帧的前 200 位后开始转发。

答:

传输延迟 $T_1 = 5\,000\text{ b}/10\text{ Mb/s} = 0.5\text{ ms}$

传播延迟 $T_2 = 10\text{ }\mu\text{s}$

(1) 延迟 $= 2 \times T_1 + 2 \times T_2 = 1.02\text{ ms}$

(2) 延迟 $= 4 \times T_1 + 4 \times T_2 = 2.04\text{ ms}$

(3) 延迟 $= 2 \times T_1 + 200\text{ b}/10\text{ Mb/s} + T_2 = 0.54\text{ ms}$

5.13 为什么 ATM 信元采用固定长度,并且信元长度比较小,只有 53 B?

答:

采用比较小的信元长度是为了降低网络延迟和延迟抖动,一个发送站点最多只需要等待一

个信元时间。采用固定长度是为了使得交换更加简单,有助于通过硬件实现来提高转发速率。

5.14 为什么在 ATM 网络实现 ARP 比较困难? 一个可能的实现 ARP 的方法是什么? 它有什么缺点?

答:

ATM 是一种连接方式的技术,因此实现广播并不是简单直接的,而 ARP 要求广播。一种实现 ARP 的机制是定义一个 ARP 服务器,所有站点都向 ARP 服务器登记,所有的 ARP 请求都发给 ARP 服务器。这种机制要求所有站点都要支持登记功能,ARP 服务器成为一个单点故障。

5.15 在 ATM 交换机中采用 VP 路由的好处是什么?

答:

采用 VP 路由有以下优点:

- (1) 一旦建立虚路径,其他虚电路可以采用该 VP 路由,而不需要重新选择路由;
- (2) VP 路由只需要根据 12 b 的 VPI 字段而不是 28 b 的 VPI + VCI;
- (3) 可以通过 VP 交换来实现一组虚电路的交换。

5.16 为了保证冲突检测机制能够正常运行,10 Mb/s 以太网的最小帧长为 512 位,覆盖的范围可以为 2 500 m,而 100 Mb/s 以太网为了能够和 10 Mb/s 以太网兼容,同样也要求最小帧长为 512 位,这时覆盖的范围只有 200 m,那么在 1 Gb/s 以太网中 CSMA/CD 机制是怎样的?

答:

1 Gb/s 以太网中,由于数据传输速率增加了 10 倍,帧的传输延迟大大减少,如果不对 CSMA/CD 作任何改动,则覆盖的范围只有 20 m,这显然是不实用的,因此对 CSMA/CD 协议进行了扩展,即在 MAC 帧的尾部附加一些特殊的编码,这些编码不会出现在正常的数据传输中,使得帧的传输时间最少为 4 096 比特时间,这样保证了冲突检测机制能够正常运行。

5.17 以太网的全双工模式所适用的环境是什么?

答:

以太网的全双工模式要求两个以太网节点(主机或者交换机)通过一条点到点的链路连接,并且要求节点具有缓冲支持,由于在一个方向只有一个节点传输,因此不会出现冲突,不再需要有 CSMA/CD,节点可以任意发送,吞吐率为链路速率的一倍。

5.18 请问 ATM 信元头部占用的开销是多少?

答:

一个 53 B 的 ATM 信元有 5 B 的头部,因此开销为: $\frac{5}{53} \times 100\% = 9.4\%$

5.19 假设一个 IP 数据报总共 40 B 长,包括了 20 B 的 TCP 头部和 20 B 的 IP 头部,该数据报经过一个 ATM 网络,ATM 网络采用 AAL5 来封装 IP 分组。请问需要有多少个 ATM 信元? 如果 ATM 采用 AAL3/4 来封装呢?

答:

该 IP 数据报可以封装在一个 AAL5 CS-PDU(可以携带 48 B 的数据)中,因此只需要一个 ATM 信元。而对于 AAL3/4,虽然 CS-PDU 也是 48 B,但是每个信元只能携带 44 B 的数据,因此需要 2 个 ATM 信元。

5.20 信元交换中一般采用虚电路路由而不是数据报路由,请给出一个理由。

答:

数据报路由要求信元必须携带路由信息,而 ATM 的信元本身只有 53 B,因此路由信息占用的开销将非常大。

5.21 考虑一个共享总线交换机,I/O 总线速率为 800 Mb/s,内存带宽为 2 Gb/s。请问这样的交换机可以支持多少个 45 Mb/s 的 T3 链路?

答:

因为 I/O 总线速率小于内存带宽,所以 I/O 总线成为瓶颈。注意到通过 I/O 总线交换的分组要通过总线两次,一次从输入端口到达 DMA,另一次从 DMA 到达输出端口。因此交换机能够支持的 T3 链路的个数 $= \frac{800}{2 \times 45} = 8$

5.22 请给出一个 16×6 Knockout 集中器的设计。

答:

如图 5.1 所示。

5.23 在教材图 5.42 的 Batcher 网络中,假设从上到下 8 个输入分别为 7,6,5,4,3,2,1,0。请给出排序后的输出。

答:

如图 5.2 所示。

5.24 一个 $n \times n$ Banyan 网中每级包括 $n/2$ 个 2×2 交换单元。请推导出一个 $n \times n$ Banyan 网总共包含的 2×2 交换单元的个数。如果 $n = 8$,交换单元是多少?

答:

每级包括 $n/2$ 个交换单元,第一级之后分组被交换到网络的一半区域,第二级后路由到网络的 $1/4$ 区域,也就是说每级之后就知道了分组属于当前网络的哪一半,因此一个 $n \times n$ Banyan 网需

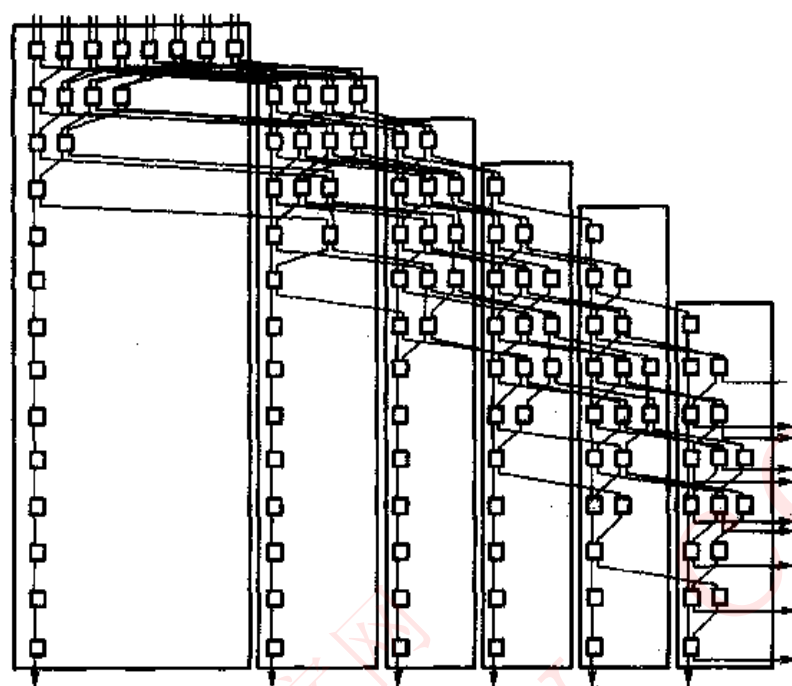


图 5.1

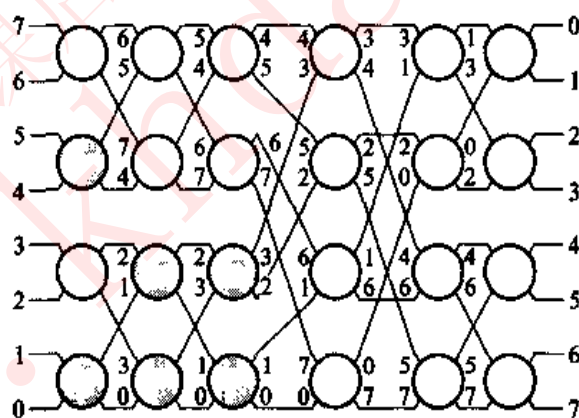


图 5.2

要 $\log_2 n$ 级。总的交换单元个数为 $(n/2)\log_2 n$ 。对于 $n = 8$, 交换单元个数为 12。

5.25 请简要描述 WDM 的工作原理。

答:

由于光电器件的限制,一般只能支持 2.5 Gb/s 的速率,而这只占了光纤媒体带宽的很小一部分。波分多路复用 WDM 系统就是一种利用不同的光波波长,将多个光信号合并并在单一的光

纤中传送的光纤传输技术。 n 个激光传输器独立产生一个波长中心各不相同的光波,这 n 束光波然后通过一个 WDM 复用器混合在一起被注入光纤中。光波在光纤传输过程中会遇到衰减,因此途中可能会有多个光放大器来放大光信号,从而可以扩展光纤覆盖的距离。光信号最后到达一个 WDM 解复用器。WDM 解复用器把信号分成 n 份,每份具有原有信号能量的 $1/n$,每份信号然后通过一个过滤器来获取相应波长的光信号。这个光信号最后由光接收器转换为电信号并获取原始的比特流。如果把 4 路光信号波分复用在一起,则支持的数据速率就可达 10 Gb/s,复用的信号越多,支持的数据速率越高。

第六章 无线网络

6.1 画图说明一个典型无线网络的拓扑结构。为什么有线网络通常作为无线网络基础设施的一部分？

答：

图 6.1 所示是一个典型无线网络的拓扑结构。

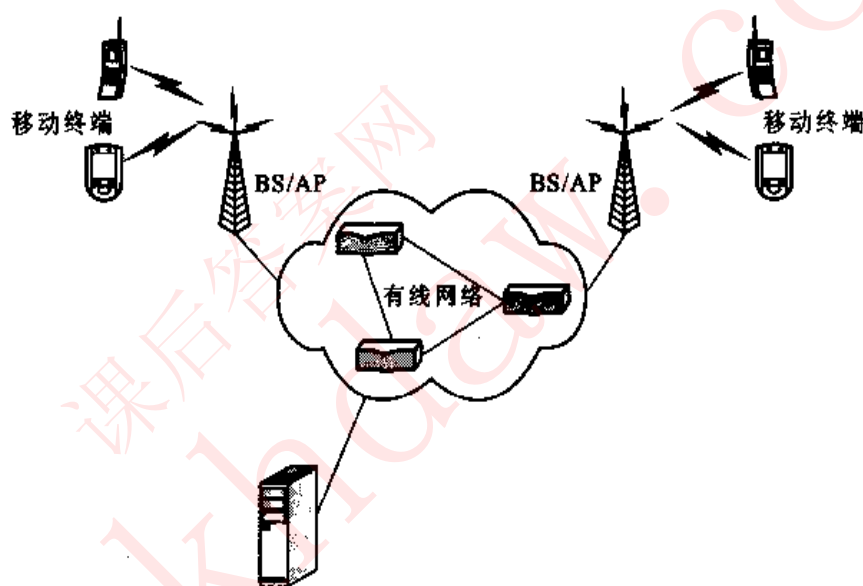


图 6.1

通常有线网络作为无线网络骨干网络用来支持移动终端 (MT) 之间和 MT 与固定终端之间的通信,无线网络主要用来 MT 的末端接入。通过有线网络的骨干连接,使用多个 BS/AP,使无线网络可以实现广域覆盖;而且大量的 MT 在 BS/AP 接入,并通过 BS/AP 之间进行通信,这需要 BS/AP 之间有高带宽、可靠的连接,这也只有有线网络才可以满足。

6.2 相对有线网络,无线网络有哪些优势？

答：

无线网络不需要网络布线,可以实现任何接入之间、随时随地的网络连接和通信。

6.3 无线网络应用从业务上分为哪两类？它们分别围绕哪种有线网络业务来发展的？无线网络应用从覆盖范围上分为哪两类？这两种划分的结合形成了无线网络应用的哪 4 个部分

(同时说明对应的4个应用实例)?

答:

无线网络应用从业务上分为:面向语音的应用和面向数据的应用。

面向语音的应用是围绕着连接 PSTN 的有线网络业务来发展的;面向数据的应用是围绕着 Internet 和计算机通信网络的有线网络业务来发展的。

无线网络应用从覆盖范围上分为:局域网应用和广域网应用。

这两种划分的结合形成了无线网络应用的4个部分:面向语音的广域无线应用(如 GSM)、面向语音的局域无线应用(如无绳电话)、面向数据的广域无线应用(如 GPRS)和面向数据的局域无线应用(如 WLAN)。

6.4 指出目前商用的三种主要无线网络业务,并指出它们首次推出的大概时间。

答:

目前商用的主要无线网络业务及其首次推出的大概时间:GSM,1991年;CDMA,1995年;CDPD,1993年;GPRS,1997年;WLAN,1997年。

6.5 无线网络分为哪几代?各代的主要特征是什么?这说明无线网络经过技术的不断进步,正朝着一个什么样的方向发展?

答:

目前来说无线网络分为三代:第一代(1G)无线网络主要特征是面向语音的模拟无线系统,使用 FDMA 技术实现。第二代(2G)无线网络主要特征是面向语音的数字无线系统,使用 TDMA 或窄带 CDMA 技术实现。第三代(3G)无线网络的主要特征是把蜂窝电话、PCS 语音业务以及移动数据业务用各种分组交换数据业务综合在一个高语音质量、高容量、高速率的网络系统中,使用宽带 CDMA 技术实现。

无线网络经过技术的不断进步和更新换代,正朝着数字化、高带宽、大容量、多媒体、综合业务的方向发展。

6.6 从工作频率、基于网络的应用和支持数据业务的速率来看,3G 无线系统和 WLAN 有何不同?

答:

工作频率上,3G 使用需要许可证频段,而 WLAN 使用免许可证频段。

在基于网络的应用方面,3G 还融合了广域移动数据业务和蜂窝电话业务,而 WLAN 只提供局域移动数据业务。

在速率方面,3G 目前还只能提供 2 Mb/s 左右的移动数据服务,WLAN 可以提供高达 54 Mb/s 的移动数据服务。

6.7 什么是 WPAN? WPAN 与 WLAN 有何不同? 列举 WPAN 的两个例子。

答:

WPAN (Wireless Personal Area Network) 即无线个人区域网络, WPAN 的目的是用于 Ad-hoc 场合, 把个人设备 (如家电、便携机、PDA、手机、耳机等) 连接起来。

WLAN 支持较高的数据速率和大的用户容量; 而 WPAN 工作在用户数量较少的场合, 通信速率相对也较低。二者都工作在免许可证频段, 但 WPAN 的覆盖范围比 WLAN 要小。因此, WLAN 通常适合于商用和办公, 而 WPAN 适合于个人和家庭。

典型的 WPAN 如蓝牙、HomeRF 等。

6.8 简述主要的三种多址无线信道接入技术的原理。同时指出使用该种多址技术实现的典型系统。

答:

主要的三种多址无线信道接入技术包括: FDMA、TDMA 和 CDMA。

FDMA (频分多址) 以 PSTN 干线网络上的多址技术 FDM 为基础, 它把通信系统的总频段划分成若干个等间隔的频道 (信道), 分配给不同的用户使用。这些频道互不交叠, 其宽度应能传输一路话音信息, 而在相邻频道之间无明显串扰。发送端将发送的信息调制到某个频道上, 接收端通过频率选择 (滤波), 从混合信号中选出相应的信号。模拟的蜂窝系统都采用了 FDMA/FDD, 如 AMPS 和 TACS。

TDMA (时分多址) 以 PSTN 干线网络上的多址技术 TDM 为基础, 它把时间分割成周期性帧, 每一帧再分割成若干个时隙, 然后根据一定的时隙分配原则, 使移动台在每帧中按指定的时隙向基站发送信号, 基站可以分别在各时隙中接收到移动台的信号而不干扰。同时基站发向多个移动台的信号, 都按规定在预定的时隙中发射, 各移动台在指定的时隙中接收 (时间选择), 从合路的信号中提取发给它的信号。TDMA 的格式是完全数字化的, 并且缓冲和多路复用功能都可灵活配置, 所以多个用户的时隙分配随时可以调整, 为不同的用户提供不同的接入速率。当前应用 TDMA 的典型系统如 GSM。

CDMA (码分多址) 的每个用户 (收发信机) 具有特定的地址码, 用于发射信号的扩展频谱调制, 从而实现在公共信道上传输信息。CDMA 的特征是代表各信源信息的发射信号在结构上各不相同, 并且其地址码相互具有准正交性, 以区别地址, 而在频率、时间和空间上都可能重叠。这些码的设置使得它们在同一时间、同一频带被使用时, 了解某一用户的码的接收器可以从所有收到的信号中检测到该用户信息。典型的 CDMA 应用如窄带 CDMA 系统或 3G 系统。

6.9 描述两种基本的无线网络拓扑及它们适应的应用场合。并试从可伸缩性、灵活性、可控性、路由的复杂性、覆盖范围、可靠性以及存储转发延迟和介质使用效率等特征比较这两种拓扑。

答:

两种基本的无线网络拓扑是基础结构集中式(或称中心辐射式)拓扑和 Ad-hoc(或称分布式)拓扑。

在基础结构拓扑中,固定(有线)基础结构用来支持移动终端(MT)之间和 MT 与固定终端之间的通信。基础结构网络通常设计用于大的覆盖区域和使用多个 BS/ AP 的情况。BS/AP 作为基础结构网络的边沿节点用于 MT 的无线接入,任何一个 MT 与另一个 MT 之间的通信,都要通过 BS/AP。在基础结构拓扑中设置有中心站用于管理 MT 接入网络,它控制 MT 并监控每个 MT 传输的内容。

很显然,基础结构拓扑的无线网络投资大、建设周期长、系统容量大、通信速率高,它适用于用户多且较稳定的场合。所有标准蜂窝移动电话和无线数据系统都支持基础结构的使用。

Ad-hoc 拓扑中 MT 不需要固定的基础结构就能工作,是“纯无线”的网络,而且通常 Ad-hoc 网络可自动重组,因此 Ad-hoc 拓扑适于在移动或固定情况下快速部署无线网络。Ad-hoc 拓扑又分为单跳 Ad-hoc 和多跳 Ad-hoc,前者用于 MT 之间近距离的直接通信,后者用于由于无线信号的覆盖限制、MT 接力实现远距离通信的情况。一些 WLAN 和 WPAN 支持 Ad-hoc 拓扑。

基础结构网络拓扑与 Ad-hoc 网络拓扑的比较如下:

- 可伸缩性

在 Ad-hoc 网络中,无线收发器的覆盖范围通常限制了网络的扩展,而且没有简单的方法可以用来扩展网络所能支持的覆盖范围或容量。多跳 Ad-hoc 中,可以通过多个 MT 增加网络的覆盖范围,但网络的流量处理能力仍保持不变。基础结构网络本来就具有可伸缩能力。为了扩展无线基础结构网络,增加了大量的 BS 或 AP 用来扩展覆盖区域或增加容量,而且使用了相同的频率。因此,基础结构网络总是用于广域覆盖和可变通信量的应用中。

- 灵活性

运行基础结构网络需要部署网络基础设施,这通常十分费时而且昂贵。Ad-hoc 网络则天生就具有灵活性,可以快速组网。因此,Ad-hoc 网络通常用于更看重灵活性的临时应用中。

- 可控性

为了协调无线网络的正常工作,需要集中控制它的某些特征,如时间同步、在确定区域工作的移动台发射功率等。在一个基础结构网络中,所有对这些特征的控制自然在 BS 或 AP 中实现。在 Ad-hoc 网络中,实现对这些特征的控制则需要更复杂的结构,要求改变所有的终端。

- 路由的复杂性

在多跳 Ad-hoc 网络中,每个终端都可以发送信息给其他终端。要具备这种能力需要每个终端都监视相邻的其他终端的存在,并且能够与它们连接。要做到这一点,就需要有一种路由算法可以发送信息给相邻的下一个合适的终端。这些特性的实现增加了终端和网络工作的复杂性。在基础结构网络和单跳 Ad-hoc 网络中就不存在这个问题。

- 覆盖范围

在单跳 Ad-hoc 网络中,两个终端之间的最大距离就是终端无线接口的覆盖范围。在基础结构网络中,两个无线终端之间的通信是依靠一个(或多个)BS/AP。这两个终端之间的最大距离

是单个无线调制解调器覆盖范围的 2 倍(或更大),因为它们可以位于 BS/AP 覆盖区域的边缘。实际中的 BS/AP 通常寻机使用高架固定,以增加无线调制解调器的覆盖范围,这使得在基础结构网络中两终端之间的最大距离比 Ad-hoc 网络中使用相同无线调制解调器的最大距离要大得多。

- 可靠性

基础结构网络是“单点破坏”网络,如果 BS 或 AP 失效,整个通信网络都会瘫痪。而这个问题在 Ad-hoc 网络中就不存在。

- 存储转发延迟和介质使用效率

在单跳 Ad-hoc 网络中,信息只传输一次,不存在存储转发过程。在基础结构网络中,数据传输两次,一次从信息源到 BS/AP,另一次从 BS/AP 到目标,BS/AP 会存储信息然后转发。这就增加了数据分组的延迟。多跳 Ad-hoc 网络可能会有几次传输和几次存储转发延迟,这与当时的拓扑和数据从源端到目的端的跳数有关。

6.10 典型的蜂窝拓扑中,频率复用因子是多少?

答:

典型的蜂窝拓扑中,频率复用因子是 7。

6.11 极限情况下,可以使蜂窝拓扑中的小区无限小而使系统总的容量无限大。但实际上是不可能的,为什么?

答:

这是因为:① 基站的发射功率不能无限小;② 小区太小,很难获得较好的频率分隔效果和较小的串扰;③ 小区越小,用户的越区切换越频繁,通话中断的可能性越大,系统管理开销也越大。

6.12 简述码分多址的工作原理。

答:

每个站点有一个唯一的 m 位码片序列,所有的码片序列都满足两两正交的要求,即满足:① 任意两个不同的码片序列 S 和 T 的内积($S \cdot T$)为 0;② 任何码片序列与自己的内积($S \cdot S$)为 1。

我们以基站到站点的传输来说明 CDMA 的工作原理(反向的传输原理类似):

(1) 当发送比特 1 时,基站就发送其码片序列,发送比特 0 时,站点就发送其码片序列的反码(码片序列都以双极型表示,二进制 0 为“-1”,二进制 1 为“+1”);

(2) 若需向多个站点同时传输,它们片码的双极型信号就线性相加,形成片码的混合信号;

(3) 站点收到片码的混合信号(设为 S')和自己的片码(设为 C)计算内积 $S' \cdot C$,就可以还原发送到该站点的比特数字:结果是“+1”表示 1,“-1”表示 0,“0”表示没有信号。

由于片码满足两两正交的要求,第(3)点总是可以做到的。

由 CDMA 的工作原理可知,一个比特的数字需要 m 位码片序列的宽度去传输,这对传输的带宽有较高的要求,但也正是通过这 m 位的码片序列去识别不同的站点,提高系统容量。

6.13 窄带 CDMA 的峰值传输速率为 384 kb/s,系统带宽为 1.25 MHz,根据香农公式,要到达峰值速率,该系统能在信噪比为多少的情况下工作?

答:

$$\text{信噪比} \frac{S}{N} = (384 \times 10^3) \div 1.44 \div (1.25 \times 10^6) = 0.2134$$

6.14 说明两种基本的扩展频谱通信(直扩和跳频)的原理。

答:

- 直接序列扩频

图 6.2 给出了直接序列扩频系统的原理图。

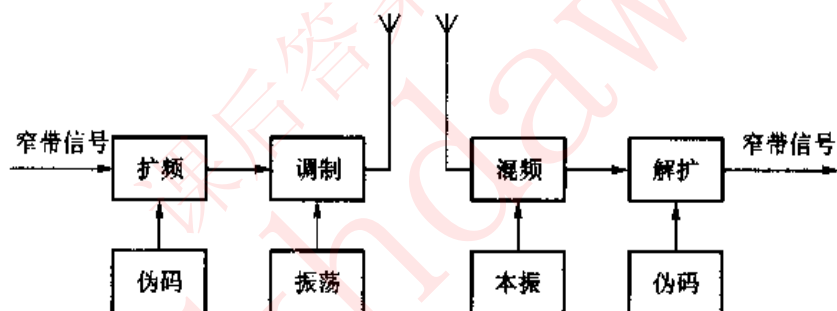


图 6.2

窄带的基带信息码是要传输的信息,它通过与速率很高的编码序列(通常用伪随机序列)进行调制将其频谱展宽(扩频过程)。频谱展宽后的序列被进行射频调制,其输出则是扩展频谱的射频信号,经天线辐射出去。

在接收端,射频信号经混频后变为中频信号,对它用与发送端相同的编码序列进行反扩展,将宽带信号恢复成窄带信号(解扩过程)。解扩后的中频窄带信号经普通信息解调器进行解调,恢复成原始的信息码。

- 跳频扩频

图 6.3 给出了跳频扩频系统的原理图。

跳频扩频利用伪随机码来控制频率的变化,发射机的频率将在一个很宽的范围内不断改变,从而射频载波的频率也在很宽的范围内变化,由此形成了一个宽带离散频谱。

接收端用相同的伪码制定本地频率合成器,使其与发送端的频率做相同的改变,经与接收到

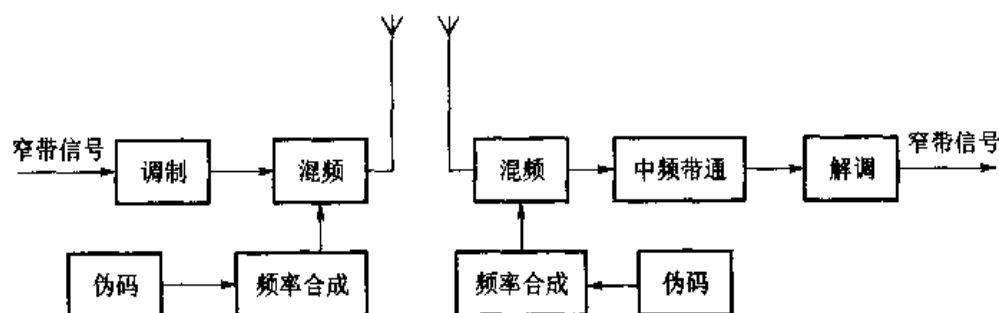


图 6.3

的信号混频,得到一个中频信号,再经过带通滤波,得到一个窄带信号,即我们需要的信号。

6.15 对跳频图案的要求是什么?

答:

跳频图案是指跳频系统的频率跳变规律,它是时间和频率的函数。跳频图案直接影响跳频系统的保密、抗干扰、多址等能力。一般要求如下几点:

(1) 跳频图案的周期要长,长到实际通信中跳频图案不重复,这就要求用于控制跳频图案的伪随机码的周期要长,如位数 $N=32$ 或 64 ;

(2) 跳频图案的保密性要好,不是简单地用伪随机码去控制跳频图案,还要考虑采用别的措施,增强跳频图案的保密性,如加密钥、加时间变量等;

(3) 跳频图案的随机性要好,即由伪随机码控制频率合成器产生的频率,在选定的跳频图案中均匀出现;

(4) 跳频图案在允许的时延情况下,各种跳频图案间的可能重叠的时隙数最小。两个跳频图案重叠,即两个发射电台同时跳到同一个频率上去,则接收端无法正常接收;

(5) 对于给定一个允许的重叠规则,其构成的跳频图案的数目应是最大的。

6.16 当前 CDMA 成为了 3G 系统最流行的选择,它具有哪些特点?

答:

3G 的特点如下:

1. 具有很强的抗干扰能力

由于 CDMA 采用了频谱扩展的原理,从而大大提高了通信抗干扰的能力。它可以在接收到的信号比干扰信号的强度小得多的情况下,仍能进行可靠的通信。

2. 用户可随机接入系统进行多址通信

CDMA 用户用不同的正交编码区分地址,多个电台共同使用一个频带,网内不需严格同步(只需在收发端同步),所以可随机接入通信网进行通信;还可以用软件随机改变地址号码,这比

频分和时分多址通信要灵活得多。

单纯地从窄带信息被扩展为宽带信号的角度看,CDMA 的频带利用率似乎降低了,但实际上由于采用码分多址,地址数相当大。

3. 安全通信

由于扩频后信号功率密度很低,近似于白噪声性能,且信噪比可低到 $-20 \sim -30$ dB,这样采用一般方法难于检测,所以信息传输具有隐蔽性。即使被检测到,由于它具有很强的抗干扰性,尤其对人为的瞄准干扰抵抗能力更强,所以它具有很高的通信可靠性。

4. 越区软切换

由于无线单元采用同一无线信道,仅有的差别是伪码序列。因此移动台从一个单元进入另一个单元时不需要从一个频率切换到另一个频率,只需更改伪码,实现越区软切换。

5. 系统性能的自适应性

每一个电台都将不需接收的信号看成干扰信号(包括其他电台发射的信号),而 CDMA 具有很强的抗干扰能力。所以在系统用户负荷满载时,即达到设计时额定用户量时,再增加用户只会引起话音质量轻微降低,而不会形成阻塞现象。

6. 抗衰落能力强

扩频信号频谱所占据的频带很宽,当由于某种原因引起衰落时,只会使小部分的频谱衰落受影响,而不会使整个信号产生严重畸变,因为它在任意窄的频率范围内所发送的功率都很小,所以扩频信号的功率谱密度远比普通调频信号功率谱密度小得多,因此,具有强的抗频率衰落能力。

6.17 有哪四类移动数据网络? 它们的主要特征是什么?

答:

移动数据网络分为四类:独立网络、共享网络、重叠网络和一体网络。

独立网络拥有自己的频段,该频段不被其他的服务所使用,并且独立网络有自己的基础网络,该基础结构也不与别的服务所共享。

共享网络与现有的面向语音的模拟服务共享频谱和部分基础结构。这些服务在模拟语音所使用的相同的无线信道上传输,但数据网络有自己的空中接口和 MAC 协议。移动数据服务除了使用专用的数据信道外,还可占用空闲的语音信道。

重叠网络与现有网络重叠,也就是数据服务不但利用分配给其他服务的频谱,还利用现有的面向语音的数字蜂窝系统的 MAC 层和空中接口,重叠网络只是使用数字蜂窝系统中业务信道和信令信道空闲的时隙。

一体网络在设计和系统实施之初,就将话音、数据和综合多媒体服务等进行一体化设计。也就是说,这种网络本身就潜在支持语音和数据等服务,不存在谁先实现、谁后加载的问题。

6.18 共享或重叠型的移动数据网络其有线网络基础设施部分可以共享吗? 为什么?

答：

一般不可以共享。因为通常面向语音无线网络的有线网络基础设施是针对语音网络而设计的,是以语音信道的电路交换技术实现,不适合数据传输的分组交换网络使用。

6.19 为支持 CDPD,在 AMPS 基础结构中需增加什么部件?

答：

主要需增加:① 移动数据基站(MDBS)用于 CDPD 的空中接口;② 移动数据中间系统(MD-IS)用于移动管理和数据转发。

6.20 简述 CDPD 中 M-ES 的数据传输(接收和发送)和移动管理过程。

答：

CDPD 数据传输和移动管理过程与移动 IP 很相似,如图 6.4 所示。

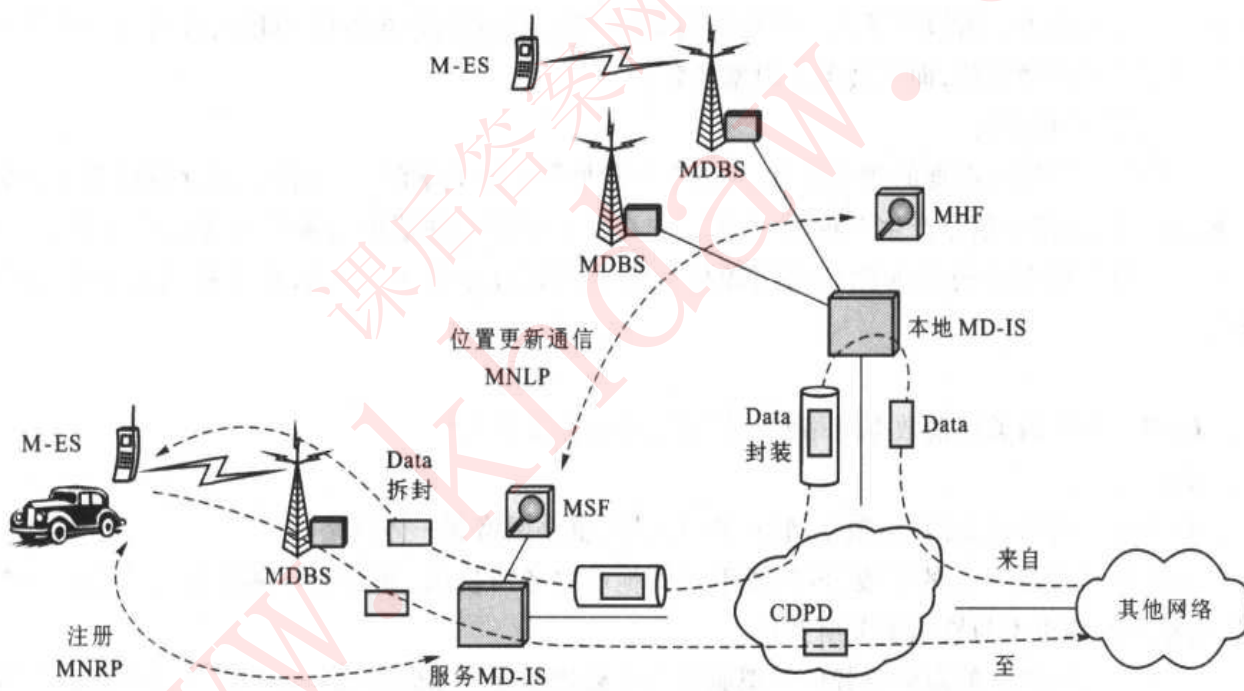


图 6.4

MD-IS 为这个过程的核心部件,MD-IS 在逻辑上分为本地 MD-IS 和服务 MD-IS。

本地 MD-IS 拥有 MHF(移动本地功能),MHF 掌握的主要信息一方面包括在本地 MD-IS 注册的所有用户的数据库,其中有分配给用户的 IP 地址。另一方面,MHF 还掌握着在本地注册的 M-ES 的当前位置信息。如果 M-ES 在本地活动,就不涉及移动管理,数据传输也很简单,由 MD-IS 通过 MDBS 和 M-ES 直接进行通信就可以了。

如果 M-ES 移动到外地,首先它通过 MNRP(移动网络注册协议)在当地服务 MD-IS 进行注

册,服务 MD-IS 拥有的 MSF(移动服务功能)使用 MNLP(移动网络位置协议)将该 M-ES 的信息通知给其 MHF。如果期间 M-ES 涉及系统间的越区切换,新的服务 MD-IS 将把其位置更新信息通知给旧的 MSF 和其 MHF。这样就完成了 M-ES 的移动管理。

移动到外地的 M-ES 在进行数据传输时,对于接收,MHF 使用隧道技术封装传输给该 M-ES 的所有分组,然后再定向转发给 M-ES 在该服务区域的服务 MD-IS 的 MSF。MSF 对转发数据拆封后,重新路由到本服务区正确的信道中去。而对于发送,为了高效地实现,MSF 将分组通过 CDPD 基础网络直接路由到目的端。

6.21 在移动 IP 中与 CDPD 的 MHF 和 MSF 对应的分别是_____和_____。

答:

在移动 IP 中与 CDPD 的 MHF 和 MSF 对应的分别是 HA 和 FA。

6.22 CDPD 有哪些主要特点?

答:

CDPD 具有以下主要特点:

(1) 64 kb/s 的有线线路和固定网连接,19.2 kb/s 的空中信道用以连接移动台,全双工实时无线数据通信;

(2) 高质量、无差错的无线数据传送,采用可避免碰撞的 DSMA/CD 无线信道多址接入技术;

(3) 采用先进的动态密钥管理、正反信道密钥不同、以每呼叫为基础进行动态密钥更换,保护用户数据不被窃听;

(4) 无连接方式的分组通信,可以 TCP/IP 为基础。由于协议的开放性,任何以 TCP/IP 为基础的软件产品,都可用于 CDPD,使 CDPD 有很好的应用基础;

(5) 适用于信息量较小但实时性要求较高的数据传输。网络中的移动终端可一直处于在线模式,随时随地发送数据、图像、文字等数据信息。

(6) 使用专用 CDPD 信道的网络中,其通信信道有保障;而对于寄生在现有语音系统上的 CDPD 网络,其数据信道有可能被突然抢占而使通信中断。

6.23 为支持 GPRS,在 GSM 基础结构中需增加什么部件?

答:

主要需要增加的部件是 GPRS 支持节点(GSN),包括两类 GSN:GPRS 服务支持节点 SGSN 和 GPRS 网关支持节点 GGSN。此外,在 GSM 的 HLR 中,需要设立 GPRS 寄存器。

6.24 GPRS 如何提供不同的数据速率?

答:

GPRS 每个 TDMA 帧中可分配 1~8 个无线接入时隙,系统可以提供 1 到多个时隙给一个用户,这样用户就可以得到理论上最大达 164 kb/s 左右的不同数据速率。

6.25 描述 GPRS 的越区切换过程。

答:

MS 侦听广播控制信道 BCCH,并与邻接小区的 BCCH 的 RSS(接收信号强度)相比较,确定它应选择的小区(这其中通常还使用了其他一些参数,包括小区等级、路径损耗、功率预算等)。MS 可独立选择小区,也可选择让 BSS 请求 MS 返回报告,过程与 GSM 相同。

这种类似于移动 IP 的越区切换过程主要分为 3 步:

(1) 当 MS 改变 RA(路由区域)时,向新 SGSN 发送 RA 更新请求,其中包括小区身份和先前 RA 的身份;

(2) 当新 RA 接受相同的 SGSN 的服务时,也有可能发生 SGSN 间路由区域的更新,新 SGSN 请求旧 SGSN 发送 MS 的路由环境(GGSN 地址和隧道信息);

(3) 新 SGSN 根据新 SGSN 地址和新隧道信息更新本地 GGSN,新 SGSN 同时也更新本地 HLR。本地 HLR 删除旧 SGSN 中的 MS 环境信息,并将用户数据加载到新 SGSN 中。新 SGSN 再给 MS 发送确认消息,请求旧 SGSN 将未发送的数据传送给新 SGSN。

6.26 简述 GPRS 协议栈结构及其各组成部分。

答:

GPRS 的协议栈结构如图 6.5 所示。

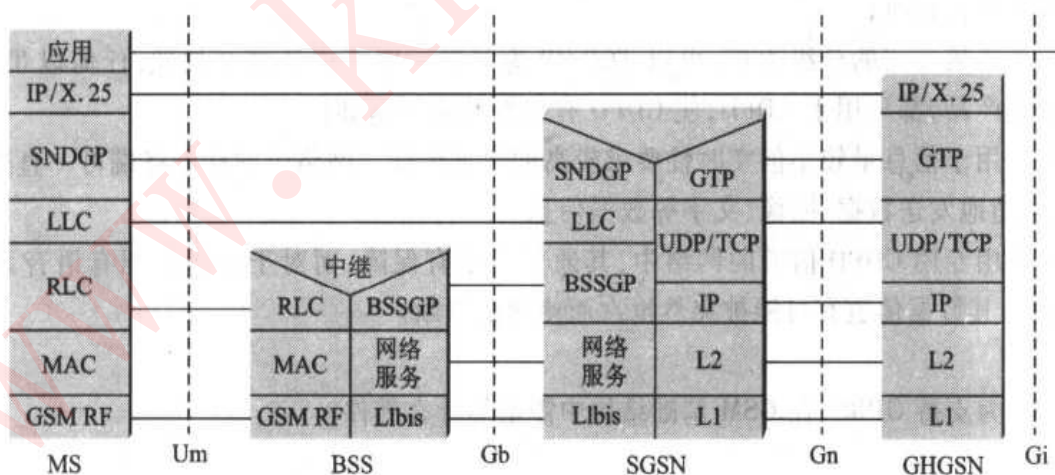


图 6.5

其中各组成部分介绍如下:

1. GTP(GPRS 隧道协议)

GTP 是 GPRS 骨干网中 GSN 节点之间的互连协议,它是为 G_n 接口和 G_p 接口定义的协议。

2. TCP/UDP

在 GPRS 骨干网中如果需要一个可靠的数据链路(如 X.25)进行 GTP PDU 的传输,就使用 TCP 协议。如果不要求一个可靠的数据链路(如 IP),就使用 UDP 协议。

3. IP

GPRS 骨干网的网络协议使用的是 IP(IPv4/IPv6),用以用户数据和控制信令的路由选择。

4. SNDCP(子网相关融合协议)

SNDCP 的主要作用是完成传送数据的分组、打包,确定 TCP/IP 地址和加密方式。

5. LLC(逻辑链路控制)

LLC 层负责从 SNDCP 数据单元上形成 LLC 地址、帧字段,从而生成完整的 LLC 帧。此外,LLC 还可以实现一点对多点的寻址和数据帧的重发控制。

6. BSSGP(GPRS 基站系统协议)

BSSGP 用来传输在 BSS 和 SGSN 之间与路由选择服务质量有关的信息。

7. NS(网络服务)

NS 层用来传输 BSSGP PDU。NS 以 BSS 和 SGSN 之间的帧中继连接为基础,而且有多跳功能,并能横贯有帧中继交换节点的网络。

8. RLC(无线链路控制)/MAC(介质访问控制)

RLC 功能能提供一条独立于无线解决方案的可靠链路。MAC 功能定义和分配空中接口的 GPRS 逻辑信道,使得这些信道能被不同的移动台共享。MAC 还实现将 LLC 帧映射到 GSM 物理信道中去。

9. GSM RF

Um 接口的物理层为 GSM RF 射频接口。

10. Relay(中继转发)

在 BSS 中,Relay 功能中继转发 U_m 和 G_s 接口间的 LLC PDU,在 SGSN 中,Relay 功能转发 G_s 和 G_n 接口间的 PDP PDU。

6.27 GPRS 有哪些优势和缺陷?

答:

GPRS 有如下优势:

(1) 资源利用率高:GPRS 以分组交换模式传输数据,用户只有在发送或接收数据期间才占用信道资源,这样多个用户可高效地利用有限的无线信道资源,同时只需要按自己的流量进行付费。

(2) 传输速率高:GPRS 可提供高达 164 kb/s 左右的传输速率。这使一些有高速率要求的移动多媒体应用成为可能。

(3) 接入时间短:GPRS 能提供快速即时的连接,用户随时“在线”。可大幅度提高事务效

率,更便捷地操作应用。

(4) 支持 IP 协议和 X.25 协议:GPRS 支持分组数据网络中最广泛的 IP 和 X.25 协议,方便地实现分组数据网随时随地的接入。

GPRS 也存在如下一些缺陷:

(1) 会发生包丢失现象:GPRS 的分组交换连接会发生包丢失现象。而且,在话音优先的 GSM 网络中其信道时隙会被话音抢占而使数据分组阻塞或丢失。

(2) 实际速率比理论值低:GPRS 数据传输速率要达到理论上的最大值需要一个用户占用所有的 8 个时隙,这几乎是不太可能的。

(3) 终端不支持无线终止功能:目前 GPRS 终端还不支持无线终止接收来电的功能,无法拒绝未经授权的内容。

(4) 调制方式不是最优:GPRS 采用基于 GMSK 的调制技术,这种调制方式不是最优的。

6.28 CDPD 或 GPRS 能支持有 QoS 要求的实时移动数据服务吗?为什么?

答:

不能。由于它们是共享或叠加在已有话音网络上,而话音可以优先使用无线信道,这样当一个小区的话音太忙时,将没有空余的无线信道提供 CDPD 或 GPRS 作数据服务使用。当然也可以保留部分无线信道给 CDPD 或 GPRS 专用,这样将损害话音服务的利益。

6.29 第三代移动通信系统(3G)的主要组成部分有哪些?

答:

主要组成部分包括:接入部分、宽带通信网和智能化网络管理。

6.30 国际上主要的 3G 标准有两个:_____和_____,它们分别起源于欧洲的_____和美国的_____。我国自主的 3G 标准是_____。

答:

国际上主要的 3G 标准有两个:W-CDMA 和 CDMA2000,它们分别起源于欧洲的 GSM 和美国的 IS-95/CDMA。我国自主的 3G 标准是 TD-SCDMA。

6.31 说明 WAP 设计模型和 WAP 实现过程。和 WWW 模型相比较,WAP 有什么特点?

答:

WAP 设计模型如图 6.6 所示。

一般情况下,WAP 包括源数据服务器、WAP 代理(网关)服务器和 WAP 客户端。其实,WAP 主要就是在 Internet/WWW 模型上增加了一个中间代理(网关)层来实现的。WAP 代理主要包含以下功能:

(1) 协议网关:将来自 WAP 协议栈(WSP、WTP、WTLS 和 WDP)的请求转换为 WWW 协议

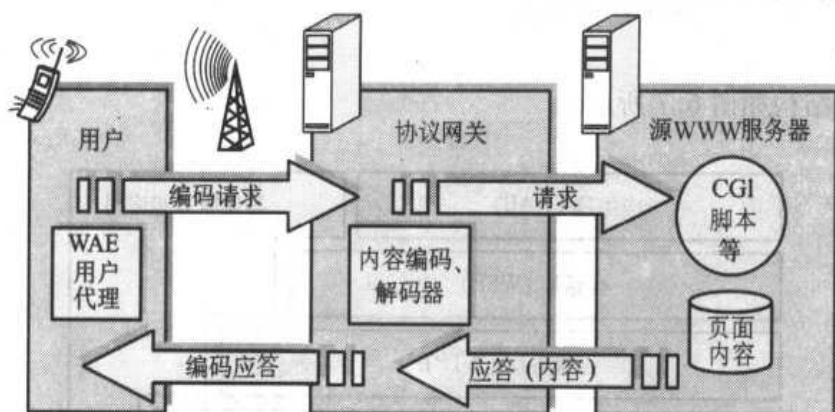


图 6.6

栈(HTTP 和 TCP/IP)的请求。

(2) 内容编、解码器:内容编码器将 WAP 内容压缩编码,以减少它在无线网络上的数据流量。

(3) 用户代理能力管理:用户代理能力描述了客户端的能力及个人偏好。

(4) 高速缓存代理:缓存一些频繁访问的资源,加快无线网络端的访问速度。

也就是说,WAP 利用代理技术连接 WWW 和无线区域,并实现其访问过程:

(1) WAP 代理以 WAP 协议接受 WAP 客户端的经编码压缩过的请求。

(2) WAP 代理将请求解压后,以源数据服务器的访问协议(如 HTTP)向源数据服务器发出请求。

(3) 源数据服务器返回内容,WAP 代理以源数据服务器端的访问协议接受内容。

(4) WAP 代理将内容进行编码压缩后,以 WAP 协议返回给 WAP 客户端。

当然,要高效实现这个过程,WAP 代理还具备有 WAP 客户端管理和 WAP 内容缓存等方面的能力。

和 WWW 模型相比较,WAP 特点主要表现在以下几点:

(1) WAP 模型是为无线访问而设计的,为了满足无线环境的要求,其协议、内容编码等都进行了很大的简化、压缩。

(2) WAP 浏览器比 WWW 浏览器“瘦”得多,其客户机(手机)的处理能力比 WWW 客户机的处理能力也差很多,因此,整个 WAP 对内容的表达能力上要比 WWW 差很多。

(3) 针对 WAP 访问的内容服务器目前还很少,WAP 的访问主要是通过 WAP 代理进行,多数代理是将 WWW 的内容进行裁剪后提供给 WAP 客户端的。

(4) WAP 实现了随时随地的 Internet 访问,但目前来说,其网络载体是 GSM,因此,其数据传输速率和资费是阻碍其发展的很大障碍。

6.32 简述 WAP 协议栈各层的功能。

答：

WAP 协议栈结构如图 6.7 所示。

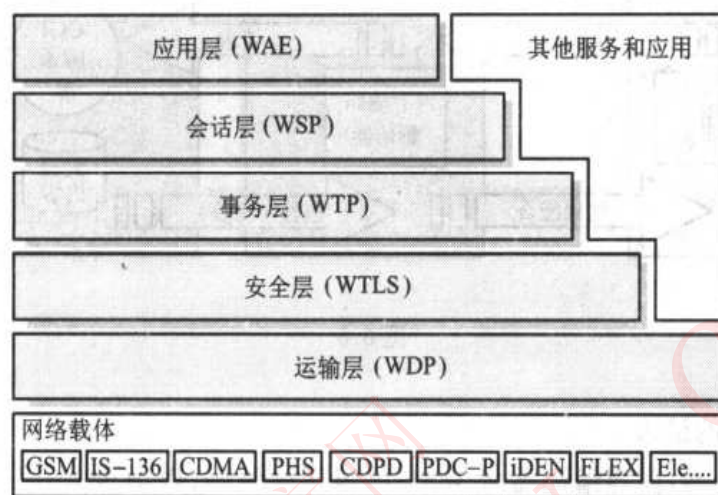


图 6.7

由于它采用了分层设计模型,所以 WAP 协议栈为移动通信设备提供了一个可缩放和可扩展的应用开发环境,外部的应用程序能直接访问会话层(WSP)、事务层(WTP)、安全层(WTLS)和运输层(WDP)。

各层的功能描述如下：

(1) 网络载体 (Bearers): WAP 能在各种不同的网络载体提供的服务上工作,包括 GSM、IS-136、CDMA、PHS、CDPD 等,甚至未来的网络载体。

(2) 运输层 WDP(无线数据报协议): WDP 对 WAP 的上层协议提供一致服务,使得它们能透明地进行数据传输而意识不到下层载体的区别,这样使得上层的安全层、会话层以及应用程序能独立于无线网络完成自己的功能。

(3) 安全层 WTLS(无线运输层安全性): WTLS 是 TLS 上的一个安全协议,是为与 WAP 运输层协议一起使用而设计的,并针对低带宽通信信道进行了优化。WTLS 能提供数据完整性、保密性、鉴权和拒绝服务保护等方面的特性。

(4) 事务层 WTP(无线事务协议): WTP 在数据包服务之上运行,提供适合在无线环境下、“瘦”客户端上执行的面向事务的简化的协议。

(5) 会话层 WSP(无线会话协议): WSP 在传输服务基础上增加了一些协调会话的功能,为应用层用户提供两种会话服务:面向连接和面向非连接服务的统一接口。

(6) 应用层 WAE(无线应用环境): WAE 建立一个可互操作的环境,该环境允许操作人员和服务提供者用有效的方式创建能访问广泛不同的无线平台的应用和服务。目前的 WAE 主要就

是一个微浏览器,通过它实现 WAP 访问浏览。

(7) 其他服务和应用:WAP 协议栈的每一层协议不仅能被 WAP 使用,它们还能直接用来开发无线领域的应用程序。

6.33 i-Mode 和 WAP 的主要区别是什么?为什么 i-Mode 能取得巨大的成功,而 WAP 的应用却不尽如人意?

答:

i-Mode 和 WAP 的主要区别在于 i-Mode 的内容是用 Compact HTML 写成的,因此可以兼容现行的大部分网络内容;而 WAP 使用的是 WML,现有的网络内容必须转化为 WML 内容才能被 WAP 所使用。

目前来说,WAP 的应用不尽如人意,其原因是多方面的。从技术角度来说,首先,上述主要区别使 i-Mode 拥有无穷无尽的 Internet 信息资源几乎可以直接访问,而 WAP 需要服务方针对 WAP 进行改造才可以让 WAP 终端访问。其次,i-Mode 是通过分组网络来进行数据传输的,因此它是根据分组传输收费,而 WAP 基本上是根据连接的时间收费,这就使得用户使用 i-Mode 的价格更低。此外,i-Mode 终端(手机)在设计之初就是针对无线浏览访问来进行的,如它有更大的屏幕来显示图片和小文本信息,而多数 WAP 终端是在原来的语音终端基础上再改进以适应 WAP 应用,这使用户对 i-Mode 有更大的兴趣。另外,i-Mode 在商业运作上要比 WAP 好得多。

6.34 试画出 WLAN 典型的应用结构。对比有线 LAN,WLAN 的主要差别在哪里?对比移动数据网络,WLAN 的主要差别在哪里?WLAN 具有哪些主要优势?

答:

图 6.8 是基于有线网络基础结构的、典型的 WLAN 应用结构。

对比有线 LAN,WLAN 的主要差别有以下两点:

(1) 传输媒体:WLAN 以无线信道(RF 射频技术)作为传输媒体,因此有线 LAN 需要布线,而 WLAN 不需要布线。

(2) 传输速率:目前 WLAN 理论上的最大传输速率是 54 Mb/s,而有线 LAN 现在已经有 1 000 Mb/s 的产品。因此 WLAN 主要适合终端用户接入,LAN 骨干和服务器等连接还得依赖有线 LAN。

对比移动数据网络,WLAN 的主要差别有以下几点:

(1) WLAN 及其用户属于拥有此网络的机构,而移动数据网络属于系统运营商。因此通常用户使用移动数据网络是需要付费的,而使用 WLAN 则不需要。

(2) 传输速率:目前 WLAN 理论上的最大传输速率是 54 Mb/s,而移动数据网络中速率最大的 3G 也只能提供 2 Mb/s。

(3) WLAN 工作在免许可证的 ISM 频段上,而多数的移动数据网络工作在需要许可证的频段上。

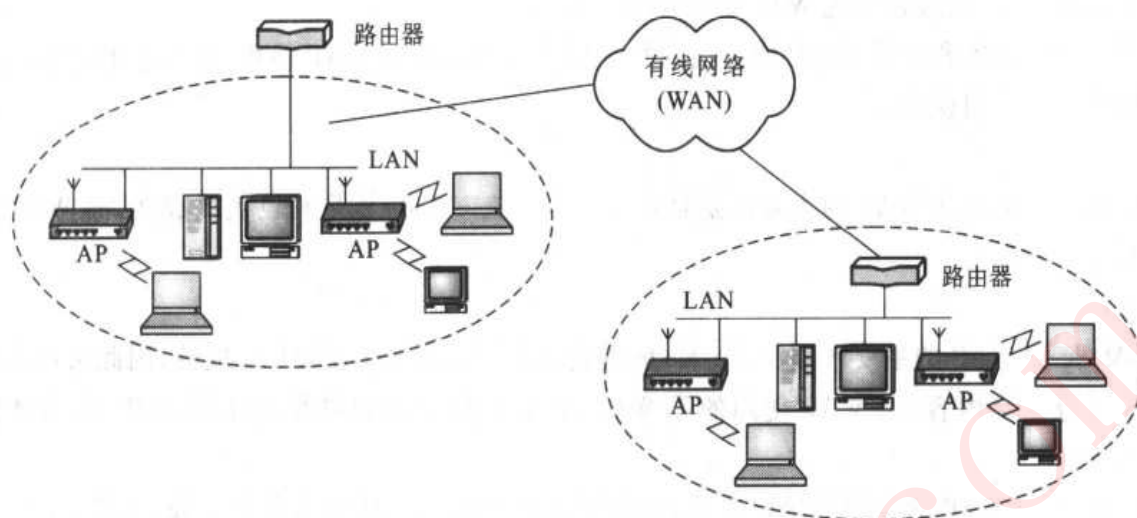


图 6.8

(4) WLAN 在系统规模、投资、建设周期上比移动数据网络都要小得多。因此 WLAN 主要面向机构,而移动数据网络主要面向运营商。

WLAN 的主要优势有以下几点:

(1) 安装便捷、维护方便:WLAN 不需要布线,只要在合适的位置安装接入点 (AP) 设备,就可快速建立 LAN。

(2) 使用灵活、移动简单:在 WLAN 的无线信号覆盖区域内任何一个位置都可以方便地接入网络。

(3) 经济节约、性价比高:节约了布线费用,尤其在一些不适合布线的环境下(如装修好的高档办公区、厂区)。

(4) 易于扩展、大小自如:WLAN 能灵活配置以满足规模经常变化的 LAN 应用。

6.35 IEEE 802.11 WLAN 的 MAC 层以下使用了哪三种基本的传输技术? 它们各有何先进性和局限性?

答:

三种基本的传输技术及其先进性和局限性:

1. 扩频

扩频先用要传送的信息数据对载频进行调制,再使用比发送信息数据速率高许多倍的伪随机序列对载有信息数据的基带信号的频谱进行扩展。扩频在信号接收质量、抗干扰、保密性、增加系统容量等方面有其突出的优点,其缺点是设备较复杂,且即使采用同样的扩频技术,各种产品实现的方法也可能不相同,不能兼容。

2. 窄带技术

窄带技术中数据基带信号的频谱不做任何扩展即被直接搬移到射频发射。窄带方式占用频带少,频带利用率高。但窄带方式的无线局域网一般用于需要许可证的专用频段,若使用 ISM 频段,则彼此间的干扰会影响通信质量。

3. 红外线技术

红外线局域网采用小于 $1\text{ }\mu\text{s}$ 波长的红外线作为传输媒体,具有非常高的频率,有较强的方向性。因此其优点和缺点是很明显的:使用不受无线电管理部门的限制,窃听困难,对邻近区域的类似系统不会产生干扰;受日光、环境照明等影响较大,传输距离受限制,高度定向的红外线对移动的用户不切实际。

6.36 何谓 AP? 它是如何应用的? AP 怎样实现 WLAN 和有线 LAN 的桥接?

答:

AP(Access Point):访问接入点。AP 连接在有线网络上,同时接受用户终端通过无线信道的接入。这样,无线用户终端通过 AP 的桥接,访问有线网络及其提供的服务。

要实现 AP 的 WLAN 和有线 LAN(如 802.3)桥接,AP 在其两端网络接口处实现了 802.11 协议与 802.3 协议,同时在其内部实现这两种协议的转换。如图 6.9 所示。

6.37 什么是 WLAN 应用中的“隐藏终端”问题? IEEE 802.11 中如何解决这个问题?

答:

“隐藏终端”问题是指:彼此间无法感知的多个终端(或 AP)向同一个终端(或 AP)发送信号,引起信号冲突,最终导致发送至这个终端(或 AP)的信号都丢失了。

IEEE 802.11 中引入 RTS/CTS(请求发送/允许发送)握手协议来解决这个问题,即发送终端(或 AP)在正式发送信号前需要先得到接收终端(或 AP)的允许;接收终端(或 AP)在收到某个发送终端(或 AP)请求发送信号后,会先通知其他终端(或 AP)不要向自己发送信号,然后许可这个发送终端(或 AP),数据传输结束后,接收终端(或 AP)再对其他终端(或 AP)“解禁”。

6.38 你如何看待 WLAN 的前景? 你认为 WLAN 下一步发展的重点是哪些方面?

答:

WLAN 的前景是非常美妙的:用户终端可以在 WLAN 覆盖区域自由移动,随时随地享受有线 LAN 的速率和资源,而且是免资费的。WLAN 的应用范围非常广泛,除办公环境外,校园、家庭、军用、会议、餐饮、娱乐等方面同样具有巨大的应用价值。

WLAN 下一步发展重点主要是两个方面:高带宽和安全性。即使是目前 54 Mb/s 的 WLAN,满足人们高带宽、多媒体的应用仍然不够;而安全性是无线网络的通病,WLAN 同样需要强化。

6.39 设计 WATM 的目标是什么? 为达到这个目标,设计者会碰到哪些困难?

答:

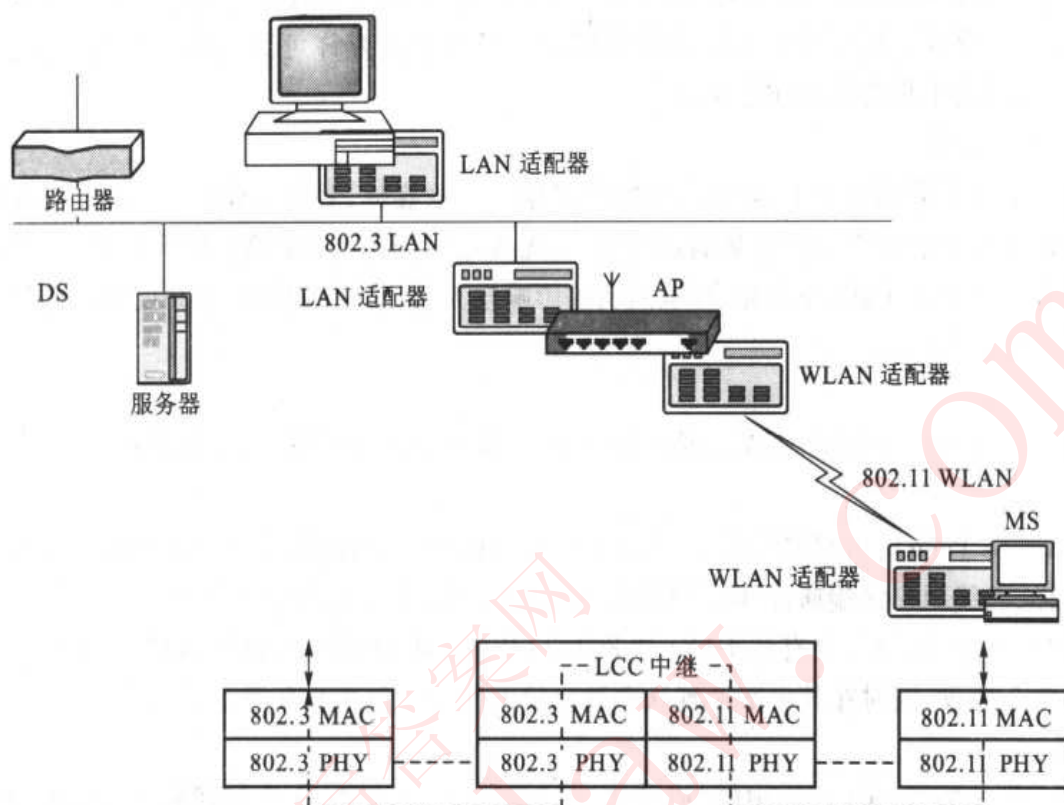


图 6.9

设计 WATM 的目标是支持多媒体业务在无线环境下对 QoS 的要求。

要实现这个目标,设计者碰到的困难主要有以下几点:

(1) ATM 网络是为超带宽的快速交换连接和可靠光传输信道设计的,然而无线传输介质是很不可靠的,而且对于带宽的操作有非常大的限制。这是 WATM 实现中最根本性的困难。

(2) ATM 所支持的 QoS 是基于和用户终端协商、在整个会话期间都保持不变的。无线环境下因为信道状态的变化,要连续支持协商后不变的 QoS 几乎是不可能的。

(3) 此外,WATM 还面临其他一些困难,如必须提出一些方法来实现更为可靠和更快速的空中接口;要发明一些技术能在网络元素中分解附加的网络复杂性;在无连接的环境下如何有效处理 IP 应用等。

6.40 解释说明 ATM 和 WATM 网络的数据帧格式的一般区别。

答:

WATM 帧格式是在 ATM 帧上再加上无线帧头及进行差错控制的 CRC 校验帧尾封装而成,如图 6.10 所示。

无线帧头	ATM 帧头 (5 字节)	ATM 有效载荷 (48 字节)	无线帧尾 (CRC 校验)
------	------------------	---------------------	------------------

图 6.10

这样做的好处是:可以将 WATM 和 ATM 之间进行帧格式转换的处理时间减少到最小。不足之处是:由于 ATM 的帧较小,其有效载荷率也就较小,不利于提高无线信道的利用率。

6.41 比较 WATM 和 IEEE 802.11 WLAN 的协议栈,说明它们之间的区别。

答:

图 6.11 和图 6.12 分别是 WATM 和 IEEE 802.11 WLAN 的协议栈结构。

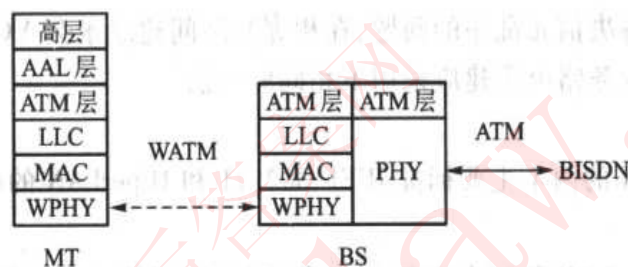


图 6.11



图 6.12

首先,最大的区别同 ATM 和 LAN 的区别一样,WLAN 只提供低两层的服务,而 WATM 在低两层的服务上还提供了 ATM 网络层的服务。其次,WLAN 在物理层、站管理层上分得更细,明确划分了一些子层。

6.42 解释说明 WATM 涉及哪些关键技术?

答:

WATM 涉及的关键技术:

(1) 物理层的调制解调技术使用了 OFDM、QAM 和 OQPSK 技术。

(2) WATM 的 MAC 协议都采用了信道动态分配算法来支持业务速率的可变,信道利用率较高。包括:基于 FDD 的 DQRUMA、PRMA/DA、DSA++、DTDMA/PR 和 DSAMA 等技术和基于 TDD 的 MASCARA、PRMA/ATDD 和 DTDMA/TDD 等技术。

(3) WATM 的差错控制应针对不同的业务采用不同的方式,其中包括使用了 FEC(前向差错控制)技术。

(4) 要满足 WATM 越区切换时 QoS 不恶化的要求,WATM 的方法是将 ATM 的 VP/VC 连接分为静态段和动态段。当发生越区切换时,在动态 VP/VC 连接段实现路由重建。同时,在无线信元中增加序号字段以解决信元乱序的问题;在相邻基站间建立永久 VC 连接,用于切换时交换信息、重建路由,而各种业务路由重建应采用不相同的方法。

6.43 试比较 WLAN 的两个主要标准 IEEE 802.11 和 HiperLAN 的相似和相异之处。

答:

IEEE 802.11 和 HiperLAN 相同点主要有:面向 LAN 应用的无线网络、使用免许可证的频段以及和有线基础网络结构的连接方式。

IEEE 802.11 和 HiperLAN 不同点主要有:IEEE 802.11 是无连接的、而 HiperLAN 是面向连接的,因此 HiperLAN 支持 QoS;HiperLAN 支持自动频率配置和多跳特性,在安全性上也做得更好;此外,在协议结构上,HiperLAN 增加了一层汇聚层以适配高层不同的网络。但 IEEE 802.11 在与 IEEE 802.3 Ethernet LAN 的互连、融合上会做得更好。

6.44 解释说明 HiperLAN-2 的协议栈结构。并说明设立汇聚层的益处。

答:

HiperLAN-2 的协议栈结构如图 6.13。

HiperLAN-2 协议包括 3 个基本层:物理层(PHY)、数据链路层(DLC)和汇聚层(CL)。

(1) 物理层实现无线信道的调制解调,HiperLAN-2 采用了 OFDM 调制方案。

(2) 数据链路层包括 MAC 协议、差错控制协议(EC)和无线链路控制协议(RLC)3 个子层。MAC 协议用于发送数据时控制对物理介质的访问;EC 有助于提高无线链路的可靠性,HiperLAN-2 采用可选择重复(ARQ)机制。RLC 协议为信令实体提供传输服务,主要包括无线资源控制(RRC)、关联控制(ACF)和 DLC 用户连接控制(DDC)等三种控制功能。

(3) 汇聚层的目的是使来自高层的服务请求与 DLC 层提供的服务相适配。

设立汇聚层(CL)的益处在于 CL 使 DLC 和 PHY 层脱离开主干网独立存在而不必关心 HiperLAN 接入何种网络中(如以太网、IP 网、ATM 和 UMTS 等),系统实现相对简单、灵活、适应

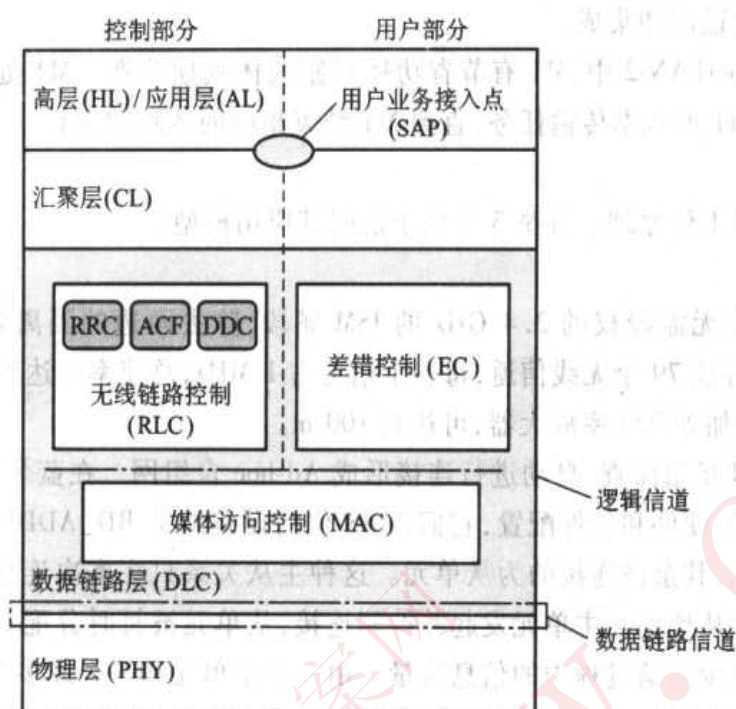


图 6.13

性强。

6.45 HiperLAN-2 具有哪些主要的特点？

答：

HiperLAN-2 的主要特点包括以下几个方面：

- (1) 高传输速率：HiperLAN-2 具有很高的传输速率，它的物理层的传输速率最高可达到 54 Mb/s。
- (2) 面向连接：在 HiperLAN-2 网络中，需要先使用 HiperLAN-2 控制面的信令功能，在 MT 和 AP 之间建立连接，然后才能进行数据传输。
- (3) QoS 支持：HiperLAN-2 的面向连接的特性使它很容易满足 QoS 要求，可以为每个连接分配一个指定的 QoS。
- (4) 自动频率分配：HiperLAN-2 的 AP 支持为在其覆盖范围内进行的传输选择最适当的无线信道。
- (5) 安全支持：HiperLAN-2 支持认证和加密。
- (6) 移动性支持：MT 能够自动从最近的 AP 发射和接收数据。即 MT 使用的 AP 所提供的无线信号具有最佳的信噪比。
- (7) 网络和应用独立：HiperLAN-2 协议栈增加的汇聚层使其具有一个灵活的结构，很容易

与多种固定网络进行适配和集成。

(8) 功耗低: HiperLAN-2 中, MT 有节省功耗机制的休眠期状态。MT 处于休眠期间, AP 将延迟所有发送给该 MT 的数据传输任务, 直到 MT 结束相应的休眠期为止。

6.46 叙述蓝牙工作原理。并举 3 个例子说明其应用模型。

答:

蓝牙使用国际上无需授权的 2.4 GHz 的 ISM 频段, 除去两边的隔离频带, 实际使用的为 2 402 ~ 2 480 MHz, 分成 79 个无线信道, 每一个信道占 1 MHz, 总速率可达到 1 Mb/s。通信距离一般为 10 m, 如果附加外部功率放大器, 可达到 100 m。

蓝牙设备之间可互相探查, 自动进行连接形成 Ad-hoc 自组网。在蓝牙网络里, 每个设备都是对等的, 具有相同的硬件和软件配置, 它们以 48 位的设备地址 BD_ADDR 来区别。发起连接的那个设备叫主单元, 其余被连接的为从单元。这种主从关系只在本次连接有效, 并且, 在本次连接中也可更换其主从地位。主单元发起并控制连接, 从单元被暂时分配一个 3 位的活跃成员地址 AM_ADDR 以减少通信过程中的信息流量。由一个主单元和一个或多个从单元组成的自组网称为微微网, 一个微微网最多只可以有 7 个从单元。由多个这种微微网结合形成了散射网, 其中有一个从单元同时属于两个微微网, 它可起一个桥的作用, 将两个微微网连起来, 从而可将多于 8 个的设备连接起来形成一个大网, 同时也延长了通信的距离。

通信时, 单个单元的峰值传输速率可达到 721 kb/s 或最多 3 个话音信道 (每个信道传输速率为 64 kb/s), 也能同时传输声音和数据。而在充分扩大的散射网里, 总的吞吐率可超过 10 Mb/s 或 20 个话音信道。

目前已有许多蓝牙应用案例, 如, 蓝牙支持的无线耳机, 将音源和耳机组成蓝牙网络, 实现语音的数字传输; 蓝牙支持的笔记本电脑通过蓝牙支持的手机 (作为 Modem) 上网; 家庭中, 蓝牙支持的家电之间形成一个智能的控制网络。

6.47 简述蓝牙设备组网时的查询过程。

答:

在一个蓝牙微微网中, 主单元在和从单元建立连接前可能并不知道其余设备的存在以及它们的地址。在这种情况下, 主单元需要先执行查询操作。

查询的目的是得到别的设备的设备地址及其时钟, 这个过程是由主单元不断地重复发送查询信息, 请求其他设备应答, 直到正确地收到其他设备的回复消息。

举个例子: 假设在连接范围内共有 4 个设备 A、B、C、D。A 为主单元, 发起连接, 发出查询信息, 请求别的设备的设备地址及其时钟, 首先 C 回复 A 一个 FHS 包 (包括 C 的设备地址和时钟), A 然后将自己的设备地址及时钟回复给 C; A 再次发出查询信息, 假设 B 和 D 同时回复, 则发生冲突, 所以 A 将不回复; 冲突使得 B 和 D 得不到回复消息, 它们将各自等待一段随机长度的时隙后再侦听信道, A 再发查询信息, B 重复 C 的过程; A 再发查询信息, D 又重复 C 的过程。

6.48 蓝牙技术目前的优缺点有哪些?

答:

蓝牙技术的优点主要体现在以下几点:

(1) 作为低成本和短距离无线连接的解决方案,可以消除不同数字装置之间的界限。当两个装有蓝牙芯片的数字设备相互距离在 10 m 之内并满足一定要求时,它们就能快速建立可靠的无线联系。

(2) 蓝牙可以消除千头万绪的电缆线,对于诸如移动电话、计算机和 PDA 等各种数字设备可以随心所欲地无线连通。

(3) 蓝牙使用的是国际上无需授权的 ISM 频段,所以可以在全世界范围内建立一个统一的标准,这使得蓝牙设备在全世界通用。

(4) 蓝牙即插即用的自组网方式使任意蓝牙设备一旦搜寻到另一个蓝牙设备,马上就可以建立联系,而无需用户进行任何设置。

(5) 蓝牙采用了跳频技术,具有扩频通信的优点,尤其显著的是它的抗干扰能力。

蓝牙技术的缺点主要体现在以下几点:

(1) 目前蓝牙模块的成本较高,导致推广蓝牙有一定障碍。

(2) 蓝牙不支持漫游,在微微网或散射网之间切换时需要重新建立连接,这期间会导致数据传输中断。

(3) 由于 ISM 频段的公用特征,多个 ISM 频段上的设备间会互相干扰,从而影响蓝牙通信速率,或导致数据传输中断。

6.49 什么是 HomeRF? 它的设计目的是什么?

答:

HomeRF 即家庭射频技术,是由 HomeRF 工作组 (HomeRF WG) 开发的在家庭区域范围内的任何地方、在 PC 和用户电子设备之间实现无线数字通信的开放性工业标准。

因此 HomeRF 的设计目的就是为在家用电器设备之间传送语音和数据,并且能够与 PSTN 和 Internet 进行交互式操作。

6.50 为了满足 HomeRF 的设计目标,它使用了怎样的协议栈结构和关键技术?

答:

HomeRF 的协议栈称为 SWAP (Shared Wireless Access Protocol, 共享无线访问协议)。如图 6.14 所示。

在 SWAP 中,其协议层次与 OSI 网络模型有一定的映射关系,MAC (介质访问层) 对应于数据链路层,在其上的协议层则根据开展的业务不同而有所差异,它用 TCP/IP 承载数据业务、UDP/IP 承载流业务 (诸如视频数据流等),同时为了提供高质量的语音业务,还集成了 DECT

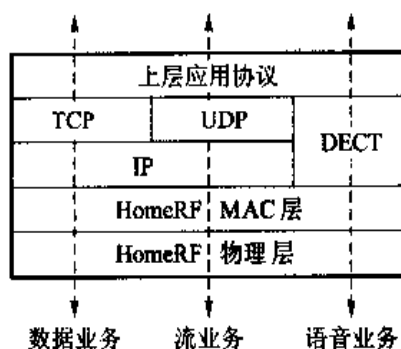


图 6.14

协议。

关键技术如下：

(1) HomeRF 的物理层采用数字跳频扩频技术,调制方式为恒定比例的 FSK 调制。最新版 HomeRF2.x 中,采用了 WBFH 技术来增加跳频带宽,其数据峰值高达 10 Mb/s。

(2) 在 HomeRF 一个跳频点上,大部分时间用于数据通信,同时根据激活的话音信道数目,动态地为语音业务预留一部分资源。而为了满足实时通信的要求,HomeRF 为流业务定义了高级别的优先权,可以随时占用信道资源。

(3) HomeRF 中融合了 IEEE 802.11 的 CSMA/CA 方式和 DECT 的 TDMA 方式,对于数据通信应使用前者,而语音业务则使用后者。从而确保高效的数据传输和高质量的语音通信。

(4) 在网络层上,HomeRF 采用了 Internet 的 TCP/IP 协议,TCP 协议应用于一般的数据通信,而 UDP 协议用于开展流业务。

(5) 此外,HomeRF 使用随机跳频、低功率传输和 Blowfish 加密算法来保障数据传输的安全性。

6.51 比较 HomeRF、蓝牙和 WLAN 的主要特点和应用场合。

答：

HomeRF 的主要特点是融合了数据和数字语音两种通信模式,非常方便地实现与 PSTN 和 Internet 的互操作,因此,较适合在家庭区域范围内以及在 PC 和家用电器设备之间进行无线传送话音和数据。

蓝牙是低成本和短距离无线连接的解决方案,其主要特点是植入蓝牙芯片的不同数字装置之间能非常方便地实现自发现、自组网,这比较适合于需要消除短距离连线的应用场合,如手机和耳机、主机和鼠标之间等。

WLAN 基本上是 LAN 的无线版本,其主要特点是规模较大、覆盖范围较大、速率较高、建设成本也较高,侧重是主机间的数据传输应用,因此,较适合在商用、局域无线连网进行数据传输的应用场合。

第七章 网 络 层

7.1 试讨论分组交换的数据传输特点。

答:

分组交换(Packet Switch)是一种数据传输方式。相对于另外一种数据传输方式——电路交换而言,分组交换在数据传输之前,通信双方不需要像电路交换那样先建立一条专用的物理通信链路(尽管在面向连接的分组交换中建立了逻辑上的专用通信链路)。分组交换在分组交换网络中通过多个中间节点传输数据分组,分组可能被延迟或阻塞。然而,分组交换带来的一个明显优势在于物理通信线路可以被许多通信节点所共享以提高通信线路的利用率;另外,分组交换在网络的容错能力上也有很大的提高。

分组交换有的两种内部交换方式:虚电路分组交换和数据报分组交换。虚电路分组交换在传输前先建立一条逻辑链路(路径、连接),以后这条虚电路上的所有分组将采取相同的路由,这样分组的传输延迟非常短。但是和电路交换相比,由于电路交换中信道供某条链路专用,所以不用考虑拥塞的问题;而对虚电路交换而言,实际上相当于信道供所有正在传输的站点共享使用,仍然需要有存储转发,从而传播延迟相对来说要长一些,同时可能会出现网络的拥塞情况。

数据报分组不需要预先建立连接,每个分组都必须重新进行路由选择,分组每次可能沿不同的路径到达目的地。由于每次都要进行路由选择,会影响分组传播的延迟,并且到达目的地的分组可能会丢失、重复、失序,但是由于路由选择更加灵活,所以有可能会绕过网络拥塞的区域。

7.2 虚电路为什么是“虚”的?如何区分一个网络节点所处理的多个虚电路?

答:

“虚”是相对电路交换而言的,指的是逻辑电路,而不是专用的。分组仍然在每个节点处首先缓存起来,并排队等待转发。每个节点到其他节点之间可能同时有若干条虚电路,也可能同时与多个节点间存在虚电路。每条虚电路支持特定的两个端系统之间的数据传输,两个端系统之间也可以有多条虚电路为不同的进程服务,这些虚电路的实际路由可能相同,也可能不同。

网络节点所处理的多个虚电路是靠虚电路号来区分的,这个虚电路号仅仅是一个局部标识,即使用动态的虚电路号。节点对于每一条虚电路包含了一个表项,其中包括虚电路号、虚电路上的前一节点和后一节点的标识。

7.3 数据报与虚电路操作方式各有什么特点?数据报(无连接)服务与虚电路(面向连接)服务各有什么特点?

答:

数据报方式没有呼叫建立和释放过程,从而免去了电路建立的延迟,但是因为每个分组都必须进行路由选择,这可能会影响分组传输的延迟。同时如果某个节点失败或者拥塞,以后的分组传输就可以绕过故障区域,当然这样可能会导致分组的丢失、重复、失序。

虚电路方式必须首先建立一条虚电路,但是一旦建立好虚电路后,分组传输就沿已经建立好的路径传递,此时分组传输延迟要小一些,分组的传递不会失序。同时节点必须保存有关虚电路的信息,一旦节点失效,原有的经过该节点的虚电路就没法继续了。

数据报服务和虚电路指的是网络层所提供的服务。数据报服务是指无连接的网络层服务,而虚电路服务是指有连接的网络层服务。

虚电路服务是网络层向运输层提供了一种可靠的数据传送方式,所有分组按照发送顺序到达。进行数据交换的两个端系统之间有一条虚电路(网络连接)为它们服务。

数据报服务认为通信子网是不可靠的,必须自己进行差错控制和流量控制,分组在网络中可能经过不同的路径,有可能会出现失序、重复、丢失等问题。

虚电路服务与数据报服务的本质差别表现在于是把顺序控制、差错控制和流量控制等通信功能交给通信子网去完成,还是由端系统自己来完成。

7.4 有没有可能一个虚电路服务会以乱序的方式递交分组的情况?

答:

有可能。考虑紧急数据的情况,这些紧急数据可能会比那些正常传输的数据早到达。比如一个典型的终端仿真应用,如果用户键入退出键,此时产生的分组属于紧急数据,应该优先到达。

7.5 考虑下面虚电路服务实现涉及的设计问题。如果虚电路用在子网的内部,每个数据报文必须有一个 3 B 的报头,每个路由器必须留有 8 B 的空间来标识虚电路。如果内部使用数据报,则需要使用一个 15 B 的分组头。假定每站段传输带宽的费用为每 10^6 B 要支付 1 个货币单位;路由器存储器的价格为每字节 1 个货币单位,并且在未来两年价格会下降。平均会话期为 1 000 s,传输 200 个分组;分组平均需传送 4 个站段。请问:子网内部采用虚电路还是采用数据报更便宜一些?便宜多少?

答:

分组平均需要传送 4 个站段,也就是说途中要经过 5 个路由器。

首先考虑虚电路方式的开销,包括 3 个部分,路由器中虚电路表项所占用存储空间开销、分组头部用掉的带宽的开销和分组数据在途中传输和存储转发的开销为

$$Cost_{circuit} = 5 \times 8 \text{ B 的存储空间被使用 } 1\,000 \text{ s 的开销} + \frac{3 \times 200 \times 4}{10^6} + Cost_{data}$$

而对于数据报方式,只有分组数据在途中传输和存储转发的开销和分组头部的带宽开销为

$$Cost_{datagram} = \frac{15 \times 200 \times 4}{10^6} + Cost_{data}$$

这样两者开销的差别等于

$$\begin{aligned} Cost &= Cost_{\text{datagram}} - Cost_{\text{circuit}} \\ &= \frac{12 \times 200 \times 4}{10^6} - 40 \text{ 字节的存储空间使用 } 1000 \text{ s 的开销。} \end{aligned}$$

路由器存储器的价格为每字节一个货币单位,可以用两年,考虑每年有 52 个星期,每个星期工作 40 小时,这样每个字节用 1 s 的开销为

$$\frac{1}{2 \times 52 \times 40 \times 3600} = 6.7 \times 10^{-8}$$

这样,虚电路方式要便宜一些

$$Cost = 9600 \times 10^{-6} - 6.7 \times 400 \times 10^{-6} = 6920 \times 10^{-6}$$

7.6 假定所有的路由器和主机都正常工作,路由器和主机的软件也没有错误。请问有没有可能(无论这种可能性会非常小)把一个分组递交到错误的目的地?

答:

有可能。因为 IP 的校验和并不能检查所有的错误,因此有极小的可能性会发生 IP 地址在传输过程中改变,但是整个分组的检验和校验仍然正确,因而会将分组递交到错误的目的地。同样对于虚电路方式的服务也有类似的情况。

7.7 考虑教材图 7.1 中的网络,如果节点 1 通过扩散发送一个分组到节点 6。把一个分组沿其中一条链路传输产生的负载记为 1。在下面两种情形下,总的负载为多少?

- (1) 每个节点丢弃重复的扩散分组;
- (2) 使用站点计数机制,并且站点计数初值为 5。

答:

对于(1)的情况,每个节点会丢弃重复的扩散分组:

第一轮:1→2/3

第二轮:2→3;4→5/7

第三轮:3→5;5→3/6;7→6

以后 5、3 会检测到重复的分组,不会扩散。所以总的负载为 9。

对于(2)的情况,采用站点计数机制:

第一轮:1→2/3

第二轮:2→3;4→5/7

第三轮:3→5;5→3/6;7→6

第四轮:5→4/6;3→2

第五轮:4→1/7;2→1

此时站点计数值到,所以总的负载为 15。

7.8 主机 A 要向主机 B 传输一个 512 KB 的消息,途中要经过 8 个路由器。链路是可靠的,数据速率为 50 Mb/s。每个链路长 1 000 km,信号传播速度为 2×10^8 m/s。假设每个路由器的排队延迟为 1 ms。请问在下列情况下该消息要多长时间到达目的地?

- (1) 采用消息交换(即消息不被分段),消息头部为 32 B;
- (2) 采用分组交换,每个分组携带的数据为 2 KB,头部为 32 B。

答:

(1) 消息大小为 $(512 \times 1024 + 32) \times 8 \text{ b} = 4\,194\,560 \text{ b}$

消息传输时间 = 消息大小/传输速率 = 0.083 891 s

排队延迟 = 0.001 s

每个链路的传播延迟 = $\frac{(1\,000 \times 1\,000)}{2 \times 10^8} = 0.005 \text{ s}$

因此需要的时间 = $9 \times \text{消息传输时间} + 9 \times \text{传播延迟} + 8 \times \text{排队延迟}$
= 0.808 s

(2) 分组大小 = $(2 \times 1024 + 32) \times 8 \text{ b} = 16\,640 \text{ b}$

分组传输时间 = 分组大小/传输速度 = 0.000 333 s

分组个数 $N = 512/2 = 256$

因此需要时间 = $(N - 1) \times \text{传输时间} + 9 \times \text{传输时间} + 9 \times \text{传播延迟} + 8 \times \text{排队延迟} =$
0.140 648 s

7.9 数据报子网允许路由器在必要时扔掉分组。一个路由器扔掉分组的概率为 P 。考虑一端主机连接到源端路由器,源端路由器又连接到目的地路由器,它又连接到目的地主机。如果其中一个路由器扔掉一个分组,源端主机最后会超时并且重传该分组。如果主机到路由器及路由器到路由器的线路都算作一个站段。那么:

- (1) 一个分组在每次传输中所经过的平均站段数为多少?
- (2) 一个分组的平均传输次数是多少?
- (3) 每次收到的分组所需的平均站段数为多少?

答:

(1) 求分组每次传输经过平均站段数:

一个分组从源端主机到源端路由器,再到目的端路由器,最后到达目的端主机,其中源端路由器和目的端路由器可能会丢弃分组,概率为 p 。这样一个从源端主机出来的分组在传输过程中只到达源端路由器的概率为 p ,经过站段数为 1;分组到达目的端路由器,说明源端已经让该分组通过,因而分组只到达目的端路由器的概率为 $(1 - p)p$,经过站段数为 2;最后分组到达目的主机的情况说明源端和目的端路由器都让其通过,因而概率为 $(1 - p)(1 - p)$,经过站段数为 3,则

$$\text{经过的平均站段数} = p + 2p(1 - p) + 3(1 - p)^2 = p^2 - 3p + 3$$

(2) 求一个分组的平均传输次数:

一个分组在一次传输过程中传输成功的概率为 $(1-p)^2$, 这样一个分组的平均传输次数就为

$$\sum_{i=1}^{\infty} iP(\text{经过 } i \text{ 次传输成功的概率}) = \sum_{i=1}^{\infty} (1 - (1-p)^2)^{i-1} (1-p)^2 = \frac{1}{(1-p)^2}$$

(3) 分组每次传输经过平均站段数:

这实际上等于分组的平均传输次数和每次传输经过的平均站段数的乘积

$$\frac{p^2 - 3p + 3}{(1-p)^2}$$

7.10 证明 Dijkstra 算法的正确性, 即该算法的确能够找到一条最短路径的路由。

答:

要证明算法的正确性, 只需证明对于 M 中的任意 w , 有 $C(w)$ 为从 S 到 w 的最小花费即可。由 Dijkstra 算法步骤(2)可知, $C(w) = \min C(j)$, j 不属于 M 。用反证法, 假设 $C(w)$ 不是从 S 到 w 的最短路径的花费, 由 Dijkstra 算法步骤(3)可知, 必然存在一条从 S 经过 j 到 w 的路径(j 不在 M 中), 且花费小于 $C(w)$ 。显然, 从 S 到 j 的花费小于从 S 到 w 的花费, 而这与 w 是不属于 M 的节点中的最小花费节点相矛盾。证毕。

7.11 Dijkstra 算法可以计算两个节点间的最短路径, 请给出一种简单的试探方法, 寻找两个节点间的两条通路(如果确实存在), 以便在任一条通信线路失效的情况下, 两个节点仍然可以进行通信。假定路由器足够可靠, 无需考虑路由器崩溃的可能性。

答:

首先采用 Dijkstra 算法找出一条最短路径, 然后删除其所对应的所有链路, 对这个新的拓扑再利用 Dijkstra 算法计算一条最短路径, 这两条路径可以作为节点之间的两条通路, 在保证任意通信线路失效的情况下, 节点之间仍然通信。

7.12 在教材图 7.4 中, 计算从 B 到 H 的最短路径。

答:

如图 7.1 所示。

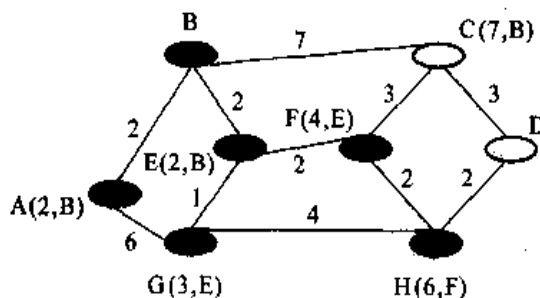


图 7.1

从 B 到 H 的最短路径为: $B \rightarrow E \rightarrow F \rightarrow H$, 长度为 6。

7.13 在随机路由算法中, 每个分组在任何时刻只有一个拷贝存在。但是即便如此, 使用一个站点计数器效果要更好一些。请解释为什么。

答:

在随机路由算法中, 路由选择是随机选择一条线路转发的, 这样某个分组可能会一直在网络中传递而无法到达目的地, 通过使用站点计数器可以尽早地消除这样的分组。

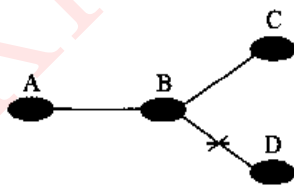
7.14 比较固定路由方法和集中路由方法的相同点和不同点。

答:

相同点: 二者都是根据网络中所有节点的网络信息来计算、选择路由的, 并且网络中每个节点都保存了一张当前的路由表(这个路由表通常由一个中心负责计算、分发)。

不同点: 固定路由用于静态路由方式, 而集中路由用于动态路由方式。因此, 固定路由中路由表的建立多数是手工计算完成并分发的; 而在集中路由中, 网络中设置了一个路由控制中心(RCC), 每隔一段时间, RCC 就会收集整个网络信息, 利用这些信息使用最短路径算法计算出每对节点之间的最优路径, 构造出路由表并自动分发给对应的每个节点。

7.15 RIP 协议的缺省设置是每 30 s 广播一次路由信息, 站点数距离大于 16 的目的地是不可达的。如教材图 7.41 所示的网络, 如果路由器间使用 RIP 协议但没有水平分隔处理能力, 而此时 B 和 D 间的链路出现故障, 那么 A 要多少时间后才能感知 D 断开了? 假设路由器的时间严格同步。



教材图 7.41

答:

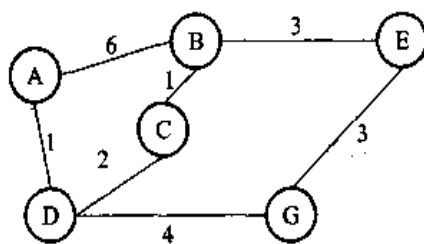
考虑 A 到连接在 D 上网络的路由距离: $(B, 3)$, $(B-D \text{ 断开}), (B, 4), (B, 5) \dots (B, 16)$, (Unreach)。因此 A 感知 D 断开的的时间: $(16 - 3) \times 30 = 6 \text{ 分 } 30 \text{ 秒}$ 。

7.16 有一个网络如教材图 7.42 所示, 图中的节点为运行距离向量路由算法的路由器, 边上的权值为链路的距离。对于下面两种情况, 给出算法达到稳定状态后节点 C 的路由表:

(1) 不采用水平分割时节点 C 的路由表;

(2) 采用水平分割时节点 C 的路由表。

答:



教材图 7.42

(1) 不采用水平分割时节点 C 收到来自于 B 和 D 的路由更新消息,然后花费更小的路径作为当前的路由表,用下划线字描述,参见表 7.1。

表 7.1

目的地	通过 B	通过 D
A	5	<u>3</u>
B	<u>1</u>	5
D	4	<u>2</u>
F	<u>4</u>	8
G	7	<u>6</u>

(2) 采用水平分割算法,节点 C 的路由表参见表 7.2。

表 7.2

目的地	通过 B	通过 D
A	∞	<u>3</u>
B	<u>1</u>	∞
D	∞	<u>2</u>
F	<u>4</u>	∞
G	7	<u>6</u>

7.17 在链路状态路由算法中,路由器发送的链路状态分组使用顺序号标识其信息的“新鲜”程度。有什么办法解决路由器重启后顺序号的不确定性?又如何解决顺序号溢出后的归 0 问题?

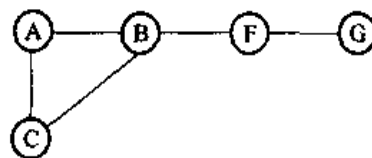
答:

解决第一个问题的一种方法是路由器设置一个硬件顺序号发生器,这个顺序号发生器不受路由器重启影响。

要解决第二个问题,必须使链路状态分组信息会过期,即在一定的生命期后,该链路状态分组信息将被丢弃。这样路由器收到顺序号小的链路状态分组,且该顺序号比老顺序号小很多

(旧的、小顺序号分组都已过期),则被认为是顺序号溢出归 0 后的新链路状态分组而被采纳。

7.18 考虑如教材图 7.43 所示的一个采用链路状态路由的网络,假设 BF 链路断开,然后首先节点 H 连接到 G 的右方,接着节点 D 连接到 C 的左方,然后 DA 间建立了一条新的链路,最后断开的 BF 链路被重新恢复。请给出在这个过程中发送的链路状态分组。假设所有节点的初始顺序号都为 1,并且每个节点当本地链路状态有变化时增加顺序号。



教材图 7.43

答:

在链路 BF 恢复前,每个节点发送的 LSP 参见表 7.3。

表 7.3

节点	LSP 顺序号	LSP 分组内容(邻居节点)
A	2	B、C、D
B	2	A、C
C	2	A、B、D
D	2	A、C
F	2	G
G	2	F、H
H	1	G

在 BF 链路恢复后,B 和 F 互相交换数据库描述信息,然后 B 和 F 分别把它的一个方向的 LSP 通过 BF 链路扩散给对方,这些 LSP 实际上就是上表描述的 LSP,除了节点 B 和 F 发送的 LSP 由于新增了邻居节点而改变,此时它们的顺序号应该为 3。

7.19 考虑一个有 200 个路由器的网络,每个路由器平均连接到 4 个其他路由器。每 100 ms 交换路由信息。请问采用链路状态或距离向量路由所使用的网络带宽各为多少?在解答时请说明你采用的路由信息大小的长度的理由。

答:

对于链路状态:

对于每个链路,每个路由器发送 $\langle \text{neighbor id}, \text{link state} \rangle$,这样 2 B 就足够了。每个路由更新还要包括一个顺序号(4 B),因此每个路由更新消息为

$$4 \text{ 链路} \times 2 \text{ B/链路} + 4 \text{ B} = 12 \text{ B}$$

每个路由消息经过带顺序号的扩散经过每个链路传递: $200 \text{ 路由器} \times 4 \text{ 链路/路由器} / (\text{每个链路有两端}) = 400 \text{ 链路}$,而路由信息每 100 ms 发送,因此发送速率为 $1/100 \text{ ms} = 10 \text{ 消息/秒}$ 。

所以,总的负载为 $200 \text{ 路由器} \times 10 \text{ 消息/秒} \times 400 \text{ 链路} \times 12 \text{ 字节/消息} = 9.6 \text{ MB/s}$ 。

对于距离向量路由:

每个路由表包括 $\langle \text{dest}, \text{hop count}, \text{nexthop} \rangle$, 共 3 B, 每个主机路由表项为 200 个。每次路由更新, 包括了完整的路由表。

因此总的负载为 $200 \text{ 路由器} \times 10 \text{ 更新/秒} \times 4 \text{ 链路/路由器} \times 3 \times 200 \text{ 字节/链路} = 4.8 \text{ MB/s}$ 。

7.20 对 4 800 个路由器进行分层路由, 若采用 3 层分层结构, 应该选择多大的区和簇才能最大限度地减少路由表的长度?

答:

选择 15 个簇, 每簇再分为 16 个区, 每个区包括 20 个路由器。注意也可以有其他的组合, 比如 20 个簇, 每簇再分为 16 个区, 每个区包括 15 个路由器, 主要是 3 层的取值分别为 15、16、20。此时路由表的长度最小, 为 $15 + 16 + 20 = 51$ 。

7.21 何为 AS? 划分 AS 有什么实际意义?

答:

AS (Autonomous System) 即自治系统, 它是由一个独立的管理实体控制的一组网络和路由器。

划分 AS 可以将一个大的网络进行分区、分块 (以统一的 AS 编号进行标识), 以简化路由: 分组先在源 AS 内路由, 再在 AS 间路由, 最后在目的 AS 内路由。AS 内的路由使用 IGP 协议, AS 间的路由使用 EGP 协议, 而 AS 内的路由信息不向 AS 外传播。另外, AS 的划分和实际的网络分区、分块管理也是一致的。

7.22 一个 AS 应该广播有关到它内部所有网络的路由信息吗? 如果不需要, 举出一个例子。

答:

不需要。在 BGP 协议中, 当某个自治系统内的主机要给另一个自治系统中的某台主机发送分组时, 只需将分组发送到目的自治系统的边界路由器, 自治系统内的路由由该边界路由器利用内部网关协议实现。

7.23 一台主机的 RIP 协议可以是主动方式吗? 给出相应的理由。

答:

所谓主动方式是指除了接收邻居路由器发送的 RIP 更新消息外, 也定期传输 RIP 更新消息, 主机通过接收更新消息, 也能够了解到那些目的网络的路由情况, 但是它不应该传输更新消息, 因为这些路由信息正是从网络接口中接收的, 否则就会违背水平分割规则。因此我们说主机的 RIP 协议不应该是主动方式。

7.24 请描述 RIP 中可以用水平分割法解决无穷计数问题的网络的特征。

答：

无穷记数问题的产生原因之一是两点之间有一个路由回路，而在路由交换中，把从邻居了解到的到某个目的网络的路由再传递给该邻居是毫无意义的。这就是水平分割的基本思想。即当通过一个特定的网络接口发送 RIP 更新信息时，不包括通过那个网络接口学习到的路由信息，所以水平分割算法解决了两点之间的路由回路问题。

7.25 请说明 RIP 所使用的各个无穷计数解决方案所解决的问题以及带来的问题。

答：

水平分割解决网络两点间存在回路而导致的无穷记数问题，它会增加路由信息的大小，从而占用更多的带宽。触发更新机制能够防止发生在 3 个或 3 个以上路由器之间的路由回路。如果触发更新足够快的话，能够在系统定期发送正常更新信息之前完成，则不会出现无穷记数问题。但这种状况并不会总是能够满足。另外，该机制有可能导致一个广播风暴。这也可以通过等待随机时间的方法来设法避免。

7.26 直接连接在路由器上的主机在 OSPF 的拓扑有向图中如何表示？

答：

直接连接到路由器的主机用中断网络表示，网络掩码为 0xffffffff，表示该中断网络只有一个节点。

7.27 在教材图 7.14 的有向图表示中，给出路由器 R6 的生成树，并且给出 R6 的路由表的情况。

答：R6 的生成树如图 7.2 所示。

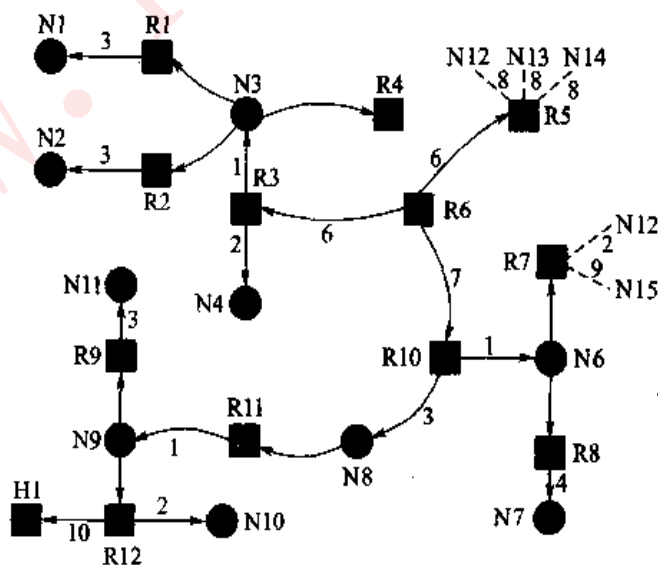


图 7.2

R6 的路由表参见表 7.4。

表 7.4

目的网络	下一跳段	距离
N1	R3	10
N2	R3	10
N3	R3	7
N4	R3	8
N6	R10	8
N7	R10	12
N8	R10	10
N9	R10	11
N10	R10	13
N11	R10	14
H1	R10	21
R5	R5	6
R7	R10	8
N12	R10	10
N13	R5	14
N14	R5	14
N15	R10	17

7.28 在 OSPF 中,一个广播网络的状态信息是由连在该网络中的选取路由器负责传播的。在教材图 7.13 的配置中,网络 N3 的链路状态消息中包含什么? 用一个拓扑图来表示。

答:

网络的链路状态信息给出了该网络所连接的路由器的信息,如图 7.3 所示。

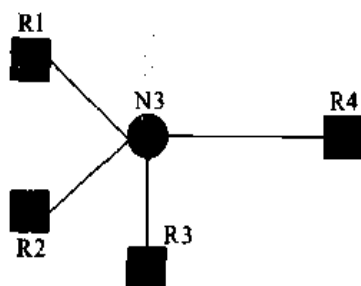


图 7.3

7.29 BGP 哪些主要的特征使它能成为真正的外部网关协议?

答:

BGP 协议在 EGP 协议的基础上作了许多改进(尤其突破了 EGP 协议在距离定义上的限

制),它是真正的外部网关协议。BGP 协议实际上是一种基于路径向量的距离向量协议,每个 BGP 记录的是确切的路由,从而可以消除路由的回路,因而对于网络的拓扑结构没有任何限制。同时 BGP 协议另一个最重要的特征是支持策略路由机制,这使它成为最为广泛使用的外部网关路由协议。

7.30 假设希望来自于 A 的所有通信都不准经过另外一个区域 B,什么路由协议可以完成该功能,怎么样实现?

答:

可以采用 BGP 协议。A 的自治系统可以进行策略控制:A 的通信不许经过区域 B,因为 BGP 把完整的路径通告出去,因此可以进行控制不违反该策略。

7.31 考虑教材图 7.17 中,如果自治系统内部路由协议都使用 OSPF,各链路的花费都一样,那么从路由器 R_8 传输分组到路由器 R_2 的最佳路径是什么?叙述这条路径被发现的过程。

答:

最佳路径: $R_8 \sim R_9 \sim R_5 \sim R_1 \sim R_2$ 。

发现过程:首先通过 AS 内部的 OSPF 协议,使 R_1 和 R_8 发现了 R_2 和 R_9 ,同时 R_8 知道目的地不在 AS3 的分组将通过边界路由器 R_9 传输;在 AS 间, R_1 通过 BGP 协议的 Update 消息,使 R_5 知道了 AS1 的 NLRI(其中包含了 R_2 的存在),同样 R_5 使 R_9 知道了到达 AS1 的 NLRI 的最佳路径是经过 AS2 和 AS1。这样, R_8 通过 OSPF 知道传输给不在 AS3 的 R_2 的分组是先传输给 R_9 ,而 R_9 通过 BGP 知道传输给 R_2 的分组是传输给 AS2 的 R_5 、然后是 AS1 的 R_1 , R_1 通过 OSPF 当然知道 R_2 的存在。

7.32 假设用 20 位而不是 16 位来标识一个 B 类 IP 地址,可以有多少个 B 类网络?

答:

因为 B 类地址的网络部分前两个比特是固定的,即为 10,而且网络部分全 0 或者全 1 有特殊意义。所以采用 20 b 标识,总共有 $2^{18} - 2 = 26142$ 个 B 类网络。

7.33 用带点十进制标记法,写出十六进制数 c22f1582 的 IP 地址。

答:

用带点十进制标记法表示的 IP 地址为 194.47.21.130。

7.34 CIDR 作为一种路由技术,它的意义是什么?

答:

CIDR 抛弃了 IP 地址类的边界(不再指定网络号或主机号的比特位数),这样使 IP 子网地址划分更自由、灵活,地址利用率也更高。同时,在分组路由定位目标子网时,将不再根据子网地址

类而形成差别,而是进行同样的处理。

7.35 Internet 上的一个 B 类网络的子网屏蔽为 255.255.240.0。每个子网上最多有多少台主机?

答:

子网屏蔽 255.255.240.0 可以表示为:

11111111 11111111 11110000 00000000

所以,IP 地址后面 12 位用于主机部分,而其中全 0 和全 1 有特殊意义,所以每个子网上最多的主机个数为 $2^{12} - 2 = 4094$ 。

7.36 主机 IP 地址为 192.168.151.100,子网掩码为 255.255.252.0,对应的网络号是:_____。

答:

主机 IP 地址为 192.168.151.100,子网掩码为 255.255.252.0,对应的网络号是:192.168.148.0。

7.37 在虚电路方式的网络互连中需要分段吗?还是只在数据报方式的系统中需要?

答:

都需要。在虚电路方式的互连的网络中,可能有些网络能够接收 2 KB 大小的分组,而有些网络只能处理 1 KB 大小的分组,这时就需要进行分段。

7.38 IP 分组经过路由器进行传输时如果不被分段,其首标的基本部分会变化的字段是:____、____。当____时,IP 分组将会被路由器丢弃。

答:

IP 分组经过路由器进行传递时如果不被分段,其首标的基本部分会变化的字段是:TTL、Header Checksum。当路由器发现 TTL = 0时,IP 数据包将会被路由器丢弃。

7.39 假设一个 IP 分组在某个路由器处被分段,问需要拷贝哪些 IP 字段,哪些字段需要重新计算?

答:

IP 头部字段包括:版本、IHL、TOS、标识、总长度、DF、MF、分段偏移、TTL、协议、头部检验和、源和目的 IP 地址、IP 选项。这样在某个路由器分段后分组的长度有变化,从而需要重新计算的字段包括:总长度、MF、分段偏移、头部检验和以及 TTL 字段,IP 选项字段根据选项的不同可能会需要拷贝,或者不再需要,从而 IHL 字段也可能会需要重新计算。而其他字段则需要重新进行拷贝。

7.40 大多数 IP 分组重组算法都有 1 个计时器,以避免因为丢失分段分组而永久占用重组缓冲区。如果一个分组被分为 4 段,前 3 段都成功到达目的地,但最后 1 段迟迟不能到达。最后计时器超时,接收者将该分组中前 3 段丢弃。随后,最后 1 段姗姗来迟。怎么处理这一迟到的分组?

答:

最后一段到来后,因为重组缓冲区中没有对应的分段,接收者认为是一个新的分段,进入重组缓冲区中,并且等待是否有新的分段到来,最后超时,这个分段也被丢弃。

7.41 我们知道 IP 分组经常需要进行分段和重组,请问在目的节点重组相比在中间节点重组有什么优势?

答:

在目的节点重组可以避免途中可能出现的不断重组和分段,允许分段采用不同的路由来到达目的地。

7.42 IP 分段允许把一个大的 IP 分组通过那些只能容纳较小分组的网络。但是由于重传是在高层实现的,分段也带来了性能的下降,请解释。

答:

关键的问题是高层只有在收到完整的 IP 分组后才会发送确认,重传的也是完整的 IP 分组而不是分段后的碎片,因此分段后的任何碎片丢失都会导致该 IP 分组完全丢失。比如,假设分组丢失的概率为 0.05,假设 IP 分组被分段成两个,因此其中任何一个丢失的概率 $= 1 - (1 - 0.05)(1 - 0.05) = 0.0975$ 。

7.43 把一个 1 024 B 的 IP 分组划分为若干个分段。各分段要通过一个分组最大长度为 128 B 的 X.25 网。问需要多少个分段? 不计更低层的开销,仅考虑 X.25 和 IP 分组的开销,传输的效率是多少?

答:

设 IP 数据包没有 IP 选项,即 IHL = 5,头部 20 B,X.25 分组的头部长度为 3 B,因此一个 MTU 为 128 B 的 X.25 中每个 X.25 分组可以携带的数据长度为 $128 - 20 - 3 = 105$ B,由于 IP 分组的分段偏移以 8 B 为单位。所以实际数据长度为 104 B,共需要有 10 个分段。传输的效率为

$$\frac{1\,024}{(104 + 20 + 3) \times 9 + (88 + 20 + 3) \times 1} \times 100\% = \frac{1\,024}{1\,254} \times 100\% \approx 81.7\%$$

7.44 IP 分组在转发过程中,源和目的 IP 地址会改变吗? 那么它在经过不同的子网段时是如何利用子网的 MAC 地址实现传递的?

答:

IP 分组在转发过程中,源和目的 IP 地址不会改变。

IP 分组在经过不同的子网段时,发送者(可能是源,也可能是转发者)把目的 IP 地址和该子网掩码进行比特与运算来取得目的 IP 所在的子网号,然后与本机的子网地址进行比较,如果相同,则表示在同一个子网中,从而可以直接递交给目的主机;如果不同,则由发送者查找路由表来选择一个最合适的、直接连接的(即本子网的)路由器,将分组递交给该路由器,由该路由器进行下一步转发。递交时先利用 ARP 协议进行地址解析,将目的主机或转发路由器的 IP 地址转换成 MAC 地址,然后发送 IP 分组。

7.45 移动 IP 中是如何实现 IP 地址的移动的? 一个主机移动到了外地,它如何实现数据传输?

答:

为了实现 IP 地址的移动(即移动主机保留其地址),移动 IP 为每个移动主机设置了两个 IP 地址,第一个地址是主机的主地址,它是永久固定的、原来的 IP 地址,由本地网分配,这是应用程序和运输层所用的地址。第二个地址是主机的辅地址,它是临时的,随着主机的移动而改变,由外地网分配,用于 IP 分组转发时的隧道传输。当主机移动到一个外地网并获得辅地址时,移动主机把辅地址发送给位于本地网的一个 HA 进行登记,以便该 HA 随后截取发送给移动主机主地址的分组,并使用 IP-in-IP 封装,把每个分组以隧道方式传输到辅地址,由辅地址的 IP 软件将封装内的 IP 分组以主地址传输到移动主机的应用程序。

目的地址是移动主机的数据传输以上面的 IP-in-IP 隧道方式进行,而源地址是移动主机的数据传输有两种情况:如果外地网络同意用任意源地址传输分组,那么移动主机可以直接沿着外地网到目的之间的最短路径进行传输,这样传输效率最高;否则,仍然使用 IP-in-IP 隧道方式先传输到 HA,再由 HA 提取原始 IP 分组进行转发。

7.46 移动 IP 的规范允许单个路由器既作为一个网络的 HA,又作为支持网络上来访者的 FA。使用单个路由器完成两种功能的优点和缺点是什么?

答:

优点是该路由器可以将移动到外地的主机主地址充分利用,作为辅地址分配给来访者使用。同时方便实现来访者从本地移动到外地主机之间的路由优化。

缺点是该路由器需要能区分目的地址是本地的 IP 分组是隧道分组还是原始 IP 分组,以实现不同的转发。同时路由优化将使路由器设计复杂化。

7.47 IGMP 并未包括确认或重传的策略,即使在尽最大努力交付的网络上也是如此。丢失询问消息会造成什么后果? 丢失报告消息呢?

答:

丢失询问消息可能会使某个网段及其下级网络上的主机失去组成员资格。丢失报告消息可

能会使某个主机失去组成员资格。

7.48 讨论构造组播分布树的两种方法。它们各有什么优缺点？

答：

构造组播分布树的两种方法是：源分布树方法和共享分布树方法。

源分布树是以组播源为根节点构造到所有组播组成员路径都最短的生成树，通常也称为最短路径树(SPT)。如果组中有多个组播源，则必须为每个组播源构造一棵组播树。共享分布树也称为 RP 树(RPT)或基于核心的树(CBT)。它的构造方法是以网络中的某一个指定的路由器为根节点，该路由器称为汇聚点(RP)或核心，由此节点生成包含所有组成员的组播树。使用共享分布树时，同一组播组的组播源需要首先把组播分组单播到 RP，再由 RP 转发给其他组成员。

SPT 的优点：由于不同组播源发出的分组被分散到各自分离的组播树上，因此采用 SPT 有利于网络中数据流量的均衡；同时，因为从组播源到每个接收者的路径最短，所以端到端的时延性能较好，有利于流量大、时延性能要求较高的实时媒体应用。SPT 的缺点是：要为每个组播源构造各自的分布树，当数据流量不大时，构造 SPT 的开销相对较大。

RPT 的优点：RPT 在路由器所需存储的状态信息的数量和路由树的总代价两个方面具有较好的性能。当组的规模较大，而每个成员的数据发送率较低时，使用 RPT 比较适合。RPT 的缺点：当通信量大时，使用 RPT 将导致流量集中及出现 RP 附近的瓶颈。

7.49 两个支持组播的路由器由一个不支持组播的中间路径连接(即路径中有不支持组播的路由器)，有什么办法使它们间的组播分组能够相互传输？

答：

可在两个支持组播的路由器之间建立一个 IP 隧道，以 IP-in-IP 隧道的方式将 IP 组播分组传输给对方。

7.50 讨论 IPv6 的主要特征。它带来了哪方面的一些优势？

答：

IPv6 具有如下的一些主要特性和优势：

- (1) 每个 IPv6 地址有 16 B 长。这解决了长期困扰 IPv4 的地址空间和主机容量问题。
- (2) 简化的 IPv6 协议头部仅包含 7 个字段。这使路由器处理分组的速度更快，因此提高了吞吐率。
- (3) 对选项提供更好的支持。除基本的协议头部字段外其他的协议字段在 IPv6 中只是选项，且选项出现的方式也不同。这使路由器可以简单地跳过选项而不必关心它们，从而加速了分组处理的过程。
- (4) 强大的安全性支持。IPv6 提供很强的身份认证和隐私保护等安全性支持。
- (5) 具备 QoS 保障能力。IPv4 几乎没有 QoS 保障，而 IPv6 的 QoS 保障能力满足未来实时、

多媒体通信的要求。

(6) 具备 IPv4 的平滑过渡和升级能力。这使 IPv4 向 IPv6 过渡时可以逐步进行,这可以保护已有的 IPv4 网络投资,而且 IPv6 本身在未来还可以发展。

7.51 IPv6 协议头部中不再包含协议字段,为什么?

答:

IPv6 协议中“下一个头部”字段说明了最后一个 IP 头部后面是什么协议,这也包括了 IPv4 协议中“协议”字段所需表达的运输层协议,如 TCP 或 UDP 等,因此无需再包含“协议”字段。

7.52 IPv6 具有较强的路由器处理性能和协议灵活性,为什么?

答:

IPv6 具有较强的路由器处理性能主要表现在:

(1) 固定长度的基本协议头部。这使路由器在处理 IPv6 分组时可以固定格式进行(甚至以硬件实现),这极大地加快了路由器处理和转发分组的速度。

(2) 基本协议不支持分段。路由器可以不需要处理分段和分段重组,使路由器集中精力处理分组路由和转发。

(3) 没有校验和。路由器不必再为每个分组计算校验和,这极大地提高了分组处理能力。

IPv6 的协议灵活性主要表现在:

(1) 很好的选项支持使现在和未来的协议内容都可以通过选项来表达。

(2) “链式”的协议头结构使 IPv6 具备几乎无限的协议灵活性和可扩展能力。

7.53 IPv6 的地址有____位,一个 IPv4 的地址 202. 224. 120. 9,其 IPv6 的地址标识可以是_____。

答:

IPv6 的地址有 128 位,一个 IPv4 的地址 202. 224. 120. 9,其 IPv6 的地址标识可以是 202. 224. 120. 9。

7.54 简述从 IPv4 向 IPv6 过渡期间双协议栈路由器的的工作原理。

答:

双协议栈路由器同时支持 IPv6 和 IPv4 两种协议,其协议栈结构如图 7.4 所示。

应用程序	
TCP/UDP 协议	
IPv6 协议	IPv4 协议
物理网络	

图 7.4

这样的路由器就既能支持 IPv4 协议,又能支持 IPv6 协议。即该路由器既能互连 IPv4 网络,又能互连 IPv6 网络以及 IPv4 和 IPv6 混合的网络。也就是说,这种路由器既能路由和转发 IPv4 或 IPv6 分组,又能将 IPv4 分组转换成 IPv6 分组或将 IPv6 分组转换成 IPv4 分组进行路由和转发。

具体实现上,路由器根据接收端口的协议处理并分离出相应的 IP 协议头部,进行路由分析后,开始转发时,根据发送端口的协议添加相应的 IP 协议头部。

第八章 运 输 层

8.1 运输实体是如何实现运输层协议功能并向运输服务用户提供服务的?

答:

运输实体接收网络层上传(或向网络层发送)的 TPDU,通过对 TPDU 协议头的处理,实现运输层协议功能。运输实体通过运输服务原语向运输服务用户提供服务,即通常所说的 Socket 服务。

8.2 运输层提供哪两类运输服务?在 TCP/IP 中分别对应哪两个协议机制?在实际的应用中,二者的侧重面是什么?

答:

运输层提供两种类型的服务:面向连接和无连接的运输服务。在 TCP/IP 中分别对应 TCP 和 UDP 协议。TCP 侧重要求可靠通信的应用,如银行转账;而 UDP 侧重要求高效通信的应用,如实时图像传送。

8.3 简述运输层 QoS 的内容和运输实体间的 QoS 协商过程。

答:

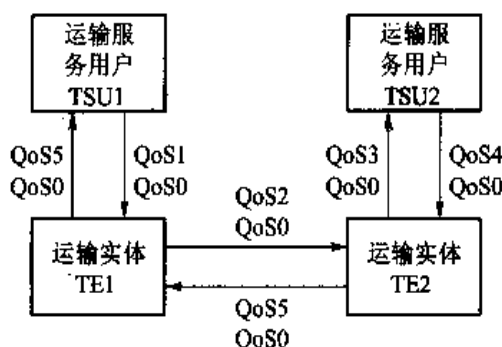
运输层 QoS(Quality of Service,服务质量)是运输实体双方能够提供的运输层的总体性能,通常通过一些可数值化的参数来表达运输层 QoS,主要包括:连接建立延迟、连接建立失败概率、吞吐量、残留差错率、传输延迟、保护性、优先权和回弹率等。

这些反映运输层 QoS 的参数通过运输实体间的 QoS 协商来确定,协商过程如下:源运输实体向目的运输实体发出连接请求,同时传递相应的 QoS 参数(包括期望值和最低值),如果目的运输实体不能接受这个 QoS 的期望值,但能接受一个比 QoS 最低值高的 QoS 值,则目的运输实体以它自己可接受的 QoS 值来响应这个连接请求。如果源同意这个 QoS 值,则双方使用这个 QoS 值;否则失败。如图 8.1 所示。

8.4 TCP/IP 协议中,通过_____和_____进行标识,可以确保数据传输到正确的主机和主机上正确的用户进程。

答:

TCP/IP 协议中,通过目的 IP 地址 和目的 TCP/UDP 端口号 进行标识,可以确保数据传输到正确的主机和主机上正确的用户进程。



QoS0: 最低 QoS QoS3: TE2 建议的 QoS
 QoS1: TSU1 期望的 QoS QoS4: TSU2 可接受的 QoS
 QoS2: TE1 建议的 QoS QoS5: 各方都能接受的 QoS
 通常: $QoS1 \geq QoS2 \geq QoS3 \geq QoS4 \geq QoS5 \geq QoS0$

图 8.1

8.5 何为复用？解释两种形式的复用。

答：

复用是指一个协议实体(或一个服务)被多个用户所共享,同时通过相应的 SAP(Service Access Point, 服务访问点)加以区分。有两种形式的复用：

(1) 向上复用使多个用户共用一个服务实体。如多条运输连接复用一条网络连接可以在满足吞吐量的情况下尽量减少费用。

(2) 向下复用使一个服务实体同时使用多个低层的服务。如一个运输连接同时复用多条网络连接可以提高对单一网络失效的恢复能力及提高吞吐量。

8.6 何为流控？解释层内流控和层间流控。

答：

流控即流量控制,是指通信时数据的接收方根据自己的资源情况(如缓存、转发处理能力等)对发送方的发送速度进行约束的控制机制。

一般有两种类型的流量控制：

(1) 层内流控,也称对等流控,是指同一层内的实体之间的流量控制。如发送方运输实体和接收方运输实体之间进行的流量控制。

(2) 层间流控是指不同层实体之间的流控。如发送方的运输实体和它的网络层实体之间进行的流量控制。

8.7 对比并分析滑动窗口和信用量方案的有利点和不利点。

答：

滑动窗口更加简单,只需要考虑窗口的边界参数,另外当窗口增加然后再减小的时候,不会

出现 TPDU 失序的情况;但是信用量方案更加灵活,考虑到了对于缓冲区的动态管理。

8.8 为什么必须使 TPDU 序号空间足够大,以保证不仅仅 TPDU 本身,而且 TPDU 的确认也全部消失才行?

答:

TPDU 序号空间应足够大,否则有可能产生错误。例如,A 发了 DT TPDU₀,没有得到确认,于是重传了 DT DPDU₀。B 收到两个 DT TPDU₀,先后发回了两个 AK TPDU₀。其中一个被收到,另一个延迟了。A 继续发送 TPDU,如果序号空间不够大,等序号空间很快用完后又重新回到 0 号。这时 A 收到前面那个延迟了的 AK TPDU₀,它将错误的以为收到了 B 关于这个新的 TPDU₀ 的确认。

8.9 对于有序的可靠网络访问,TPDU 序号一定需要吗?如果没有序号,可能会损失什么功能?

答:

因为网络是有序的、可靠的,所以 TPDU 的序号不一定需要。但若没有 TPDU 序号,则难以实现流量控制。

8.10 IEEE 802.11 无线网络通过确认和重传实现链路层的可靠机制,TCP 也在运输层实现了自己的确认和重传机制。如果 TCP 用在 802.11 网络中,请问可以去掉其中一个确认和重传机制吗?为什么?

答:

不能去掉其中任何一个确认和重传机制,因为二者确保可靠性的侧重面不一样。

TCP 的确认和重传机制提供端到端的运输层可靠性,因为 802.11 只是保护了链路,但是分组可能会在交换机丢失,路由可能会变化,分组会损坏等,而 802.11 并不能处理这个问题。

尽管 802.11 的可靠机制不能保证端到端的可靠性,但是它可以更快地对分组丢失作出响应。比如 TCP 将把分组丢失作为拥塞指示,但是 802.11 可以知道分组是否由于暂时的无线电干扰而导致的丢失,因此马上重传它。因此 802.11 机制可以提高性能。

8.11 使用两次握手方法建立连接是否可能产生死锁?请举例说明。

答:

两次握手方法建立连接可能造成死锁。比如:A 发了 CR TPDU_x。在连接释放后到达 B。B 发 CC TPDU_y 确认。而此时 A 又发了一个 CR TPDU_z 要求建立一个新的连接。B 正在期待序号为 x 的 DT TPDU 的到来,收到 CR TPDU_z 后认为非法并将之丢弃。A 在收到 B 发送的 CC TPDU_y 后以为 CR TPDU_z 已获得确认,于是发送了 DT TPDU_z。B 收到后发现序号是 z 而不是 x,所以把它丢弃。这样 A 会持续的发送 DT TPDU_z,而 B 总是不予理睬,从而陷入死锁。

8.12 讨论在教材图 8.13 中,CR 和 CC 都被延迟的情况。

答:

如图 8.2 所示。

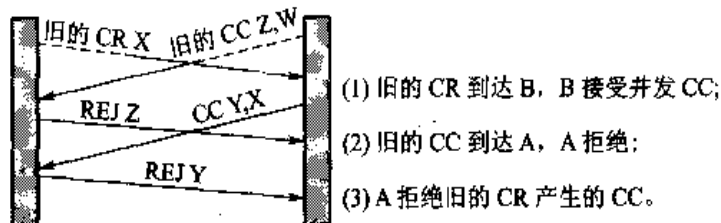


图 8.2

其中第(2)步和第(3)步可以互换。

8.13 三次握手的连接释放中会出现半开通状态吗?若会出现,怎样处理?

答:

在 ACK 或 DC 丢失的情况下,会出现半开通状态。

处理半开通状态的一种方法是引入一个计时器:

(1) (ACK 丢失的情况下)发送 DC 后在一定的时间内若没有收到 ACK,再次发送 DC 并等待 ACK……连续若干次收不到 ACK,即计时器超时,则自动释放连接;

(2) (DC 丢失的情况下)类似(1)。

8.14 数据报的分段与重组是由 IP 控制的,对 TCP 协议而言是透明的,这是否意味着 TCP 不用担心数据以错误的次序到达?为什么?

答:

不是。尽管数据报的分段与重组由 IP 控制,但是因为 IP 提供的是不可靠的网络服务,数据报到达的顺序可能会不同,因此 TCP 必须提供差错控制功能,处理失序的情况。

8.15 为什么 TCP 协议的序号是对每一个 TCP 数据字节都要进行编号?

答:

因为 TCP 是面向字节流的,即用户数据没有边界,接收方并不知道发送者的边界,需要特殊机制来实现。此外,编号也便于数据的确认。

8.16 在 100 Mb/s 的 LAN 环境下,至少多少时间后才可能出现重复编号的 TCP 数据段?

答:

$$2^{32} \times 8 / 100 \text{ Mb/s} \approx 5.5 \text{ min.}$$

8.17 当 TCP 发送一个 $\{\text{SYN}, \text{SequenceNum} = x\}$ 或者 $\{\text{FIN}, \text{SequenceNum} = x\}$, 相应的 ACK 中包含的确认号为 $x+1$, 也就是说 SYN 和 FIN 也占用了顺序号空间, 请问这是必要的吗?

答:

FIN 占用顺序号是必要的, 这样 FIN 的发送者可以通过 ACK 来确认接收方收到的到底是 FIN 还是以前发送的数据。而对于 SYN, 对后面数据的确认实际上隐含着确认该 SYN (连接没有建立之前不能对数据进行确认), 因此 ACK 中的顺序号为 $x+1$ 实际上不是必需的, 而是考虑到一致性。

8.18 请说明为什么 TCP 头部中最多只能有 44 B 的 TCP 选项?

答:

TCP 头部包括了一个 4 b 的头部长度的字段, 以 32 b 的字为单位, 因此头部最多为 $16 \times 4 = 64$ B, 而固定长度的头部字段为 20 B, 因此 TCP 选项最多为 $64 - 20 = 44$ B。

8.19 TCP 段携带的用户数据最多为 65 495 B, 为什么?

答:

TCP 段至少要求包括 20 B 的固定头部, 这个 TCP 头部封装成一个 IP 分组, IP 分组的头部也要 20 B, 而 IP 分组的最大长度为 65 535, 因此 TCP 段携带的用户数据最多为 $65\,535 - 20 - 20 = 65\,495$ B。

8.20 数据字段长为 0 的 TCP 段其意义是什么?

答:

数据字段长为 0 的 TCP 段其意义是:

- (1) 可以保持 TCP 连接的活跃状态;
- (2) 可以避免如下情况的死锁: 接收方发送一个窗口为 0 的确认, 发送方收到该确认而停止发送; 此后, 接收方有了接收缓冲区, 发送一个窗口非 0 的确认, 而该确认丢失了。

8.21 TCP 协议中, $\text{SYN} = 1, \text{ACK} = 1$, 表 _____; 在 RFC 1106 中定义了 NAK, 其意义是: _____。

答:

TCP 协议中, $\text{SYN} = 1, \text{ACK} = 1$, 表示 连接确认; 在 RFC 1106 中定义了 NAK, 其意义是: 允许接收方指定发送数据段。

8.22 假设一个应用程序要通过一个 TCP 连接发送一个 L B 的消息。TCP 段包括了 20 B 的头部, IP 头部字段为 20 B。IP 分组然后封装到以太网帧中, 头部长度为 18 B。请问在物理层传输的效率是多少? 考虑 $L \approx 10$ B 和 1 000 B 两种情况。

答:

一个 L 字节的消息,要附加 20 B 的 TCP 头部 + 20 B 的 IP 头部 + 18 B 的以太网头部,因此

$$\text{效率} = \frac{L}{L + 20 + 20 + 18} = \frac{L}{L + 58}。$$

所以对于 $L = 10$ B, 则效率 $= 10/68 = 14.7\%$ 。

而对于 $L = 1\,000$ B, 则效率 $= 1\,000/1\,058 = 94.5\%$ 。

8.23 假设一个 TCP 发送方连续收到许多重复的对 TCP 段 x 的确认,这个时候可以推断出什么情况出现? 为什么?

答:

TCP 采用累加 ACK, ACK 字段给出了最近收到的在序的段,因此多个重复的 ACK 意味着很有可能 TCP 段 x 后面的那个段丢失了。

8.24 如教材图 8.20 所示,指示客户方(即连接的主动建立方)和服务方(即连接的被动建立方)的状态变化路径。并叙述进入状态 SYN Received 的两种途径。

答:

正常的状态变化路径:

(1) 客户方: Closed $\rightarrow$ SYN sent $\rightarrow$ Established $\rightarrow$ FIN Wait1 $\rightarrow$ FIN Wait2/Closing $\rightarrow$ Timed Wait $\rightarrow$ Closed

(2) 服务方: Closed $\rightarrow$ Listen $\rightarrow$ SYN Received $\rightarrow$ Established $\rightarrow$ Close Wait $\rightarrow$ Last Ack $\rightarrow$ Closed

进入状态 SYN Received 的两种途径:

(1) (正常情况) 服务方在 Listen 状态等待连接请求时,收到了请求建立连接的 SYN 数据段;

(2) (较少情况) 客户方在发送请求建立连接的 SYN 数据段后,收到了对方请求建立连接的 SYN 数据段。此后,客户方转换为服务方。

8.25 在关闭一条 TCP 连接时,为什么 TCP 状态从 Last Ack 到 Closed 不需要等待 2 个段生命期的超时?

答:

因为 TCP 状态为 LAST_ACK 表示它等待接收该 TCP 连接的最后一个 ACK,而 2 个 MSL 的目的是发送最后一个 ACK 的发送者不知道该 ACK 是否被收到,而导致对方重传,因此等待一段时间来防止老的迟来的重复分组会对以后新建立的分组造成干扰。既然处于 LAST_ACK 状态的那方是等待 ACK 而不是发送 ACK,因此不需要等待 2 个 MSL 时间。

8.26 请简单描述一下 TCP 的流量控制机制和拥塞窗口机制,两者有什么联系吗?

答:

TCP 流量控制机制的目的是防止发送方的数据发送得过快,以致接收方来不及处理的情况。它通过一种可变大小的滑动窗口机制来实现:接收方设置一个滑动接收窗口(接收缓存),以滑动窗口的空余可接收容量指示发送方发送数据的大小。

拥塞窗口机制的目的是防止网络过载(因为网络的容量是有限的)。它通过一种可变大小的拥塞窗口机制来实现:发送方设置一个可变的拥塞窗口参数,该参数遵循慢启动算法。

而对于发送方,其发送数据的速度(即下一步发送数据的大小)必须小于接收窗口和拥塞窗口,即: $LastByteSent - LastByteAcked \leq \min(CongWindow, RecvWindow)$ 。

8.27 假设你需要设计一个类似于 TCP 的滑动窗口协议,该协议将运行在一个 100 Mb/s 网络上,网络的往返时间为 100 ms,最大段生命期为 60 s。请问协议头部的窗口和顺序号字段应该有多少位?为什么?

答:

窗口大小必须能够容纳的数据应该为延迟 $\times$ 带宽,而顺序号则应该保证不会回绕,即为生命期 $\times$ 带宽。

这样: $RTT \times \text{带宽} = 100 \text{ ms} \times 100 \text{ Mb/s} = 10 \text{ Mb}$, 而 $10 \times 10^6 < 2^{24}$, 这样窗口大小必须容纳 $\frac{2^{24}}{8} = 2^{21}$, 即窗口字段最小为 21 b。

而顺序号空间大小为

$$\frac{60 \text{ s} \times 100 \text{ Mb/s}}{8} < 2^{30}$$

因此顺序号字段最少为 30 b。

8.28 在 TCP 中,发送方如果收到一个窗口大小为 0 的 TCP 段并发送 ACK 后需要开启保持计时器,定期向另一方探测是否接收窗口被重新打开。请问如果发送方不进行探测,而是把这个探测过程转向接收者应该怎么样处理?

答:

发送方进行探测的目的是接收方重开窗口后发送的 ACK 可能丢失,而如果它转向接收者来探测,则接收方应该包括一个计时器来处理该 ACK 丢失的情况,直到接收方能够确认发送方收到了该 ACK 才结束,否则就需要重传该 ACK。而接收方要确认发送方收到了 ACK,则要求发送方有新的数据发送,所以在连接空闲的时候,接收方实际上得不断地重传。

8.29 有一个 TCP 连接,当其拥塞窗口为 64 个分组大小时超时。假设网络的 RTT 是固定的 3 s,不考虑其他开销,即分组不丢失,请问系统在超时后处于慢启动阶段的时间有多少秒?

答:

拥塞阈值为 $64/2 = 32$, 在慢启动阶段, 每次 RTT 时间拥塞窗口加倍, 我们不妨用 $W(t)$ 来表示时刻 t 的拥塞窗口, 这样 $W(0) = 1$, $W(RTT) = 2$, $W(2RTT) = 4$, $W(3RTT) = 8$, $W(4RTT) = 16$, $W(5RTT) = 32$ 。因此系统处于慢启动阶段的时间为 5 个 RTT 即 15 s。

8.30 一个 TCP 发送方的 MSS 为 1 000 B, 其初始拥塞窗口为 $1 \times \text{MSS}$, 初始的慢启动阈值为 4 000 B。在该 TCP 连接上传了一个全 TCP 段(即段携带的数据长度为 MSS), 然后从接收方收到 ACK, 假设前面 6 次传输都成功, 直到第 7 次传输 TCP 段时该 TCP 段丢失需要进行重传。请给出每次收到 ACK 时以及重传第 7 个 TCP 段时拥塞窗口和慢启动阈值的变化情况。

答:

表 8.1 给出了每次收到 ACK 的拥塞窗口和慢启动阈值的变化情况, 注意在 ACK 1 ~ 4 时仍然处于慢启动阶段, 所以拥塞窗口增加为 1 000 B, 而此后则属于拥塞避免阶段。

表 8.1

ACK	拥塞窗口	慢启动阈值
初始	1 000	4 000
1	2 000	4 000
2	3 000	4 000
3	4 000	4 000
4	5 000	4 000
5	5 200	4 000
6	5 392	4 000
7 丢失	1 000	2 696

8.31 考虑一条 TCP 连接的慢启动阶段, 往返传输时间为 10 ms, 最大分段大小为 2 KB, 假设网络不会出现拥塞, 接收窗口为 48 KB, 请问从建立连接开始到可以发送第一个完整的接收窗口的数据要多长时间?

答:

初始拥塞窗口为 2 KB, 每次成功的传输之后, 拥塞窗口增长到 4 KB、8 KB、16 KB、32 KB、64 KB, 因此需要 50 ms 的时间可以连续发送 48 KB 的数据。

8.32 假设一条 TCP 连接的当前往返时间 RTT 为 30 ms, 并且分别在相隔 26 ms、32 ms 和 24 ms 后收到确认。请问新的 RTT 估计是多少? 假设 $\alpha = 0.9$ 。

答:

按照 RTT 的公式有:

$$\text{EstimatedRTT} = \alpha \times \text{RTT} + (1 - \alpha) \times \text{SampleRTT}$$

$$\text{RTT} = 0.9 \times 30 + (1 - 0.9) \times 26 = 29.6$$

$$\text{RTT} = 0.9 \times 29.6 + (1 - 0.9) \times 32 = 29.84$$

$$\text{RTT} = 0.9 \times 29.84 + (1 - 0.9) \times 24 = 29.256$$

所以新的估计值为 29.256 ms。

8.33 一个 1 Gb/s 的 TCP 连接上,发送窗口大小为 65 535 B,连接两端的延迟时间为 10 ms,请问可以获得的最大吞吐率是多少?效率是多少?

答:

由于发送窗口大小为 65 535 B,因此一个 RTT 可以传输满窗口的数据,即最大吞吐率为

$$\frac{65\,535 \times 8\text{ b}}{2 \times 10\text{ ms}} = 26.214\text{ Mb/s.}$$

而效率为

$$26.214\text{ Mb/s} / 1\text{ Gb/s} = 2.6\%$$

8.34 为什么需要 UDP? 为什么用户不能直接访问 IP?

答:

这是因为 UDP 作为运输层协议,提供了多路复用的功能。IP 分组包括了 IP 地址,给出了 IP 分组的目的主机的地址。当 IP 分组到达时,IP 模块不知道到底应该递交给哪个进程。UDP 分组包括了一个目的端口号,从而可以知道上层用户的进程。

8.35 简述 Socket 编程的基本概念和 Socket 通信模型。

答:

Socket 编程是在 TCP/IP 网络上为不同主机上不同应用程序(进程)间提供进程间通信的一种实现方法。它是运输层为应用程序提供端到端通信的一种服务形式。

Socket 编程中,进程被绑定在以(地址,端口号)标识的 Socket 上,而一个完整的网间进程通信链路用一个五元组来标识:(协议,本地地址,本地端口号,远地地址,远地端口号)。通过调用 Socket 原语(函数)来建立 Socket 链路、进行数据收发和关闭 Socket 链路。这样,进程通信就被简化为对一个被绑定的本地 Socket 进行文件读写式的操作过程:打开→读写(数据收发)→关闭。

Socket 通信以 C/S 模式实现:通信的两端各有一个 Socket,其中一个 Socket 以服务方的形式预先启动并等待客户方的 Socket 链路建立请求,客户方在通信之前先向服务方请求建立 Socket 链路,一旦建立好链路,双方就都可以进行数据的收发,通信结束前需要关闭 Socket 链路。

因此,从网络的角度看,Socket 是通信连接的端点;从程序设计者的角度看,Socket 是编写网络通信程序的 API。

8.36 在 Unix 下利用 Socket 接口实现一个聊天室的功能,允许多人加入该聊天室,每人所说的话应该能够被现在正在聊天室里面的其他人听到,并且允许用户动态地加入和退出该聊天室。

答:

(参见实验指南)。

8.37 利用 Winsock 实现一个支持 TCP 和 UDP 方式的 ECHO 服务,服务方接收请求,把收到的数据原封不动地送回去。要求能够支持多个用户同时使用该服务,并且可以用 TCP 或者 UDP 方式来连接。

答:

(参见实验指南)。

第九章 高层应用

9.1 什么是主机名、域名、FQDN?

答:

用平面名字为每台连网的主机取一个唯一的字符名字,以此来标识连网的主机,该名字称为主机名。主机名没有与名字相关联的分层或纵向结构,也就是说,主机名没有分区/分块的命名和管理能力。

以树型分层结构来命名域或主机,该名字称为域名。域名的书写从域名树中当前节点开始直到根,用“.”隔开,靠右的是上级域名。域名非常适用于大型网络(如 Internet)的主机命名和管理。

FQDN(Fully Qualified Domain Name),即完全合格的域名。它是从域名树中叶子节点开始直到根节点组成的一个完整域名。在同一个网络及其域名管理体系中,FQDN 是唯一的。

9.2 为什么要引入域名的概念? 域名以什么机制进行命名,其好处是什么?

答:

IP 地址很难记忆,引入域名之后,便于人们记忆和识别,域名查找可以把域名转换为 IP 地址。

域名以一种层次、分级、树型的命名机制进行命名。其有多方面的好处:

1. 命名的方便性和表意性

分层域名机制有大得多的名字资源,它有好得多的命名方便性和表意性。

2. 扩展性

树型分层域名机制的树有足够高,它能够满足实际的子域扩展需要。

3. 管理性和维护性

域名的管理和维护可以简单地参照命名树分布进行,由当地的授权管理机构进行属下的域名管理和维护,域名的更新有很强的即时性。

4. 分布式实现

域名从名字到地址的转换也可以参照命名树分布式实现,域名转换的可靠性和效率有很大的保障。

9.3 什么是域名服务(DNS)? 简述 DNS 的工作原理。

答:

域名服务 DNS(Domain Name Service)是一种组织成域层次结构的网络设备命名服务系统。DNS 用于 TCP/IP 网络(如 Internet),用来通过用户的名字定位网络设备和服务。当用户在应用程序中输入 DNS 名字时,DNS 服务可以将此名字转换(或称解析)为与此名字关联的数字地址信息(如 IP 地址)。

DNS 服务包括两方面的内容:DNS 管理和 DNS 解析。前者用于维护 DNS 域名树,后者用于为用户提供从域名到 IP 地址的转换服务。其大致工作过程如下:

当一台用户主机要查找一个域名时,它会向本地的域名解析服务器发送一个请求。如果该解析服务器有这个域名 IP 地址的映射缓存数据,就返回一个回答;如果没有,就通过递归或迭代查找方法,向上一级的域名解析服务器发送请求(任何一次成功解析后就将返回)……直到根服务器。根服务器将沿域名树依次层层向下请求域名管理服务器中该域名的权威资源记录,直到找到最后结果(然后返回)或失败。

9.4 在域 foo.com 的 DNS 记录中有一行为

```
www      IN      A      192.168.4.4
```

这是一个什么记录?为什么在 www 后没有句点?

答:

没有句点代表它是一个相对域名,即代表 www.foo.com 的 IP 地址为 192.168.4.4。

9.5 一个域名(fudan.edu.cn)和 IP 子网地址(202.120.224.0)之间的关系是什么?子网中的所有主机都必须由同一个域名服务器来维护其映射吗?

答:

从理论上来说两者之间的关系非常少,不过在实践中一个组织或者公司经常会让它的域名服务器来维护其子网中所有主机的资源映射。但是某个域的 DNS 服务器并不一定要属于自己的 IP 子网,完全可以在不同的网络中。同样,域 fudan.edu.cn 下面的主机可以在不同的网络中,同时 IP 子网中的主机也可以由不同的域名服务器来维护其映射。

9.6 你在本地用浏览器访问 http://www.yahoo.com/default.htm。进行域名解析时,假设在到达根 DNS 服务器前使用递归查找方式,其后使用迭代查找方式。说明域名的解析过程。

答:

假设在复旦大学校园网中。域名 www.yahoo.com 的解析过程:

先向复旦的 DNS 解析服务器发出请求,该服务器如果有这个域名的缓存,则成功返回;否则该服务器向上一级(如 edu)的解析服务器发出请求;再失败则向 cn 的解析服务器发出请求。这期间任何一个解析服务器成功命中的话,就沿请求的路径依次返回(递归查找)。若 cn 的解析服务器还不能解析,它就把请求转给根 DNS 服务器。

此后使用迭代查找方法。根 DNS 服务器将这个请求发送给 com 域管理服务器,若该管理服

服务器有权威资源记录,则直接返回给用户;否则将这个请求转给其 yahoo 子域名管理服务器,该管理服务器应该有权威资源记录,然后直接返回给用户;如果没有,就直接返回解析失败信息给用户。

9.7 你在遥远的一个南太平洋岛国通过当地的 ISP 访问 bbs.fudan.edu.cn,试说明解析这个域名时多种可能的过程。

答:

首先,通过当地的 ISP 提供的 DNS 解析服务器试图解析这个域名,若不成功(即该解析服务器没有缓存),则层层向上级解析服务器发出请求。如果直到 DNS 根服务器还不成功,则依次向 cn 域名管理服务器、cn 的 edu 子域名管理服务器、edu 的 fudan 子域名管理服务器请求该域名的权威资源记录,然后返回。

在每次提交解析请求时,可以指定使用递归或迭代查找方法,因此会形成多种返回的过程。

9.8 如果一个 Web 服务器的 IP 地址改变,请问 DNS 缓存会带来什么问题? 怎么样降低其影响?

答:

DNS 服务器维护的 DNS 资源记录包括了一个 TTL,给出了一个 DNS 记录在缓冲中的时间,因此在 IP 地址改变后,有的用户还使用老的地址。可以在改变 IP 地址之前首先把 TTL 设为一个小一点的值,从而使得 DNS 记录缓冲的时间非常短,然后在改变之后把 TTL 恢复到正常的值。

9.9 Internet 上提供客户访问的主机一定要有 IP 地址吗? 一定要有域名吗? 域名是 www.abc.com.cn 的主机一定位于中国吗?

答:

Internet 上提供客户访问的主机一定要有 IP 地址,但不一定要有域名,域名是 www.abc.com.cn 的主机不一定位于中国。

9.10 试举例说明域名和 IP 地址的对应关系可以是多对多的关系。

答:

如域名 www.yahoo.com 在不同时间解析出不同的 IP 地址,这是为了服务器负载均衡的目的。也可以把多个域名指向同一个主机 IP 地址,如 www.sohu.com 和 www.sohu.net 就可能是同一个主机。

9.11 想一个你认为好的 Internet 域名,说明其价值,看它是否被别人注册了。

答:

别指望像 business.com 或 email.com 之类的域名还可以使用,还有创意很好的域名吗? 组

合一些数字或 cn 等试试看。

9.12 什么是 URL? 写一个完全的 URL。

答:

URL (Uniform Resource Locator) 即统一资源定位器, 俗称“网址”, 是 Internet 上定位所访问资源的一个统一格式, 它包括访问所使用的应用层协议、主机 (及端口)、主机上的路径 (目录)、路径下的文件名等。格式: protocol://host[:port][/path][/filename]。

一个完全的 URL 如: http://www.fudan.edu.cn/fudannews/main.php。

9.13 当前, Internet 上最著名的服务是 WWW, WWW 的基本要素有哪些? 简述 WWW 服务模型。

答:

WWW 的基本要素包括两个部分: HTTP 协议和 HTML 语言。HTTP 用于 WWW 客户和服务之间进行信息传输, 而 HTML 用来控制如何把信息显示给用户。

WWW 服务模型称为浏览器/服务器 (Browser/Server, B/S) 模型, 本质上也是客户机/服务器 (C/S) 模型。只不过 WWW 客户机是一个标准的网络应用软件——浏览器, 浏览器处理标准格式的输出, 同时与 WWW 服务器的请求/应答也遵循标准的协议。即, WWW 浏览器是 HTML 的解释器和内嵌代码的执行容器, 它向 WWW 服务器发出 URL 请求, 接受 HTML 输出返回, 这个请求/应答过程使用 HTTP 协议。同样, WWW 服务器使用 HTTP 协议接受浏览器的 URL 请求, 返回 HTML 输出响应 (如图 9.1 所示)。

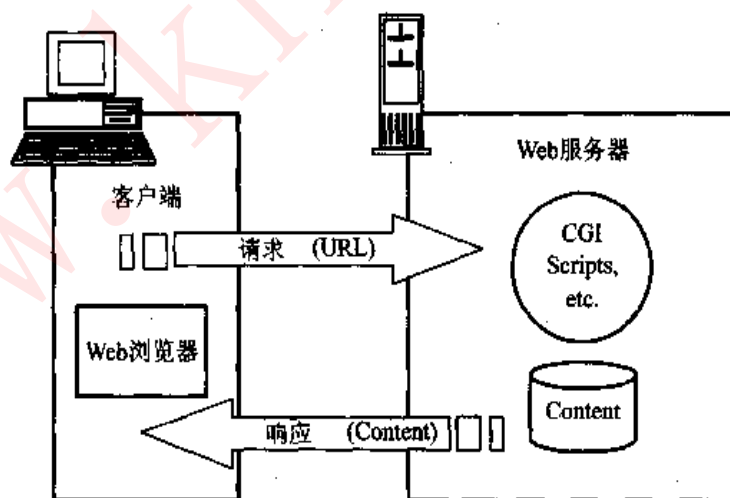


图 9.1

9.14 请使用某种理论 (如排队论等) 或模拟形式分析 HTTP 1.1 协议的性能。

答:

HTTP 1.1 协议系统近似于一个 $M/M/1$ 排队系统。以 HTTP 1.1 服务方对客户方的请求 - 响应的平均延迟时间 T 作为 HTTP 1.1 协议系统的主要性能来分析。

根据 $M/M/1$ 排队系统的结论

$$T = \frac{1}{\mu C - \lambda}$$

其中, μC 为服务方的平均服务率, $\frac{1}{\mu}$ 为请求或响应的平均内容长度, C 为服务方的服务容量, λ 为请求的平均到达率。

HTTP 1.1 几个方面的改进都使 T 变小。其中:

① “持续连接”和“大块编码”尽管使 $\frac{1}{\mu}$ 变大, 但使 λ 显著变小, 整个收益仍然使 $\mu C - \lambda$ 变大;

② “字节范围操作”和“内容协商”都使 $\frac{1}{\mu}$ 变小, 那么 $\mu C - \lambda$ 将变大;

③ “快存”使 λ 显著变小, 那么 $\mu C - \lambda$ 将显著变大。

因此, HTTP 1.1 在性能上较 HTTP 1.0 有显著提高。

9.15 Web 服务器有时也被称为 HTTP 服务器, 为什么?

答:

Web 服务器的作用是提供 WWW 服务, 而 WWW 服务是建立在应用层协议 HTTP 上的, 因此, Web 服务器有时也被称为 HTTP 服务器。这也意味着能提供 HTTP 协议服务的主机都可以作为 Web 服务器。

9.16 一个用户正在通过 HTTP 下载一个网页, 该网页长度为 14 个分组, 没有内嵌对象。假设 TCP 的慢启动阈值为 30 个分组, 用户到 Web 服务器间的 RTT 时间为 1 s, 不考虑其他开销 (比如报文丢失、处理)。请问用户下载该网页大概要多少时间?

答:

第 1 秒: 连接建立, 发送 SYN, 收到 SYN + ACK;

第 2 秒: 拥塞窗口为 1, 用户发送 HTTP 请求, 并且收到第一个分组;

第 3 秒: 拥塞窗口为 2, 收到 2 个分组;

第 4 秒: 拥塞窗口为 4, 收到 4 个分组;

第 5 秒: 拥塞窗口为 8, 收到 7 个分组;

因此用户下载该网页需要 5 s。

9.17 假设你点击某个超链接来访问某个网页,该 URL 对应的 IP 地址没有被缓存,因此需要通过 DNS 查询来获得其 IP 地址。假设采用 n 个不同的 DNS 服务器,每个 DNS 服务器和当前机器的 RTT 时间分别为 $RTT_1, \dots, RTT_n$ 。同时假设网页没有内嵌对象,大小为 500 B,当前主机到 Web 服务器的 RTT 时间为 RTT_0 。请问从你点击超链接到接收到该对象的时间最长为多少?

答:

点击该超链接,首先需要查找其对应的 IP 地址,分别询问 n 个 DNS 服务器,最坏的情况下需要的时间为 $RTT_1 + RTT_2 + \dots + RTT_n$ 。然后与服务方建立连接,需要 RTT_0 时间,然后发送 HTTP 请求,由于网页只有 500 B,因此只需要一个 TCP 段就可以了,接收网页的时间为 RTT_0 。因此总的时间最长为: $2 \times RTT_0 + RTT_1 + RTT_2 + \dots + RTT_n$ 。

9.18 在 HTTP 1.0 中,服务方通过关闭连接来表示传输结束。请从 TCP 的角度来分析这种方法带来的问题。HTTP 1.1 中采取什么方法来避免这个问题?

答:

当服务方首先关闭连接时,最后服务方的 TCP 将进入 TIME WAIT 状态,也就是需要等待 2 个 MSL 才能进入 CLOSED 状态,在这期间服务方必须维护相应的状态,这会导致服务器性能的下降。在 HTTP 1.1 中,HTTP 响应中给出了消息大小的信息,因此服务方在传输完之后并不关闭连接,而是由客户方发起连接关闭过程。

9.19 一个用户在浏览一个网站,每个网页平均包含了 4 个内嵌的对象。请问用户如果分别采用非持续连接和持续连接来浏览该网站时,所打开的 TCP 连接数目的比为多少?

答:

采用持续连接,即获得一个对象后并不关闭该连接,后面的内嵌对象也可以通过该连接传输,因此只有 1 条 TCP 连接。

而采用非持续连接,每收到一个对象后关闭该连接,网页有 4 个内嵌对象,再加上网页自身,因此总共有 5 条连接。

所以打开的 TCP 连接的数目比为 5:1。

9.20 HTML 是 Internet 浏览的最重要的语言,它的主要作用是什么?

答:

HTML 是一种信息显示的格式语言,控制如何把信息显示给用户,其作用是将图形、文字、表格、Java Applet 等各种元素组成一个可见的网页。

9.21 在一个 HTML 文件中包含如下代码:

```
<H1> <IMG ALIGN = MIDDLE ALT = "Fudan University" SRC = "Icons/fudan_logo.gif" >
```

请问 ALT 参数有什么作用?

答:

如果用户关闭了图像的自动显示,或者由于其他原因不能显示图像,那么将用 ALT 参数里给出的正文来代替该图像显示。

9.22 在 HTML 中,怎样使得一个图像成为一个超链接? 怎样使得一段文字成为一个超链接? 给出相应的例子来说明。

答:

一个超链接由 `<A HREF = "...">` 和 `</A>` 组成,中间可以是一段文字,也可以是一个图像。比如:

```
<A HREF = "http://www.foo.com"> <IMG SRC = "http://www.foo.com/images/logo.gif">
</A>
```

```
<A HREF = "http://www.acm.org"> ACM </A>
```

9.23 目前已经应用的动态 HTML 是什么含义?

答:

动态 HTML 语言是基于动态对象模型的一种技术。它使网页的每一个元素都可以动态操作,从而使网页更加生动灵活。

9.24 某单位在网络上有一个问卷调查,调查内容有姓名、单位名、单位地址、联系电话等,请设计这样一个调查页面,用户按“发送”按钮后,Web 服务器把有关信息以电子邮件的形式发送给某特定人(假设电子邮件地址是 guest@fudan.edu.cn)。

答:

```
<HTML>
```

```
<HEAD>
```

```
<TITLE> 问卷调查 </TITLE>
```

```
</HEAD>
```

```
<BODY>
```

```
<H1> 问卷调查 </H1>
```

```
<FORM ACTION = "cgi-bin/survey.pl" METHOD = POST>
```

```
姓名 <INPUT NAME = "name" SIZE = 20> <P>
```

```
单位 <INPUT NAME = "org" SIZE = 20> <P>
```

```
单位地址 <INPUT NAME = "orgaddress" SIZE = 50> <P>
```

```
联系电话 <INPUT NAME = "phone" SIZE = 20> <P>
```

```
<INPUT TYPE = SUBMIT VALUE = "发送"> <P>
```

```
</FORM>
```

</BODY>

</HTML>

9.25 用浏览器/Web 服务器(简称 B/S)方式实现数据库的开发,有什么优缺点?

答:

优点:客户端为“瘦模型”,安装方便,容易操作。

缺点:响应速度慢,安全问题复杂。

9.26 B/S 方式能(或者会)取代传统的 C/S 方式吗?为什么?

答:

不会的。二者各有优缺点,分别适用于不同的应用环境。

9.27 请分析 B/S 方式和传统 C/S 方式下的数据库系统的性能特点,并且分析它们对硬件要求的不同点。

答:

B/S 数据处理的大部分依赖于 Web 服务器,而 C/S 可以将计算在 Server 和 Client 之间做一个平衡。因此 B/S 对服务器的要求较高。

9.28 什么是 XML? 为什么有了 HTML 后还需要 XML?

答:

XML(eXtensible Markup Language)即可扩展标记语言,它是 SGML 语言的子集。使用 XML 可以开发出具有适用于专门行业要求的元素和属性的标记语言。当该语言开发出来后,就可以像使用 HTML 一样使用该语言创建文档。

相对 HTML,XML 有如下特点和功能:

- (1) XML 字符集不但包括特殊字符、标点和数学符号,而且包括非英语语言的字符和字母。
- (2) 样式表单上,XML 支持全部的文档风格符号和标记语言(DSSSL)及重叠样式表单 2 (CSS2)。
- (3) XML 允许多种类型的文档不仅显示在多种计算机平台上,而且可以显示在其他仪器上。程序员可以使用任意的程序语言或脚本语言定义文档。
- (4) XML 支持但并不必须要求 DTD。如果使用 DTD,可以根据不同浏览器和用户需要定义特殊的文档类型。
- (5) XML 所支持的标准(Xlink 和 Xpointer)使用户可以建立比 HTML 更细致的连接。XML 支持的标准允许同时使用多个链接,甚至是链接组。
- (6) XML 标准支持印刷文档、电子文档和根据用户需要建立的其他形式的文档。
- (7) XML 允许在服务器或客户端处理文档,这使开发者可以分配或任意存储资源。

9.29 试描述 XML 中正确构成文档和有效文档的区别。

答：

XML 支持两类文档：正确构成文档和有效文档。

正确构成文档必须包含至少一个根元素，且必须在根元素中嵌套子元素。

有效文档符合关于正确构成文档的所有标准，但有效文档总具有一个相关的 DTD。DTD 定义了文档内容的规则，也就是定义了创建文档的标记语言的元素和属性。

9.30 你发一封 E-mail 给你的朋友。假设使用 SMTP 和 POP3 进行 E-mail 的发送和接收，叙述这个 E-mail 的传输过程。

答：

如图 9.2 所示。

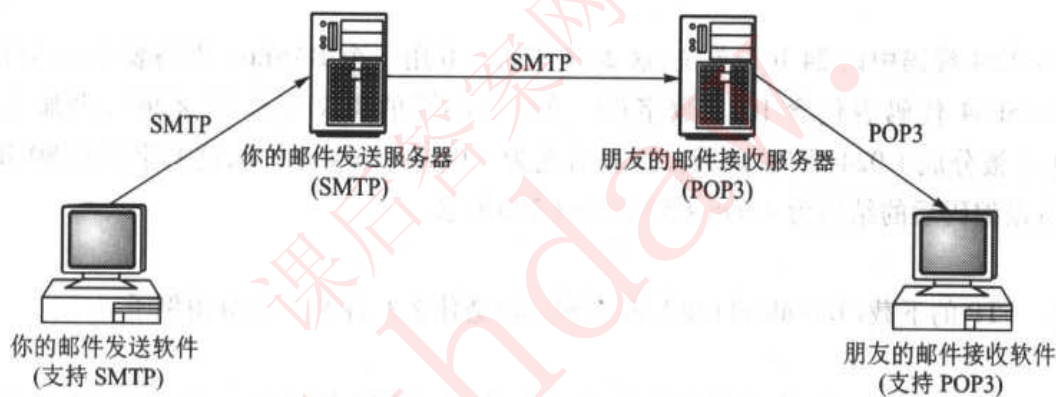


图 9.2

(1) 通过支持 SMTP 的邮件发送客户端软件使用 SMTP 协议向自己的 SMTP 邮件发送服务器发送邮件；

(2) 发送服务器使用 SMTP 协议向朋友的 POP3 邮件接收服务器发送邮件（假设中间没有经过邮件网关或邮件代理之类的设备转发）；

(3) 朋友通过支持 POP3 的邮件接收客户端软件使用 POP3 协议从他的 POP3 邮件接收服务器接收邮件。

9.31 SMTP 是 Internet 上的标准 E-mail 传输协议，请分析 SMTP 在安全性方面的弱点。

答：

SMTP 是一个 ASCII 协议，使用明码传输，因为在接收或者转发邮件时一般不对发送者进行验证，这也正是垃圾邮件泛滥的原因之一，同时也很容易冒充别人来发送电子邮件。

9.32 考虑 SMTP、POP3 协议,请问这些协议是无状态的协议吗?为什么?

答:

SMTP 是一种无状态的协议,因为邮件服务器不需要维护任何与客户方的控制连接,不需要存储任何有关客户方的信息。如果一个邮件被发送两次,则邮件服务器也会发送两次,而不会认为该邮件以前发送过而不需要再次发送。

而考虑到不同的 POP3 会话来说,POP3 是一个无状态的协议,因为它只是一个非常简单的协议,不同的 POP3 会话之间没有任何共同的状态信息。而如果考虑一个 POP3 用户和 POP3 服务器之间的一次 POP3 会话,POP3 维护了一些状态信息,特别的它记录了那些已经标记为删除的邮件消息,当发送 QUIT 命令后才进行实际的删除操作。

9.33 一个二进制文件有 3 072 B,如果采用 BASE64 进行编码,则编码后的结果有多长?假设为了阅读方便,BASE64 编码后的数据中每 80 B 插入一个回车换行 CRLF。

答:

在 BASE64 编码中以 24 b 为单位,这 24 b 中每 6 b 用一个 BASE64 代码表中的 ASCII 字符来表示,BASE64 代码表包含 10 个数字(0~9)、大小写的英文字母(2×26),再加上 + 和/。3 072 B 正好被分成 1 024 个 24 b,因此编码后变为 $1\,024 \times 4 = 4\,096$ B,这些字节每 80 B 插入 2 个字符,所以编码后的结果为 $4\,096 + 52 \times 2 = 4\,200$ B 长。

9.34 FTP 的下载(Download)和上传(Upload)是什么?各举一个应用例子。

答:

FTP 下载是指通过 FTP 客户端软件使用 FTP 协议从 FTP 服务器上取(Get)文件到客户机上;相反的,FTP 上传是指通过 FTP 客户端软件使用 FTP 协议把客户机上的文件放(Put)到 FTP 服务器上。如从服务器上取一个 MP3 音乐文件和远程更新网站上一个 HTML 文档就各是 FTP 下载和上传的典型例子。

9.35 为什么 FTP 发送控制信息是带外传输?

答:

在 FTP 协议中,FTP 客户方首先建立一条到服务方的 21 端口的连接,通过该连接发送 FTP 命令,而在要进行数据传输时必须打开一条新的 TCP 连接,因此控制信息和数据不是在同一条连接上传递,也就是 FTP 的控制信息的传输是带外传输。

9.36 为什么 HTTP、SMTP、POP3、FTP、Telnet 等运行在 TCP 而不是 UDP 之上?

答:

因为 HTTP、FTP、SMTP、POP3 协议都要求数据传输的可靠性,而 TCP 提供了面向连接的可靠数据传输服务,这样使得高层协议不需要考虑数据传输的可靠性。而如果采用 UDP 协议,由

于 UDP 提供不可靠的数据传输,这样高层协议就需要采取比较复杂的机制来进行确认、重传以保证可靠性,因此 TCP 更加适合。

9.37 如何用一种简单的方式在防火墙中设置禁止对内部网络的 FTP 访问? Telnet 呢?

答:

可在防火墙中设置:丢弃外部到内部的应用层协议是 FTP 的 IP 分组。

同样,丢弃外部到内部的应用层协议是 Telnet 的 IP 分组。

9.38 RPC 的作用是什么?

答:

RPC(Remote Process Call)即远程过程调用,它为一个进程提供了访问其他进程(往往在别的计算机上)服务的能力。RPC 是一种客户机/服务器形式的服务,一个客户机进程可以执行另一台计算机上的进程,向这个进程提供参数,获取这个进程运行的结果。

9.39 考虑一个 SUN RPC 的客户方发送一个请求给服务方。

(1) 在什么情况下客户方能够确信该请求正好只执行一次?

(2) 假设希望在 SUN RPC 中增加“最多执行一次”的支持,需要做什么改动?

答:

(1) 如果客户方只发送了一次请求,并且收到了一个回答,而且下层网络从来不会出现重复分组,则客户方能够确认该请求正好只执行一次;

(2) 为了使得请求最多执行一次,服务方必须把该请求对应的相应缓冲起来,直到收到客户方的确认为止,而为了能够和请求对应,请求和响应中应该包含一个唯一的 XID。因此为了支持这种语义除了增加 ACK 机制之外,服务方必须把结果缓冲。

9.40 和 C 语言相比,Java 语言有什么优点?

答:

Java 是由 C++ 发展而来的,但 Java 去掉了 C++ 中的一些复杂的内容,并且实现了自动垃圾收集等功能,它是一种面向对象的语言,其最大的特点就是平台独立性和可移植性,编译后产生的是字节码指令,由 Java 虚拟机解释执行。Java 采用了和 C 类似的控制结构,但是不支持指针、结构、枚举和联合类型,更加简洁。

9.41 设计一个 Java 程序,功能是显示一个 gif 格式的图片。

答:

```
// Specify the classes to import.
import java.awt.image.renderable.ParameterBlock;
```

```

import java.io. File;
import javax. media. jai. JAI;
import javax. media. jai. PlanarImage;
import javax. media. jai. RenderedOp;
import javax. media. jai. widget. ScrollingImagePanel;
public class display extends WindowContainer {
    // Specify a default image in case the user fails to specify one at run time.
    public static final String DEFAULT_FILE = ". /images/test. gif" ;
    public static void main( String args[ ] ) {
        String fileName = null;
        // Check for a filename in the argument.
        if( args. length == 0 ) {
            fileName = DEFAULT_FILE;
        } else if( args. length == 1 ) {
            fileName = args[0];
        } else {
            System. out. println( "\nUsage: java " + ( new FileTest( ) ). getClass( ).
                getName( ) + " [file] \n" );
            System. exit(0);
        }
        new FileTest( fileName );
    }
    public void FileTest( ) {}
    public void FileTest( String fileName ) {
        // Read the image from the designated path.
        System. out. println( "Creating operation to load image from'" + fileName + "'" );
        RenderedOp img = JAI. create( "fileload" , fileName );
        // Set display name and layout.
        setTitle( getClass( ). getName( ) + " : " + fileName );
        // Display the image.
        System. out. println( "Displaying image" );
        add( new ScrollingImagePanel( img, img. getWidth( ), img. getHeight( ) ) );
        pack( );
        show( );
    }
}

```

9.42 在 C 语言中并没有定义整数占用的位数,而在 Java 语言中则给出了整数总共 32 位。请讨论这两种方式。

答:

C 语言允许编译器来定义整数占用的比特数,64 位机器的编译器将采用 64 b 的整数,而 32 位机器的编译器将采用 32 b 的整数。Java 语言则与平台无关,在 64 位机器上整数仍然为 32 位。

第十章 网络管理与信息安全

10.1 请概述网络管理的主要功能。

答:

根据 ISO 提出的网络管理模型,网络管理有五大功能,即:

- (1) 配置管理(Configuration Management):管理所有的网络设备,含对各设备的配置参数与设备间关系的管理;
- (2) 故障管理(Fault Management):找出故障的位置并进行恢复;
- (3) 性能管理(Performance Management):监测和统计网络的使用状况,根据网络的使用情况产生报告或采取相应的措施;
- (4) 安全管理(Security Management):限制非法用户窃取或修改网络中的重要数据等;
- (5) 计费管理(Accounting Management):记录用户使用网络资源的数量,调整用户使用网络资源的配额大小和记账收费。

10.2 简述 SNMPv1、SNMPv2、SNMPv3 的特点。

答:

SNMPv1 是最初的版本,通过以下 3 个文档进行定义:

RFC 1155:定义了管理信息结构(SMI),它是描述和命名管理对象的基本体制;

RFC 1212:给出了更精确的描述机制,但是,从总体上而言,与 SMI 是一致的;

RFC 1157:定义了 SNMP 协议本身。

SNMPv1 最大的优点是设计简单。简单化的直接好处是网管容易实施,并且不会对现有的网络增加太大的负荷。用户可以很容易地根据网管需要对网管变量进行编程。SNMPv1 的可扩展性是它的另一个优势。由于它的设计简单,协议容易更新,使得它可以扩展功能以满足用户将来新的管理需求。

SNMPv1 目前已经被广泛使用。几乎所有的网络设备(例如网桥、路由器等)供应商,都将他们的产品设计成可以支持 SNMPv1,使得网络管理容易实现。这也是 SNMPv1 一直未被替代的原因。SNMPv1 也存在一些缺点。首先是安全性较差,使得非法用户可以得到在网络中传递的网管信息,对网络设备进行非法操作。另外,由于 SNMP 太简单,以致于它能够处理的信息既不详细,也没有得到很好的组织,无法满足网络日益发展的管理需要。

SNMPv2 在 RFC 1902 到 RFC 1907 中定义,RFC 1908 定义了 SNMPv1 和 SNMPv2 共存及转换等问题。相对于 SNMPv1,SNMPv2 的改进主要包括如下:

(1) 在 SNMPv2 中增加了一些安全机制,用于对付下列 3 个最大的安全问题。

- ① 数据加密:防止非法用户得到在网上传递的网管信息;
- ② 鉴别权限:防止非法用户在网上传送错误的网管消息;
- ③ 访问控制:限制用户访问一些特殊的变量,以此消除由于用户的误操作引起网络故障的可能性。

(2) SNMPv2 在网管信息方面得到改进,允许更详细的变量描述,包括使用表数据结构以方便数据提取,而且增加了两个新的 PDU 用于对表结构的对象进行操作。SNMPv2 具有以下优点:

- ① 扩展的数据类型:64 位计数器;
- ② 改进的效率与性能:get-bulk 操作;
- ③ 确认的时间通知:inform 操作;
- ④ 更丰富的差错处理:errors 和 exceptions;
- ⑤ 更精细的数据定义语言。

实际上,SNMPv2 增加了许多特性和功能,甚至许多人认为 SNMPv2 失去了 SNMP 原有的简单性,虽然这些改动大部分都是必需的。SNMPv2 使得 SNMP 协议变得复杂了,对于厂家来讲,要编制能够完全满足 SNMPv2 所要求功能的管理进程和代理进程比较困难。SNMPv2 还有待进一步完善,例如,在 SNMPv2 中,曾经采用了现代的加密技术,大大地提高了安全性。但是,这就使得原来就很庞大的协议更加庞大,因此,后来就将它抛弃了。这样一来 SNMPv2 并没有达到原来设计 SNMPv2 时的最初目的:提供所谓的商业级安全和管理机制。

SNMPv3 由 RFC 2271 到 RFC 2275 定义,描述了 SNMPv2 中所缺乏(或者讲不完善)的安全和管理方面的问题。SNMPv3 体现了模块化的设计思想,可以简单地实现功能的增加和修改。其特点是:

适应性强:适用于多种操作环境,既可以管理最简单的网络,实现基本的管理功能,又能够提供强大的网络管理功能,满足复杂网络的管理需求。

扩充性好:可以根据需要增加模块。

安全性好:具有多种安全处理模块。

SNMPv3 主要有 3 个模块:信息处理和控制模块、本地处理模块和用户安全模块。

• 信息处理和控制模块

信息处理和控制模块(Message Processing and Control Model)在 RFC 2272 中定义,它负责信息的产生和分析,并判断信息在传输过程中是否要经过代理服务器等。在信息产生过程中,该模块接收来自调度器(Dispatcher)的 PDU,然后由用户安全模块在信息头中加入安全参数。在分析接收的信息时,先由用户安全模块处理信息头中的安全参数,然后将解包后的 PDU 送给调度器处理。

• 本地处理模块

本地处理模块(Local Processing Model)的功能主要是进行访问控制,处理打包的数据和中断。访问控制是指通过设置代理的有关信息使不同的管理站的管理进程在访问代理时具有不同

的权限,它在 PDU 这一级完成。常用的控制策略有两种:限定管理站可以向代理发出的命令或确定管理站可以访问代理的 MIB 的具体部分。访问控制的策略必须预先设定。SNMPv3 通过使用带有不同参数的原语使用来灵活地确定访问控制方式。

- 用户安全模块

与 SNMPv1 相比,SNMPv3 增加了 3 个新的安全机制:身份验证、加密和访问控制。其中,本地处理模块完成访问控制功能,而用户安全模块(User Security Model)则提供身份验证和数据保密服务。身份验证是指代理(管理站)接到信息时首先必须确认信息是否来自有权的管理站(代理)并且信息在传输过程中未被改变的过程。实现这个功能要求管理站和代理必须共享同一密钥。管理站使用密钥计算验证码(它是信息的函数),然后将其加入信息中,而代理则使用同一密钥从接收的信息中提取出验证码,从而得到信息。加密的过程与身份验证类似,也需要管理站和代理共享同一密钥来实现信息的加密和解密。

10.3 SNMP 消息一般是通过 UDP 而不是 TCP 传递的,为什么采用这种设计?

答:

因为一个管理工作站经常会要管理许多设备,都采用 TCP 会增加管理工作站的开销。同时,由于管理工作站会定期去查询所管理设备的状态,因此一个 UDP 分组的丢失并不一定会带来灾难性的后果。

10.4 简述 CMIP 和 SNMP 的异同。

答:

二者都是网络管理协议,采用面向对象的技术。CMIP 是由各大公司资助的一个复杂而庞大的协议,具有灵活性和强大的功能,安全性更好,但是其实现非常复杂。SNMP 简单易用,易于扩充,但它的安全性不够好。

10.5 CMIP 和 SNMP 都使用面向对象的方法来描述协议,这样做有什么优点?

答:

面向对象的方法为所有被管理的元素提供了一个统一的界面,具有很强的灵活性,实现起来也很方便,可以更好地体现被管理网络设施的特性。

10.6 简述网络安全的多种访问控制策略。

答:

访问控制是网络安全防范和保护的主要策略,它的主要任务是保证网络资源不被非法使用和非常访问。它也是维护网络系统安全、保护网络资源的重要手段。各种安全策略必须相互配合才能真正起到保护作用,但访问控制可以说是保证网络安全最重要的核心策略之一。具体的访问控制策略又可区分为:

(1) 入网访问控制:入网访问控制为网络访问提供了第一层访问控制。它控制哪些用户能够登录到服务器并获取网络资源,控制准许用户入网的时间和准许他们在哪台工作站入网。

(2) 网络的权限控制:网络的权限控制是针对网络非法操作所提出的一种安全保护措施。用户和用户组被赋予一定的权限。网络控制用户和用户组可以访问哪些目录、子目录、文件和其他资源。可以指定用户对这些文件、目录、设备能够执行哪些操作。

(3) 目录级安全控制:网络应允许控制用户对目录、文件、设备的访问。用户在目录一级指定的权限对所有文件和子目录有效,用户还可进一步指定对目录下的子目录和文件的权限。对目录和文件的访问权限一般有八种:系统管理员权限(Supervisor)、读权限(Read)、写权限(Write)、创建权限(Create)、删除权限(Erase)、修改权限(Modify)、文件查找权限(File Scan)、存取控制权限(Access Control)。

(4) 属性安全控制:当用文件、目录和网络设备时,网络系统管理员应给文件、目录等指定访问属性。属性安全控制可以将给定的属性与网络服务器的文件、目录和网络设备联系起来。网络的属性安全控制可以保护重要的目录和文件,防止用户对目录和文件的误删除、执行修改、显示等。

(5) 网络服务器安全控制:网络允许在服务器控制台上执行一系列操作。用户使用控制台可以装载和卸载模块,可以安装和删除软件等操作。网络服务器的安全控制包括可以设置口令锁定服务器控制台,以防止非法用户修改、删除重要信息或破坏数据;可以设定服务器登录时间限制、非法访问者检测和关闭的时间间隔。

(6) 网络监测和锁定控制:网络管理员应对网络实施监控,服务器应记录用户对网络资源的访问,对非法的网络访问,服务器应以图形或文字或声音等形式报警,以引起网络管理员的注意。

(7) 网络端口和节点的安全控制:网络中服务器的端口往往使用自动回呼设备、静默调制解调器加以保护,并以加密的形式来识别节点的身份。自动回呼设备用于防止假冒合法用户,静默调制解调器用以防范黑客的自动拨号程序对计算机进行攻击。网络还常对服务器端和用户端采取控制,用户必须携带证实身份的验证器(如智能卡、磁卡、安全密码发生器)。在对用户的身份进行验证之后,才允许用户进入用户端。然后,用户端和服务器端再进行相互验证。

(8) 防火墙控制:防火墙是确保基础设施安全的一种常用方法。它通过在网络边界上建立起来的相应网络通信监控系统来隔离内部和外部网络,控制进/出两个方向的通信流。它制定一系列规则来准许或拒绝不同类型的通信,并执行所做的路由决策,以阻挡外部的侵入。

10.7 Windows 系统和 UNIX 系统各实现了什么安全策略?

答:

以 Windows 2000 Server 为例来说明 Windows 系统的安全策略。该操作系统的安全服务能识别网络用户并控制它们对资源的访问。操作系统的安全模型使用信任域控制器身份验证、服务之间的信任委派以及基于对象的访问控制。其核心功能包括了与 Windows 2000 Active Directory 服务的集成、支持 Kerberos 版本 5 身份验证协议(用于验证 Windows 2000 用户的身份)、验证外

部用户的身份时使用公钥证书、保护本地数据的加密文件系统 (EFS) 以及使用 Internet 协议安全性 (IPSec) 来支持公共网络上的安全通信。此外,开发人员可在自定义应用程序中使用 Windows 2000 安全性元素。

UNIX 系统经过 40 年左右的发展,已使其自身具备了良好的安全性能,按照可信计算机评价标准达到 C2 级,它具有以下安全特征:

- (1) 访问控制;
- (2) 对象的可用性;
- (3) 个人身份标识与认证;
- (4) 审计记录;
- (5) 操作的可靠性等;

在此我们不再详述。

10.8 实现一个安全的网络(系统),需要从哪些方面去考虑?为什么说一个完善的管理制度是网络安全的最重要的方面之一?

答:

应该从物理安全、硬件、软件安全、管理制度等各个方面去考虑。管理制度是网络安全的一个重要方面。有效的管理是解决内部安全问题的重要方法,同时还可以时刻使系统工作人员保持高度警惕,以便对来自外部的恶意侵害迅速作出反应。

10.9 网络审计的作用是记录网络上发生的事件,以备发生问题(如系统被破坏等)后查询发生事件时的系统状况,试设想网络审计系统有什么功能?

答:

网络审计系统记载用户在网络上的活动细节,特别是对系统的重大事件和与安全有关的事件进行记录,以备查找系统的漏洞和查明事件原因。

10.10 概述病毒的原理及防范方法。

答:

病毒是一个附着于可执行文件之上,并执行原可执行文件设计以外的动作的指令集。PC 机中,病毒常附着于扩展名为 .EXE 或 .COM 的文件之上。当病毒附着于文件之上时,我们就说该文件被感染 (Infect) 了。未感染文件包含可执行代码,在被调用时运行。而在被病毒感染后的文件中,病毒放置了一条分支命令,指向病毒代码。当用户调用感染病毒后的文件执行某些任务时,分支命令将控制权先移交给病毒代码。病毒通常在执行它的操作后执行另一个分支回到开始请求的任务。就用户而言,请求的任务完成了。除非病毒执行如删除硬盘的明显操作,否则用户可能不知道病毒的存在。蠕虫 (Worm) 也是一种病毒,但它们经常以独立的程序出现。与病毒类似,它们是对系统的入侵,是对系统安全的潜在威胁。病毒亦可将自己拷贝到内存并等待一

个可执行文件存储到内存中。当文件进入时,病毒就攻击它。这就是内存常驻病毒。

反病毒程序通过查找病毒签名(Signature)来工作,病毒签名是包含于病毒中的二进制序列(相当于机器语言指令)。病毒程序开始认识到为了成功,它们必须隐藏病毒的签名,或至少将它伪装使之难以辨认。加密是他们开始使用的工具。通过将病毒代码经过一个加密算法(即使是简单的密码),它的签名就改变了,反病毒软件就不能将它检测出来。更有多态病毒使用一种免疫引擎,每感染一个文件,解密算法就改变代码。任何有经验的程序员都知道,完成指定的任务可有很多种代码生成方法。多态病毒正好利用了这个事实。所以,多态病毒的每个拷贝不仅用不同的密钥加密,也用不同的解密例程解密,以对付查找病毒或解密算法的特殊二进制序列特征的反病毒程序。反病毒程序在分析免疫引擎产生的比特串和检测模式方面,获得了一些成功。而且,任何计算机病毒最终都必须解密并将自己暴露出来方可执行。这是成功对付多态病毒的关键。现在的一些病毒检测技术使用一种叫做通用解密(Generic Decryption, GD)的技术。在一定程度上,它是一种愚弄病毒,使病毒在有机会破坏之前暴露自己的技术。更准确地说,GD反病毒软件包含一个CPU仿真器。检查文件时,它执行一个模拟该文件执行的软件。若文件有多态病毒,它就会将自己解密,它的签名模式就会暴露。通过周期性地触发签名搜索例程,病毒就可检测出来。即使在检测出来之前病毒被掩盖和执行了,它只是个模拟,并不造成实际伤害。病毒和防范者之间的战斗仍在继续。

10.11 对下列密码进行解密,明文是从一计算机教科书中摘录而成,所以,可能出现诸如“computer”之类的词汇,明文只包含字母,没有空格。加密方法是变位(Transposition)法,为了阅读方便,密文被分成5个字母一组。

aaauan cvlre runnn dlme aeepb ytust iceat nrmey iicgo

gorch srsoc nntii imiha oofpa gsivt tpsit lbolr otoex

答:

a digital computer is a machine that can solve problems for people by carrying out instructions given to it

10.12 某个黑客采用已知明文攻击单字母表(26个英文字母)加密算法,通过尝试各种不同的映射组合来解密,假设它已经知道了7个字母的映射关系,则需要检查的映射组合减少的倍数约 10^9 ,请解释。

答:

由于已经知道7个字母映射,因此只剩下19个字母,可能的组合为 $19!$;而在不知道7个字母映射时可能的组合为 $26!$ 。因此,减少的倍数为 $\frac{26!}{19!} \approx 10^9$ 。

10.13 简述数据加密的历史。

答:

作为保障数据安全的一种方式,数据加密起源于公元前 2000 年。埃及人是最先使用特别的象形文字作为信息编码的人。随着时间推移,巴比伦、美索不达米亚和希腊文明都开始使用一些方法来保护他们的书面信息。早在 Julius Caesar(凯撒大帝)时代就使用了替换密码,在历次战争中,包括美国独立战争、美国内战和两次世界大战,都广泛使用了密码。最广为人知的编码机器是 German Enigma 机,在第二次世界大战中德国人利用它创建了加密信息。此后,由于 Alan Turing 和 Ultra 计划以及其他人的努力,终于对德国人的密码进行了破解。还有变位(transposition)加密算法,都是传统加密算法。现代加密算法则可以分为私密密钥算法(或称为“对称密钥算法”)和公开密钥算法(或称为“不对称密钥算法”)。

10.14 DES 算法中,若 $K_1 = 000000000000$, $R_0 = D8D8DBBC$, 求 $S(K_1 \wedge E(R_0))$ 。

答:

按 DES 算法计算可得

$$S(K_1 \wedge E(R_0)) = 5BB997F3。$$

10.15 DES 算法是一种 64 位的分组加密算法,当要加密的消息超过 64 位时可以采用 ECB 和 CBC 模式,请给出两种方法的基本原理。

答:

ECB 方法把加密的消息以 64 b 为单位组成多个块,每个块再利用 DES 算法进行加密,不同块之间的加密是完全独立的。这种方法很容易受到攻击,攻击者可以把其中的某块替换,而接收者可能无法觉察到。

而 CBC 方法也是把加密的消息以 64 b 为单位组成多个块,第一个块与一个初始向量 IV 进行异或后再用 DES 算法进行加密,第二块的明文首先与第一块加密后的密文进行异或后再用 DES 算法进行加密,如此继续。这种方法块与块之间的加密是有关系的,因此如果途中某一块被替换,会导致后面的块也无法解密出来。

10.16 RSA 算法中,设 $p = 11$, $q = 23$, 则 $n =$ _____, $z =$ _____, 若取 $d = 49$, 则最小的 $e =$ _____。

答:

$n = p \times q = 253$, $z = (p - 1) \times (q - 1) = 220$, 若取 $d = 49$, 要求 $49e \text{ MOD } 220 = 1$, 可以选择最小的 $e = 9$ 。

10.17 RSA 算法中,设 $p = 5$, $q = 11$ 。

1) 求 n 和 z ;

2) 若取 $d = 27$, 则最小的 e 是多少?

3) 此时, 对应于明文 $P=11$ 的密文 C 是多少? (都为十进制数)

答:

1) $n=55, z=40$;

2) 若取 $d=27$, 对于 e , 要求 $27e \text{ MOD } 40 = 1$, 可以选择最小的 $e=3$;

3) 对应于明文 $P=11$ 的密文 $C = 11^3 \text{ MOD } 55 = 11$ 。

10.18 对称密钥密码体系和公开密钥密码体系的基本原理各是什么?

答:

按照收发双方密钥是否相同来分类, 可以将密码体系分为对称密钥密码体系和公开密钥密码体系。

在对称密钥密码体系中, 收信方和发信方使用相同的密钥, 即加密密钥和解密密钥是相同或等价的。比较著名的对称密钥密码算法有: 美国的 DES 及其各种变形, 比如 Triple DES、GDES、New DES 和 DES 的前身 Lucifer; 欧洲的 IDEA; 日本的 FEAL-N、LOKI 91、Skipjack、RC4、RC5 以及以替换密码和变位密码为代表的古典密码等。在众多的对称密钥密码算法中影响最大的是 DES 算法。对称密钥密码算法的优点是有很高的保密强度, 且经受住时间的检验和攻击, 但其密钥必须通过安全的途径传送。因此, 其密钥管理成为系统安全的重要因素。

在公开密钥密码体系中, 收信方和发信方使用的密钥互不相同, 而且几乎不可能从加密密钥推导解密密钥, 或者反之。比较著名的公开密钥算法有: RSA、背包密码、McEliece 密码、Diffie-Hellman、Rabin、Ong-Fiat-Shamir、零知识证明的算法、椭圆曲线、ElGamal 算法等。最有影响的公开密钥算法是 RSA 算法。公开密钥密码算法的优点是可以适应网络的开放性要求, 且密钥管理问题也较为简单, 尤其是可方便地实现数字签名和验证。但其算法复杂, 加密数据的速度较慢。

10.19 假设在一个安全系统中总共有 N 个节点, 这些节点之间都会要进行通信, 并且任何节点都可以看到别的节点传输的数据流, 为此需要对通信进行加密。如果采用公开密钥算法, 则至少需要多少个密钥对, 而如果采用共享密钥算法, 则至少需要多少个密钥对。

答:

采用公开密钥算法, 发送者用对方的公开密钥对消息进行加密, 这样只有拥有该私人密钥的用户才能解开。因此只需要 N 个密钥对就可以了。而采用共享密钥算法, 任意两个节点之间的通信都必须采取完全不同的密钥, 因此总共需要的密钥对至少为 $N(N-1)/2$ 。

10.20 常见的身份鉴别技术有哪些? 它们各有什么特征?

答:

常见的身份鉴别技术有:

(1) 只有该主体了解的秘密, 如口令、密钥。口令是相互约定的代码, 假设只有用户和系统

知道。数字签名和数字证书则属于利用密码技术的身份鉴别方法。

(2) 主体携带的物品,如智能卡(Smart Card)。

(3) 只有该主体具有的独一无二的特征或能力,如指纹、声音、视网膜或签字等。利用个人特征进行鉴别的方式具有较高的安全性。

10.21 试述数字签名的基本原理。

答:

基本原理是:对传输的明文用摘要函数进行摘要,然后将摘要用自己的私钥进行加密,最后把明文和加密后的摘要一起发送。对方接收后,解密取得摘要,同时计算另一个摘要。二者比较,从而可以确定信息在传输的过程中是否发生了变化。

10.22 请估计总共 N 个消息,其中有两个消息具有相同的 MD5 消息文摘的概率是多少?

假设 $N = 2^{63}, 2^{64}, 2^{65}$ 。提示:利用 $\log(1 - \frac{k}{2^{128}}) \approx \frac{-k}{2^{128}}$ 来进行估计。

答:

MD5 文摘为 128 b,我们定义 P_N 为 N 个消息中 MD5 文摘各不相同的概率,则有:

$$P_N = (1 - \frac{1}{2^{128}})(1 - \frac{2}{2^{128}}) \cdots (1 - \frac{N-1}{2^{128}})$$

对两边取对数,并利用 $\log(1 - \frac{k}{2^{128}}) \approx \frac{-k}{2^{128}}$ 有:

$$\begin{aligned} \log P_N &\approx -(\frac{1}{2^{128}} + \frac{2}{2^{128}} + \cdots + \frac{N-1}{2^{128}}) = -(1 + 2 + \cdots + (N-1))/2^{128} \\ &\approx \frac{-N^2}{2^{129}} \end{aligned}$$

因此有: $P_N \approx e^{\frac{-N^2}{2^{129}}}$, 分别取 $N = 2^{63}, 2^{64}, 2^{65}$:

$$P_{2^{63}} = e^{-1/8} \approx 0.8825$$

$$P_{2^{64}} = e^{-1/2} \approx 0.6065$$

$$P_{2^{65}} = e^{-2} \approx 0.1353$$

这样当 $N = 2^{63}, 2^{64}, 2^{65}$ 时有两个消息相同的概率为 $1 - P_N$, 分别为 0.1175、0.3935、0.8647。

10.23 简述数字证书的工作流程。

答:

首先向证书认证中心(CA)取得证书,证书包括了电子签证机关的信息、公钥用户信息、公钥、权威机构的签字和有效期等。

甲想和乙通信时,甲首先取得乙的数字证书,然后对乙进行认证。

在数据传输过程中,为了保证信息传送的真实性、完整性和不可否认性,需要对要传送的信息进行数字加密和数字签名。

10.24 防火墙有哪些基本组件?

答:

防火墙的主要组件有网络策略(Network Policy),高级认证机制(Advanced Authentication Mechanisms),数据包过滤(Packet Filtering)和应用网关(Application Gateway)等。

10.25 简述主要的防火墙类型及其特征。

答:

一般采用的防火墙类型主要包括:

(1) 数据包过滤防火墙(Packet Filtering Firewall),在小的站点上应用得最普遍,也最容易实现。

(2) 双宿网关防火墙(Dual-homed Gateway Firewall),双宿网关防火墙是更好的选择。它由一个带两块网络接口卡的主机系统组成,并使主机的IP转发功能禁用(即在缺省条件下主机不再为两个相连网络的数据包寻径),它是一个应用网关。

(3) 屏蔽主机防火墙(Screened Host Firewall),屏蔽主机防火墙是比双宿网关更灵活的防火墙,但牺牲了一些安全性,适用于需要较多灵活性的内部网络。

(4) 屏蔽子网防火墙(Screened Subnet Firewall),是双宿网关防火墙和屏蔽主机防火墙的变形。它将防火墙的各组件放置在分离的系统上,因而获得更大的吞吐量和灵活性。每个防火墙组件系统只需实现某一特定任务,减少了配置的复杂度。

10.26 SSL 协议是一种比较有效的安全协议,设想在 Web 浏览中如何使用 SSL 协议?

答:

SSL 安全协议可以利用数字证书在寻求安全通信的双方之间建立安全的“管道”。多数主要 Web 浏览器和商用 Web 服务器中都使用 SSL。如果一位客户打算与一个采用 SSL 加密的 Web 站点建立连接的话,他的浏览器向这台 Web 服务器发出“客户机呼叫”信息,请求一次 SSL 加密会话。这台 Web 服务器通过向客户发送服务器的证书进行答复。客户的浏览器将验证服务器的证书是否有效,是否由一个可信赖的 CA 所签署。这一确认两个实体打算建立一次安全的 SSL 连接的过程被称为 SSL“握手”。

为了启动这次握手,客户的浏览器将生成一个用服务器公共密钥进行加密的特殊的一次性会话密钥,并向服务器发送这个加了密的会话密钥。服务器利用私有密钥解密收到的信息,恢复出会话密钥。这种交换证实了 Web 站点的身份,保证了只有这个浏览器和这台 Web 服务器才有这个会话密钥。然后,Web 服务器使用这个会话密钥向客户发送加密的信息。

第十一章 多媒体网络

11.1 网络多媒体应用可以分为哪些类型,请简要比较一下这些不同类型的应用的不同点。

答:

网络多媒体应用可以分为现场流媒体、点播流媒体和交互式会议应用三种。点播多媒体应用中多媒体信息预先录制并保存在媒体服务器上,客户向媒体服务器发送请求,要求进行点播,在接收了一小部分信息后就开始进行播放,在播放的同时多媒体信息继续通过网络传输。同时也支持用户交互功能,用户可以进行暂停、回退、快进等。从用户发出播放请求到实际播放之间的间隔大约在 1~5 s 范围内都是可以接受的。

现场流媒体应用中,多媒体信息是实时采集和压缩的,然后通过网络传输出去,一般有许多用户在同时收听或者收看,音频和视频信息的分发可以通过多个不同的单播来进行,也可以通过更加高效的组播方式来完成,由于是实时录制,一般不支持用户交互功能。从媒体服务器发出声音或者视频信号到接收者开始播放之间的时延在大概 10 s 或者更多一些都可以接受。

交互式多媒体应用中,用户可以通过音频和视频信息的传输来进行相互通信,与会者在任何时候都可能说话或者移动。与会者说话或者移动的动作到达接收者的时延应该小于几百毫秒才能为用户接受。人的听觉系统对延迟小于 150 ms 的声音感觉不到有时延,在 150~400 ms 之间的时延可以接受,而时延超过 400 ms 的会话就会令人感觉异常。

11.2 请问逐行扫描和隔行扫描有什么不同,计算机显示器是采用哪种扫描方式?为什么?

答:

在逐行扫描中,电子束从显示屏的左上角一行接一行地扫到右下角,在显示屏上扫一遍就显示一幅完整的图像。而在隔行扫描中,电子束扫完第 1 行后回到第 3 行开始的位置接着扫,然后再在第 5、7 等奇数行上扫,奇数行扫完后接着扫偶数行,这样就完成了一帧的扫描。计算机显示器一般都采用逐行扫描,这是因为隔行扫描主要用于消除闪烁,但是显示器一般都可以以 75 次/秒或更高的速率显示,这样不需要采用隔行扫描的方法,而是可以通过连续 3 次以上重画保存在内存中的帧来消除闪烁现象。

11.3 假设一个信息源生成的符号来自于字母集 {a,b,c},假设每个字母出现的概率都相等。

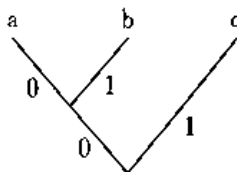
(1) 如果每个字母代表了某个信息,请给出相应的霍夫曼编码。

(2) 如果每 2 个字母代表了某个信息,请给出相应的霍夫曼编码。

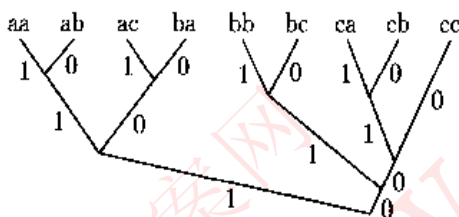
答:

(1) 每个符号用一个字母来表示,每个字母出现的概率为 $1/3$,这样霍夫曼编码为:

a	00
b	01
c	1



(2) 每个符号用两个字母表示, 包括 $\{aa, ab, ac, ba, bb, bc, ca, cb, cc\}$, 每个符号出现的概率为 $1/3 \times 1/3 = 1/9$, 这样霍夫曼编码为:

$$\{111, 110, 101, 100, 011, 010, 0011, 0010, 000\}$$


注意问题(1)和(2)的答案并不只有这两种。

11.4 假设一个源生成的二进制串中每个比特 b_i 是相互独立的, 其中 $P[b_i = 0] = p, P[b_i = 1] = 1 - p$ 。假设 n 代表在 1 出现前的连续的 0 的个数。请问:

(1) n 等于 i 的概率是多少, 其中 $i=0, 1, 2, \dots$

(2) 假设 L 为一个正整数, 请问 n 大于 L 的概率是多少?

答:

(1) n 等于 i 即前面出现了 i 个 0, 然后紧跟一个 1, 因此:

$$P\{n=i\} = p^i(1-p)$$

(2) n 大于 L 相当于 1 前面出现的 0 的个数大于 L , 也就是说至少出现了 $L+1$ 个 0, 因此:

$$P\{n > L\} = p^{L+1}$$

11.5 假设一个基本 PCM 音频编码的采样频率为 44 kHz, 每个采样用 16 位编码, 请问 20 s 的音频文件的大小是多少?

答：

一个采样频率为 44 kHz 的音频编码的数据速率为

$$44k \frac{\text{samples}}{s} \times 16 \frac{b}{\text{samples}} = 704 \text{ kb/s}$$

20 s 的音频文件大小为 $704 \times 20 \text{ kb} = 14\,080 \text{ kb}$ 。

11.6 RTP 是一种运输协议吗？为什么？

答：

RTP 不是一种运输协议，这是因为各种网络多媒体应用的需求各不相同，很难独立出一个完全的层次来处理多媒体信息的传输，比如有的应用多媒体分组丢失时可能采用重传，而有的应用则不需要进行重传，而是由接收者来恢复。这样 RTP 只是把多媒体信息传输的一些通用的功能抽象出来，定义了一个基本的框架结构，而大多数的协议细节在开发具体的多媒体应用时再定义，这一般是在相应的脚本和格式文档中定义的。从多媒体应用的开发者的角度来看，开发者必须把 RTP 集成到自己的应用中。比如在发送多媒体信息时，首先把 RTP 数据封装进 RTP 报文中，然后通过 UDP 的套接字接口把 RTP 报文传送出去，而接收方从套接字中接收 UDP 报文，然后解析其中封装的 RTP 报文。

11.7 请问 SIP 是否能够用于下列情形，如果能，请描述大致的过程。

(1) 呼叫显示：被呼者应答之前屏幕上显示呼叫者的号码或者名字。

(2) 呼叫回答：如果被呼者正忙或者响铃一段时间后，网络通知呼叫者并且允许呼叫者语音留言。

答：

(1) 能。被呼者处有一个 UAS 来处理到来的呼叫，在收到 SIP INVITE 请求之后，该 INVITE 请求中包括了呼叫者的 SIP 标识，从而 UAS 可以把呼叫者的号码或者名字显示出来，提醒用户谁正在进行呼叫。

(2) 能。用户首先通过 DNS 查询到被呼者所在的 SIP 服务器，然后向其发送 INVITE 请求，SIP 服务器收到该请求后会该请求转发给被呼者，如果一定时间收不到应答，或者收到一个忙响应，SIP 服务器可以把 INVITE 请求转发给语音服务器，从而允许呼叫者进行语音留言。

11.8 大学里面的一个教授有 3 个办公地点，分别是教授办公室、实验室和负责的研究中心办公室。

(1) 请问如果采用 SIP 提供的 fork 功能，一个 Internet 电话呼叫如何到达教授？

(2) 假设教授在外参加一个活动，并且也可以使用 Internet 电话，请描述怎样通过 SIP 的 REGISTER 消息来把呼叫转向教授的当前位置。

答：

(1) 教授在 3 个办公地点都通过相应的多媒体软件注册了所在的位置，一个 Internet 呼叫首先到达大学的 SIP 服务器，SIP 服务器收到该 INVITE 请求之后，同时把该请求转发给前面注册过的 3 个办公地点，一旦教授在其中一个办公地点接受了该呼叫，发送了一个成功的响应，SIP 服务器收到该响应后转发给呼叫者，从而可以与教授进行进一步的交谈。

(2) 教授外出到达新的地点后，首先教授使用的多媒体软件通过发送 SIP REGISTER 消息给大学的 SIP 位置服务器，这样位置服务器就知道了教授的当前位置。一个用户在呼叫教授时

首先发送 INVITE 请求给大学的 SIP 服务器, SIP 服务器查询位置服务知道教授目前在外面, 因此发送一个 302 消息把教授的当前 SIP 位置告诉呼叫者, 呼叫者然后与教授当前所在的 SIP 服务器或者直接与教授的 SIP 终端进行通信来建立呼叫。

11.9 RTP 时间戳时钟可以有不同的精度, 如果采用每视频帧加 1 或者每个音频采样加 1, 显然这是为了能够保证准确回播而可能采用的最低时钟精度。但是实际环境下很少采用这种方法, 为什么?

答:

这是为了能够更加精确地测量 jitter, 而 jitter 可以用于指示网络早期出现的暂时拥塞, 同时也可以用于进一步调整播放缓冲区的大小。

11.10 假设接收者发送的 RTCP 负载的带宽不能超过 RTP 流所占带宽的 5%, 如果每个 RTCP 消息长 84 B, RTP 负载带宽为 20 kb/s, 总共有 1 000 个接收者, 请问接收者发送 RTCP 报告的频率是多少? 如果有 10 000 个接收者呢?

答:

总共有 1 000 个接收者, 因此每个接收者的 RTCP 分组占用的带宽为

$$\frac{1}{1\,000} \times 5\% \times 20\text{ kb/s} = 1\text{ B/s}$$

而 RTCP 消息长 84 B, 因此接收者发送 RTCP 报告的频率为每 84 s 一个 RTCP 分组。如果有 10 000 个接收者, 则频率为每 840 s 一个分组, 即每 14 min 一个分组。

11.11 H.323 系统中网关和网守分别完成什么功能?

答:

网关主要用于 H.323 终端要与别的系统的终端进行通话时进行呼叫信令和多媒体分组的转换, 比如与传统的 PSTN 电话通信。

而网守则用于管理 H.323 网络, 一旦一个 H.323 网络有网守, 则该网络的所有 H.323 设备都必须通过 RAS 信道进行注册, 这样网守支持地址翻译机制、准入控制、带宽管理和区域管理等, 除此之外网守还支持一些高级的功能, 包括支持采用网守路由的呼叫信令, 从而提供了呼叫授权、账目、账单和付费、负载均衡、呼叫转发等。

11.12 请问流式音频或者视频更合适用 TCP 还是 UDP 来传输, 为什么?

答:

多媒体分组更加适合采用 UDP 来传输。这是因为多媒体信息对时间比较敏感, 要求分组能够及时到达接收者, 并且允许少量的分组丢失。而 TCP 提供了数据的可靠传输, 但是这些超时重传等可靠性机制并不一定需要, 同时超时重传也会导致多媒体分组到达接收者的时间较长, 可

能由于已经超过播放时序而丢弃,同时 TCP 在遇到网络拥塞时会减少发送的速率,而这对于多媒体信息传输可能并不是非常合适,因为多媒体信息要求一定的数据速率,而且 TCP 连接建立初始要进入慢启动阶段,这个时候的数据速率较小,这对于多媒体的传输不是很合适。

11.13 本章介绍了三种通过 Internet 访问音频或视频文件的方法,请描述每种方法的优点和缺点。

答:

第一种方法通过 HTTP 协议直接访问相应的多媒体文件,这样只有在文件全部下载到本地之后才能开启相应的媒体播放器进行播放。

第二种方法也是通过 HTTP 协议来访问,不过现在访问的是描述了多媒体文件所在位置以及相应参数的元文件,这样获得元文件后开启相应的媒体播放器,然后媒体播放器通过 HTTP 协议来获得多媒体文件,在下载的过程中就可以进行播放。这种方法不需要像第一种方法那样必须首先全部下载多媒体文件后进行播放,而是采用流媒体技术进行播放,但是多媒体信息的传递是通过 HTTP 协议的,我们知道 HTTP 协议是在 TCP 之上实现的,而多媒体信息具有时间敏感和容许少量分组丢失的特性,并不是特别适合采用 TCP 进行传输。

第三种方法则和第二种方法类似,可以通过 HTTP 协议访问元文件,但是媒体播放器不再与 HTTP 服务器进行通信,而是与一个媒体服务器进行通信,从而多媒体信息的传输可以采取合适的协议比如 RTP 来进行传输。

11.14 端到端延迟和延迟抖动有什么不同? 延迟抖动是什么原因导致的?

答:

端到端的延迟指的是分组通过网络从发送者到达接收者途中经历的时间,而延迟抖动指的是端到端延迟的变化情况。由于网络的状况是动态变化的,不同的分组可能经过不同的路由到达接收者,不同的分组在每个路由器处的排队延迟可能各不相同,从而导致分组经历的端到端的延迟也可能各不相同。

11.15 请问在教材图 11.15 的 H.323 终端协议栈中,哪些 H.323 信道是运行在 TCP 上,又有哪些是运行在 UDP 上的。

答:

音频、视频和 H.225 RAS 分组采用不可靠的 UDP 协议,而 H.245 控制信令信道和 H.225.0 呼叫信令信道则采用可靠的 TCP 协议来传输。

11.16 RTSP 和 HTTP 有什么共同点? HTTP 协议可以用于请求播放媒体流吗?

答:

实时传输流协议 RTSP 吸纳了 HTTP 协议的设计思想,是一种控制一个或者多个时间同步

的实时音频、视频等连续媒体流的递交的客户/服务方协议。RTSP 采用和 HTTP 协议类似的语法,包括请求和响应两种类型的文本消息,支持多种方法。HTTP 协议可以用于请求播放媒体流,通过 HTTP 协议获得元文件,然后开启相应的媒体播放器进行播放。

11.17 RTSP 和 HTTP 有什么不同之处?

答:

HTTP 是一种无状态的协议,而 RTSP 引入了会话标识的概念,当用户请求服务器上的某个多媒体表演时,RTSP 服务器返回一个会话标识,以后用户发送的 RTSP 请求都包含该会话标识。因此会话标识代表了客户方和服务方共享的状态。和 HTTP 协议中只有服务方可以发送请求不同,RTSP 也允许服务器发送请求,同时 RTSP 也提供了相应的协议扩展机制,允许加入新的请求类型。RTSP 是一种带外协议,即 RTSP 消息和多媒体数据是通过不同的协议和信道来传递的。

第十二章 服务质量控制 QoS

12.1 课本中的 Max-Min 算法假设所有流都是平等的,如果每个流具有不同的权重,请给出相应的 Max-Min 算法描述。

答:

考虑每个流具有不同的权重,流 n 的权重为 w_n 。这个时候 Max-Min 算法可以这样修改:

考虑有 N 个流,流 n 要求的资源为 r_n ,首先把每个流要求的资源按照权重进行归一,即得到每个流的加权资源要求 x_n 。

$$x_n = \frac{w_n}{\sum_{i=1}^N w_i} r_n$$

然后按照所要求的加权资源按序排列,假设 $x_1 \leq x_2 \leq \dots \leq x_N$,资源的总量为 C ,用 m_n 表示实际分配给流 n 的资源。首先根据剩下的资源按照权重来分配的方法计算,用 M_n 表示可以分配给流 n 的资源数,显然

$$M_n = \frac{w_n}{\sum_{i \in \text{尚未满足需求的流}} w_i} \times \text{剩余资源数} = \frac{w_n}{\sum_{i \in \text{尚未满足需求的流}} w_i} (C - \sum_{i=1}^{n-1} m_i)$$

而分配给流 n 的资源不能超过其实际要求的资源数,因此

$$m_n = \min(x_n, M_n) \quad \text{其中 } 1 \leq n \leq N$$

12.2 一个采用 PGPS 的路由器有 3 个输入流和一个输出端口,每个流的服务速率为每秒 1 个数据单元,假设队列 1:一个长度为 2 的分组在时刻 0 到达,一个长度为 1 的分组在时刻 4 到达;队列 2:一个长度为 3 的分组在时刻 1 到达,一个长度为 1 的分组在时刻 2 到达;队列 3:一个长度为 5 的分组在时刻 3 到达。请问分组传输的顺序是什么?

答:

如表 12.1 所示:我们用 P_{ij} 表示第 i 个队列在时刻 $t=j$ 到达的分组,比如 P_{14} 表示队列 1 到达的第 2 个分组。

因此分组传输的顺序依次是:队列 1 在时刻 0 到达的分组、队列 2 在时刻 1 到达的分组、队列 1 在时刻 4 到达的分组、队列 2 在时刻 2 到达的分组、队列 3 在时刻 3 到达的分组。

表 12.1

时刻	事件	虚拟时间 $V(t)$	发送	Finish Tag	队列 1	队列 2	队列 3
$t=0$	P10 到达	$V(0)=0$	发送 P10	$0+2=2$	P10		
$t=1$	P21 到达	$V(1)=1$		$1+3=4$	P10	P21	
$t=2$	P10 完成 P22 到达	$V(2)=1+1/2=1.5$	发送 P21	$4+1=5$		P21P22	
$t=3$	P33 到达	$V(3)=1.5+1=2.5$		$2.5+5=7.5$		P21P22	P33
$t=4$	P14 到达 P21 完成	$V(4)=2.5+1/2=3$	发送 P14	$3+1=4$	P14	P22	P33
$t=5$	P14 完成	$V(5)=3+1/3=10/3$	发送 P22			P22	P33
$t=6$	P22 完成	$V(6)=10/3+1/2$	发送 P33				P33

12.3 假设队列 1、2、3 的权重分别为 2、3、5,其余与问题 12.2 的假设相同。

答:

与 12.2 类似,时序如表 12.2 所示。

表 12.2

时刻	事件	虚拟时间 $V(t)$	发送	Finish Tag	队列 1	队列 2	队列 3
$t=0$	P10 到达	$V(0)=0$	发送 P10	$0+2/2=1$	P10		
$t=1$	P21 到达	$V(1)=0+1/2=0.5$		$0.5+3/3=1.5$	P10	P21	
$t=2$	P10 完成 P22 到达	$V(2)=0.5+1/5=0.7$	发送 P21	$1.5+1/3=1.83$		P21P22	
$t=3$	P33 到达	$V(3)=0.7+1/3=1.0$		$1.0+5/5=2.0$		P21P22	P33
$t=4$	P14 到达 P21 完成	$V(4)=1.0+1/8=1.125$	发送 P14	$1.125+1/2=1.625$	P14	P22	P33
$t=5$	P14 完成		发送 P22			P22	P33
$t=6$	P22 完成		发送 P33				P33

因此分组传输的顺序依次是:队列 1 在时刻 0 到达的分组、队列 2 在时刻 1 到达的分组、队列 1 在时刻 4 到达的分组、队列 2 在时刻 2 到达的分组、队列 3 在时刻 3 到达的分组。

12.4 传统的 Internet 路由器当缓冲区用完时采用 Drop-tail 丢弃策略,即最近到来的分组被

优先丢弃。考虑一个路由器的丢弃策略是当队列满时丢弃具有最高花费的分组,而分组的花费定义为分组大小与该分组剩余的排队时间的乘积。

- (1) 请问这种丢弃策略的优缺点是什么?
- (2) 这种丢弃策略并不是丢弃那些最大的分组,请给出一个例子来说明;
- (3) 给出一个例子来说明两个分组的花费的大小关系可能随着时间的进行而变为相反。

答:

(1) 这种丢弃策略的好处是丢弃的分组是那些占用的资源比较多的分组,而其缺点是每次队列前进时都需要重新计算花费。

(2) 假设队列有 3 个分组,大小分别为 5、15、5。某个分组在队列中剩余的时间用它前面的分组的大小之和来描述,这样第 3 个分组的花费为 $5 \times 20 = 100$,第 2 个分组的花费为 $15 \times 5 = 75$,因此第二个分组虽然更大,但是其花费要比第 3 个分组小。

(3) 和(2)类似,分组的花费通过分组的大小来度量,即假设需要 1 个时间单位来传输 1 个大小的分组。假设一个大小为 3 的分组在 $T=0$ 到达,该分组将在 $T=5$ 时刻到达队列头部发送。此时一个大小为 1 的分组随后到达。

在 $T=0$,这两个分组的花费分别为 $3 \times 5 = 15$ 和 $1 \times 8 = 8$;

在 $T=3$,花费分别为 $3 \times 2 = 6$ 和 $1 \times 5 = 5$;

在 $T=4$,花费分别为 $3 \times 1 = 3$ 和 $1 \times 4 = 4$;

在 $T=5$,花费分别为 0 和 3。

我们可以看到在 $T=4$ 时花费的大小关系正好相反。

12.5 在 RED 路由器中,为什么 MaxThreshold 设置为小于缓冲池的实际容量?

答:

根据 RED 算法,只有在平均队列长度大于 MaxThreshold 时,新到来的分组丢弃的概率为 1。如果平均队列长度小于 MaxThreshold,到来的分组的丢包概率不等于 1,也就是说尽管此时实际的队列长度超过 MaxThreshold,但是平均队列长度可能还是小于 MaxThreshold,因此新到来的分组可能要进行排队,这样就要求 MaxThreshold 设置为小于缓冲池的实际容量。

12.6 考虑一个 RED 路由器,其中 $\text{MaxP} = 0.02$,队列平均长度正好位于两个阈值的正中间。

- (1) 请问当 count 分别等于 1 和 50 的时候的分组丢弃概率是多少?
- (2) 请给出前面 50 个分组都没有被丢弃的概率。

答:

(1)

$$\text{TempP} = \frac{\text{AvgLen} - \text{MinThresh}}{\text{MaxThresh} - \text{MinThresh}} \times \text{MaxP}$$

平均队列长度正好位于两个阈值的正中间,这样 $\text{Temp}P = \text{Max}P/2 = 0.01$ 。

而分组丢弃概率 $P_{\text{count}} = \frac{\text{Temp}P}{1 - \text{count} \times \text{Temp}P} = \frac{1}{100 - \text{count}}$, 这样 count 等于 1 时的丢弃概率为 $1/99$, count 等于 50 的时候丢弃概率为 $1/50$ 。

(2) 前面 50 个分组都没有丢弃的概率 $P = (1 - P_1)(1 - P_2) \times \dots \times (1 - P_{50})$, 而 $P_{\text{count}} = \frac{1}{100 - \text{count}}$, 这样 $P = \frac{98}{99} \times \frac{97}{98} \times \dots \times \frac{50}{51} \times \frac{49}{50} = \frac{49}{99} \approx 0.495$

12.7 考虑这样一个网络, 其中每条链路的容量都超过网络中所有端系统的输入速率之和。请问:

(1) 拥塞控制机制需要吗? 为什么?

(2) 流量控制机制需要吗? 为什么?

答:

(1) 因为每个队列的输入速率和要小于传输速率, 因此不会出现排队, 无需拥塞控制。实际上可能会需要排队, 但是非常短, 这是考虑到在某个时刻有突发数据到来。

(2) 流量控制仍然需要, 因为接收者可能来不及处理发送者发送的数据。

12.8 一个 ATM 网络采用令牌桶算法进行通信量整形, 每 $5 \mu\text{s}$ 往桶中加入一个新令牌。最大可持续网络数据速率 (即包括头部比特在内) 是多少?

答:

每个信元大小为 53 B, 所以最大可持续数据速率为

$$\frac{53 \times 8}{5 \times 10^{-6}} = 84.8 \text{ Mb/s}$$

12.9 一个 6 Mb/s 的网络中有一台由令牌桶算法控制的计算机。令牌桶以 1 Mb/s 的速率注入, 其容量为 8 Mb, 最初令牌桶是满的。问该计算机能以 6 Mb/s 的速率全速传送多长时间?

答:

$$M = 6 \text{ Mb/s}, C = 8 \text{ Mb}, p = 1 \text{ Mb/s}$$

$$S = \frac{C}{M - p} = 1.6 \text{ s}$$

所以该计算机能以 6 Mb/s 的速率全速传送 1.6 s。

12.10 请问令牌桶相比漏桶算法有什么优点?

答:

令牌桶更加灵活,因为它允许一定的突发数据,只要有足够的令牌,数据就能很快地发送出去;另外一个好处是令牌桶当桶满时丢弃令牌但是不丢弃分组。

12.11 漏桶可以用于限制分组流的峰值速率和突发长度,请问如何通过采用两个漏桶来限制分组流的平均速率?

答:

假设分组流的峰值速率为 p ,突发长度为 b ,平均速率为 r ,则第一个漏桶允许漏出的速率为峰值速率 p ,桶的容量 c 要求满足: $pc/(p-r) = b$,即 $c = b(p-r)/p$ 。第二个令牌桶的令牌速率为 r ,桶的容量为 $c = b(p-r)/p$ 。这样经过两个漏桶后的分组流的平均速率为 r ,同时也限制了分组流的峰值速率和突发长度。

12.12 一个漏桶允许每秒一个分组通过,并且漏桶的容量足够大。假设 9 个分组分别在时刻 0、2、3、3.2、3.4、3.6、3.8、4、5 s 到达该漏桶,请问这些分组实际经过漏桶注入网络的时间是多少?

答:

由于每秒只有一个分组通过,所以 9 个分组最后的离开时间分别为 0、2、3、4、5、6、7、8、9 s。

12.13 假设一个用户注入网络的分组情况是这样的,第 1 s 发送的分组为 5,第 2 s 为 5,第 3 s 为 1,第 4 s 为 0,第 5 s 为 6,第 6 s 为 0。采用令牌桶机制来规范用户流。假设令牌桶初始为满的,请问对于下面两种令牌生成速度,令牌桶的容量要求为多少?

(1) 每秒 2 个分组;

(2) 每秒 4 个分组。

答:

(1) 假设令牌桶容量为 N ,则第 1 秒之后桶中的令牌仍然有 $N-5+2=N-3$,第 2 秒后为 $N-3-5+2=N-6$,第 3 秒后为 $N-6-1+2=N-5$,第 4 秒后为 $N-5+2=N-3$,第 5 秒后为 $N-3-6+2=N-7$,第 6 秒后为 $N-7-1+2=N-6$,因此令牌桶的容量最少为 7。

(2) 和(1)类似,第 1 秒后为 $N-5+4=N-1$,第 2 秒后为 $N-1-5+4=N-2$,第 3 秒后为 $N-2-1+4=N+1$,第 4 秒后为 $N+1+4=N+5$,第 5 秒后为 $N+5-6+4=N+3$,第 6 秒后为 $N+3-1+4=N+6$,因此令牌桶的容量最少为 2。

12.14 假设一个路由器目前支持 3 个用户的流,每个流通过一个令牌桶来规范,令牌桶可以通过(令牌产生速度,令牌桶容量)来描述,其中令牌产生速度以每秒多少个分组来度量,令牌桶容量以分组数来度量,这 3 个用户流的令牌桶分别为(1,10)、(2,4)、(4,1)。路由器的转发速度为每 0.1 s 一个分组。

(1) 一个分组经历的最大延迟是多少?

(2) 在 2.0 s 内路由器转发的第 3 个流的分组数最小为多少? 假设其他流以最大可能的速率发送分组。

答:

(1) 如果路由器队列为空, 所有 3 个流在同一时刻以最大速率(即所有桶中的分组)传输, 则突发的数据总共为 $10 + 4 + 1 = 15$ 个分组, 最大延迟为 1.5 s。

(2) 2.0 s 内路由器最多可以转发 20 个分组。如果第 1 个流在 $T=0$ 发送 10 个突发分组, 在 $T=1$ 发送 1 个分组, 而第 2 个流在 $T=0$ 发送 4 个突发分组, 在 $T=1$ 发送 2 个分组。因此此时第 3 个流最多只能发送 3 个分组。当然从长期来看, 2 秒内第 3 个流平均发送的分组数仍然为 8。

12.15 一个采用漏桶算法的 155 Mb/s 的 ATM ABR 连接的峰值速率 PCR 为 20 000, 而可以忍受的到达时间的变化量 $L = 25 \mu\text{s}$, 请问该连接的最大突发长度为多少个信元?

答:

ATM 采用漏桶算法, 每个信元到达都要往漏桶加入水: $1/\text{PCR}$, 假设最大突发长度为 N 个信元, 总共注入水 N/PCR 。假设信元传输时间为 T , 而在这个期间漏走的水为 $(N-1)T$, 这样桶中水增加的数量为 $N/\text{PCR} - (N-1)T$, 而桶的容量为 $1/\text{PCR} + L$, 因此

$$\frac{N}{\text{PCR}} - (N-1)T = \frac{1}{\text{PCR}} + L$$

化简后有

$$N = 1 + \frac{L}{\frac{1}{\text{PCR}} - T}$$

根据题意, 有 $\frac{1}{\text{PCR}} = \frac{1}{200\,000} = 5 \mu\text{s}$, $L = 25 \mu\text{s}$

而信元传输时间 $T = \frac{53 \times 8}{155.52} \approx 2.73 \mu\text{s}$

代入后有 $N = 12.01$, 因此最大突发长度为 12 个信元。

12.16 考虑 RSVP 中的资源汇集支持, 假设只有一个发送者, 该发送者到 2^n 个接收者的组播树为一个深度为 n 的二叉树, 即该二叉树的每个叶子路由器节点连接了一个接收者。请问为了到每个接收者的路径预留资源, 总的预约请求为多少?

答:

首先 2^n 个接收者会发送 2^n 个 RESV 请求, 在上一层的路由器处会将接收者的请求汇集, 然后进一步向前转发, 这时发送的 RESV 请求个数为 2^{n-1} , 如此继续直到 RESV 请求通过深度为 n 的二叉树到达发送者。因此总的预约请求数为

$$2^n + 2^{n-1} + \dots + 2^2 + 2 = 2(2^{n-1} + \dots + 2^2 + 2 + 1) = 2^{n+1} - 2$$

12.17 请描述 RSVP 是怎样通过软状态的机制来处理网络中的故障的。

答:

在 RSVP 中发送者定期发送 PATH 消息让接收者知道目前到发送者的路径,同时接收者通过定期发送 RESV 消息来刷新预留状态信息。每个节点在收到 PATH 和 RESV 消息时会记录相应的路径状态信息和资源预留情况,但是这些状态信息是暂时的,具有一定的期限。如果路由没有变化,定期发送的 PATH 消息仅仅用于保证原有的路径状态信息一直有效,如果采用了一条新的路由,接收者就可以了解到一条新的路径,接收者可以通过发送 RESV 请求来在这条新的路径处预留资源,而老的路径上的状态信息则会因为超时而过期,从而老的资源被释放掉。接收者定期发送 RESV 状态信息,这些 RESV 用于刷新老的资源预留请求,如果一个接收者退出应用,则由于没有定期刷新,原先预留的资源被释放,如果一个新的接收者加入,发送的新的 RESV 消息就会预留相应的资源。注意到软状态的策略提供了相当的鲁棒性,当路径变化或者网络暂时的故障时只不过导致资源的暂时预留失败,这个时候网络会为这些流提供尽力递交的服务,一旦资源预留恢复正常,就可以提供相应的服务质量保证了。

12.18 请问下列应用应该最合适采用哪种 RSVP 的预留方式? 并请说明原因。

(1) 一个多用户应用,其中每个成员的视频流都要递交给所有其他成员。

(2) 一个音频会议,同一个时刻最多只有两个成员同时发言。

答:

(1) 由于每个成员都会收到来自于所有其他成员的视频流,也就是说接收到的视频流来自于所有其他用户,因此应该采用共享显式预留方式 SE,指定对于所有其他成员预留资源。

(2) 同一个时刻最多只有两个成员同时发言,从而我们能够知道在任一个时刻每个成员收到的流的带宽,同时我们又不知道到底是哪两个成员发言,因此应该采用通配预留方式 WF。

12.19 EF PHB 要求什么样的负载调节机制? AF PHB 呢?

答:

EF PHB 提供了在 DS 域内的低延迟、低丢失、低抖动、确保带宽的端到端服务,对于端系统来说提供了类似于点到点连接或者虚拟专线服务。为了实现 EF PHB,需要配置 DS 域节点使得 EF 分组汇集后有一个已知的与节点的其他负载无关的最小汇集服务速率,同时要求对 EF 分组通过通流量控制和整形等机制进行调节使得 DS 节点的到达率小于节点配置的最小服务速率。后一个任务一般是由 DS 域的边缘节点来完成的,而 DS 节点通过相应的排队调度机制来保证 EF 分组的低延迟,EF PHB 并没有限定具体的调度机制,比如可以采用优先级调度机制,EF 分组放到最高优先级的队列中,或者可以采用 WFQ 机制,把 EF 分组和别的分组分隔开来,保证 EF 分组获得预先配置的服务速率。

AF PHB 用于提供比尽力递交更好的服务,但是用户不需要预先预留资源,只要来自于某个用户的汇集速率不超过预先协商的负载特性,DS 域将保证大多数分组的递交,用户仍然可以发送超过预定的负载特性的数据,只是这些分组的递交不会得到很好的保证。AF 支持的 4 种负载类型是完全独立的,每个 AF 类通过汇集的数据速率和突发程度来描述,DS 节点为这些 AF 类分别分配了不同的带宽和缓冲区,一般采用 WFQ 或者类似的方法来实现,每个 AF 类的分组分别放在不同的队列上,每个 AF 类获得不同的服务速率。用户根据应用的要求选择合适的 AF 类来发送分组,对于每个 AF 类的分组,DS 域的边缘节点会通过 Meter 来检测来自于用户的分组的特性,根据用户与网络预先协商的 SLA 中的 TCA,分组赋予不同的丢弃优先级,每个 AF 类支持 3 种优先级,在遇到网络拥塞时低优先级的分组丢弃的概率比高优先级的分组大,在每个队列一般采取类似于 RED 算法的主动队列管理机制。

12.20 请比较集成服务和区分服务的特点。

答:

集成服务提供的是基于流的服务,它通过预留相应的网络资源来为每个流提供相应的服务质量保障,进行实际的数据传输之前,用户首先必须把自己的 QoS 要求告诉给网络,RSVP 协议用于传递相应的信令信息来进行资源的预留,是 IntServ 模型的核心部分。集成服务要求每个路由器为每个流维护相应的状态信息,并且这些状态信息要通过定期刷新来进行维护,如果网络中的流非常多时,每个路由器需要大量的空间和时间来维护这些流的状态,同时这些 RSVP 消息也会占用相当大的带宽;其次集成服务模型中要求每个路由器实现 RSVP 信令、准入控制、分组分类器和分组调度器,这使得路由器的实现变得相当复杂;集成服务只支持尽力递交、确保和控制负载等少数几个预先定义好的服务类,这种方式无法提供更加灵活的服务,比如把用户希望的服务分为三大类,分别是金卡、银卡和普通卡的服务等。

区分服务为由多个流汇集在一起的负载类提供服务质量保证,路由器利用 IP 分组头部中的字段把 IP 分组映射成相应的服务类,那个 IP 头部的字段被称为 DSCP,DSCP 在 IPv4 中通过 ToS 字段来表示,而 IPv6 中则通过负载类型来表示。区分服务中的路由器根据 DSCP 字段来对于属于不同类的 IP 分组进行分别处理,不再需要 IntServ 模型中的信令协议,不需要考虑这个分组是哪个主机发送,准备递交给那个主机的,也就是说 DS 路由器不需要为每个流维护状态信息,而只需要指定对于属于每个负载类的分组如何处理就可以了。每个路由器采取的流量控制操作被称为逐跳行为 PHB,网管人员通过配置来对每个 DS 路由器的 PHB 进行设置。区分服务并没有限定到底支持哪些服务或者服务类,区分服务只是提供了相应的体系结构和功能模块,以后根据需要可以定义新的 PHB,或者放弃老的 PHB,从而提供了相当大的灵活性,可以支持各种类型的服务。

12.21 请说明为什么 DS 域的边缘出口路由器也需要整形的功能。

答:

因为 DS 服务的用户可能是另外一个 DS 域中的节点,DS 域的带宽代理与邻居 DS 域的带宽代理之间维护了双边协议,对于那些目的地位于别的 DS 域的请求,带宽代理首先检查这个请求是否符合与邻居 DS 域的双边协议,然后通知邻居的带宽代理需要分配的带宽,邻居带宽代理接着配置相应的边缘路由器来通过整形来处理这个新的流,使得其满足与下一个 DS 域协商的 SLA。

12.22 考虑 EF PHB,请问 EF 分组一定比普通的分组经历更短的延迟吗?为什么?

答:

EF PHB 提供类似于虚拟专线的服务,要求在 DS 路由器处 EF 分组的到达速率小于服务速率,从而使得分组经历较短的延迟。但是 EF 分组和其他分组都是通过同一个路由器来转发,一般对于 EF 分组分配高优先级,而其他分组的优先级要低一些,但是并不一定所有的 EF 分组会比普通分组经历更短的延迟,考虑一个路由器首先收到一个普通的分组,由于 EF 队列为空,因此这个分组被调度器选择传输,我们假设这个分组的长度非常大,虽然马上有一个 EF 分组到来,但是由于前面的分组已经在开始传输,因此 EF 分组必须等到前一个普通分组传输完毕后才能进行传输,因此该 EF 分组会经历一个比较大的延迟。

第二部分

实验指南

实验一 异步串行通信接口实验

描述

使用异步串行通信接口在两台计算机之间进行通信操作。

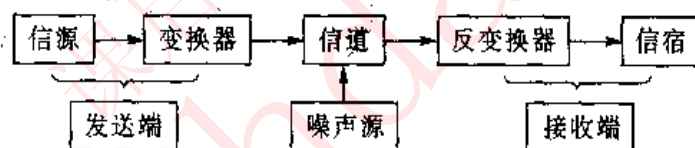
目的

熟悉串行通信概念,理解串行通信中直接电缆连接和 Modem 拨号这两种不同的工作方式,熟悉计算机中异步串行通信的组建工作原理。

基础知识

• 串行通信的一般概念

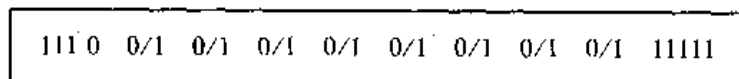
通信即信息交换,在计算机中主要为电子信息交换,其基本模型如实验图 1.1 所示。



实验图 1.1

计算机中的通信方式主要有两种:并行通信和串行通信。串行通信即把数据的字节分解成单个的二进制比特流依次传输,其最大优势在于它能方便地利用现有的通信信道(如电话线路等),促进了计算机技术和通信技术的结合。数据通信和计算机网络就是这一结合的产物。

根据时钟控制数据发送和接收的方式,串行通信可分为异步通信和同步通信。异步通信要求在发送每一字符时都要在数据位的前面加上起始位“0”,在数据位的后面要有停止位“1”,在字符之间允许有不定长度的空闲位“1”,例如,如实验图 1.2 所示。



实验图 1.2

在发送前双方设备需要设定采用的数据格式:数据位长度、停止位长度、奇偶校验位、数据传输速率(即波特率)。传输开始后,接收设备不断检测线路,当测到一系列的“1”之后检测到一个

“0”时就可确认一个字符开始传输,然后按照传输速率将接收到的数据位和校验位拼装成一个字符。由此可见异步通信是按字符传输的,接收设备在收到起始位之后只要在字符传输时间内与发送设备保持同步就能正确接收。若接收设备和发送设备两者的时钟略有偏差,则可以通过字符之间的停止位和空闲位进行调整。

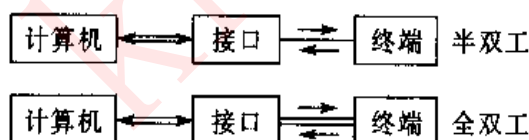
异步通信由于在每个字符前后附加起始位、停止位,因此传输效率不高。同步通信则去掉每个字符的起始位和停止位,将字符顺序连接起来,以一定长度的字符组成一个数据块,然后在每个数据块前附加1或2个同步字符。数据块内部划分成若干段,尾部是校验字段,通信时只有约1%的附加数据,提高了传输效率。

对于同步通信,接收设备首先检测同步字符,然后装配字符数据,在传输过程中双方要保持同步。在近距离情况下,可以在传输线中加一根时钟信号线,用同一时钟发生器驱动接收和发送设备;而远距离时可以通过解调器从数据流中提取同步信号用锁相技术得到数据。如实验图1.3所示。



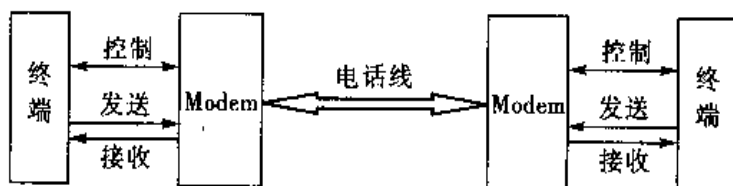
实验图 1.3

串行通信有两种传送模式:全双工和半双工。一般来说,当信号的接收和发送分别采用不同的传输线,即为全双工通信,系统在同一时刻既能发送又可接收。若使用同一根传输线既用作发送又用作接收,则为半双工通信,此时系统只能处于接收或发送两种状态之一,如实验图1.4所示。



实验图 1.4

串行接口发送的串行数据位流所能传送的最远距离取决于传输速率和传输线的电气特性。传输速率与传输距离成反比关系。在一定的传输速率和距离内,不需要中间设备,通信连接只需要3根线:发送线、接收线和公共信号地线就能组成。若超过一定距离,则需要加入通信设备,如实验图1.5所示。



实验图 1.5

通常需要同样的通信设备置于通信链路的两端,用以将数据调制到通信链路上或将链路上的信号解调成数据,这即是我们常见的调制解调器(Modem)。

• 串行总线标准 RS-232C

标准化的通用总线能大大简化系统软件和硬件的设计,使系统结构模块化和标准化,从而得到普遍使用。RS-232C 是美国电子工业协会(EIA)颁布的串行总线标准,它采用 25 引脚的连接线,其中定义了 20 个引脚,是通用的串行通信接口。常用 RS-232C 标准的引脚定义:

脚号	符号	方向(对计算机)	功能	缩写名
2	BA	输出	发送数据	TXD
3	BB	输入	接收数据	RXD
4	CA	输出	请求发送	RTS
5	CB	输入	清除待发送	CTS
6	CC	输入	数据设备准备好	DSR
7	AB		信号地	
8	CF	输入	数据载波检测	DSD
20	CD	输出	数据终端准备好	DTR
22	CE	输入	振铃指示器	RI

(1) 若在近距离进行简单的串行通信,只需 3 根线就可以了(2,3,7)。

(2) 若使用一条专设电话线进行通信,则需要使用 1~8 号引脚。对于发送,终端将一个“请求发送”(CA)信号送至 Modem;若收到“清除发送”(CB)信号,则发送被认可,于是开始通过“发送数据”(BA)线发送数据。而对于接收,当“数据载波检测”(CF)信号有效时表明 Modem 正在接收数据,并将接收的数据通过“接收数据”(BB)线送至接口或终端,无论是接收还是发送,“数据设备准备好”(CC)线都必须有效,表明 Modem 是“打开”的并已不在测试模式中,而是工作在数据模式下。

(3) 如果通信链路是交换式电话系统的一部分,则至少还需要“振铃指示器”(CE)信号线来通知计算机接口或终端。作为应答,接口或终端将启动“数据终端准备好”(CD)信号线,这个信号送到 Modem,控制它的转换设备,以建立通信链路。

• 调制解调器

调制解调器(Modem)是用于把数字设备连到电话线及其他模拟信道的设备。它具有以下功能:

(1) 将数字数据流转换成模拟数据流,并控制发送。

(2) 将模拟数据流转换为数字数据流

(3) 控制数据流量。

Modem 的工作过程如下:

① 呼叫。在 Modem 内部有一个继电器,当开始拨号时,该继电器闭合,计算机就被连到电话系统上(DSR 有效)。Modem 告诉计算机它已被连到线上,然后由计算机控制 Modem 进行自动拨号。Modem 中的扬声器可把线路上的声音信号放出,当电话拨到对方时,Modem 给计算机发出一个“回铃”信号来响应,同时可从扬声器中发出回铃音,如果对方电话没空,则会返回“占线”信号,扬声器发出“忙音”。当振铃结束时(连接已经完成),Modem 开始做信号交换和载波检测。

对于发出呼叫的一端,DTR 有效,RTS 有效。

而对于被呼方,调制解调器通过 22#振铃指示有效来通知计算机,如果 DTR 有效,Modem 使线路继电器闭合,振铃停止,此时 DSR(6#)有效,Modem 告诉计算机 Modem 被连到电话线上并将进行信号交换阶段。

② 信号交换和载波检测。两个 Modem 虽已摘机,但它们还没有“联机”。回答方的 Modem 发送它自己的回答音,呼叫方的 Modem 的 CXR(8#载波检测)有效,并一直保持到呼叫方的 Modem 检测到合适的音为止。这时 CXR 向计算机指出回答 Modem 已联机了,计算机可以在接收通路上接收到。在呼叫 Modem 检测由应答 Modem 发出的特殊频率之后的几个毫秒中,呼叫方 Modem 的 CTS 有效,然后发起方 Modem 发送它自己的回答音。当应答 Modem 检测到该回答音时,应答 Modem 的 CXR 有效,CXR 保持有效直至另一端一个音被检测到为止,它把 CTS 置为有效,并让计算机知道呼叫 Modem 已经做好通过线路发送数据的准备。

③ 信息交换。此时,在两台 Modem 联机后,两台计算机之间就可通过电话线路进行串行数据交换。

④ 断接。可以有几种方法,其中一种是通过发送相应的 Modem 的 AT 指令来断开。另外一种是通过设置信号线的状态,即让 DTR(20#)无效,从而使得调制解调器从电话线上掉线,或者通过重置 Modem,使其变为空闲,8#无效。

• INS 8250 的程序设计

由一个程序在异步通信环境中完成发送和接收数据字符,是一件十分复杂的事,这一般是通过一个叫做通用异步接收发送器(UART)的专用微处理器来完成多数串行协议的转换,在 PC 机中的 UART 是 8250 异步通信组件。UART 允许用户作串行的数据发送和接收而不必做任何并串转换或串并转换。使用 UART 之前,首先必须通过相应的输出指令来确定所要的波特率、数据格式(数据位数、奇偶类型和停止位数),一旦用所要的串行协议特征对 UART 进行编程,就可以用来发送和接收串行数据。

INS 8250 内部有 7 个 8 位寄存器可备 CPU 访问:

3F8H:数据发送保持寄存器和数据接收缓冲器,它们共用同一个地址。

3FBH:通信线控制寄存器 LCR,用于控制通信线路的数据通信格式,可以进行读写。如实

验图 1.6、实验图 1.7 和实验图 1.8 所示。

DLAB	SB	SP	EPS	PEN	STB	WLS1	WLS0
------	----	----	-----	-----	-----	------	------

实验图 1.6

DLAB:除数寄存器访问允许,取值为允许(1)、禁止(0);

SB:中止设定,中止(1)、正常(0);

SP:附加奇偶标志位选择,附加一位(1)、不附加(0);

EPS、PEN:偶校验选择和奇偶校验允许;

STB:停止位选择,1 位停止位(0)、1.5 位或 2 位停止位(1);

EPS	EPN	功能
1	1	偶校验
0	1	奇校验
X	0	无校验

实验图 1.7

WLS1	WLS0	字符长度
0	0	5
0	1	6
1	0	7
1	1	8

实验图 1.8

WLS1、WLS0:字长选择位。

3FDH:通信线状态寄存器 ISR,为 CPU 提供 8250 芯片的内部状态。如实验图 1.9 所示。

X	TSRE	THRE	BI	FE	PE	OR	DR
---	------	------	----	----	----	----	----

实验图 1.9

TSRE:发送移位寄存器空; THRE:发送保持寄存器空; BI:中止识别;

FE:接收格式错;

PE:数据奇偶错;

OR:数据重叠;

DR:数据就绪。

3F9、3F8H:除数寄存器(只有当 DLAB=1 时,CPU 才能对其进行读写)。3F9、3F8 分别存放除数的高 8 位和低 8 位。由于 8250 芯片使用的是 1.843 2 MHz 的时钟输入、因此需要采用分频的方法产生所要求的波特率;

$$\text{除数} = 1\,843\,200 \div (\text{波特率} \times 16)$$

3FAH:中断标志寄存器 IIR(可读写)。指出有无待处理的中断发生及其类型,并且封锁比此类型优先级低的所有类型中断。该寄存器 3~7 位总是为 0。第 0~2 位意义如实验图 1.10 所示。

2 1 0	中断类型	优先级
0 0 1	无中断	
1 1 0	接收出错	最高
1 0 0	接收缓冲区满	次高
0 1 0	发送保持寄存器空	低
0 0 0	Modem 中断	最低

实验图 1.10

3F9H:控制寄存器最高位为 0 时表示中断允许寄存器 IER。该寄存器的 0~3 位分别代表中断类型 2、1、3、0,置位为允许,复位为屏蔽。

3FCH:Modem 控制寄存器 MCR。如实验图 1.11 所示。

000	LOOP	OUT2	OUT1	RTS	DTR
	环路检测	输出 2	输出 1	请求发送	数据终端就绪

实验图 1.11

3FEH:Modem 状态寄存器 MSR。如实验图 1.12 所示。

RLSD	RI	DSR	CTS	ΔRLSD	ΔRI	ΔDSR	ΔCTS
------	----	-----	-----	-------	-----	------	------

实验图 1.12

RLSD:接收线载波检测;RI:振铃指示;DSR:数据机就绪;CTS:清除发送;

ΔRLSD:RLSD 位改变;ΔRI:RI 位改变;ΔDSR:DSR 位改变;ΔCTS:CTS 改变。

有关 Modem 中常用的 AT 指令可以参见相应的 Modem 参考手册,在此就不作介绍了。

要求

使用串行电缆直接连接两台机器或者通过 Modem 拨号连接,在这两种情况下分别实现下述应用:

(1) 实现一个模拟终端程序,即两台计算机间通信,使对方的机器都成为对方的终端,以全双工方式工作。

(2) 实现两台机器间的文件传送,以全双工方式运行,通过该程序可以互传文件。

实验二 数据链路层的帧封装协议

描述

模拟一个简单的数据链路层的封装功能,即从网络层获得一段用户数据,然后添加帧的头部和尾部(包括检验和),然后在接收方把帧的头部和尾部去掉,把用户数据传递给高层。

目的

通过该实验课题,了解数据链路层为了向网络层提供服务并且保证物理层传输的比特流无差错而使用的方法;了解检错码、通过比较各种成帧的方法,了解传输中的相关问题。

基础知识

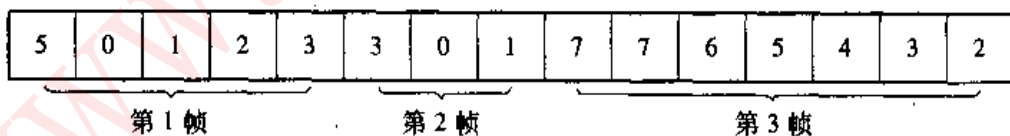
为了向网络层提供服务,数据链路层必须利用物理层提供的服务。物理层所做的工作就是接受一个原始的比特流并准备把它交给目的地,但是并不能保证这个比特流无差错。对于数据链路层,通常需要将比特流分成离散的帧,并对每一帧计算出校验和。当一帧到达目的地后重新计算校验和,如果校验和不相符,数据链路层就知道数据出错了,从而采取相应的差错处理措施。

除了依赖计时确定每一帧外,人们提出了其他的方法。一般有四种:

1. 面向字符计数的帧同步法

这种同步方法的帧格式如实验图 2.1 所示。

该帧同步法在帧头使用一个字段来表明帧内的字符数。



实验图 2.1

该方法面临的问题是,如果该字段在传输过程中被“篡改”,接收方和发送方就不同步,接收方无法知道下一帧的开始位置。而且接收方向发送方发送请求重传一帧也没有用,因为接收方也不知道最近的正确的帧的开始位置,故很少使用。

2. 面向字符的帧同步法

这种同步方法的帧格式如实验图 2.2 所示。

SYN	SYN	SOH	标题	STX	数据块	ETB/ETX	块校验
-----	-----	-----	----	-----	-----	---------	-----

实验图 2.2

其中 SYN、SOH、STX、ETB、ETX 等特殊字符的二进制位模式随采用的字符编码集不同而不同,当然,还有其他的一些如 EOT、DLE(Data Link Escape)、ACK 等字符。

面向字符的帧同步还有一些较复杂的问题要处理。假如在数据块中刚好有上述特殊字符的二进制位模式,那么就可能发生误解。其解决方法是将每个独立的控制字符都作为普通的数据字符来处理,所以它们可以出现在数据块中。对于具有控制含义的上述字符,可在其前面插入一个 DLE 进行转义。若数据块中本身出现了 DLE 字符,则同样在其前面插入一个 DLE 字符。

这种成帧方法绕过了字符计数法出错后再同步的问题,而且其中有了“标题”字段,从而具备了更为复杂的通信控制能力,校验能力也增强了。

当然这种帧同步的方法和特定的字符编码集关系密切,从而导致了其兼容性差。

3. 面向比特的帧同步法

该方法使用一种特殊的位模式 01111110 作为帧开始和结束的标志,如果数据中包含该位模式,则在 5 个连续的 1 后插入一个 0,在接收方删除该填充的 0。采用该位填充模式,两帧的边界就可以通过位模式而唯一识别。如果接收方失去同步,只需在输入流中扫描该位模式,即可重新获得同步,因为这些标志只能是帧界,而绝不应该出现在数据中。

4. 起止式异步法

适用于在物理介质的编码策略中采用了冗余技术的网络。例如,一些局域网用两个物理位表示数据位的一位。通常,数据位 1 编码成高-低电平,数据位 0 编码成低-高电平时,高-高电平、低-低电平在数据编码中没有使用。这种方案意味着每个数值位都要在中间进行一次变换。

网络设计者已经研究出两种基本的策略来处理差错。一种方法是在每一个要发送的数据块上附加上足够多的冗余信息,使得接收方能够推导出已发送的字符是什么,另一种方法是只加入足够的冗余位,使得接收方只知道有差错发生,但不知道是什么样的差错,然后向发送方请求重传。前者的策略是纠错码,后者的策略是检错码。在实际应用中,多项式编码,也叫循环冗余码被广泛采用。循环冗余码是基于将位串看成系数为 0 或 1 的多项式的原理,一个 k 位帧可以看成是从 $x^{(k-1)}$ 到 x^0 的 k 次多项式的系数序列。如 110001 有 6 位,表示成多项式就是 $x^5 + x^4 + x^0$ 。多项式以 2 为模运算。进行数据传输前,发送方和接收方商定一个生成多项式 $G(x)$,其高位和低位都是 1。要计算 m 位的帧 $M(x)$ 的校验和,生成多项式 $G(x)$ 必须比该多项式短。其基本思想是将校验和加在帧的末尾,使这个带校验和的帧的多项式能被 $G(x)$ 除尽。当接收方收到带校验和的帧时,用 $G(x)$ 去除它,如果有余数,则传输出错。计算校验和的算法如下:

设 $G(x)$ 为 r 阶,在帧的末尾附加 r 个 0,使帧为 $m+r$ 位,则相应的多项式 $x'M(x)$ 。

按模 2 除法用对应于 $G(x)$ 的位串去除对应于 $x'M(x)$ 的位串。

按模 2 减法从对应于 $x'M(x)$ 的位串中减去余数。结果就是要传送的带校验和的帧。

要求

1. 校验和的计算和数据透明支持。
2. 帧同步方法自己选择。

实验三 数据链路层的滑动窗口协议

描述

模拟滑动窗口协议,第一题的基础上加入流量控制和差错控制的支持,模拟丢包的情形。

目的

通过该实验课题,了解滑动窗口协议及通过该协议来保证发送方所有的帧最终都能按正确的发送顺序交付给接收方的网络层及处理发送方的发送能力比接收方处理能力大的问题。顺便了解(和考试相关的)协议描述和验证方法。如 Petri 网。

基础知识

滑动窗口协议类似于一个协议族,包括 3 个不同的协议。在三种滑动窗口协议中,每一个要发出的帧都包含了一个顺序号,范围一般是从 0 到一个通常为 $2^n - 1$ 的数,因而顺序号能恰好放在 n 位字段中。

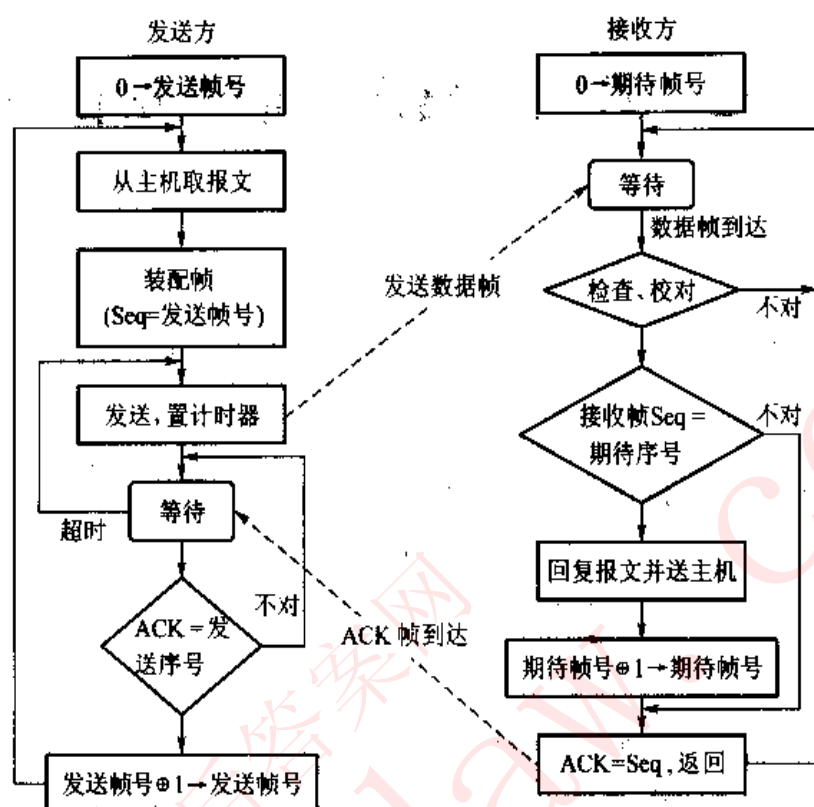
所有滑动窗口协议的关键在于:任何时刻发送过程都保持着一组顺序号,对应于允许发送的帧,称为发送窗口。同样接收过程也维持一个接收窗口。发送过程的窗口和接收过程的窗口不需要有相同的窗口上限和下限,甚至不必具有相同的窗口大小。同时有下述要求:协议必须按分组从网络层传送给发送机器上的数据链路层的顺序把这些分组递交给目的机器的网络层。

在发送窗口中的顺序号代表已发送的但未确认的帧。接收过程的数据链路层窗口对应于允许接收的帧,任何落在窗口外面的帧都不加说明的丢弃,当顺序号等于窗口下限的帧收到后,把它交给网络层,产生一个确认,且窗口整个向前移动一个位置。根据发送窗口的大小和对出错帧的处理方法,滑动窗口协议分为 1 位滑动窗口协议、使用退后 n 帧的协议、选择性重传协议三种。

1 位滑动窗口协议:

该协议是最简单的一个滑动窗口协议。该协议的发送窗口大小为 1,此协议使用了停-等方式,它规定发送方每发送一帧后就要停下来,等待对方已正确接收的确认返回后,才能继续发送下一帧。因为发送窗口大小为一,所以只用一位二进制数(0 和 1)来编号。其发送方和接收方的流程如实验图 3.1 所示。

考虑到帧到达接收方的时间及确认帧的传输时间,为了提高信道利用率,采用一种称为管道化的技术,在等待确认帧的时间内,发送方继续发送数据帧。但是当帧流中间的某一帧出现错误



实验图 3.1

的时候,如何处理后续帧?

一种是用退后 n 帧的协议

由于发送方在收到确认帧之前可以连续发送多个未确认的帧,所以帧号就不能仅使用一位二进制数来区分。

对于发送方,由于发送的帧可能丢失或出错,所以要求发送方有较大的发送缓冲区保留准备重发的帧。对于接收方,应以正确的顺序将收到的报文向上送到主机,但是由于发送方在未收到确认的情况下,有可能已发送了多帧,接收方由于中间的这一帧出错,不能正确接收该帧并送主机,所以对后面发送的这些帧均不接收,而是直接抛弃所有的后续帧。对丢弃的帧,不发送确认。

另一种是选择性重传协议:在退后 n 帧的协议中,将已经正确收到的帧丢弃,然后等待重传一遍显然是一种浪费。一种更好的方法是选择性重传协议。在该方法中,丢失或出错帧后面正确接收到的帧被暂时放在一个接收方的缓冲区中,同时只要求发送方重新发送丢失或出错的那一帧。一旦重新收到了这一帧,则和原来存储在缓冲区的后续帧一起向上送到主机。

要求

实现停-等协议、后退 n 帧的协议和选择性重传协议,并比较其带宽利用率。

提示

下面给出一个停-等协议算法的伪代码,实验时仅供参考:

```
seq_nr next_frame_to_send = 0;    //设定发送帧序号为 0
seq_nr frame_expected = 0;    //设定期望接收帧号为 0
packet buffer;    //存储即将发送的数据
frame s;    //临时变量
event_type event;
from_network_layer(&buffer);    //从网络层得到一个数据包
s.info = buffer;
s.seq = next_frame_to_send;
s.ack = 1 - frame_expected;
to_physical_layer(&s);    //传送帧
start_timer(s.seq);    //开始计时
while(true)    //循环处理
{
    wait_for_event(&event);
    if(event == frame_arrival)    //有一帧到达
    {
        from_physical_layer(&r);    //从物理层得到新的数据
        if(r.seq == frame_expected)
        {
            to_network_layer(&r.info);    //上传该帧给物理层
            frame_expected = 1 - frame_expected;    // 改变期望得到的帧号
        }
        if(r.ack == next_frame_to_send)
        {
            from_network_layer(&buffer);    //从网络层取得新的数据
            next_frame_to_send = 1 - next_frame_to_send;    // 通知发送的序号
        }
    }
    s.info = buffer;
```

```
s.seq = next_frame_to_send;  
s.ack = 1 - frame_expected;  
to_physical_layer(&s);  
start_timer(s.seq);
```

实验四 局域网模拟:模拟 CSMA/CD

描述

通过程序实现 CSMA/CD 的三种坚持算法的一种,最好实现 IEEE 802 标准采用的 1 坚持方式。

目的

通过该实验课题,了解局域网中 CSMA/CD 规则的具体内容,并进一步加深对 CSMA/CD 规则与 CSMA 规则的异同以及它的优点的了解。

基础知识

载波监听多路访问/冲突检测 CSMA/CD 是针对载波监听多路访问 CSMA 的缺点而提出的。由于在 CSMA 中,一旦有两个帧发生冲突,在这两个坏帧传输的这段时间内,其他站点就不能传输,如果帧比较长,就严重影响了系统的工作效率,相对于传播时间来说被浪费的容量是相当可观的。但是如果站点在传输的同时继续监听,这种浪费可以减少。因此,CSMA/CD 又叫边听边说(LWT)。下面就是 CSMA/CD 的规则:

- (1) 若媒体空闲,则传输;否则,转第(2)步;
- (2) 若媒体忙,一直监听直到信道空闲,然后立即传输;
- (3) 若在传输中监听到冲突,则发出一个短小的人为干扰信号,让所有的站点都知道发生了冲突并停止传输;
- (4) 发完人为干扰信号等待一段随机的时间,再次试图传输(从第(1)步开始重复)。

采用 CSMA/CD 技术后,被浪费的带宽减少为检测冲突所花费的时间。对于基带系统的最坏情况,此时用于检测一个冲突的时间为从信道的一端到另一端的传播延迟的两倍。对于宽带总线来说,最坏情况发生在与头端离得最远的两个相邻站点间,此时用于检测冲突的时间等于从头端到电缆尾部的传播延迟的 4 倍。这样一来,若是帧不够长,以至于冲突在帧传输完毕后还没有被检测到。那么 CSMA/CD 就与 CSMA 协议没有区别了。

CSMA/CD 也可以选用三种坚持算法,非坚持方式、1 坚持方式、P 坚持方式,同 CSMA 中一样,非坚持方式要浪费部分带宽。而在 P 坚持方式中,为了保持稳定,P 必须取得足够小,结果在轻负荷下延迟时间过长,因而可能产生较大的浪费。而采用 1 坚持方式,则上述对带宽的浪费没有了,然而却付出了一些冲突的时间。

IEEE 802 标准采用 1 坚持方式,而为了确保这种退避保持稳定,IEEE 802 标准采用了一种

叫二进制指数退避的技术:

二进制退避算法如下所描述:当冲突发生后,时间被分成离散的时槽。时槽长度等于在传输媒体上来回传输的时间。第一次冲突发生后,每个站点等待 0 或 1 个时槽后尝试重新发送。如果每个站点等待的时槽数相同,这样它们将再次冲突。这一次,它们会从 0、1、2、3 中随机挑选一个数作为等待的时槽数。如果又产生第三次冲突(概率为 0.25),它们将从 $0 \sim 2^3 - 1$ 中随机挑一个等待的时槽数。一般而言, i 次冲突后,等待的时槽数从 $0 \sim 2^i - 1$ 中选,但是如果冲突次数达到 10 次后,随机等待的最大时槽数固定为 1023。在 16 次冲突后,站点放弃传输,并报告一次错误。

本实验模拟 CSMA/CD 协议,预先给定网络拓扑结构(总线型/树形或星形),各节点可在随机时刻或者事先预定的时刻发出一个帧,给定帧长和帧的传播时间,网络各节点将按照 CSMA/CD 规则运行,可观察运行状况是否与规则相符,并将网络各节点具体运行情况写入记录文件。

要求

1. 给出一个网络拓扑结构,并以图形方式模拟各节点的运行状况以及帧在网络中的传输情况(坚持算法采用二进制指数退避算法)。
2. 要求有记录网络运行情况的文件作日志。

提示

1. 可用按钮或者画板来代表各个节点,并用文字表示节点的各种状况:传输中、监听中、空闲中等,用连线表示信道,信道状态也可用文字来表示空闲或者忙。帧则可用一定长度的与信道不同颜色的线段来表示。
2. 帧传播的时间可扩大到几秒钟(为了便于观察),根据信道(连线)的长度来决定帧(线段)的移动速度。
3. 信道的状态可用标志来表示(0 表示空闲,1 表示忙),节点状态也同理。

实验五 模拟网络层路由算法

描述

写出一个网络拓扑的路由处理程序,根据一个给出的网络拓扑,能够计算出任意两点的最优路由。最好实现距离向量路由选择算法。

目的

由于数据链路层仅将数据帧从导线的一端送到另一端,为了将源端发出的分组送到目的端,网络层选择通过子网的合适路径。通过该实验课题,了解现代计算机网络中通常使用的动态路由算法。

路由选择算法是网络层软件的一部分,负责确定所收到的分组应传送的外出路径。对于路由选择算法,都希望其具有以下特性:正确性、简单性、健壮性、稳定性、公平性和最优性。

基础知识

路由选择算法可以分为两大类,非自适应的和自适应的。非自适应的算法不根据实测或估计的网络的当前通信量和拓扑结构来做路由选择。相反的,自适应的算法根据拓扑结构,通常还有通信量的变化来改变其路由选择。

• 扩散法

该算法是一个很简单的算法,其原理大致如下:一个网络节点从某条线路上收到一个分组之后,就把该分组向除了分组到来的线路外的所有其他相连的线路上发送。结果最先到达目的节点的一个或多个分组肯定是经过了最短的路径,并且所有可能的路径都被同时尝试过了。

显然该方法会产生大量的重复分组,如果不采取某些措施来抑止这种过程,则产生的分组数可能会非常的大,从而浪费了大量的网络带宽。一般有三种方法来抑止产生大量的分组。

一种方法是让每个分组的头部包含一个站点的计数,每到达一个站点,该计数就减1,直到该计数为0,就丢弃该分组。

第二种方法是记录下分组扩散的路径,防止它第二次在扩散到已扩散过的路径中。

第三种方法是选择性扩散法。该算法并不是简单的向所有其他的节点转发分组,而是选择那些可能向目的节点的方向上的节点发送。这里的方向可以是地理上的方向,也可以是网路拓扑上的方向。

• 最短路由选择

为了便于对算法的研究,首先建立一个子网图,图中的每个节点代表一个路由器,每条弧线

代表一条通信线路。为了选择一对路由器间的路由,算法要找出其间最短的路径。

有关计算该最短路径的算法有多种,下面简单介绍 Dijkstra 提出的前向搜索算法。每个节点用从源点沿着找到的最佳路径到本节点的距离来标注。一开始,一条路径也不知道,故所有的节点都标注为无穷大。随着算法的进行,不断找到新的路径,标注也随之改变,使之反映出较好的路径。其正式定义如下:

定义: N = 子网图中所有节点的集合

S = 源节点

M = 已由算法归并的节点的集合

$L(i, j)$ = 节点 i 与 j 之间链路的权值;若两个节点间没有直接连接,则为无穷大。

$C(n)$ = 算法求得的从当前 S 到 n 的最少花费路由

算法步骤:

(1) 初始化

$M = \{S\}$

$C(n) = L(S, n)$ for $n \neq S$

(2) 从不在 M 中的相邻节点中找出一个具有和节点 S 的最少花费路由的节点,并把该节点规约进 M 中。可以表示如下:寻找不属于 M 的 w ,使得 $C(w)$ 等于所有未规约的节点 j 中 $C(j)$ 最小的一个。

(3) 更新最少花费路径

对所有的不属于 M 的节点 n , $C(n) \approx \text{Min}[C(n), C(n) + L(w, n)]$, 如果后一项为最小值, 则从 S 到 n 的路径改变为从 S 到 w 的路径在加上从 w 到 n 的路径。

(4) 重复(2)到(3),直到 $M = N$

• 基于流量的路由算法

上述算法没有考虑网路地负载因素。基于流量的路由算法(Flow-based Routing)综合地考虑网络的拓扑结构和负载因素。由于在某些网络中,每对节点间的平均数据流是相对稳定并且可以预测的。在预先知道节点 i 和 j 的平均通信量的情况下,可以对流量进行数学分析,以优化路由选择。

这种分析的基本思想为:对某一给定的线路,如果已知负载量与平均流量,那么可以根据排队论来计算出该线路上的平均分组延迟。由所有的线路平均延迟,可以直接计算出流量的加权平均值,从而得到整个网络的平均分组延迟。这样路由选择问题就归结为如何找到产生网路最小平均延迟的路由选择算法。

根据排队论计算网络图中各线路的平均延迟:

$$T = \frac{1}{uC - \lambda}$$

其中 $1/u$ 是以 b 为单位的分组平均长度, C 是以 b/s 为单位的容量, λ 是以分组/秒为单位的平均流量。

• 距离矢量路由算法

该算法是让每个路由器维护一张路由表,表中每一项对应这一个目的地。路由表中每一项包括到对应的目的地路由中下一个节点的标识。另外,它还包括了一个测量地到目的地距离的度量值。

每个节点定期地把它的路由表向它相邻的节点传递。通过与相邻的路由其交换信息来更新表的信息。

其更新算法如下:

更新消息由一系列二元组 (V,D) 组成,其中 V 表示目的地,叫做向量, D 是到该目的地的距离。

(1) 初始化,对每个节点 G ,对所有的它直接连接的目的地 N ,路由表中的表项用三元组 $(N,G,0)$ 来表示,即从节点 G 到目的地 N 无需转发。

(2) 节点 G 定期的发送它的路由表给相邻的节点,更新信息中对应着每一个目的地 N ,用一个三元组来表示 (N,V,D) ,即目的地 N 的路由上的下一个节点为 V , G 到 N 的距离为 D 。

(3) 节点 G 收到 G' 送来的路由信息,对于更新信息中给出的每个目的地,在 G 的路由表中查找相对应的表项,设它为 (N,V,D) ,而更新信息中的三元组为 (N',V',D') , C 为节点 G 和 G' 之间的距离。

如果找不到相应的表项,则在 G 的路由表中增加一项 $(N,G',D' + C)$;

如果 $V = G'$,则在 G 中路由表对应的表项更新为 $(N,G',D' + C)$,否则比较 $D' + C$ 和 D ;

如果 $D' + C < D$,则 G 中表项更新为 $(N,G',D' + C)$,否则, G 中表项任保持原状。

要求

1. 最好将上述算法都通过程序模拟实现。
2. 相关参数自己设定。

实验六 NetBIOS 编程

描述

使用特别的命令序列来调用各种 NetBIOS 功能,实现数据源地与数据目的地之间的信息交换。

目的

理解 NetBIOS,熟悉两个工作站之间的信息收发和会话功能,要求工作站把键盘输入的一行文字或文件传送至对方,同时把收到的字符串显示出来;能传输任意长度的二进制文件。

基础知识

• NetBIOS 概述

NetBIOS 是 Network Basic Input/Output System 的缩写,即网络基本输入输出系统,它是一种应用程序接口,适应于多种不同的网络适配器,用户的应用程序可以使用特别的命令序列来调用各种 NetBIOS 功能,而无需涉及各种最底层的通信协议,从而实现数据源地与数据目的地之间的信息交换。

NetBIOS 是 IBM 公司于 1984 年首次推出的,后来形成了 TCP/IP NetBIOS 工业标准,它扩展了 NetBIOS 原有的功能,解决了网络互连环境中的路由、名字等问题。NetBIOS 不但适用于 IBM 公司提出的各种 PCLAN 适配器环境,而且通过各种仿真程序,它也能适用于 3COM 的 Ethernet 和 Novell 网络等。

从 ISO/OSI 参考模型来看,NetBIOS 大体上位于该模型的表示层与会话层之间。编写 NetBIOS 接口应用程序基本上与其他层次无关。两个 NetBIOS 应用程序可以使用 IBM PC 网络适配器进行通信,基本的通信工作可由适配器中的内部会话管理规程(SMP)来完成,应用程序也可以使用 IEEE 802.2 逻辑链路控制(LLC)规程的 IBM PC LAN 支持程序。但在任何情况下,应用程序都与所使用的规程毫无关系。NetBIOS 提供如下四种服务:名字支持、数据报支持、会话支持和一般命令。

• 名字支持

NetBIOS 是面向名字的,网络上的每一个站点(或 LAN 适配器)都被标识为一个或多个网络名字。每一个网络名都由 16 个字符组成。一个站点使用某名字之前,必须获得注册的权利和在 LAN 中使用该名字的权利。NetBIOS 中有一个名字表用来记录这些名字。

对每个注册成功的名字都有一个对应的名字号,它可用于 NetBIOS 中与名字有关的命令中。

此外,还有一个永久节点名,它在网络上总是唯一的,所有 IBM LAN 都有一个与该适配器有关的 6 字节数,用于保证适配器的唯一性,永久节点名就是由这 6 个字节数加上 1 个字节的二进制组成。永久性节点名总是适配器的 NetBIOS 名字表中的第一项,其名字号总是 1(0 表示无效的名字号)。永久性节点名用作 LAN 地址,使用此地址就可以打印出适配器传送的所有信息,并且可以把它作为传送给某适配器的所有信息的标志支撑点。它也被用于告诉适配器通信线路,应该忽略哪些信息,应该把哪些信息接收到工作站中。

应用程序可以查名字表,除了永久节点名外,应用程序可以根据需要增加或删除名字,也可以在名字表中增加一个组名,与一般的名字不一样,组名并不要求在网上唯一,不同的站名可以在同一时刻使用相同的组名。建立会话时,可以将这些名字(可能是加入的,也可能是永久节点名)作为目的地址和源地址。名字号被用来发送数据报。

NetBIOS 中与名字管理相关的命令为 ADD Name, ADD Group Name 和 Delete Name。NetBIOS 的 Reset 命令清除名字表中所有的名字(永久节点名保留)。在运行 IBM PC LAN 时,不可使用 Reset 命令,因为 Reset 将会删除 DOS 文件重定向使用的所有名字与应用程序加入的名字。

为了保证加入到名字表中的名字是唯一的,NetBIOS 首先检查本地名表,然后检查整个网络,以检查该名字是否被用过。摸索整个网络时,NetBIOS 广播一个名字请求帧,若这个请求帧被接收了,则就表明这个名字已经被用了。

• 数据报支持

一旦适配器在网络中开始工作,工作站中的应用程序就可以使用 NetBIOS,与驻留在相同或不同工作站中的应用程序进行通信。应用程序可以使用数据报进行通信,也可以使用会话进行通信。

发送数据报无需先行连接,它不保证数据报信息的安全到达,目的站点可能不在工作,也可能虽在工作但不想接收数据报,甚至目的站点根本就不存在,在这些情况下,数据报就无法送到目的站点。NetBIOS 支持广播式数据报和普通数据报,普通数据报要求应用程序在发送数据报时,指出接收站点的 NetBIOS 的名字。在这两种情况下,NetBIOS 数据报传输命令均涉及已有的本地 NetBIOS 名字号(即永久节点名),把它用作数据报的源地名。这个名字号可以与本地唯一名(指其他适配器不能使用的注册名)或组名联系起来。可见 NetBIOS 可把数据报发送给某个名字、一个组名或网上全部站点,数据报是一种短信息,装载的记录长度最大为 512 B。数据报的主要优点是消耗系统资源少。

NetBIOS 用来发送与接收数据报的命令有:Send Datagram、Send Broadcast Datagram、Receive Datagram 和 Receive Broadcast Datagram。

• 会话支持

NetBIOS 支持在网络上的两个名字之间创建一个会话,两个名字之间可以有多个会话,一个会话也可以在同个站点的两个名字之间创建。在会话过程中,发送的信息将以正确的顺序到达正确的目的地。会话通信包括建立会话、传送信息和结束会话的完整过程。

创建一个会话时,一个站点发一个 Listen 命令(可以用也可以不用 wait 状态),Listen 命令指

定收听任意或一些特定名字的呼叫。另一站点发一个呼叫(Call)命令,并指定呼叫的名字。当每个站点的命令完成时,NetBIOS 给每个应用程序返回一个局部会话号 LSN(Local Session Number),每个站点返回的 LSN 不一定相同,LSN 可循环取值 1,2,3,...,254。这样,每个站点都使用指定的 LSN 来使用已打开的活动的会话。

一旦会话建立成功,双方都可以发送和接收信息,使用的发送信息命令有:Send 和 Chainsend(后者用于发送多个信息),接收命令为 Receive。在一个会话中发送和接收信息记录的最大长度为 65 535 B。当会话结束,会话的一边或两边发出 NetBIOS Hang Up 命令,即可终止本次 LSN 的会话。

• 一般命令

NetBIOS 一般命令所提供的服务有:复位命令(Reset),它强行把适配器设置为初始状态;适配器状态命令(Adapter Status),它可以使用户对 NetBIOS 适配器进行查询和重试操作信息;取消及断开链路命令(Cancel),它可以使应用程序取消没有执行完的命令;查找名字命令(Find Name),使用在该命令中给出的符号名对适配器进行定位;跟踪命令(Trace),它提供一种对诊断程序的支持。

• 网络控制块 NCB

为了调用某个 NetBIOS 命令,应用程序应先建立一个网络控制块 NCB(Network Control-Block),然后执行 DOS 中断 5CH。NCB 占用 64 B,由 13 个字段(或域)和作为保留区的 14 B 所组成。在 C 语言中,NCB 结构定义如下:

```
#define USGC unsigned char
#define USGI unsigned
#define USGL unsigned long
struct NCB {
    USGC NCBCommand;           /* Command Code */
    USGC NCBRetCode;           /* return code */
    USGC NCBLSn;               /* local session number */
    USGC NCBNum;               /* Datagram NAME table entry */
    char * NCBBufferOffset;    /* I/O buffer offset */
    USGI NCBBuffersegment      /* I/O buffer segment */
    USGI NCBLength;            /* length of data in I/O buffer */
    char NCBCallName[16];      /* remote system name for CALL */
    char NCBName[16];          /* local adapter network name */
    USGC NCBRto;               /* receive timeouts in 1/2 second units */
    USGC NCBSto;               /* send timeouts in 1/2 second units */
    char * NCBPostRtnOffset;   /* offset of post routine */
    USGI NCBPostRtnSegment;    /* segment of post routine */
};
```

```
USGC NCBLanaNum;          /* network adapter number to execute cmd */
USGC NCBCmdCplt;           /* 0xFF => Command pending, else completed */
char NCBReservedArea[14]; /* work area for network card */
```

其中:后处理例程的位移和段这两个域亦可定义为

```
void interrupt( * post-Func)( void)
```

对于 NCB 结构中各个域的含义简单介绍如下:

(1) 命令(NCB_Command)。占 1 B,是要求 NetBIOS 执行命令的命令码,该域的最高位确定命令的等待(wait)或非等待(no-wait)状态。最高位值为 0 表示等待状态。为 1 表示非等待状态。Reset、Cancel 和 Unlink 命令必须保证执行完,它们的最高位必须置 0。当适配器完成命令后,所选的等待状态要求 NetBIOS 返回到控制器,在这种意义上,非等待的 NetBIOS 命令的使用就像普通应用程序的子程序调用一样。选择非等待状态,在允许几条同时挂起的命令之前获得最大的吞吐量。然而命令挂起时,应用程序既不改变 NCB 也不改变与命令有关的任何数据域。对于非等待状态,也许返回两个返回码,第一个返回码(即立即返回码),它是 NetBIOS 最初扫描此命令时的返回码;另一个返回码(即最后返回码),是在命令完成时的返回码。若第一个返回码不是 00h,NetBIOS 丢弃这条命令,并不再处理它。因而,也不返回第二个返回码,也不执行后置(或后处理)例程。

当 NetBIOS 接受该命令时,就让该命令排队,它允许同时挂起多个请求,并在命令完成域中置 FFh 标志,以表示命令已排队但未执行,当命令完成时,NetBIOS 把最后的返回码送至 NCB 的返回码域和命令完成域中。在 NetBIOS 程序中可以通过传递一个函数指针来指定一个后处理例程,事件结束时 NetBIOS 调用这个例程,也可以通过循环不停的询问事件是否结束,直到 NCB_Cmd_Cplt 域从 FFh 变成实际的返回代码为止。

用十六进制数表示的 NetBIOS 命令定义如下:

```
#define Chain_send_NO_ACK          0xf2
#define Chain_send_NO_ACK_Wait     0x72
#define Receive                     0x95
#define Receive_wait                0x15
#define Receive_Any                 0x96
#define Receive_Any_wait            0x16
#define Session_Status              0xb4
#define session_status_Wait         0x34
#define Send_Datagram               0xa0
#define Send_Datagram_Wait          0x20
#define Send_Broadcast_Datagram     0xa2
#define Send_Broadcast_Datagram_Wait 0x22
```

```
#define Receive_Datagram          0x1
#define Receive_Datagram_Wait     0x21
#define Receive_Broadcast_Datagram 0xa3
#define Receive_Broadcast_Datagram_Wait 0x23
```

上述定义中,凡命令中带有 wait 者表示该命令选择等待状态,否则表示选择非等待状态,这两类命令所定义的值均相差 0x80。

(2) 返回代码(NCB_RetCode 和 NCB_Cmd_Cplt)。NetBIOS 给应用程序返回一个立即返回代码(NCB_RetCode)、一个最后返回代码(NCB_Cmd_Cplt)。

如果没有确定一个后处理例程而选择非等待项,则这两个返回代码域在命令完成后均含有最后的返回码。返回码为 0x00,表示命令顺利完成。命令执行失败时,返回码为特定的错误代码号(从 0x01 ~ 0xFE)。应用程序不应该循环查询 NCB_RetCode 域来确定命令是否完成,而应该循环查询 NCB_Cmd_Cplt 域,这是因为在设置 NCB_Cmd_Cplt 域之前先设置 NCB_RetCode 域。

设置完 NCB_RetCode 域后,NCB 可能置于内部 NetBIOS Post 后处理队列,直到 NCB 从 Post 后处理队列退出后才设置 NCB_Cmd_Cplt,当后处理例程的地址是 0000:0000 时,NCB 从队列中退出,并作出申请一个后处理例程的决定。

(3) 本地会话号(NCB_LSN)。LSN 的值必须在 0x1 ~ 0xFE 之间,仅当 Call 和 Listen 命令顺利完成 LSN 的值才有效。应用程序也许是本地的或者是远地的,分配给会话一方的 LSN 值与分配给对话的另一方的 LSN 值是相互独立的,在同一个机器内的两个适配器,也许使用一个 LSN 值会话,但可通过它们的适配器的 NCB_Lana_Num 值来区分。

Send、Receive 和 Hang Up 命令需要 NCB_LSN 域,但数据报不需要该域,LSN 的值用以确定 NetBIOS 允许适配器最大的会话号。

(4) 名字号(NCB_Num)。在 Add Name 或 Add Group Name 命令执行后,NetBIOS 返回一个字节长的名字号码,并送入该域中。当使用与数据报相关的命令或 Receive Any 命令(其功能是接收来自会话伙伴的数据,即与一个特定的本地名有关的会话上,Send 或 Chain Send 命令发送的数据)时,可以使用这个号,而不使用名字。

(5) 缓冲区地址域(段:位移)。共占 4 B,用于存放应用程序数据缓冲区地址,即存放与发送和接收操作有关的长指针。缓冲区地址域亦可定义为:NCB_Buffer_Ptr,它是一个远指针。

(6) 缓冲区长度(NCB_Length)域。指出应用程序接收与发送数据缓冲区的长度(字节数)。

(7) 呼叫名(NCB_CallName)域。占 16 B,用于存放欲与之通信的站点的名字。

(8) 本地名(NCB_Name)域。占 16 B,用于存放应用程序应知道的本地名字表中的名字(或为永久节点名)。

(9) 接收超时(NCB_Rto)域。占 1 B,Call 和 Listen 命令用它来确定与一个请求的、但仍未建立的会话有关的 Receive_Any 和 Receive 命令的超时时间,此时间以 0.5 s 为间隔,例如,值 0xFF 表示超时 127.5 s,00 表示未超时。

(10) 发送超时(NCB_Sto)域。与 NCB_Rto 类似,相当于等待 Send 命令的情况。

(11) 后处理例程地址(段:位移)域。占 4 B,指向在 NetBIOS 命令完成时调用的一个后处理例程,该后处理例程的地址用长指针表示(段地址:位移),这个域仅用于非等待状态的命令,若它为 0(或 NULL),NetBIOS 就不调用后处理例程,这时应用程序应根据 0xFF 值来检查 NCB_Cmd_Cplt 的变化。PostOff 和 PostSeg 亦可总称为 Post_Func。

换句话说,后处理例程地址非 0 时,NetBIOS 就调用这个后处理例程作为中断,NetBIOS 把寄存器 AL(指向返回码)、CS:IP(指向后处理例程)和 ES:BX(指向完成的 NCB)送入例程,把后处理程序入口置为撤销状态,并以 IRET 命令返回 NetBIOS。

后处理例程并不存储和恢复寄存器,可以发送 NetBIOS 请求。它不应发出 PC_DOS 请求,因为 PC_DOS 是不可重入的,况且这样的请求可能被中断,进而调用后处理例程。

(12) 适配器号(NCB_Lana_Num)域。占 1 B,它指出命令应该由那个适配器来处理,0 表示使用主适配器,1 表示为替换适配器卡,2 和 3 是为 IBM PC LAN 适配器保留的。

(13) 保留区(NCB_Reserve)域。占 14 B,NetBIOS 用它来存放返回的扩大的信息错误。另外,在请求处理时,NetBIOS 还把它用作中间存储器。应用程序中不能使用它。

要求

1. 利用 NetBIOS 数据报功能,编写两个工作站之间的信息收发功能,要求工作站把键盘输入的一行文字传送至对方,同时把收到的字符串显示出来。

2. 利用 NetBIOS 会话功能,编写两个工作站之间文件传送程序,要求能传输任意长度的二进制文件。

提示

本提示中对 NetBIOS 编程作进一步的深入介绍。

• 远指针及有关函数

在微机中有一个段结构,每一个结构和功能的地址都被指定为“段:位移”这种格式。因为网络支持在应用程序代码的不同段中,可以使用远指针告诉网络支持软件结构功能的位置。因此,远指针常用于网络程序设计。

远指针(far)不是让编译程序把程序数据段地址作为指针的段地址部分,而是把指针的段地址与指针的偏移量直接存放在指针内。因此,远指针由 4 B 构成,它可以指向内存中的任一目标,可以用于任一编译模式,另外,仅在紧凑、大、巨三种模式下远指针才是缺省的数据指针。

Borland C 等多种 C 语言都支持远指针,尽管它们在具体的处理上存在某些差别。Borland C++ 等为远指针提供了 3 个宏指令,这 3 个宏指令的用法和功能如下:

1. 设置一个远指针 MK_FP

调用格式为

```
#include <dos.h>
```

```
void far * MK_FP(unsigned seg,unsigned ofs);
```

其中 seg 表示段;ofs 表示位移。该函数的功能是通过段值(seg)和偏移量(ofs)建立一个远指针的宏。其原型在 dos.h 中。它返回一个远指针。

2. 获取远地址段值 FP_SEG

调用格式为

```
#include <dos.h>
unsigned FP_SEG(void far * p);
```

其中,FP_SEG 用于取得和设置远指针 *p 的段地址值,其原型在 dos.h 中。它返回一个无符号整数代表的段地址值。

3. 获取远地址偏移量 FP_OFF

调用格式为

```
#include <dos.h>
unsigned FP_OFF(void far * p);
```

该宏指令的功能是取得或设置远指针 *p 的偏移量(或位移),其原型在 dos.h 中。该宏指令返回一个无符号整数代表的偏移量。

• 后处理例程

在 NCB 结构中,有一个后处理例程的地址域(总的名称是:POST_Func),当该域为空(NULL)时,表明此 NCB 无相应的后处理例程。若执行一个非等待状态(no_wait)NetBIOS 命令,则程序将一直循环到 NCB_Cmd_Cplt 从 0xFF 变成一个实际的返回代码为止。因此,对于一条 no_wait 命令,必须编写一个后处理程序,将其地址送入 POST_Func 中,以供操作结束时 NetBIOS 调用。例如,应用程序可以编写下列程序段:

```
void interrupt answer_the_call()
{
}

listen_NCB_POST_Func = answer_the_call;
```

这里 answer_the_call()同说明中使用的 Interrupt 字一样,后处理例程(post)好像是一个中断处理程序。在调用时:中断被关闭,寄存器对“ES:BX”为指向完成事件 NCB 的远指针,最后返回代码送入 NCB_Cmd_Cplt 域和 AL 寄存器,在后处理程序中进行 DOS 功能调用(如磁盘文件操作等)可能产生错误。

后处理例程的执行速度必须很快,后处理完成后使用 IRET 机器指令退出例程,当使用关键字 Interrupt 时,编译程序将自动生成该指令。

• 调用 NetBIOS 命令

当应用程序希望调用 NetBIOS 的某一条命令来完成相应的功能时,就需要编写调用 NetBIOS 的程序,为此应用程序应先设置一个网络控制块 NCB,填写有关域的内容,将此 NCB 远指针放入“ES:BX”寄存器对中,然后调用 DOS 的中断 0x5C。下面给出具体调用的程序。

用 C 语言调用 NetBIOS 的程序。该程序直接向内部 NetBIOS 中断系统发出 NetBIOS 中断。

对于 IBM PC 机来说,也可以采用另一种方法,即使用 PC-DOS INT 2AH 功能的 01H 子功能(它的功能是执行 NetBIOS 请求,不进行错误重试,它等价于调用 INT 5CH),调用者应提供两个入口参数:AH=01H 以及 ES:BX 指向 NCB。可见在这两种情况下,ES:BX 都必须指向一个有效的 NCB。调用 NetBIOS 的 C 语言程序如下所示:

```
#define USGC unsigned char
#define USGI unsigned
#define USGL unsigned long

void NetBIOS ( NCBpointer)
struct NCB * NCBpointer;
{
    struct SREGS SegRegs;
    Union REGS InRegs, OutRegs; /* define in dos.h */
    struct NCB far * NCBPtr = (struct NCB far *)NCBpointers;
    segread( &SegRegs);
    SegRegs.es = FP_SEG( NCBPtr);
    InRegs.x.bx = FP_OFF( NCBPtr);
    int86x( NetbiosInt5C, &InRegs, &OutRegs, &SegRegs);
}
```

• 增加名字

NCB 中有一个 NCB_Name 域,它表示你在本地名字表中的名字,用于标识你自己(对收听或呼叫命令而言)。它是通过 NetBIOS 的加名字命令加入到本地名字表中的,由 NetBIOS 加名命令(和加组名命令)将加名所返回的 NetBIOS 名字号送入 NCB_Num 域中,可供发送数据报时使用。该名字不同于永久节点名。

加入本地名字表中的名字应为 15 B,并且在最后一个字节后为一个空字节。必须确认它是一个正常的字符串,并为名字表中正确的名字项。步骤一般如下:

- (1) 检查名字的正确性;
- (2) 确保这个新加入的名字是唯一的。

• NetBIOS 数据报服务

编写 NetBIOS 数据报程序时,本地站点名就是加名操作返回的名字号(即 NCB_Num 域的内容),数据报的目的地址指定为一个名字(或一个组名),被放入 NCB_CallName 域中。

NetBIOS 数据报有点对点方式和广播方式两种。假定有 A、B 终端以数据报方式通信其主要步骤如下:

工作站 A

增加名字 A

工作站 B

增加名字 B

发送报文→B

接收报文(A)

接收报文(B)

发送报文→A

删除名字 A

删除名字 B

• NetBIOS 会话服务

当应用程序要进行 NetBIOS 的会话通信时,首先必须在会话的两个站点之间创建一个会话,会话成功地建立后,双方才可以收发信息,且保证正确的接收次序,当会话结束时,还必须终止这次会话。其主要过程如下:

工作站 A

工作站 B

增加名字 A

增加名字 B

监听呼叫

呼叫 A

发送报文→B

接收报文(A)

接收报文(B)

发送报文→A

请求挂断 B

请求挂断 A

删除名字 A

删除名字 B

实验七 UNIX 下的 Socket 编程

描述

在 UNIX 环境下使用 Socket 编制网络通信程序。

目的

熟悉 Socket 基本概念、Socket 编程流程、I/O 多路复用,能使用 Socket 编制相关软件。

基础知识

• Socket 接口的概念

20 世纪 80 年代初,美国国防部的高级研究计划局(ARPA)给 Berkeley 的 California 大学提供了资金,让他们在 UNIX 操作系统下实现 TCP/IP 协议。在这个项目中,一组 Berkeley 研究人员为 TCP/IP 网络通信开发了一个 API。设计人员将这个 API 称为 Socket 接口。

Socket 接口是 TCP/IP 网络的 API。也就是说,Socket 接口规定了许多函数和例程,程序员可以用它们来开发 TCP/IP 网络上的应用程序。今天,Socket 接口是 TCP/IP 网络最为通用的 API 之一。

网络通信包括两台主机或两个进程,通过网络传递它们之间的数据。网络专家将网络对话的每一端称为一个端点。当使用 Socket 接口对网络通信编程时,Socket 是网络通信过程中端点的抽象表示。为了进行网络通信(通过 Socket 接口),程序在网络对话的每端都需要一个 Socket。两个 Socket 之间的通信可以是面向连接的(点对点连接),也可以是无连接的。

• Socket 处理过程

Socket 接口使用“打开→读→写→关闭”的处理过程。为了和一个 TCP/IP 网络通信,程序首先打开一个到网络的连接。接着,程序可以通过这个连接读和写数据。最后,程序必须关闭这个连接。

许多网络调用都需要一个变元,是指向 socket 地址结构的指针,在 <sys/socket.h> 中定义。

```
struct sockaddr {
    u_short sa_family;    /* address family; AF_XXX value */
    char sa_data[14];     /* up to 14 bytes of protocol-specific address */
};
```

其中 sa_data 是按照地址的类型进行解释的。对于 Internet 协议集,在 <netinet/in.h> 中:

```
struct in_addr {
```

```

        u_long s_addr;           /* 32 bit netid/hostid , network byte ordered */
    };
    struct sockaddr_in {
        short sin_family;         /* AF_INET */
        u_short sin_port;         /* 16 bit port number, network byte ordered */
        struct in_addr sin_addr;  /* 32 bit netid/hostid, network byte ordered */
        char sin_zero[8];        /* unused */
    };

```

一般说来,在使用 TCP/IP 协议时,应用程序在调用 Socket API 时可以直接使用 struct sockaddr_in 结构,它实际上是等同于 sockaddr。

• 建立 Socket

为了进行网络 I/O,第一件事就是调用 Socket 函数,指定所要求的协议的类型(TCP/UDP/IP 等)。Socket 函数返回一个类似文件描述符的句柄:

```
Socket_handler = socket(int family, int type, int protocol);
```

其中 family 可以选:AF_UNIX(UNIX socket)、AF_INET(Internet)等。

Socket 接口使用符号常量 SOCK_DGRAM 表示数据报,用 SOCK_STREAM 表示字节流。Socket 接口还定义了叫做 Raw Socket(原始 Socket)(SOCK_RAW)的第三种通信类型。Raw Socket 让程序使用网络自己的低层协议。

对于 TCP/IP 网络,协议的符号常数是以前缀 IPPROTO_开始的。例如,IPPROTO_TCP,IPPROTO_UDP 分别表示 TCP 和 UDP 协议。

调用 Socket 函数时,Socket 执行体将建立一个 Socket,并返回一个指向描述符表入口的 Socket 句柄(或描述符)。Socket 执行体为用户管理描述符表。访问描述符表的唯一途径是通过 Socket 描述符。

在通信完成之后,用户应该通过调用 close 函数来关闭 socket,释放所分配的资源。

```
Result = close(int sock_handle)
```

• 配置 Socket

面向连接的客户程序使用 connect 函数来配置 Socket。connect 函数要求用户指定一个 Socket 句柄、一个包含远地主机信息的地址结构和此地址结构的长度。

```
Result = connect(int socket_handle, struct sockaddr * servaddr, int addrlen);
```

connect 函数的第一个参数,Socket 句柄,是 Socket 函数返回的 Socket 描述符。第二个参数,远地 Socket 地址,是一个指向特定 Socket 地址结构的指针,其内容依赖于程序使用的协议族。第三个参数,地址长度,简单地指明 Socket 执行体远地 Socket 地址结构(第二个参数)的字节数。

由于无连接客户程序没有和远地主机建立直接连接,所以它必须对协议端口进行侦听,以便接收对它的服务请求所产生的数据报应答。同样,为了接收客户的请求,服务器程序必须对一个特定的协议端口进行侦听。Socket API 中的 bind 函数让程序将一个本地地址(包括主机地址和

协议端口)和一个 Socket 联系起来。

```
Result = bind(int socket_handle, struct sockaddr * myaddr, int addrlen);
```

• 通过 Socket 发送数据

(1) 通过一个连接的 Socket 发送数据。

使用面向连接数据的 Socket_API 函数不需要目的地址当作其参数。send、write 和 writev 函数只和连接的 Socket 工作,它们要求指明一个 Socket 句柄作为第一个参数。

```
Result = write(int socket_handle, char * message_buffer, int buffer_length);
```

Write 的第二个参数指向一个包含传输信息的数据缓冲区。第三个参数简单地指明缓冲区的大小,和 UNIX 下普通的文件 I/O 类似。

```
Result = writev(int socket_handle, struct iovec io_vector[], int iovcount);
```

```
struct iovec {
```

```
    caddr_t iov_base;          /* starting address of buffer */
```

```
    int iov_len;              /* size of buffer in bytes */
```

writev 的第二个参数指明一个包含一系列指针的数组的地址。当 writev 函数传输数据时,它将发送由指针数组指定的每一内存位置内包含的数据。Writev 的第三个参数指明数组的大小。Writev 提供了标准系统调用 write 的变形,可以写入不连续的缓存。

```
Result = send(int socket_handle, char * buffer, int buflen, int special_flags);
```

Send 函数可以通过可选标志, special_flags, 对传输进行控制,它或者为 0 或者为下面几种常数的“或”:MSG_OOB(接收加急数据)、MSG_DONTROUTE(旁路路由选择)。例如,你可以使用一个标志 MSG_OOB,告诉接收进程你在传送带外数据,它将具有比其他数据更高的优先级。

这 3 个函数都返回一个整数值,如果正确执行,函数将返回传送的字节数;如果发生错误,函数返回 -1。

(2) 通过一个无连接的 Socket 发送数据。

为了通过一个无连接的 Socket 发送数据,可以使用 sendto 和 sendmsg 这两个 Socket API 函数。

Sendto 函数要求 6 个参数,前 4 个参数和 send 函数相同。它的第五个参数确定目的地址,第 6 个参数指明目的地址的大小(以 B 为单位)。

```
Result = sendto(int socket_handle, char * buffer, int buflen, int flags, struct sockaddr * to, int addrlen);
```

sendmsg 函数传输数据时,让你使用一个报文结构而不是一个简单数据缓冲区。

```
Result = sendmsg(int socket_handle, struct msghdr * msg, int special_flags);
```

```
struct msghdr {
```

```
    caddr_t msg_name;          /* optional address */
```

```
    int msg_namelen;           /* size of address */
```



```

struct iovec * msg_iov;          /* scatter/gather array */
int msg_iovlen;                  /* # elements in msg_iov */
caddr_t msg_accrights;          /* access rights sent/received */
int msg_accrightslen;

```

其中的 `msg_iov` 为一个指针数组, 指针数组所指向的每一数据块中的数据都将被发送, 其数据结构见上文。Special_flags 和前面 `send` 函数中相应参数的意义一样。

• 通过 Socket 接收数据

Socket 接口包括 5 个与传送数据函数相对应的接收函数 (`read`, `readv`, `recv`, `recvfrom`, `recvmsg`)。对应的传送接收函数使用同样的参数。

```

Result = read(int sock_handle, char * buffer, int length);
Result = readv(int sock_handle, struct iovec iov[], int iovcount);
Result = recv(int sock_handle, char * buf, int len, int flags);
Result = recvfrom(int sock_handle, char * buf, int len, struct sockaddr * fromaddr, int fromlen);
Result = recvmsg(int sock_handle, struct msghdr * msg, int flags);

```

`Read` 中的 `buffer` 和 `length` 参数分别给出缓冲区地址以及缓冲区长度, 收到的数据将放在缓冲区中, 返回值给出了所接收的字节数, 如果出错, 返回 -1。

`Readv` 中的 `iov` 字段和 `iovcount` 一起指出了一些离散的缓冲区的位置, 这些缓冲区可以不是连续的, 收到的数据将放在缓冲区中, 返回值给出了所接收的字节数, 如果出错, 返回 -1。

`Recv` 中的 `buf` 和 `len` 指出了所使用的缓冲区, `flags` 字段和前面 `send` 中的类似, 用户通过设置这个字段来选择是否接收紧急数据 (`MSG_OOB`)、是否只看看队列中的数据而不把数据从队列中清除, 这样下次从该 Socket 接收数据时仍然接收到同样的数据 (`MSG_PEEK`)。

`recvfrom` 的 `buf` 和 `len` 指出了所使用的缓冲区, `fromaddr` 用来保存所接收的消息的发送者的地址, `fromlen` 指出该地址的长度。返回值给出了所接收的字节数, 如果出错, 返回 -1。

`Recvmsg` 的 `msg` 指出了所使用的缓冲区, 相应的数据结构在描述 `sendmsg` 时给出。flag 和前面 `recv` 中描述的一致。返回值给出了所接收的字节数, 如果出错, 返回 -1。

• 服务器的 Socket 使用

面向连接的服务器需要调用 `listen` 和 `accept` 函数。`listen` 函数先将服务器置于被动的侦听模式, `accept` 函数接着建立一个 socket 连接。

`listen` 函数将所有到达的服务请求在一个请求队列上排队, 也就是说, `listen` 函数让 socket 执行体建立一个输入数据队列, 将到达的服务请求保存在此队列上, 直到程序处理它们为止。

`listen` 函数需要两个参数: socket 句柄和队列长度。

```
Result = listen(int socket_handle, int query_length);
```

`Accept` 函数让服务器接收客户的连接请求。在建立好输入队列以后, 服务器就调用 `accept`

函数,然后睡眠,并等待客户的连接请求。

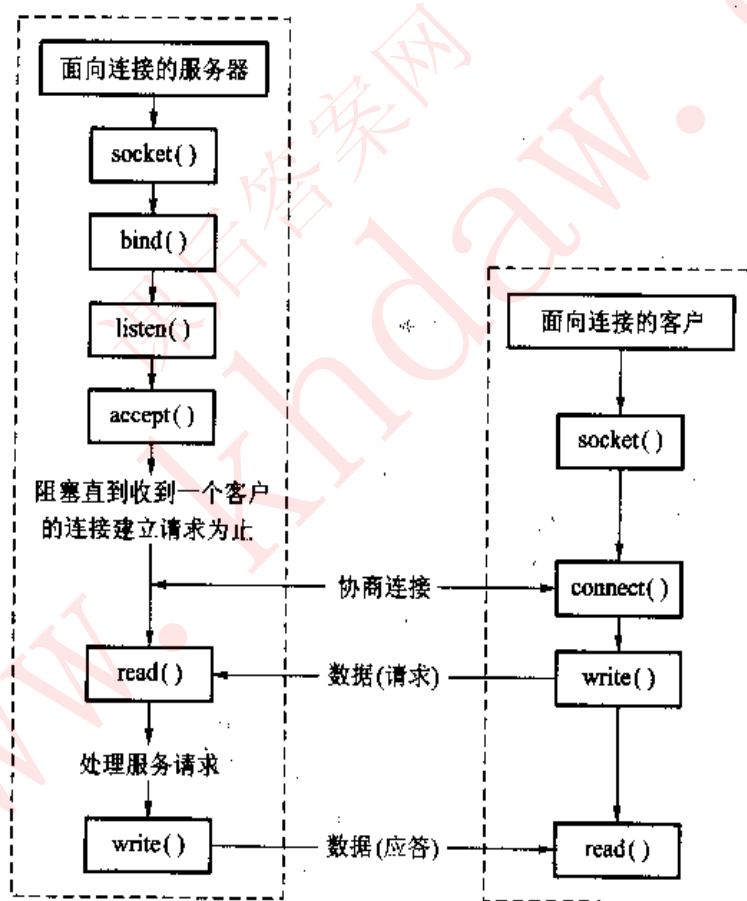
Accept 函数需要 3 个参数:socket 句柄,socket 地址和地址长度。当请求到达一个特定的 socket 时,socket 执行体把请求连接的客户地址填入第二个参数指向的 socket 地址结构中。

```
Result = accept(int socket_handle, struct sockaddr * addr,int length);
```

当 accept 函数监视的 socket 收到连接请求时,socket 执行体将建立一个新的 socket。执行体将这个新 socket 和请求连接进程的地址联系起来。收到服务请求的初始 socket 仍具有任意的网络地址,因此它可继续接收进来的服务请求。

• Socket 编程大致流程

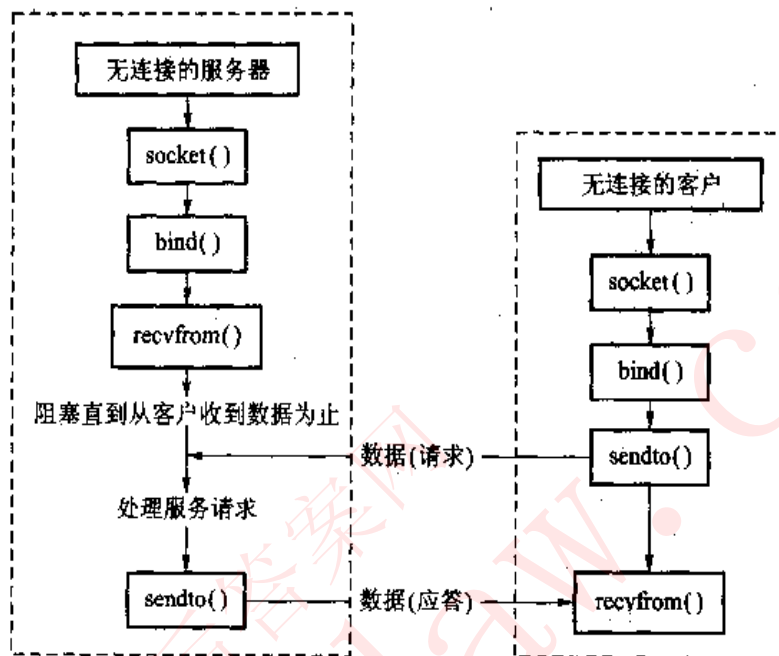
实验图 7.1 显示了程序使用面向连接协议时,进行的典型 socket 系统调用。服务器程序建立一个 socket,并调用 bind 函数将此 socket 和本地协议端口联系起来,然后用 listen 和 accept 函数将此 socket 函数置于被动的侦听模式并接收和建立连接。



实验图 7.1

客户程序也建立一个 socket,接着调用 connect 函数启动网络对话。在客户和服务端建立连接以后,就可以用 read,write 等函数进行通信了。

实验图 7.2 显示了无连接的典型 socket 通信过程。无连接的服务器和客户建立 socket 并用 bind 函数将此 socket 与一本地协议端口绑定,然后双方用 sendto、recvfrom 等函数进行通信。在每次通信中,必须指明目的地址。



实验图 7.2

• IO 多路复用

在 TCP/IP 下的网络应用中,有时需要同时打开几个 socket 来进行通信。由于这些 socket 上的通信是无法预知的,因此就要采取相应的措施。UNIX 系统中提供 select 系统调用。用户进程使用这种系统调用等待几种事件发生,一旦这几种事件中任何一种发生,内核将用户进程唤醒。

```
int select(int maxfd, fd_set * readfds, fd_set * writefds, fd_set * exceptfds, struct timeval *
timeout);
```

```
FD_ZERO(fd_set * fdset);          /* clear all bits in fdset */
FD_SET(int fd, fd_set * fdset);    /* turn the bit for fd on in fdset */
FD_CLR(int fd, fd_set * fdset);    /* turn the bit for fd off in fdset */
FD_ISSET(int fd, fd_set * fdset);  /* test the bit for fd in fdset */
Struct timeval {
    Long tv_sec;                    /* seconds */
    Long tv_usec;                   /* microseconds */
}
```

Readfds、writefds、exceptfds 分别对应于文件描述符可读、可写、例外条件三种情况,在进行网

络编程时,最关心的是让 socket 中有数据可供读出时通知进程。select 的调用有三种情况:

① 马上检查是否有指定的事件发生。timeout 参数指向的 timeval 结构,定时器值为 0。

② 等到指定的描述符中有一个已可进行 I/O 操作时才返回,但是等待时间不超过一个确定的值(由 timeout 定义)。

③ 可无限制等待,直到指定的描述符中有一个已可进行 I/O 操作时才返回。timeout 参数为空。这种等待可以被信号打断。

FD_ZERO、FD_CLR、FD_SET 宏用来设置相应的事件指示,在从 select 返回后,通过宏 FD_ISSET 来检测到底是发生了什么事情,从而调用正确的事件处理程序。

要求

利用 Socket 接口实现一个聊天室的功能;允许多人加入该聊天室,每人所说的话应该能够被现在正在聊天室里面的其他人听到,并且允许用户动态地加入和退出该聊天室。

提示

• 一些有用的库函数

对于不同计算机系统来说,它们存放数据字节的顺序是不同的。对于 2 B 整数,有的把低位字节放在前面,有的把高位字节放在前面。而 4 B 的整数则更加复杂。当这些整数通过网络传输时就可能产生问题。通过调用字节排序,在发送之前,先转换成网络顺序,当另外一台主机接收到之后,把数据从网络顺序转换到本机的顺序。

u_long htonl(u_long hostlong);	主机 - 网络转换,长整数
u_short htons(u_short hostshort);	主机 - 网络转换,短整数
u_long ntohl(u_long hostlong);	主机 - 网络转换,长整数
u_short ntohs(u_short hostshort);	主机 - 网络转换,短整数

Internet 的地址通常是由用小数点分隔的十进制数字表示的,如 192.43.235.1。下面两个函数进行地址转换:

```
unsigned long inet_addr(char * ptr);
char * inet_ntoa(struct in_addr inaddr);
```

其中,inet_addr 把以数字和点组成的字符串转变为 32 位 Internet 地址,inet_ntoa 完成相反的转变。

通常每台机器都有个可读的名字,当请求网络服务时就使用这些名字,因此需要把它们转换为适当的数字表达方式(gethostbyname)。有时已知主机的 Internet,希望找出它的名字,gethostbypaddr 函数就完成地址到名字的转换,其中 addr 指针指向含有该 Internet 地址的一个 sockaddr_in 结构,len 为这个结构的长度,type = AF_INET。Getservbyname 通过给定服务名和协议名,返回一个 servent 结构,其中给出了对应的端口号。

```
struct hostent * gethostbyname(char * hostname);
```

```
struct hostent {
    char h_name;           /* official name of host */
    char * h_aliases;      /* alias list */
    int h_addrtype;        /* host address type */
    char * h_addr_list;    /* list of addresses from name server */
                          /* a NULL terminates the list */
};
```

```
#define h_addr h_addr_list[0] /* first address in list */
struct hostent * gethostbyaddr( char * addr, int len, int type);
struct servent * getservbyname( char servname, char * protname);
```

以上介绍了 UNIX 下的 Socket 编程的基本思想,但是限于篇幅,不可能对其作一个详细的介绍,读者可以参考有关的网络编程方面的书籍。下面给出了一个简单的例子,客户方给服务方发送数据,服务方把接收到的数据显示出来。

通过 TCP 发送的客户方部分代码:

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>
#include <stdio.h>
#define DATA "This is just a test blah blah"
main(int argc, char * argv[])
{
    ...

    if ( ( sock = socket( AF_INET, SOCK_STREAM, 0) ) < 0 ) {
        perror( "opening stream socket" );
        exit(1);
    }

    server.sin_family = AF_INET;
    hp = gethostbyname( argv[1] );
    memcpy( ( char * ) &server.sin_addr, ( char * ) hp -> h_addr, hp -> h_length );
    server.sin_port = htons( atoi( argv[2] ) );
    connect( sock, ( struct sockaddr * ) &server, sizeof server );
    write( sock, DATA, sizeof DATA );
    close( sock );
    exit(0);
}
```


}
通过 TCP 接收的服务方部分代码:

```
main( int argc, char * argv[ ] )
{
    sock = socket( AF_INET, SOCK_STREAM, 0 );
    server.sin_family = AF_INET;
    server.sin_addr.s_addr = INADDR_ANY;
    server.sin_port = 0;
    bind( sock, ( struct sockaddr * ) &server, sizeof ( server ) );
    /* find out assigned port number and print it out */
    length = sizeof ( server );
    getsockname( sock, ( struct sockaddr * ) &server, &length );
    printf( "socket port # %d \n", ntohs( server.sin_port ) );
    listen( sock, 5 );
    while ( 1 ) {
        msgsock = accept( sock, ( struct sockaddr * ) 0, ( int * ) 0 );
        do {
            memset( buf, 0, sizeof ( buf ) );
            if ( ( rval = read( msgsock, buf, 1024 ) ) < 0 )
                perror( "reading stream message" );
            if ( rval == 0 )
                printf( "Ending connection \n" );
            else
                printf( " -- > %s \n", buf );
        } while ( rval != 0 );
        close( msgsock );
    }
    exit( 0 );
}
```

下面给出了通过 UDP 方式接收的部分服务方代码:

```
sock = socket( AF_INET, SOCK_DGRAM, 0 );
name.sin_family = AF_INET;
name.sin_addr.s_addr = INADDR_ANY;
name.sin_port = 0;
bind( sock, ( struct sockadr * ) &name, sizeof ( name ) );
```

```
length = sizeof(name);
getsockname(sock, (struct sockaddr *)&name, &length);
printf("socket port #%d\n", ntohs(name.sin_port));
if (read(sock, buf, 1024) < 0)
    perror("receiving datagram packet");
printf("-- > %s\n", buf);
close(sock);
```

通过 UDP 方式发送的部分客户方代码:

```
#define DATA "blah blah blah"
sock = socket(AF_INET, SOCK_DGRAM, 0);
hp = gethostbyname(argv[1]);
memcpy((char *)&name.sin_addr, (char *)hp->h_addr, hp->h_length);
name.sin_family = AF_INET;
name.sin_port = htons(atoi(argv[2]));
sendto(sock, DATA, sizeof(DATA), 0, (struct sockaddr *)&name, sizeof(name));
close(sock);
```

实验八 Winsock 编程(1)

描述

使用 Winsock 实现一个支持 TCP 和 UDP 方式的 ECHO 服务,并且可以用 TCP 或者 UDP 方式来连接。

目的

熟悉 Winsock 机制,能熟练利用 Winsock 实现 TCP 和 UDP 方式的 ECHO 服务,并可通过 TCP 或者 UDP 方式来连接。

基础知识

• Winsock API 分类

Windows Sockets (Winsock) 是一个基于 Socket 模型的 API。Windows API 只在 Microsoft Windows 操作系统类(包括 Microsoft Windows、Windows NT 和 Windows 95)中使用。Winsock 是以动态链接库(DLL)来实现 Socket 接口的。

本文的读者需要对 Socket 接口和 Windows 环境下的编程有一定的认识。

Winsock 规范将 API 库分为三类:

- ① Winsock API 包含的 Berkeley socket 函数。
- ② 检索有关域名、通信服务和协议等 Internet 信息的数据库函数。
- ③ Berkeley socket 例程的 Windows 专用的扩展函数。

Winsock 数据库函数可以让程序获得有关域名、通信服务和协议等 Internet 信息。这些函数通过访问各种数据库资源(本地和远地的)得到这些信息。Winsock 规范定义了这些可;接口以及数据库函数返回的数据。

如果你的程序需要可变数据区中的数据的话,程序必须将此数据拷贝到另一内存地址,即程序分配和维护的内存中,在程序调用另一个 Winsock 函数以前。Winsock 缓冲区中的可变数据在程序调用另一个 Winsock 函数以前是有效的。

由于 Windows 是基于消息驱动机制的操作环境,所以,为了让程序员利用 Windows 内部的消息传送机制,Winsock 开发者设计了某些 socket 函数的异步版本。这些函数以“WSA”打头。

要求

利用 Winsock 实现一个支持 TCP 和 UDP 方式的 ECHO 服务,服务方接收请求,把收到的数

据原封不动地送回去。要求能够支持多个用户同时使用该服务,并且可以用 TCP 或者 UDP 方式来连接。

提示

本提示中对 Winsock 接口编程作进一步的深入介绍。

- 包含一个新的头文件

在 Windows 环境下,编写网络程序时,需要头文件 Winsock.h。但不需要包含 Windows.h 头文件,因为 Winsock.h 头文件依赖于 Windows.h 中的定义,Winsock.h 头文件中已包含了 Windows.h

- Winsock 的初始化和清除

Windows Socket 要求 Winsock 程序都使用两个与 Windows 有关的函数,WSAStartup 和 WSACleanup。在调用任何其他 Winsock 函数以前,必须先调用 WSAStartup 函数。对每个 WSAtartup 函数调用,程序必须在后面包括一个相应的 WSACleanup 函数调用。

WSAStartup 函数让程序符合指定要求的 Winsock API 的版本,并检索有关 Winsock 执行体的信息。当程序调用 WSAStartup 时,该程序就和 Winsock.dll 开始了一个协调工作。其函数原型为:

```
Result = WSAStartup( WINSOCK_VERSION, &wsaData );
```

其中的常量 WINSOCK_VERSION 指明了 Winsock 版本,例如,0x0101 表示 Winsock 1.1 版。wsaData 是一个 WSADATA 变量,WSADATA 定义了一个包含 Winsock 执行体的版本信息、描述和 socket 信息的数据结构。若 WSAStartup 函数调用失败,将返回“-1”。

WSACleanup 函数对应 WSAStartup 函数。程序每次调用 WSAStartup 后,都必须相应地调用 WSACleanup 函数。Winsock 使用一个内部计数器。程序每次调用 WSAStartup 时,Winsock 给计数器加一。相反的,程序每次调用 WSACleanup 函数时,Winsock 对计数器减一。最后的 WSACleanup 函数告诉 Winsock 进行清除工作。

当程序最后一次调用 WSACleanup 函数时,Winsock 将断开与所有现存字节流 socket 的连接。但在调用 WSACleanup 以后,Winsock 还是会将已关闭的 socket 中未传送完的数据传出去。

- 使用 Winsock 错误处理函数

Winsock 定义了一个新的常数 SOCKET_ERROR(Winsock 定义它为 -1)来表示 socket 错误。因此当发生一个 socket 错误时,Winsock 函数将返回一个 SOCKET_ERROR 常数。程序必须调用 Winsock 专用的 WSAGetLastError 函数来确定引起错误的原因。

- 建立 socket

```
socket_handle = socket( protocol_family, socket_type, protocol );
```

同 Berkeley_socket 一样,Winsock 通过调用 socket 函数建立一个 socket。虽然两者的参数完全一样,但返回的句柄却不同。Winsock 定义了一种叫做 SOCKET 的新的无符号数据类型,将它

用作文件句柄。Winsock 的 socket 函数将返回一个 SOCKET, 常数 INVALID_SOCKET 表示无效 socket。

- 配置 socket

对于面向连接的客户程序, 可以调用 connect 函数在 socket 数据结构中保存本地和远地信息, 并与远地主机建立连接。

对于无连接的服务器程序以及无连接的客户程序, 可以调用 bind 函数将 socket 与某一本地协议端口绑定, 这样就可以对该协议端口进行侦听, 并通过该端口进行传送与接收。

对于面向连接的服务器程序, 首先需用 bind 函数将 socket 与一本地协议端口绑定, 然后调用 listen 和 accept 函数将此 socket 函数置于被动的侦听模式并接收和建立连接。

上述 Winsock 函数与相对应的 Berkeley_socket 函数使用一样的参数, 其原型如下:

```
result = connect(socket_handle, remote_socket_address, address_length);
result = bind(socket_handle, local_socket_address, address_length);
result = listen(socket_handle, query_length);
result = accept(socket_handle, socket_address, address_length);
```

- 使用 socket

Winsock 有 4 个函数用于数据传输: send 和 sendto 函数用于数据传送, recv 和 recvfrom 函数用于数据接收。如果函数执行成功, 将返回实际传输的字节数。send 和 recv 函数只用于连接的 socket, 而其他函数不受限制。程序可以在无连接的 socket 上使用 sendto 和 recvfrom 函数, 也可在连接的 socket 上使用 sendto 和 recvfrom 函数, Winsock 简单地忽视了它们的地址参数。

- 使用一个连接的 socket

下面程序语句是一个典型的 send 函数调用:

```
result = send(socket_handle, message_buffer, buffer_length, special_flags);
```

连接的 socket 数据结构中保存了远地主机的地址信息, send 函数将 message_buffer 里的数据发送给该主机地址。而 recv 函数从本地协议端口获得数据, 并将数据保存到报文缓冲区中。

```
result = recv(socket_handle, message_buffer, buffer_length, special_flags);
```

Winsock 为 recv 函数的最后一个参数 special_flags 提供了两个特殊标志。程序使用符号常数 MSG_OOB 作为标志值, 表示可以从协议端口请求带外数据。带外数据是程序必须立即处理的紧急数据。如果使用 MSG_OOB 标志调用 recv 函数, 带外数据存在的话, recv 立即将紧急数据返回给程序, 否则, 将返回常数错误值 EINVAL。

程序使用符号常数 MSG_PEEK 作为标志值, 表示可以对运输层输入队列中的数据进行分析。通常情况下, 当程序从输入队列拷贝完数据以后, 运输层将放弃缓冲区中所有的数据。但如果指定 MSG_PEEK 标志的话, 运输层不会从输入队列中移走拷贝的数据。因此, 程序可以在决定怎样处理输入数据以前对它们进行分析, 接着就可以决定是检索所有还是一部分数据。

当然, 程序可将特定标志参数指定为 0, 以进行一般的传输。

- 使用一个无连接 socket

无连接 socket 使用 sendto 和 recvfrom 函数进行数据传输。

```
result = sendto( socket_handle, message_buffer, buffer_length, special_flags,
                socket_address_structure, address_structure_length );
result = recvfrom( socket_handle, message_buffer, buffer_length, special_flags,
                  socket_address_structure, address_structure_length );
```

这些 Winsock 函数与相对应的 Berkeley_socket 函数使用一样的参数。

- 阻塞函数、非阻塞函数和异步函数

Winsock 将函数分为阻塞函数和非阻塞函数。阻塞函数在完成其网络操作之前禁止程序调用其他 Winsock 函数。非阻塞函数要么立即完成其操作要么返回一个错误信息,它不等待其工作完成。一般情况下,阻塞函数是完成网络 I/O 具有 Berkeley 风格的函数,而非阻塞函数不要求网络 I/O,它们完成的工作不用与远地主机通信,因此不会启动阻塞操作。

Winsock 专有的异步函数利用了 Windows 的消息驱动机制。异步函数调用只对启动操作负责,它也不等待操作完成,所以异步函数是不阻塞的。操作系统监视此操作的完成。当一个异步操作完成时,Windows 给程序启动该异步操作的目标(通常是一个窗口)发送一个消息。

- 使用非阻塞 socket

在缺省情况下,Berkeley 和 Winsock 都是建立阻塞的 socket,当该 socket 调用阻塞函数时,将引起阻塞。但可以调用 Winsock 的异步函数 WSAAsyncSelect 把一个 socket 规定为非阻塞 socket。这样,Winsock 将确保使用此 socket 的函数(包括阻塞函数)不会引起阻塞操作。

当用一个非阻塞 socket 调用一个会引起阻塞操作的函数时,Winsock 执行体总是在启动请求操作以前先检查此函数是否可以立即完成其操作。如果 socket 执行体不能立即执行完此函数,它将返回一个错误。如果请求的函数不会引起阻塞,此函数将正确执行完。

- 使用 WSAAsyncSelect 函数

如上所述,WSAAsyncSelect 函数可以将一个 socket 从阻塞的变为非阻塞的。下面语句显示了该函数的原型:

```
int PASCAL FAR WSAAsyncSelect( SOCKET s, HWND hWnd,
                               unsigned int wMsg, long lEvent );
```

WSAAsyncSelect 的第 1 个参数指出想监视的 socket 句柄。第 2 个参数指出接收消息的窗口。第 3 个参数是一个消息标识符。WSAAsyncSelect 的第 4 个参数是一个屏蔽位,它指明程序想监视的一组网络事件。可使用实验表 8.1 中的常数作为 WSAAsyncSelect 函数的 lEvent 参数值,也可使用位操作符 OR 来组合这些常数值。

当程序中 WSAAsyncSelect 函数调用指定的网络事件之一发生时,Windows 以标准 Windows 格式发送一条消息。标准 Windows 消息包括一个窗口句柄、一个消息标识符和两个消息参数(一个 16 位参数和一个 32 位参数)。作为 WSAAsyncSelect 的结果,Windows 发送的 16 位消息参数指明了发生此事件的 socket,32 位参数的低 16 位指明了发生的网络事件,高 16 位包含错误码。

实验表 8.1

常 数	含 义
FD_READ	请求通知读已准备好
FD_WRITE	请求通知写已准备好
FD_UDB	请求通知带外数据已到达
FD_ACCEPT	请求通知已收到连接
FD_CONNECT	请求通知已完成连接
FD_CLOSE	请求通知 socket 已关闭

现在假设程序想使用 `recv` 函数而不希望发生阻塞操作,那么程序必须调用 `WSAAsyncSelect` 函数设置 socket 句柄,并把 `lEvent` 参数指定为 `FD_READ`。`WSAAsyncSelect` 函数将告诉 Windows 当 socket 上数据可读时通知程序。当 socket 收到数据时,Windows 发送相应的消息标识符给 `WSAAsyncSelect` 指定的窗口。由于 Windows 只在数据到达后才发送消息,所以 `recv` 函数能够立即得到新来的数据而不会发生阻塞。

- 检测和取消阻塞操作

Winsock 提供了两个函数帮助程序员检测和处理阻塞操作:`WSAIsBlocking` 和 `WSACancelBlockingCall`。

```
BOOL PASCAL FAR WSAIsBlocking(void);
```

```
int PASCAL FAR WSACancelBlockingCall(void);
```

调用 `WSAIsBlocking` 函数可确定当前是否有阻塞操作,如果有的话,它将返回布尔值 `TRUE`,否则返回 `FALSE`。调用 `WSACancelBlockingCall` 函数将取消当前的阻塞操作。产生该阻塞操作的函数调用接着以错误值“`WSAEINTR`”返回。如果在阻塞操作中调用其他 Winsock 函数(不是 `WSAIsBlocking` 和 `WSACancelBlockingCall` 的其他函数),Winsock 将使该函数以错误值“`WSAIN-PROGRESS`”返回。

下面给出了一个简单的用 Winsock 进行编程的例子,允许一台机器中的用户从另一台机器了解到当前的日期和时间(`DAYTIME` 服务),注意在我们的例子中,允许客户方使用 TCP 或者 UDP 来和服务方通信以取得时间,服务方同时支持 TCP 和 UDP:

```
#include <stdio.h>
#include <time.h>
#include <winsock.h>
#define QUEUE_LEN      5
#define LINELEN         128
main(int argc, char * argv[ ])
{
```

```

char * service = "daytime";      /* service name or port number */
char buf[ LINELEN + 1 ];        /* buffer for one line of txt */
struct sockaddr_in fsin;        /* the request from address */
SOCKET tsock, usock;            /* TCP or UDP socket */
fd_set rfd;                     /* readable file descriptors */
struct sockaddr_in sin;
WSADATA wsadata;

...

WSAStartup( WINSOCKS_VERSION, &wsadata );
memset( &sin, 0, sizeof( sin ) );
sin.sin_family = AF_INET;
sin.sin_addr.s_addr = INADDR_ANY;
pse = getservbyname( service, "UDP" );
sin.sin_port = pse -> s_port;
usock = socket( AF_INET, SOCK_DGRAM, 0 );
bind( usock, ( struct sockaddr * ) &sin, sizeof( sin ) );
memset( &sin, 0, sizeof( sin ) );
sin.sin_family = AF_INET;
sin.sin_addr.s_addr = INADDR_ANY;
pse = getservbyname( service, "TCP" );
sin.sin_port = pse -> s_port;
tsock = socket( AF_INET, SOCK_STREAM, 0 );
listen( tsock, QUEUE_LEN );
FD_ZERO( &rfd );
while ( 1 ) {
    FD_SET( tsock, &rfd );
    FD_SET( usock, &rfd );
    if ( select( FD_SETSIZE, &rfd, ( fd_set * ) 0, ( fd_set * ) 0, ( struct timeval * ) 0 )
== SOCKET_ERROR )
        ...
    if ( FD_ISSET( tsock, &rfd ) ) {
        SOCKET ssock; /* TCP slave socket */
        alen = sizeof( fsin );
        ssock = accept( tsock, ( struct sockaddr * ) &fsin, &alen );
        daytime( buf );
    }
}

```

```

        send( ssock, buf, strlen( buf ), 0 );
        closesocket( ssock );
    }
    if ( FD_ISSET( usock, &rfd ) ) {
        alen = sizeof( fsin );
        rv = recvfrom( usock, buf, sizeof( buf ), 0, ( struct sockaddr * )&fsin, &alen );
        daytime( buf );
        sendto( usock, buf, strlen( buf ), 0, ( struct sockaddr * )&fsin, sizeof( fsin ) );
    }
}

WSACleanup( ); /* not reached */
exit( 0 );
}

void daytime( char buf[ ] )
{
    time_t now;
    ( void ) time( &now );
    sprintf( buf, "%s", ctime( &now ) );
}

```

通过 TCP 方式连接的客户方代码：

```

WSAStartup( WINSOCK_VERSION, &wsadata );
memset( &sin, 0, sizeof( sin ) );
sin. sin_family = AF_INET;
pse = getservbyname( service, "TCP" );
sin. sin_port = pse -> s_port;
phe = gethostbyname( argv[ 1 ] );
memcpy( &sin. sin_addr, phe -> h_addr, phe -> h_length );
sock = socket( AF_INET, SOCK_STREAM, 0 );
connect( sock, ( struct sockaddr * )&sin, sizeof( sin ) );
cc = recv( sock, buf, LINELEN, 0 );
while( cc != SOCKET_ERROR && cc > 0 ) {
    buf[ cc ] = '\0';
    ( void ) fputs( buf, stdout );
}

```

```

        cc = recv( s, buf, LINELEN, 0 );
    }
    closesocket( s );
    WSACleanup( );

```

通过 UDP 方式连接的客户方代码:

```

WSAStartup( WINSOCK_VERSION, &wsadata );
memset( &sin, 0, sizeof( sin ) );
sin. sin_family = AF_INET;
pse = getservbyname( service, "UDP" );
sin. sin_port = pse -> s_port;
phe = gethostbyname( argv[ 1 ] );
memcpy( &sin. sin_addr, phe -> h_addr, phe -> h_length );
sock = socket( AF_INET, SOCK_DGRAM, 0 );
connect( sock, ( struct sockaddr * ) &sin, sizeof( sin ) );
cc = recv( sock, buf, LINELEN, 0 );
while( cc != SOCKET_ERROR && cc > 0 ) {
    buf[ cc ] = '\0';
    ( void ) fputs( buf, stdout );
    cc = recv( s, buf, LINELEN, 0 );
}
closesocket( s );
WSACleanup( );

```


实验九 Winsock 编程(2)

描述

基于 Winsock,基本要求是实现一个简单的聊天程序,有客户端和服务端,双方可以一对一的发送消息,类似于 Windows 自带的 NetMeeting 聊天功能(如能实现多人加入,则更好)。

目的

通过该实验,使得学生深入了解 Windows 的 Winsock 编程机制,进而理解 UNIX 下的 Socket 编程机制,因为 Winsock 就是一个基于 Socket 的 API。

基础知识

Windows Sockets(Winsock)是一个基于 Socket 模型的 API。Windows API 只在 Microsoft Windows 类操作系统中使用。

• Winsock 接口编程

首先,必须包含一个新的头文件。在 Window 环境下,编写网络程序时需要头文件 Winsock.h。(现在已经升级到更高的版本了,但是本题的介绍还是以 1.1 版本为准。实际做题过程中可随意选择版本,相应的可能就需要包含 Winsock2.h 文件。)

使用 Winsock 编程的步骤为:

(1) Winsock 的初始化,加载 Winsock DLL 的相应版本。

如果调用 Winsock 之前,没有加载 Winsock 库,就会返回一个 WSANOTINITIALISED 的 SOCK_ERROR,一般加载 Winsock 库都是通过调用

```
int WSAStartup(WORD wVersionRequested, LPWSADATA lpWSADATA);
```

其中 wVersionRequested 指明 Winsock 的版本号,例如 0x0101 表示 Winsock 1.1 的版本。LpWSAData 定义了一个包含 Winsock 执行体的版本信息、描述和 Socket 信息的数据结构。

(2) 错误检查。

由于对于一个好的 Winsock 应用程序,错误的检查和控制非常重要,所以现在先介绍一下错误的检查和控制。一般地发生错误以后,调用

```
int WSAGetLastError(void);
```

就会返回所发生的特定的错误代码。

(3) 建立 Socket。

要利用 Socket 进行通信,必须先创建一个 Socket,其创建 Socket 调用的函数为

```
socket(protocol_family, Socket_type, protocol);
```

一般的通信都是基于 TCP/IP 的, 所以其创建调用为

```
SOCKET pSock = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
```

(4) 配置 Socket。

要建立连接进行通信, 连接必须在一个已知的“标示”上。在 TCP/IP 中, 这个“标示”就是本地接口的 IP 地址加上一个端口号。基于某一种协议进行通信, 首先必须将 C 中创建的 Socket 绑定到这个“标示”上, 它是通过调用 API 函数 bind 来完成的:

```
int bind(socket_handle, local_socket_address, address_length);
```

其中 socket_handle 为 C 中创建的 socket, local_socket_address 为 sockaddr_in 的数据结构, address_length 为该数据结构的长度。譬如应用 TCP/IP 协议, 8888 端口, 则可以如下定义:

```
struct sockaddr_in addr_tcp;
int port = 8888;
addr_tcp.sin_family = AF_INET;    //IP 协议
addr_tcp.sin_port = htons(port);
addr_tcp.sin_addr.s_addr = htonl(INADDR_ANY);
bind(pSock, (SOCKADDR *)&addr_tcp, sizeof(addr_tcp));
```

绑定后, 必须设置 socket 为监听模式, 其调用的 API 函数为 listen, 函数定义为

```
int listen(SOCKET socket_handle, int backlog);
```

其中 socket_handle 为指定的 Socket; backlog 则指定了正在等待连接的最大队列长度, 它实际上决定了一个服务端最多能同时为几个客户端提供服务。加入指定 3 个客户端, 则可以如下调用:

```
int result = listen(pSock, 3);
```

对于服务器端, 则首先要接受客户端的请求, 然后才能进行数据传输, 接受请求的 API 为 accept, 其定义格式为:

```
SOCKET accept(SOCKET sock, struct sockaddr FAR * addr, int FAR * addrlen);
```

其中, 参数 sock 是 (4) 中设置的处于监听模式的一个 Socket。第二个参数是一个合法的 SOCKADDR_IN 结构的地址, addrlen 是 SOCKADDR_IN 的长度。该函数调用返回的是对应于已经接受的那个客户机连接的 socket, 对该客户机所有的后续操作, 都对应于这个 socket。原来处于监听状态的 socket 保持不变。

而对于客户端, 则在 (3) 中创建 socket 后, 调用 API 函数 connect 即可, 其定义格式为

```
int connect(socket_handle, remote_socket_address, address_length);
```

其中的参数定义和 bind 函数中的定义相同, 唯一的区别就在于

```
addr_tcp.sin_addr.s_addr = htonl(ServerAddr);
```

其中 ServerAddr 是服务器的地址。

(5) 使用 Socket。

接下来就可以利用 socket 发送数据了, 其调用的函数为 send, 其定义格式为:

```
int send(SOCKET sock, const char FAR * buffer, int length, int flags);
```

sock 为已建立的 Socket,buffer 为将要发送的数据。length 为要发送的数据的长度,flags 可取 0、MSG_DONTROUTE 或 MSG_OOB。MSG_DONTROUTE 要求传送层不要将发送的数据路由出去;MSG_OOB 预示着数据应该被带外发送。

对于已连接 socket 上接收数据,一般通过调用 recv 来实现,其定义格式如下:

```
int recv(SOCKET sock, char FAR * buffer, int length, int flags);
```

buffer 为接收数据的缓冲区,length 为该缓冲区的长度,而 sock、flags 和 send 函数相同。

当作为一个无连接的 socket 使用时,socket 不需要调用 listen 和 accept,其发送、接收数据如下:

```
int sendto(SOCKET sock, char FAR * buffer, int length, int flags, struct sockaddr FAR *
aim_addr, int addr_length);
```

其中,前 4 个参数和 send 一样,aim_addr 是一个指向 SOCKADDR 结构的指针,带有接受数据方的目标地址。addr_length 为地址结构长度指针。

接收数据调用:

```
int recvfrom(SOCKET sock, char FAR * buffer, int length, int flags, struct sockaddr FAR
* source_addr, int addr_length);
```

除了 source_addr 带有的是发送方的地址外,其他参数和 sendto 相同。

(6) 关闭 socket。

通信完毕,中断连接,释放关联到该 socket 的资源,即调用 closesocket 函数,但是直接调用该函数可能会导致数据的丢失,随意建议在调用该函数之前,先调用 shutdown 函数来中断连接,然后调用 closesocket 函数。

shutdown 函数的定义如下:

```
int shutdown(SOCKET sock, int shut_style);
```

其中 sock 是要中断的 socket;shut_style 可取以下值:SD_RECEIVE、SD_SEND、SD_BOTH。

closesocket 的定义格式如下:

```
int closesocket(SOCKET sock);
```

最后调用 WSACleanup() 卸载 Winsock 的 dll。

要求

1. 对聊天屋的界面不作要求,简单、直观即可。
2. 具体代码可参考提示中的代码。

提示

下面给出一段代码,尽可能简单地实现服务器端建立 socket 并等待客户端的连接,客户端和服务端建立连接后,发送一段字符串,相应的服务器端也可以向客户端发送代码。

服务器端代码:

```
//server.c
#include <winsock.h>
//程序中 ClientThead 为线程处理的函数,如下:
DWORD WINAPI ClientThread(LPVOID lpParam)
{
    SOCKET sock = (SOCKET)lpParam;
    char szBuffer[1024];
    int ret;
    while (1)
    {
        ret = recv(sock, szBuffer, 1024, 0);
        if (ret == 0)
            break;
        else if (ret == SOCKET_ERROR)
        {
            printf("recv error! \n");
            break;
        }
        szBuffer[ret] = '\0';
        printf("Recv: '%s' \n", szBuffer);
    }
    return 0;
}

int main()
{
    SOCKET sListen;
    SOCKET sClient;
    WSADATA wsd;
    int addrsz;
    HANDLE m_hListenThread;
    DWORD dsThreadID;
    struct sockaddr_in local, client;

    if (WSAStartup(MAKEWORD(2, 2), &wsd) != 0)
```

```

    }

    AfxMessageBox( " failed to load Winsock! \n" );
    return -1;
}

sListen = socket( AF_INET, SOCK_STREAM, IPPROTO_IP );
if ( sListen == SOCKET_ERROR )
{
    printf( " create socket error! \n" );
    return -1;
}

local.sin_family = AF_INET;
local.sin_port = htons( 8888 );
local.sin_addr.s_addr = htonl( INADDR_ANY );
if ( bind( sListen, ( SOCKADDR * ) &local, sizeof( local ) ) == SOCKET_ERROR )
{
    printf( " bind socket error! \n" );
    return -1;
}

listen( sListen, 3 );
while( 1 )
{
    addrsz = sizeof( client );
    sClient = accept( sListen, ( struct sockaddr * ) &client, &addrsz );
    if( sClient == INVALID_SOCKET )
    {
        printf( " accept error! \n" );
        break;
    }

    m_hListenThread = CreateThread( NULL, 0,
                                    ClientThread, ( LPVOID ) sClient, 0, &dsThreadID );
    //创建线程来处理当前客户连接
    if ( m_hListenThread == NULL )
    {
        printf( " CreateThread error! \n" );
        break;
    }
}

```

```

        }
        CloseHandle( m_hListenThread );
    }
    closesocket( sListen );
    WSACleanup( );
    return 0;
}
//发送端程序
//client. c
#include < winsock. h >
int main( )
{
    SOCKET real_client;
    struct sockaddr_in server_addr;
    WSADATA wsd;
    char * buff = "Hello, server!";
    if ( WSStartup( MAKEWORD( 2, 2 ), &wsd ) != 0 )
    {
        printf( "failed to load Winsock! \n" );
        return -1;
    }
    server_addr. sin_family = AF_INET;
    server_addr. sin_port = htons( 8888 );
    server_addr. sin_addr. s_addr = inet_addr( "10. 11. 4. 200" );
    real_client = socket( AF_INET, SOCK_STREAM, IPPROTO_IP );
    if ( connect( real_client, ( struct sockaddr * )&server_addr, sizeof( server_addr ) )
        == SOCKET_ERROR )
        printf( "connection error! \n" );
    else
        send( real_client, buff, 14, 0 );
}

```


实验十 支持内容过滤的 HTTP 及 FTP 代理

描述

编写一个支持包括简单内容过滤在内等基本功能的 HTTP 代理服务器软件。

目的

了解 HTTP 代理服务器的基本原理以及简单的内容或网站过滤的基本方法。

基础知识

代理服务器具有以下几个优点：

1. 安全屏障

代理服务器(Proxy)是建立在应用层的软件,它的最主要功能是作为防火墙的重要组成部分,为它保护的主机起到安全屏障的作用。代理服务器还包括流量控制、包过滤、访问控制和操作系统用户集成等功能。

2. 提高网络性能

当 Internet 的用户以指数函数增长时,网络的拥挤是网络管理的一个重要问题。代理服务器就好像一个大储藏库,它有很大的缓存空间。如果通过该代理服务器有人去过了某个站点,传输了某些内容,那么代理服务器会将相关的资料(包含文字、图形、多媒体……)备份一份在代理服务器中。当用户再要通过该代理服务器访问相同地址时,可直接从缓存中获得数据。

3. 多机共享一个 IP 地址

当一个局域网只有一个 IP 时,就可以在服务器上安装代理服务器,整个局域网的用户就可通过一个 IP 访问 Internet 了,这样将会给局域网用户节省大量申请 IP 的费用。

要求

1. 内容过滤:如果用户访问的网页包含了一些特殊的关键字(比如可以保存在 keywords.txt),则 HTTP 代理把该网页的内容丢弃,并给用户发送一个网页,指明该网页被禁止访问。

2. 网站过滤:如果用户访问某些不恰当的网站(比如保存在 bannedsite.txt),则 HTTP 代理给用户发送一个网页,指明该网站被禁止访问。

提示

一个用于 WIN32 环境的 http proxy 的参考源代码:

```
#define HTTP "http://"
#define PROXYPORT 5001 //代理服务器端口
#define BUFSIZE 60000 //缓冲区大小

using namespace std;
UINT ProxyToServer(LPVOID pParam);
UINT UserToProxyThread(void * pParam);
struct SocketPair{
    SOCKET user_proxy; //socket;本地机器到代理服务器
    SOCKET proxy_server; //socket;代理服务器到远程主机
    BOOL IsUser_ProxyClosed; //本地机器到代理服务器状态
    BOOL IsProxy_ServerClosed; //代理服务器到远程主机状态
};
struct ProxyParam{
    char Address[256]; //远程主机地址
    HANDLE User_SvrOK; //代理服务器到远程主机的连接状态
    SocketPair * pPair; //维护一组 Socket 的指针
    int Port; //用来连接远程主机的端口
}; //这个结构用来代理服务器与远程主机的信息交换
SOCKET gListen_Socket; //用来侦听的 Socket
//该步骤包括初始化 Winsock,加载 Winsock 的 dll,等
int StartServer( ) //启动服务
{
    WSADATA wsaData;
    sockaddr_in local;
    SOCKET listen_socket;
    if(:: WSASStartup(0x202,&wsaData)!=0)
    {
        printf("\nError in Startup session.\n"); WSACleanup( );
        return -1;
    };
    local.sin_family = AF_INET;
    local.sin_addr.s_addr = INADDR_ANY;
    local.sin_port = htons(5000);
    listen_socket = socket(AF_INET,SOCK_STREAM,0);
    if(listen_socket == INVALID_SOCKET)
```

```

    {
        printf( "\nError in New a Socket. " );
        WSACleanup( );
        return -2;
    }
    if( bind( listen_socket, ( sockaddr * ) &local, sizeof( local ) ) != 0 )
    {
        printf( "\n Error in Binding socket. " ); WSACleanup( );
        return -3;
    }
    if( listen( listen_socket, 5 ) != 0 )
    {
        printf( "\n Error in Listen. " );
        WSACleanup( );
        return -4;
    }
    gListen_Socket = listen_socket;
    AfxBeginThread( UserToProxyThread, NULL ); //启动侦听
    return 1;
}

//关闭服务
int CloseServer( )
{
    closesocket( gListen_Socket );
    WSACleanup( );
    return 1;
}

//得到远程主机地址
int GetAddressAndPort( char * str, char * address, int * port )
{
    char buf[ BUFSIZE ], command[ 512 ], proto[ 128 ], * p;
    int j;
    sscanf( str, "%s %s %s", command, buf, proto );
    p = strstr( buf, HTTP );
    //HTTP

```

```

if( p)
{
//此处可对客户端访问的网站进行判别、处理
    p += strlen( HTTP );
    for( int l = 0; l < strlen( p ); l ++ )
    {
        if( * ( p + l ) == '/' )
            break;
    }
    * ( p + l ) = 0;
    strcpy( address, p );
    p = strstr( str, HTTP );
    for( int j = 0; j < l + strlen( HTTP ); j ++ )
        * ( p + j ) = ' ';
    * port = 80; //缺省的 http 端口
}
return 1;
}

//发送数据
UINT UserToProxyThread( void * pParam )
{
    char Buffer[ BUFSIZE ];
    int Len;
    sockaddr_in from;
    SOCKET msg_socket;
    int fromlen, retval;
    SocketPair Spair;
    ProxyParam ProxyP;
    CWinThread * pChildThread;
    fromlen = sizeof( from );
    msg_socket = accept( gListen_Socket, ( struct sockaddr * ) &from, &fromlen );
    AfxBeginThread( UserToProxyThread, pParam ); //启动另一侦听
    if( msg_socket == INVALID_SOCKET )
    {
        printf( "\nError in accept " ); return -5;
    }
}

```

```

|
| //读客户端的第一行数据
| Spair. IsUser_ProxyClosed = FALSE;
| Spair. IsProxy_ServerClosed = TRUE;
| Spair. user_proxy = msg_socket;
| retval = recv( Spair. user_proxy, Buffer, sizeof( Buffer ), 0 );
| if( retval == SOCKET_ERROR)
|
|     printf( " \nError Recv" );
|     if( Spair. IsUser_ProxyClosed == FALSE)
|     {
|         closesocket( Spair. user_proxy );
|         Spair. IsUser_ProxyClosed = TRUE;
|     }
|
| if( retval == 0 )
|
|     printf( " Client Close connection \n" );
|     if( Spair. IsUser_ProxyClosed == FALSE)
|     {
|         closesocket( Spair. user_proxy );
|         Spair. IsUser_ProxyClosed = TRUE;
|     }
|
| Len = retval;
| #ifdef _DEBUG
|     Buffer[ Len ] = 0;
|     printf( " \n Received %d bytes, data[ %s ] from client \n", retval, Buffer );
| #endif
|
| Spair. IsUser_ProxyClosed = FALSE;
| Spair. IsProxy_ServerClosed = TRUE;
| Spair. user_proxy = msg_socket;
| ProxyP. pPair = &Spair;
| ProxyP. User_SvrOK = CreateEvent( NULL, TRUE, FALSE, NULL );
| GetAddressAndPort( Buffer, ProxyP. Address, &ProxyP. Port );

```

```

pChildThread = AfxBeginThread( ProxyToServer, ( LPVOID ) &ProxyP );
WaitForSingleObject( ProxyP. User_SvrOK, 60000 ); //等待连接
CloseHandle( ProxyP. User_SvrOK );
while( Spair. IsProxy_ServerClosed == FALSE && Spair. IsUser_ProxyClosed == FALSE )
{
    retval = send( Spair. proxy_server, Buffer, Len, 0 );
    if( retval == SOCKET_ERROR )
    {
        printf( " \n send( ) failed; error% d\n", WSAGetLastError( ) );
        if( Spair. IsProxy_ServerClosed == FALSE )
        {
            closesocket( Spair. proxy_server );
            Spair. IsProxy_ServerClosed = TRUE;
        }
        continue;
    }
    retval = recv( Spair. user_proxy, Buffer, sizeof( Buffer ), 0 );
    if( retval == SOCKET_ERROR )
    {
        printf( " \nError Recv" );
        if( Spair. IsUser_ProxyClosed == FALSE )
        {
            closesocket( Spair. user_proxy );
            Spair. IsUser_ProxyClosed = TRUE;
        }
        continue;
    }
    if( retval == 0 )
    {
        printf( " Client Close connection\n" );
        if( Spair. IsUser_ProxyClosed == FALSE )
        {
            closesocket( Spair. user_proxy );
            Spair. IsUser_ProxyClosed = TRUE;
        }
    }
}

```



```

        break;
    }
    Len = retval;
#ifdef _DEBUG
    Buffer[Len] = 0;
    printf( "\n Received %d bytes, data[ %s] from client\n", retval, Buffer);
#endif
} //End While

if( Spair. IsProxy_ServerClosed == FALSE)
{
    closesocket( Spair. proxy_server);
    Spair. IsProxy_ServerClosed = TRUE;
}

if( Spair. IsUser_ProxyClosed == FALSE)
{
    closesocket( Spair. user_proxy);
    Spair. IsUser_ProxyClosed = TRUE;
}

WaitForSingleObject( pChildThread -> m_hThread, 20000);
return 0;
}

//读取远程主机数据,并发往本地客户端
UINT ProxyToServer( LPVOID pParam)
{
    ProxyParam * pPar = ( ProxyParam * ) pParam;
    char Buffer[ BUFSIZE];
    char * server_name = "localhost";
    unsigned short port ;
    int retval, Len;
    unsigned int addr;
    int socket_type ;
    struct sockaddr_in server;
    struct hostent * hp;
    SOCKET conn_socket;

```

```

socket_type = SOCK_STREAM;
server_name = pPar -> Address;
port = pPar -> Port;
if ( isalpha( server_name[0] ) )
    hp = gethostbyname( server_name );
else
{
    addr = inet_addr( server_name );
    hp = gethostbyaddr( ( char * ) &addr, 4, AF_INET );
}
if ( hp == NULL )
{
    fprintf( stderr, " Client: Cannot resolve address [ %s ]; Error %d\n",
server_name, WSAGetLastError( ) );
    SetEvent( pPar -> User_SvrOK );
    return 0;
}

memset( &server, 0, sizeof( server ) );
memcpy( &( server. sin_addr ), hp -> h_addr, hp -> h_length );
server. sin_family = hp -> h_addrtype;
server. sin_port = htons( port );
conn_socket = socket( AF_INET, socket_type, 0 ); /* 打开一个 socket */
if ( conn_socket < 0 )
{
    fprintf( stderr, " Client: Error Opening socket: Error %d\n",
WSAGetLastError( ) );
    pPar -> pPair -> IsProxy_ServerClosed = TRUE;
    SetEvent( pPar -> User_SvrOK );
    return -1;
}

#ifdef _DEBUG
    printf( " Client connecting to: %s\n", hp -> h_name );
#endif

if( connect( conn_socket, ( struct sockaddr * ) &server, sizeof( server ) ) == SOCKET_ERROR)
{

```

```

fprintf(stderr, "connect( ) failed; %d\n", WSAGetLastError( ));
pPar -> pPair -> IsProxy_ServerClosed = TRUE;
SetEvent( pPar -> User_SvrOK );
return -1;
}

pPar -> pPair -> proxy_server = conn_socket;
pPar -> pPair -> IsProxy_ServerClosed = FALSE;
SetEvent( pPar -> User_SvrOK );
while( ! pPar -> pPair -> IsProxy_ServerClosed && ! pPar -> pPair -> IsUser_ProxyClosed )
{
    retval = recv( conn_socket, Buffer, sizeof ( Buffer ), 0 );
    if ( retval == SOCKET_ERROR )
    {
        fprintf(stderr, "recv( ) failed; error %d\n", WSAGetLastError( ));
        closesocket( conn_socket );
        pPar -> pPair -> IsProxy_ServerClosed = TRUE;
        break;
    }

    //根据 buffer 内容,可以对网站内容进行过滤处理
    Len = retval;
    if ( retval == 0 )
    {
        printf( "Server closed connection\n" );
        closesocket( conn_socket );
        pPar -> pPair -> IsProxy_ServerClosed = TRUE;
        break;
    }

    retval = send( pPar -> pPair -> user_proxy, Buffer, Len, 0 );
    if ( retval == SOCKET_ERROR )
    {
        fprintf(stderr, "send( ) failed; error %d\n", WSAGetLastError( ));
        closesocket( pPar -> pPair -> user_proxy );
        pPar -> pPair -> IsUser_ProxyClosed = TRUE;
        break;
    }
}

```

```

#ifdef _DEBUG
    Buffer[ Len ] = 0;
    printf( " Received %d bytes, data [ %s ] from server\n",retval,Buffer);
#endif
}

if( pPar -> pPair -> IsProxy_ServerClosed == FALSE)
{
    closesocket( pPar -> pPair -> proxy_server);
    pPar -> pPair -> IsProxy_ServerClosed = TRUE;
}

if( pPar -> pPair -> IsUser_ProxyClosed == FALSE)
{
    closesocket( pPar -> pPair -> user_proxy);
    pPar -> pPair -> IsUser_ProxyClosed = TRUE;
}

return 1;
}

//入口函数
int main( int argc, TCHAR * argv[ ], TCHAR * envp[ ])
{
    StartServer( );
    while(1)
    {
        if( getchar( ) == 'q ' ) break;
    }
    CloseServer( );
    return nRetCode;
}

```

实验十一 模拟 FTP 的客户端

描述

基于 FTP 协议,实现一个 FTP 的客户端程序,根据指定的 FTP 服务器地址以及用户名、密码,能够从该服务器上下载选定的文件。

目的

通过该实验了解 FTP 协议,了解网络上常用的文件传输工具的基本功能并加以实现,并对高级编程语言(如 Delphi、VC、VB 等)如何实现文件传输有一定的了解。

基础知识

FTP 协议即远程文件传输协议(File Transfer Protocol)。FTP 协议是一个用于简化 IP 网络上系统之间文件传送的协议,采用 FTP 协议可使 Internet 用户高效地从网上的 FTP 服务器下载大信息量的数据文件,将远程主机上的文件拷贝到自己的计算机上。以达到资源共享和传递信息的目的。

FTP 也是一种客户机/服务器方式的应用,服务器使用 21 号端口。设计 FTP 的目的如下:

- 促进文件的共享;
- 鼓励间接和隐式地使用远程计算机;
- 隐藏主机之间文件地的差异;
- 可靠地、有效地传输文件。

FTP 协议使用简单命令响应的方式来实现,相关的命令参见教材相关章节。

根据 RFC959 标准,它采用两条连接:控制连接传输控制信息,数据则通过数据连接传输,包括目录列表、文件等。它的大致工作过程是:首先建立到 FTP 服务器的端口 21 的 TCP 连接,然后通过控制连接发送用户和口令。如果要进行文件传输,建立数据通道,然后在数据通道上传输文件。FTP 协议提供一系列的命令以及响应代码。

FTP 协议的任务是从一台计算机将文件传送到另一台计算机,它与这两台计算机所处的位置、连接的方式、甚至是否使用相同的操作系统无关。假设两台计算机通过 FTP 协议对话,并且能访问 Internet,你可以用 FTP 命令来传输文件。每种操作系统使用上有某一些细微差别,但是每种协议基本的命令结构是相同的。

FTP 的传输有两种方式:ASCII 传输模式和二进制数据传输模式。

(1) ASCII 传输方式:假定用户正在拷贝的文件包含的是简单 ASCII 码文本,当文件传输时,

FTP 通常会自动地调整文件的内容以便于把文件解释成另外那台计算机存储文本文件的格式。

但是常常有这样的情况,用户正在传输的文件包含的不是文本文件,它们可能是程序、数据库、字处理文件或者压缩文件(尽管字处理文件包含的大部分是文本,其中也包含有指示页尺寸、字库等信息的非打印字符)。在拷贝任何非文本文件之前,用 `binary` 命令告诉 FTP 逐字拷贝,不要对这些文件进行处理,这也是下面要讲的二进制传输。

(2) 二进制传输模式:在二进制传输中,保存文件的位序,以便原始和拷贝的是逐位一一对应的。即使目的地机器上包含位序列的文件是没意义的。例如,macintosh 以二进制方式传送可执行文件到 Windows 系统,在对方系统上,此文件不能执行。

如果你在 ASCII 方式下传输二进制文件,即使不需要也仍会转译。这会使传输稍微变慢,也会损坏数据,使文件变得不能用(在大多数计算机上,ASCII 方式一般假设每一字符的第一有效位无意义,因为 ASCII 字符组合不使用它。如果传输二进制文件,所有的位都是重要的)。如果这两台机器是同样的,则二进制方式对文本文件和数据文件都是有效的。

FTP 支持两种工作方式,一种方式叫做 Standard(也就是 Port 方式,主动方式),一种是 Passive(也就是 PASV,被动方式)。Standard 模式 FTP 的客户端发送 Port 命令到 FTP 服务器;Passive 模式 FTP 的客户端发送 PASV 命令到 FTP Server。

下面介绍一个这两种方式的工作原理:

• Port 方式

FTP 客户端首先和 FTP 服务器的 TCP 21 端口建立连接,通过这个通道发送命令,客户端需要接收数据的时候在这个通道上发送 Port 命令。Port 命令包含了客户端用什么端口接收数据。在传送数据的时候,服务器端通过自己的 TCP 20 端口连接至客户端的指定端口发送数据。FTP server 必须和客户端建立一个新的连接用来传送数据。

• Passive 方式

在建立控制通道的时候和 Standard 模式类似,但建立连接后发送的不是 Port 命令,而是 Pasv 命令。FTP 服务器收到 Pasv 命令后,随机打开一个高端端口(端口号大于 1024)并且通知客户端在这个端口上传送数据的请求,客户端连接 FTP 服务器此端口,然后 FTP 服务器将通过这个端口进行数据的传送,这个时候 FTP server 不再需要建立一个新的和客户端之间的连接。

很多防火墙在设置的时候都是不允许接受外部发起的连接,所以许多位于防火墙后或内网的 FTP 服务器不支持 PASV 模式,因为外部的客户端无法穿过防火墙打开 FTP 服务器的高端端口;而许多内网的客户端不能用 Port 模式登录外部的 FTP 服务器,因为从服务器的 TCP 20 无法和内部网络的客户端建立一个新的连接,造成无法工作。

基于 MFC,下面给出一些相关的代码提示,仅供参考。

1. 首先必须创建相应的 CftpConnection、CInternetSession 对象

```
CInternetSession * m_pInetSession;
```

```
CftpConnection * m_pFtpConn;
```

其中 CInternetSession 表示一种会话类,CftpConnection 表示为一种封装的 ftp 类。

2. 建立连接

```
m_pInetSession = new CInternetSession("ftp", 1, PRE_CONFIG_INTERNET_ACCESS);
```

其中第一个参数表示应用程序的名字,第二个参数标识操作的状态信息,一般置1,第三个参数表示指明访问类型。

创建了 CInternetSession 对象以后,就可以生成一个 CftpConnection 连接了:

```
m_pFtpConnection = m_pInetSession -> GetFtpConnection(serverName, UserName, Password);
```

其中第一个参数为 FTP 服务器的名字或 IP,第二个和第三个参数分别为用户名和密码。

3. 浏览目录

浏览目录的思路如下:根据当前的目录路径,去查询服务器中该目录下的目录和文件情况,一次获得下一个子目录和文件的标题、类型等属性。具体流程可参考一下代码。其中,Cf 是定义的一个类的对象,该类用来表示从服务器得到的一个文件和目录项。

```
BOOL Ret = Cf.FindFile(Dir, 0);
while(Ret)
{
    Ret = Cf.FindNextFile();
    SubDir = Cf.GetFileName();
    if(Cf.IsDots() || ! Cf.IsDirectory())
        continue;
}
```

其中 SubDir 即为当前的文件名和目录名,循环调用就得到该目录下所有的文件和目录。

要求

1. 客户端程序能够管理服务列表,并能够获得服务器上的文件,也可以上传本地文件到服务器上。

2. 如有兴趣,可以设计一个 FTP 的服务器程序,该程序能够管理登录客户端列表,处理每个客户端发来的请求、上传或下载。

实验十二 Internet /Intranet Web 编程

描述

采用 CGI 编程设计并实现一个基于 Web 数据库三层结构的具有留言簿功能的网站。

目的

熟悉 Internet/Intranet Web 服务器的整个体系结构,理解 Web 数据库的三层结构以及 CGI 编程原理,掌握 UNIX 环境下或 Windows 环境下的一种 Script 脚本编程语言。

基础知识

基于服务器端扩展的 Web 数据库技术中,目前主要的服务器端的扩展技术可分为:普通 CGI、混合 CGI 和应用程序编程接口 API 等多种技术。其中,普通 CGI 是最早使用的一种,而且也是目前仍被广泛采用的 Web 数据库实现技术之一,另外两种则是对普通 CGI 的改进和发展。由于性能上的优势,混合 CGI 和 API 技术现在已经越来越被重视和采用,大有替代普通 CGI 之势,但究其原理,混合 CGI 和 API 与普通 CGI 基本一致。下面分别介绍这三种技术。

• CGI 基础

CGI 就是公共网关接口(Common Gateway Interface),CGI 为 Web 服务器定义了一种与外部应用程序共享信息的方法。当 Web 服务器接收到来自某一客户端(Web 浏览器)的请求,要求其启动一个网关程序(通常称为 CGI 脚本或 CGI 程序)时,它把有关该请求的信息综合到一个环境变量集合中,然后,CGI 脚本(程序)将检查这些环境变量,试图找到为响应请求所必需的信息。CGI 还为自己的脚本程序定义了一些标准的方法,以确定如何为服务器提供必需的信息。

早期绝大多数的 Web 数据库应用都是采用普通 CGI 方法来实现的,用户在 Web 浏览器上对相关 HTML 页面中的表单(FORM)进行数据输入、选择等操作后发出激活 Web 服务器上与该页面相关联的 CGI 程序的信息,并将操作信息(如输入的数据等)送到 Web 服务器,这些操作信息可能是用户要在数据库中添加的记录,也可能是查询条件;CGI 程序启动后通过环境变量和标准输入从 Web 服务器获得必要信息后,将其转换为数据库服务器可接受的信息格式,发送到数据库服务器;数据库服务器执行了用户要求的数据操作后,CGI 程序又获得执行结果并将其转换为标准 HTML 数据流发送到标准输出,再由 Web 服务器将 HTML 数据流发送回 Web 浏览器。

利用普通 CGI 实现 Web 数据库访问的方法是发展最早的一种 Web 数据库技术。普通 CGI 方案的优点是:

(1) 简单。众多的用户都会设计包含表单的 HTML 文件及在文件中建立与 CGI 程序的关联;理论上可用任何编程语言来实现一个 CGI 程序,这给普通用户自行开发 Web 应用创造了便利条件。

(2) 价格便宜。CGI 作为传统 Web 交互能力的支撑技术,被几乎所有的 Web 服务器软件所支持。这意味着用户只要安装了 Web 服务器,即已具备了运行 CGI 的基本环境,不需要再投资购买、安装其他的支持软件。

CGI 方案的缺点是:

Web 服务器的瓶颈效应。由于 CGI 程序是在 Web 服务器上运行的,如果并发的对 Web 服务器的请求很多(请注意,这里的请求并不一定都是 CGI 请求,Web 服务器还要服务于用户对 HTML 页面的请求等),Web 服务器将非常忙碌,严重时可能会造成信息的阻塞。

普通 CGI 程序运行时对资源的“独占”。普通 CGI 在启动时被 Web 服务器调入存储器中,而在执行结束后会像普通的执行程序一样被清出存储器。而在一个 CGI 程序运行时,如果有其他用户需访问同一个 CGI 程序,则这个正在运行的 CGI 程序并不能为新的用户请求所共享,而必须由 Web 服务器再启动一次相同的 CGI 程序并占用另外的独立资源。可以想像,当并发的相同 CGI 请求很多时,对服务器资源的利用是非常低效的。上述对 CGI 优点和缺点的分析,实际上适用于所有利用 CGI 方法实现的 Web 应用,而并非只是 CGI 应用在 Web 数据库技术上的特点。

• 混合 CGI

混合 CGI 是对普通 CGI 的一种改进,在混合 CGI 结构中,服务器端的扩展程序分为两个部分:“瘦”CGI 程序和“胖”伙伴进程。这个伙伴进程(在 Windows NT 系统中为“系统服务”,而在 UNIX 系统中为守候进程“daemon”)通常只在操作系统启动的时候被装载一次并且在没有用户有意中断的情况下始终在后台运行。现代操作系统支持的进程间通信(如命名管道“named-pipe”等)允许多个 CGI 程序与单个系统服务或守候进程通信。利用这种机制,对于来自客户浏览器的每个 CGI 请求,Web 服务器调用“瘦”CGI 程序来接收环境变量和用户数据;然后 CGI 程序仅把数据发送给伙伴进程而不做任何其他工作;伙伴进程接收到数据后完成几乎所有主要的工作,包括建立与数据库服务器的连接,发送对数据的操作命令和获得执行结果等。这样,CGI 程序可以很“瘦”,可装载得更快,而且对 Web 服务器资源的占用更少,减轻对 Web 服务器的负载;同时,由于主要功能的实现模块——伙伴进程可以共享,又大大提高了运行的性能。另外,针对数据库操作的特点,可在伙伴进程中实现在 CGI 程序结束后保持与数据库的连接来进一步改善性能。这种对数据库连接的“缓存”减少了下一个对相同数据库请求的响应时间。

• 应用程序编程接口(API)

普通 CGI 中对同一个 CGI 程序的每一次调用都会使 Web 服务器装载该程序,从而造成普通 CGI 程序启动速度慢、资源利用效率低。针对这一缺点,一种基于共享 CGI 程序的改进方案应运而生,这就是应用程序编程接口(API)。API 对于大多数程序员来说并不陌生,而这里所指的 API 在本质上与其他的 API(如 MSWindows API)并没有不同,是某个 Web 服务器开发商为其产

品用户开发类似于 CGI 程序的服务器端扩展程序所提供的专用编程接口。其技术核心也是利用操作系统所支持的共享和动态装入。基于 API 的 Web 应用程序的结构(以 Web 数据库应用为例)为:服务器端扩展程序调用 API 实现了普通 CGI 的功能。这样的扩展程序在 MSWindows 系列操作系统下以动态链接库(DLL)形式存在,而在 UNIX 环境下则是共享对象形式。这种扩展程序的本质是一组编译后的对象代码例程,被 Web 服务器调用,看起来像是 Web 服务器内核程序的一部分。因此它与 Web 服务器结合更紧密、占用资源也很少,对多用户共享的有力支持使其运行效率大大优于普通 CGI。

不过,API 的高效是以下列的缺点为代价的:

各种 API 之间兼容性差。各种 Web 服务器所支持的 API 不尽相同,缺乏统一的标准。

基于 API 的服务器端扩展程序的开发比普通 CGI 程序的开发更困难。为了充分发挥某种 API 的效能,往往要使用特定的开发语言和工具,这对大部分不具有专门技术力量的用户来说是一个很大的障碍。

API 扩展程序作为 Web 服务器内核程序的扩展,如果自身不够健壮,一旦运行出现故障,将有可能导致整个 Web 服务器瘫痪。普通 CGI 程序则至多被从存储器中强行退出而不会影响 Web 服务器系统的运行。

要求

1. 设计一个网站,实现留言簿功能,具有填写留言簿,查看留言簿等功能。
2. 留言簿中的数据存储于数据库中,不允许存储在文件中。

实验十三 网络服务器

描述

网络服务器的管理与维护。

目的

了解局域网管理与维护的基本概念,熟悉网络服务器配置的常规操作。

基础知识

一般说来,一个局域网的管理和维护包括了以下 4 个方面:

(1) 首先是对用户的需求有一个比较清晰的了解,并且也应当考虑以后可能的应用来对网络进行规划,比如确定基本的布线方案、局域网操作系统的选择、网络服务器的配置等。

(2) 使用相应的工具进行实际的布线工作。现在,绝大多数的局域网都是使用双绞线来进行连接的,通过把计算机和 HUB 连接起来形成星形拓扑结构。

(3) 在选择好了机器的配置以及操作系统后,就开始安装操作系统。现在市场上的操作系统主要有两种,一种是 UNIX,另一种就是 Windows。这两个操作系统都各有自己的优势,选择哪一种可能需要考虑到各种因素。

(4) 管理和维护网络服务器。

要求

1. UNIX 或 Windows 系统的规划管理。
2. UNIX 或 Windows 系统的安装。
3. 用户的增加和删除。
4. 用户和文件权限的管理。
5. 与其他操作系统的互连,比如在 UNIX 和 Windows 系统之间如何共享资源等。
6. 网络资源的使用,比如如何映射网络驱动器等。
7. 安装 TCP/IP 协议,如果有条件可以进行 Web 服务、DHCP 服务以及如何把服务器用作路由功能等方面的实验。

实验十四 网络管理软件

描述

利用网络管理软件熟悉所在网络的各种基本信息。

目的

熟悉至少一种网络管理软件,能够对网络管理所要完成的功能有一个大致的概念性的认识

基础知识

随着网络技术的发展,网络的规模变得越来越大,所使用的网络设备也越来越复杂,如果单纯地靠网络管理人员手工来进行管理,这是不可想象的。网络管理软件有助于帮助网管人员来完成越来越复杂的网络管理功能。

到目前为止,许多公司都推出了各自的网络管理软件或平台,在这些产品之中,比较著名的是 HP Openview、Sun Netmanger 等,这些软件各有自己的优势,不过它们提供的功能都是大同小异。读者在完成这个实验时可以根据自己的情况选择任何一种网管软件,通过使用网管软件,能够对网络管理所要完成的功能有一个大致的概念性的认识。

要求

1. 利用网络管理软件发现和构造所在网络环境的拓扑图。
2. 监视所管理的网络设备的状态信息,比如可以把某个设备的状态改变(关机、重启或改变某些配置参数),看看如何利用网管软件来发现故障和处理故障。
3. 学会如何配置网络管理软件以使得它能够对某些重要的事件做出及时的记录、响应等。
4. 学会如何利用网络管理软件监视网络的性能,找出可能发生过载的地方,以便能够为合理地重新配置和维护网络提供参考。